Chapter 1 openGauss Product Architecture



HUAWEI

Foreword

- This chapter describes the openGauss database architecture from different perspectives to help gain an overall and comprehensive understanding of the database. System architecture: three-layer database system structure and the innovation of openGauss in resource pooling; Software architecture: functions and principles of database components; Logical architecture: working principles of the kernel, common capabilities, and OM&CM; Thread architecture: working principles of main worker threads in each database module.

HUAWEI

Objectives

- Upon completion of this course, you will be able to:
  - Describe the overall database system architecture;
  - Master functions of each component of the openGauss database;
  - Describe the innovation of openGauss in resource pooling;
  - Master the functions and working principles of threads in each database module.

HUAWEI

Contents

1. **System Architecture**
2. Software Architecture
3. Logical Architecture
4. Thread Architecture

HUAWEI

openGauss Overall Architecture

- The overall architecture of openGauss can be divided into three layers: storage layer, computing layer, and user layer. The storage layer is responsible for data storage and management, the computing layer is responsible for data computing and O&M, and the user layer serves as the interface between openGauss and user delivery.
  - The storage layer consists of three parts: storage engine, storage manager, and data files. The storage engine is responsible for data storage and access operations, and the storage manager is responsible for data storage layout, capacity management, and transaction management.
  - The computing layer consists of the computing engine and task scheduler. The computing engine is responsible for operations such as query computing and data processing, and the task scheduler is responsible for scheduling and allocating tasks.
  - The user layer consists of clients and application programming interfaces (APIs). Clients are used for users to interact with openGauss, and APIs are used for developers to interact with openGauss.



HUAWEI

Storage Layer Architecture

- The storage engine is responsible for database storage and operations. The storage engine uses a hierarchical structure and consists of physical storage and logical storage.
  - Physical storage is used to manage underlying devices and access data. Data files are the main form of physical storage, including tablespaces, system tablespaces, and user tablespaces.
  - Logical storage is data storage and management based on physical storage, that is, management of database objects such as indexes, tables, and views. Logical storage uses multi-version concurrency control (MVCC) to implement data concurrency control.
- The storage manager is mainly responsible for data file management, transaction management, and capacity management.
  - Data file management includes data storage layout, data page management, and space allocation and recycling.
  - Transaction management is responsible for data concurrency control and transaction commit or rollback.
  - Capacity management controls and schedules the database capacity.
- Data files include tablespaces, system tablespaces, and user tablespaces.
  - A tablespace is a physical storage unit used by openGauss to store data and indexes.
  - A system tablespace stores default data and objects, such as pg_catalog and pg_toast.
  - A user tablespace is a storage space created by a user.

HUAWEI

Computing Layer Architecture

- The computing engine is mainly responsible for query computing and data processing and uses multiple computing pipelines for parallel processing to improve query performance. It consists of three parts: SQL interpreter, optimizer, and executor.
  - An SQL interpreter parses SQL statements entered by users into computing models.
  - An optimizer converts SQL statements into optimal execution plans to improve query performance.
  - An executor is responsible for SQL query and data processing, and is the last process of the computing engine.
- A task scheduler schedules and allocates tasks, and manages the number of concurrent tasks to achieve efficient system utilization.
  - The task scheduler uses the multicast mechanism. When the task status changes, the task scheduler synchronizes messages to other nodes in the cluster in a timely manner for quick response.

HUAWEI

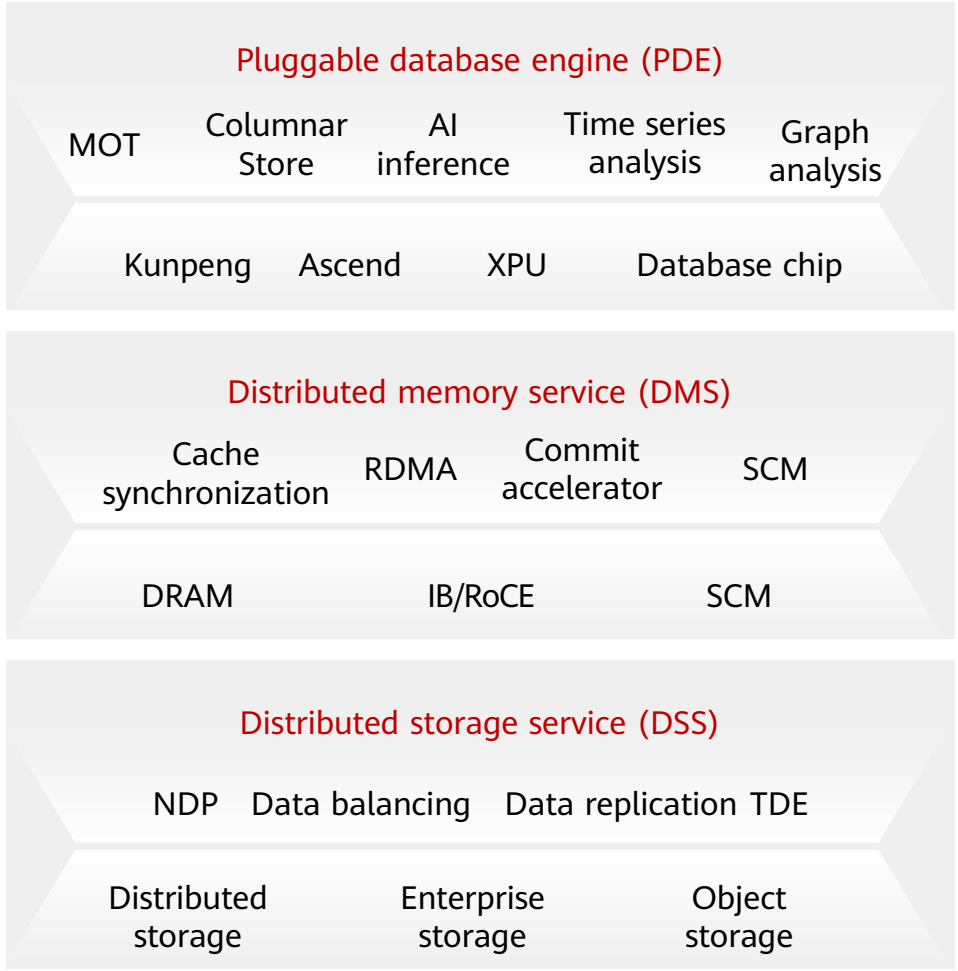
User Layer Architecture

- Clients are used for users to interact with openGauss. There are two types of clients: command line interface (CLI) client and graphical user interface (GUI) client.
  - The CLI client mainly provides a CLI for users to operate and manage data.
  - The GUI client provides a GUI for users to operate and manage the database more conveniently.
- APIs are used for developers to interact with openGauss, including JDBC, ODBC, ODP.Net, and PHP APIs. These APIs facilitate developers to access and manage the database.

HUAWEI

Resource Pooling Architecture

The openGauss resource pooling architecture consists of three pooling modes, one platform, and one standard. The three pooling modes include storage pooling, memory pooling, and computing pooling.



Main capabilities of computing pooling:

- Microkernel supporting diversified computing power
- Database chips accelerating TP and AP with a high energy efficiency ratio (improved by 10X)
- AI training and inference with Ascend processors
- AI-powered real-time trend prediction

Main capabilities of memory pooling

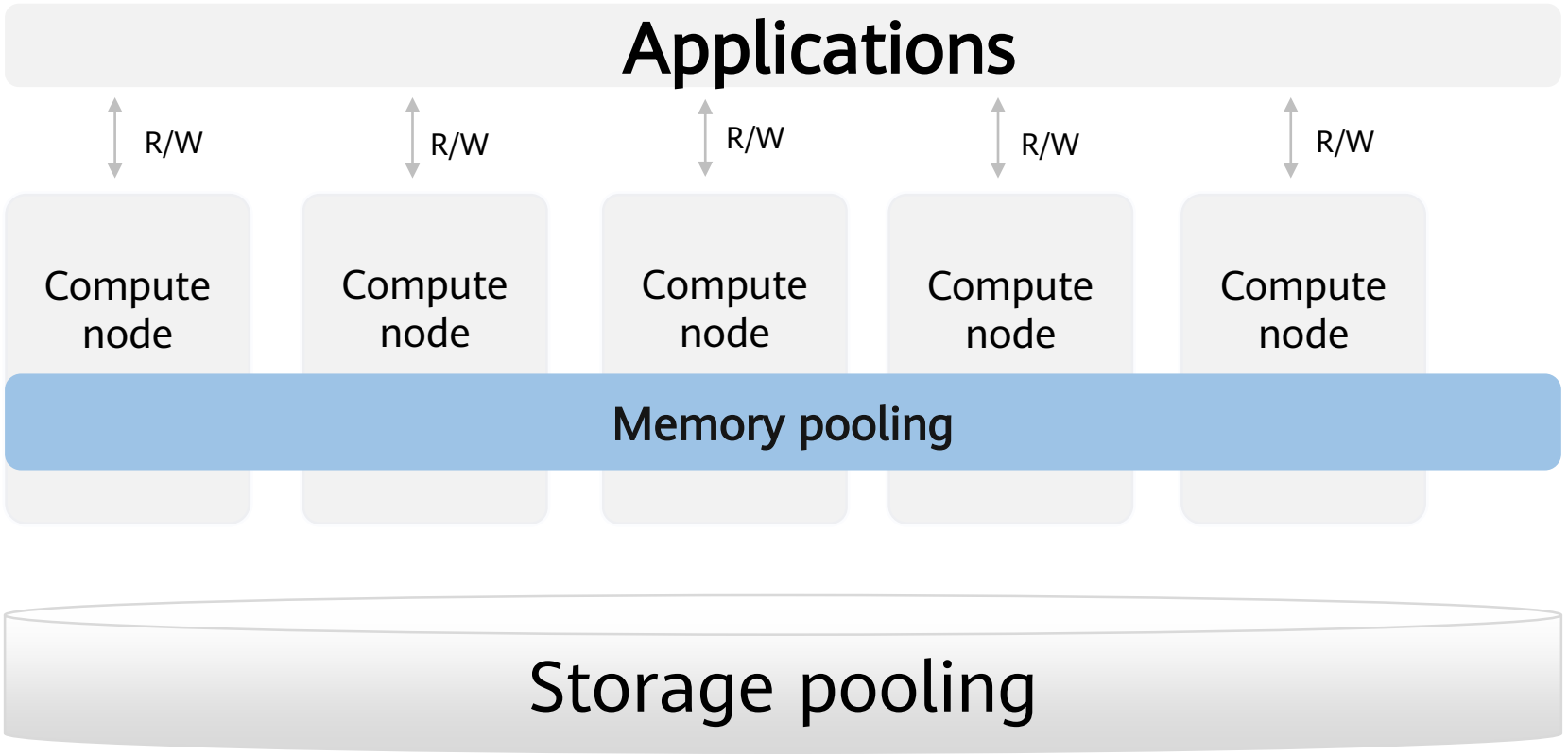
- High-speed network interconnection
- Database transaction synchronization and multi-machine memory pooling (8 nodes)
- Persistent-memory-based SCM, data cache acceleration, and large-capacity column store
- Scale-out linearity higher than 1.5 and complex query performance improved by 10X

Main capabilities of storage pooling

- Shared storage operator offloading and intra-city HA storage and replication
- Nearby data computing, supporting multiple types of storage
- Intra-city replication for enterprise-class storage; RTO < 1 minute
- Network traffic reduced by 80%, and intra-city DR within 1 minute

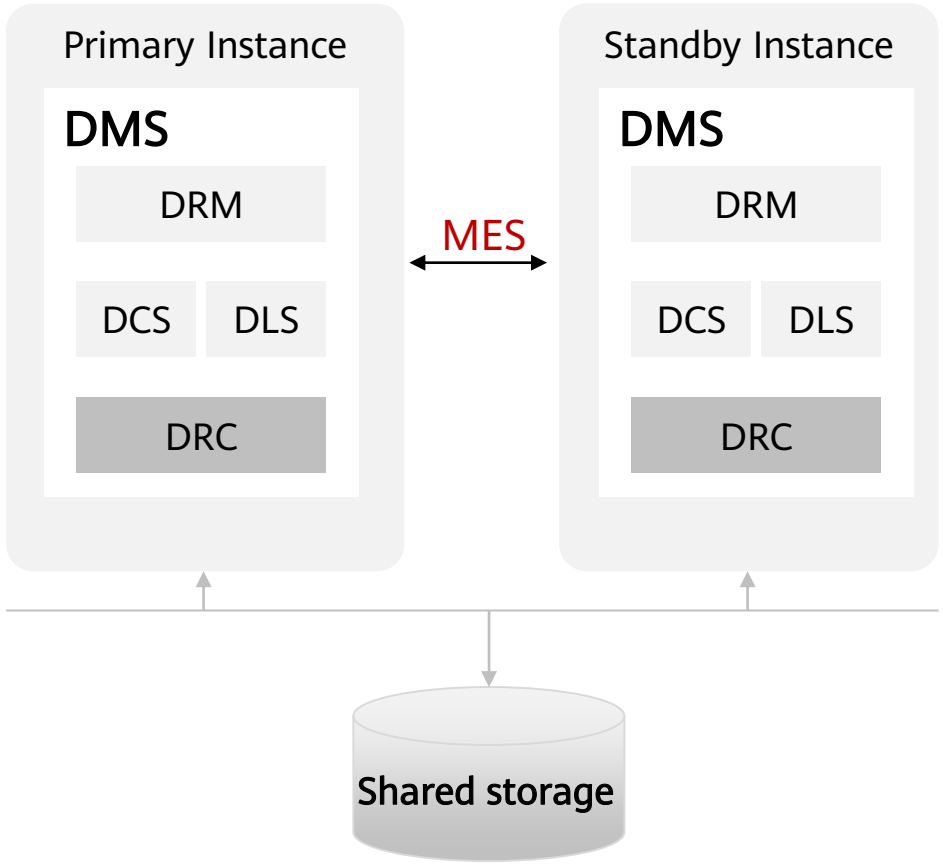
HUAWEI

Memory Pooling - Transparent Multi-Write and On-Demand Scale-Out



- Transaction snapshot information is synchronized across nodes, achieving real-time data consistency and linear expansion of read-only load performance.
- Write operations in a transaction are automatically sent to related compute nodes. Application requests can be transparently distributed to any node. The number of nodes can be scaled out as required.

Memory Pooling - Multi-Node Real-Time Consistency



Real-time page data sharing between primary and standby nodes

- Node caches are interconnected through high-speed and low-latency internal networks while transaction snapshot information and data are shared in real time.

Quick restoration of faulty node instances

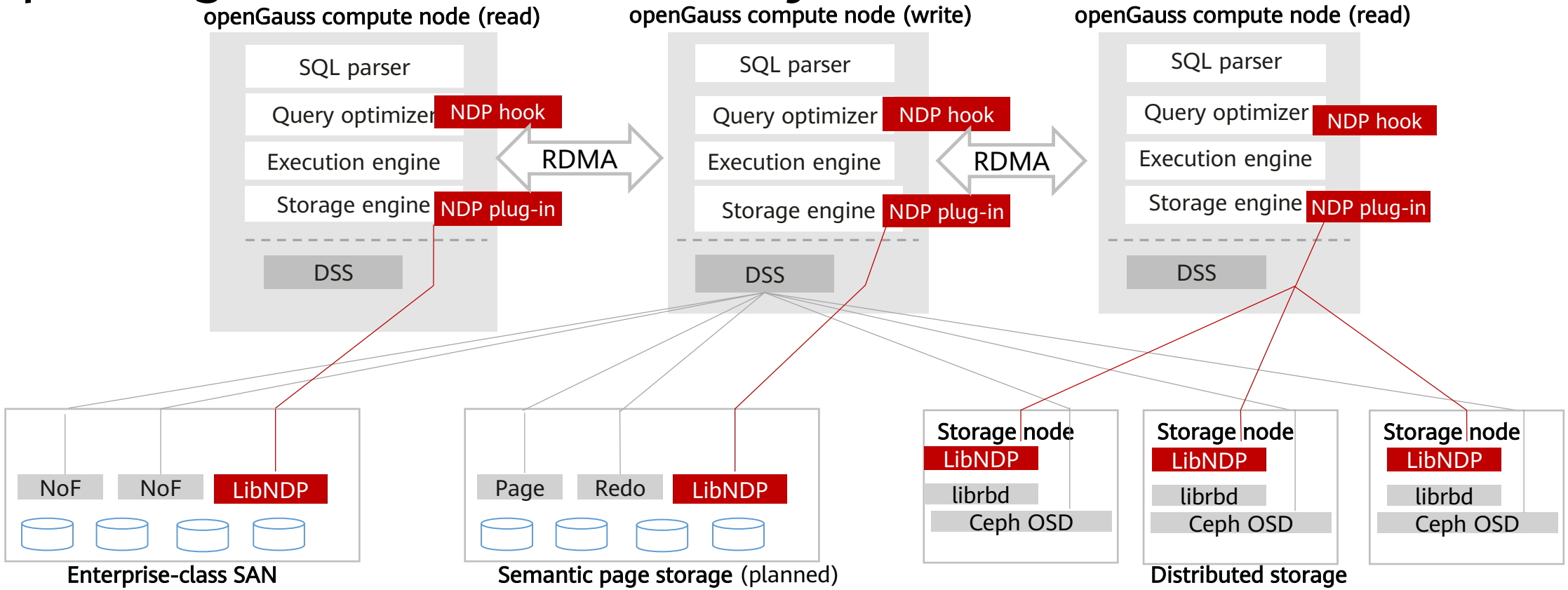
- The primary and standby nodes share the same data, eliminating the log replication overhead during normal running.
- Services are switched to the standby node when the primary node is faulty, quickly recovering the cluster.

Low-latency RDMA network

- A lightweight RPC framework is implemented based on high-performance RDMA networks.
- The CPU overhead is significantly reduced while achieving a microsecond-level network latency.

HUAWEI

Storage Pooling - Improving Storage Efficiency and Improving NDP Performance by 2X

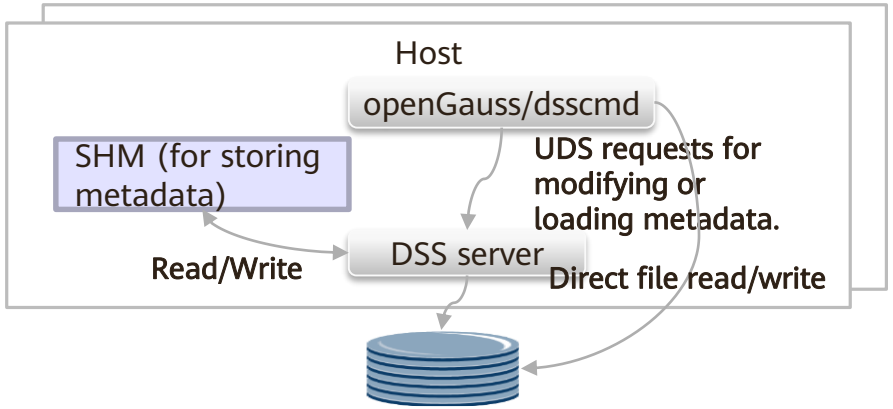


- Support enterprise-class SAN storage, page store (planned), and distributed storage, providing the best choice for different service scenarios such as TP and AP. Multiple nodes share one copy of data, eliminating log replication overheads and improving storage efficiency by more than 50%. NDP is planned, which can greatly reduce network I/O traffic at the storage and computing layers.
- Boast efficient bare metal access and database's native file system capability; support up to eight volume groups of capacity, with each volume group up to 8 PB and each file up to 8 TB.

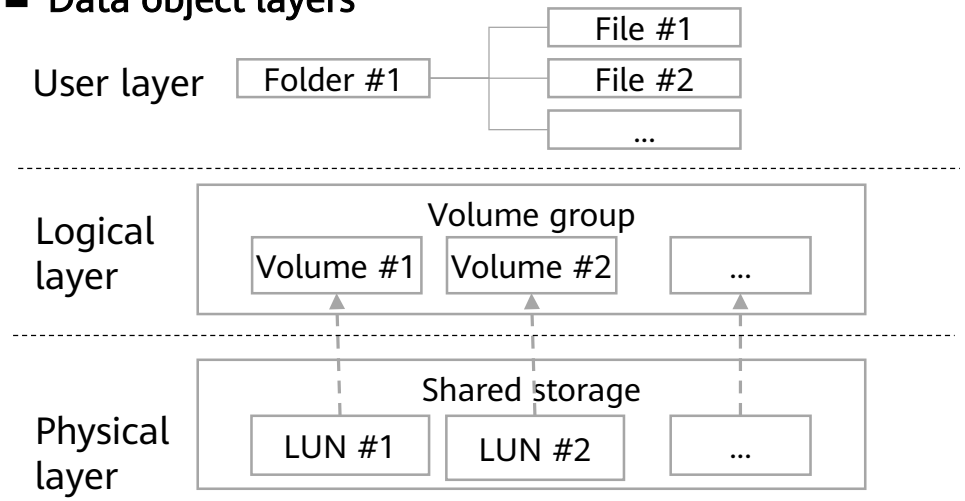
HUAWEI

Storage Pooling - Efficient Bare Metal Access and Metadata Sharing

- Multi-node file system



- Data object layers



Efficient bare metal access

- Bare metal servers are used to bypass the file system layer. Data is directly transmitted from disks to applications, reducing the file system cache overhead and improving disk throughput through asynchronous I/O.
- The original partitioning method, instead of the file system management method, is used, which reduces the overhead of maintaining the file system and improves the file access performance.

Cross-node sharing of file metadata

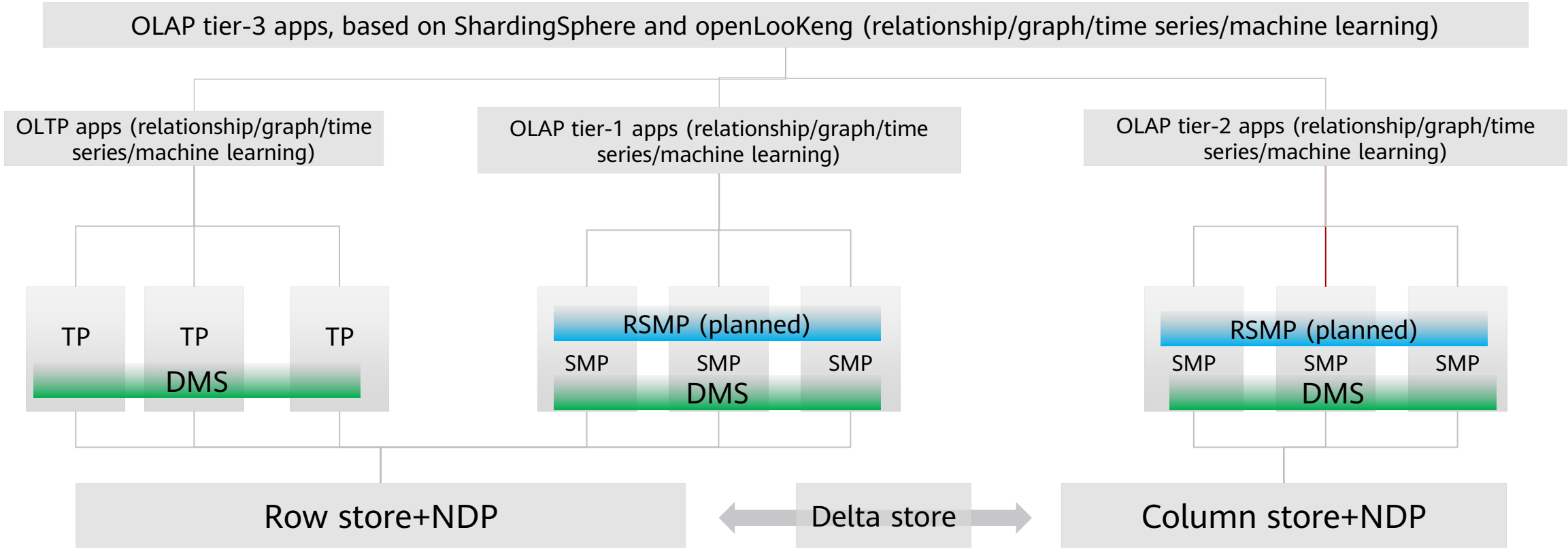
- The file metadata persistence mechanism is implemented based on redo logs, ensuring a highly reliable file system.
- Multiple nodes share metadata. The standby node is loaded in lazy mode and lightweight, improving cluster performance.

Simplified file access APIs

- APIs for creating, deleting, and accessing volumes, volume groups, directories, and files are provided, and efficient tools are developed to manage virtual file systems.
- A host supports up to eight volume groups, with each volume group up to 8 PB and each file up to 8 TB.

HUAWEI

Computing Pooling - Multi-Mode Convergence

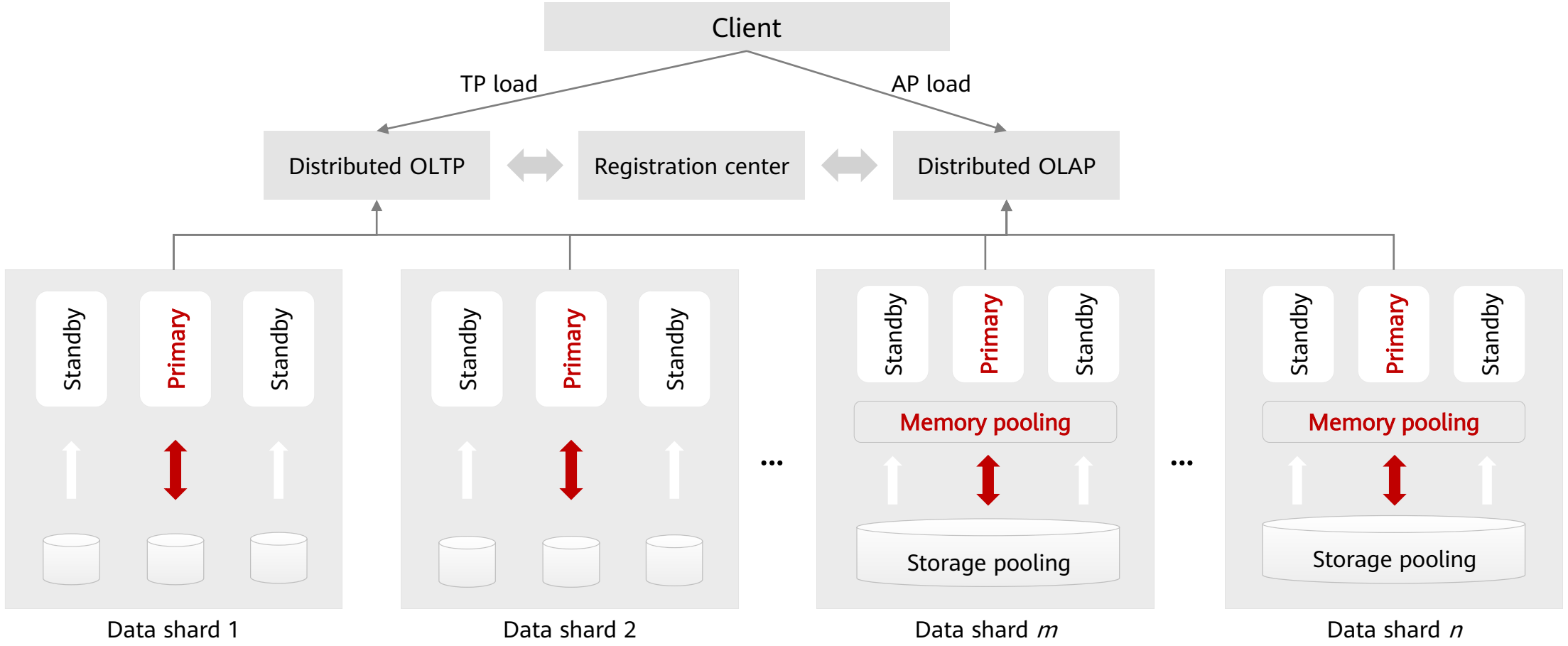


- OLTP apps: row storage engine+row computing engine; DMS for real-time data consistency among multiple nodes; NDPQ (with operator offloading)
- OLAP tier-1 apps: row storage engine+row computing engine; DMS for real-time data consistency among multiple nodes; SMP for parallel query within a single node; RSMP for parallel query among multiple nodes
- OLAP tier-2 apps: column storage engine+vector computing engine; DMS for real-time data consistency among multiple nodes; SMP for parallel query within a single node; RSMP for parallel query among multiple nodes
- OLAP tier-3 apps: Build a large-scale HTAP solution based on ShardingSphere and openLooKeng to meet enterprises' requirements for processing massive data services.

HUAWEI

Computing Pooling - Building an HTAP Solution Based on ShardingSphere and openLooKeng

openLooKeng accesses metadata from the ShardingSphere registration center, subscribes to metadata modification notifications, and supports cross-shard SQL query rewriting and optimization based on metadata.



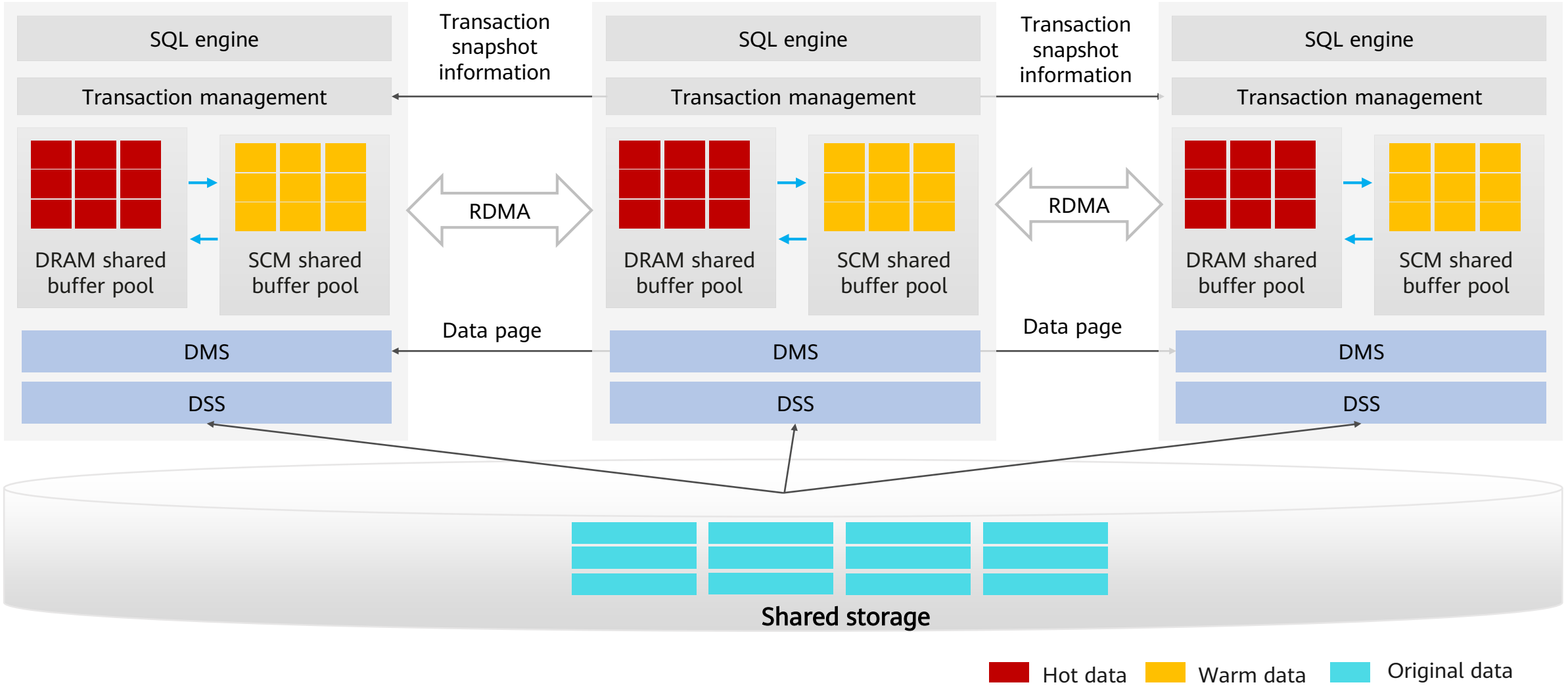
HUAWEI

Contents

1. System Architecture
2. **Software Architecture**
3. Logical Architecture
4. Thread Architecture

HUAWEI

Resource Pooling Architecture



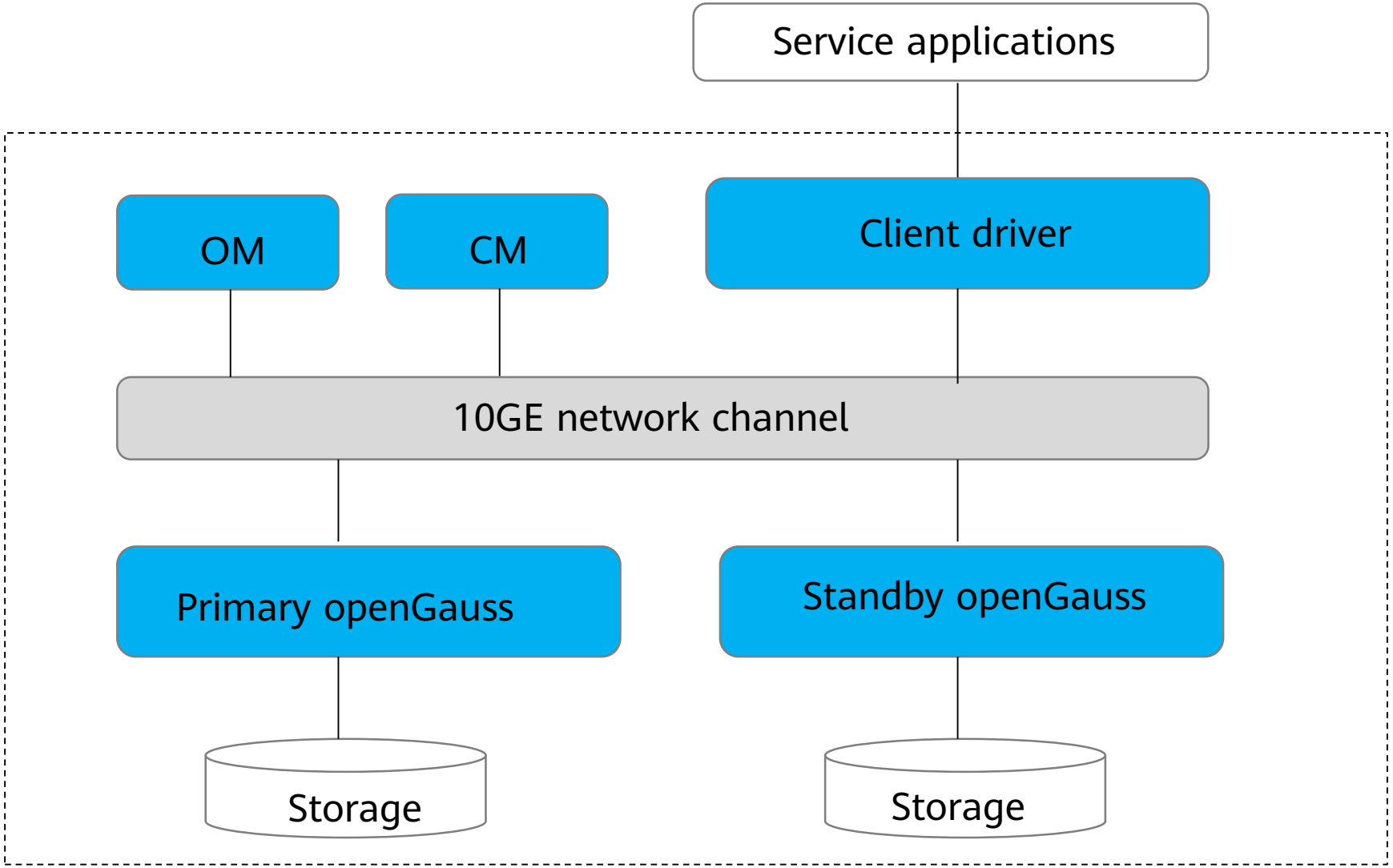
HUAWEI

Resource Pooling Architecture - Component Description

Name	Description
DMS	DMS is a dynamic library integrated in the database. It transmits page content through the TCP/RDMA network, integrates the primary and standby memories, and provides the memory pooling capability to implement real-time consistent read on the standby node.
DSS	DSS is an independent process that directly manages raw disk arrays and provides capabilities similar to distributed file systems for external systems. The shared memory and client API dynamic library provide the database with the capabilities of creating files, deleting files, expanding and shrinking files, and reading and writing files.
Shared storage	Enterprise-class storage and distributed storage are supported. Compared with traditional database creation, resource pooling classifies directories into three types: exclusively used by each instance but not shared, exclusively used and shared by each instance, and shared by all instances. The directories to be shared must be stored on the shared storage, and the directories that are not shared must be stored on the local disk. In addition, to create a database on the standby node, you only need to create directories that belong to the standby node. You do not need to create directories shared by all instances.
OCK-RDMA	OCK-RDMA can be used to reduce the latency of DMS primary/standby page switching. After OCK-RDMA is enabled, the consistent read latency on the standby node is improved by over 20%.
SCM	Memory persistence-based SCM improves performance by 30% at the same memory cost.

HUAWEI

Primary/Standby Architecture



HUAWEI

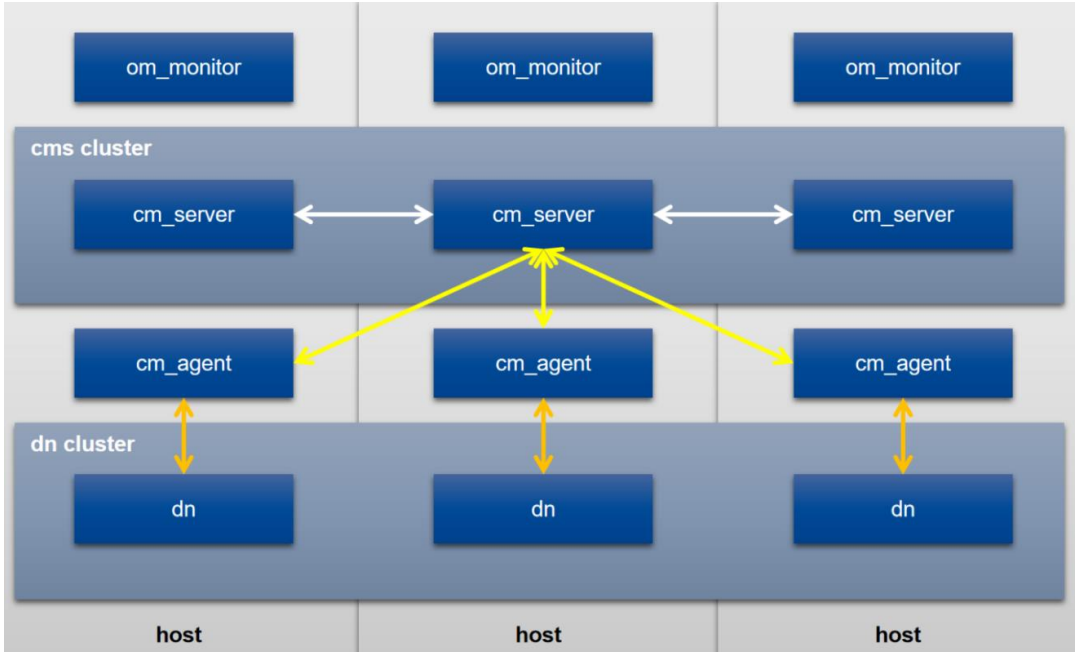
Primary/Standby Architecture - Component Description

Name	Description
OM	The operation manager (OM) provides management APIs and tools for routine maintenance and database configuration.
CM	The cluster manager (CM) manages and monitors the running status of functional units and physical resources in a database system, ensuring stable running of the entire system.
Client driver	The client driver receives the access requests from applications, and returns the execution results to the applications. It communicates with openGauss instances, delivers SQL commands from applications, and receives openGauss execution results.
openGauss (primary/standby)	openGauss primary/standby data nodes (DNs) store service data, execute data queries, and return execution results to clients. openGauss supports one primary and multiple standby instances. You are advised to deploy primary and standby instances on different physical nodes.
Storage	Functions as the server's local storage resources to store data permanently.

HUAWEI

CM - Networking

- CM includes three core modules: CM Agent, CM Server and OM Monitor (CMM). To ensure high availability (HA) of openGauss data nodes (DNs), the following networking conditions must be met:
  - The CMM and CM Agent must be deployed on each node. The CMM monitors the CM Agent instance on the local node. The CM Agent monitors the DN and CM Server instances on the local node, collects information, and reports the information to the primary CM Server.



CM - Keepalive Mechanism

- The CM component consists of the cm_server, cm_agent, and om_monitor processes, which correspond to the CM Server, CM Agent, and CMM modules, respectively. The openGauss (DN) process is gaussdb.
  - Keepalive of DNs and CM Server: The CM Agent is responsible for keeping DNs and CM Server alive. The CM Agent checks the health status of DNs and CM Server every second. If any process does not exist, it starts the corresponding process. The CM Agent checks the DN and CM Server process status and checks whether a process is a zombie process for multiple times within the specified period. If a process is a zombie process, the CM Agent kills the zombie process and restarts it.
  - Keepalive of CM Agent: The CMM is responsible for keeping the CM Agent alive. The CMM checks the CM Agent health status every second. If the CM Agent process does not exist, the CMM starts the process. If the CM Agent process is in zombie state, the CMM kills the zombie process and restarts it.
  - Keepalive of CMM: a system scheduled task is used to keep the CMM alive. The system scheduled task is executed at the 0th second every minute. If the process does not exist, the process is started when the next scheduled task is executed. If the process enters the zombie state, the zombie process is killed when the next scheduled task is executed and then started again.

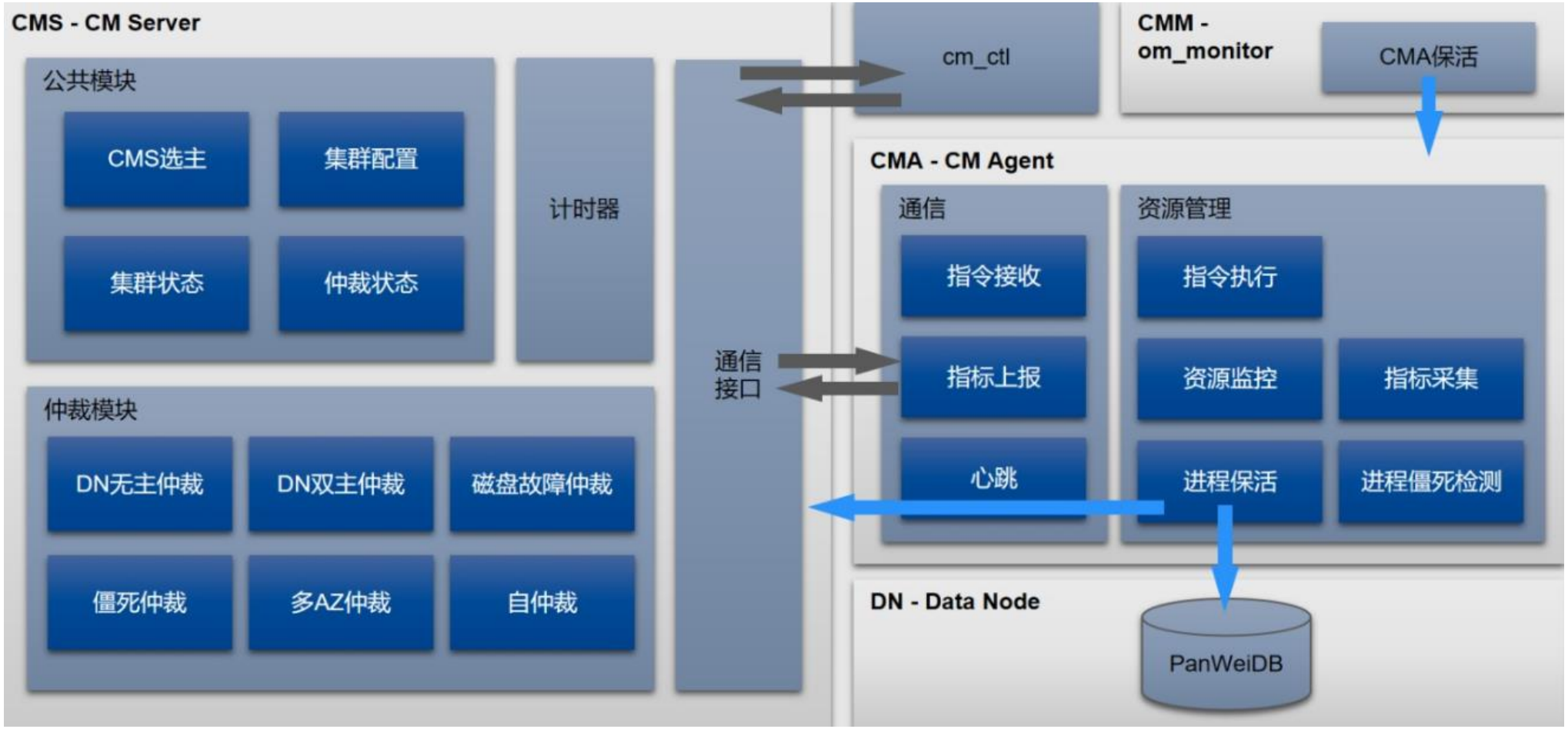
```
[                    ]$ ps ux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
pwdb      982576  0.0  0.0  25088 14592 ?        Ss   Jan13   0:01 /usr/lib/systemd/systemd --user
pwdb      982577  0.0  0.0 202880 12736 ?        S    Jan13   0:00 (sd-pam)
pwdb      999762  1.6  0.0  27264 15040 ?        S    Jan13  21:57 /home/pwdb/app/bin/om_monitor -L /home/pwdb/log/pwdb/cm/om_monitor
pwdb     1008496  8.8  0.0 810880 22400 ?        Sl   Jan13 120:39 /home/pwdb/app/bin/cm_agent
pwdb     1008513  5.6  0.0 6579200 468480 ?      Sl   Jan13  77:02 /home/pwdb/app/bin/cm_server
pwdb     1008536  7.8  0.3 8138880 1739328 ?     Sl   Jan13 107:20 /home/pwdb/app/bin/gaussdb -D /home/pwdb/data -M pending
pwdb     1008543  0.0  0.0 1398784 81536 ?       Sl   Jan13   0:00 gaussdb fenced UDF master process
pwdb     2554227  0.0  0.0 224768  6912 pts/1    S    12:15   0:00 -bash
pwdb     2555378  0.0  0.0 227264  6272 pts/1    R+   12:16   0:00 ps ux
```

```
                    ]$ crontab -l
*/1 * * * * source /etc/profile;(if [ -f ~/.profile ];then source ~/.profile;fi);source ~/.bashrc;nohup /home/pwdb/app/bin/om_monitor -L /home/pwdb/log/pwdb/cm/om_monitor >>/dev/nu
ll 2>&1 &
```


HUAWEI

CM - Architecture of Main Modules

- Overall architecture of the CM component



HUAWEI

CM - CM Agent

- In multi-thread mode, the CM Agent monitors the local DN and CM Server and interacts with the CM Server and other DNs. It provides the following functions:
  - Checks the connection to the primary CM Server.
    - The ConnCmsPMain thread of the CM Agent cyclically checks the connection to the primary CM Server at an interval specified by **AGENT_RECV_CYCLE**, which is 200 milliseconds by default. If the connection does not exist, it re-establishes the connection.
  - Checks the connections to all peer DNs.
    - The DNConnectionStatusCheckMain thread of the CM Agent cyclically pings all peer DNs at an interval specified by **agent_report_interval**, which is 1 second by default.
    - If it disconnects from all peer DNs and the local node is the primary DN, the fault is reported to the primary CM Server and the local DN process is killed. The primary CM Server starts leader election for DNs.
  - Checks and reports DN status.
    - The DNStatusCheckMain thread of the CM Agent cyclically checks DN status at an interval specified by **agent_report_interval**, which is 1 second by default.
    - If the process does not exist, the fault is reported to the primary CM Server to start the process. The CM Server starts leader election for DNs.
  - Checks and reports CM Server status.
    - The CM Agent checks the CM Server status on the local node at an interval of 1 second. If the process does not exist, the CM Agent reports the fault to the primary CM Server to start the process. The CM Server HA module starts the primary and standby arbitration.
    - If a process is in the T state, the process is regarded as a zombie process. In this case, the zombie process is reported to the primary CM Server, killed, and restarted. The CM Server HA module starts the primary and standby arbitration.
  - Detects disk usage and disk faults.
    - The CM Agent creates temporary files in the data and log paths and performs read/write tests (fopen, fwrite, and fread) to detect DN and CM Server disk faults. If a failure occurs, the CM Agent reports the "disc damage" status error to the primary CM Server.

CM - CM Server

- The CM Server interacts with DNs through the CM Agent. Based on the Quorum protocol, the CM Server cluster implements leader election for DNs and self-arbitration. The core functions are as follows:
  - Processes leader election for DNs.
    - The CM Server receiver thread receives the DN status periodically reported by the CM Agent and detects DN exceptions. If the primary DN is faulty, leader election is started. After a candidate DN is determined, the process of promoting the DN to primary is started. If the process is determined as a zombie process, zombie processing is triggered. If the zombie process is located on the primary node, leader election is triggered.
  - Processes CM Server self-arbitration.
    - The CM Server receiver thread receives the CM Server status periodically reported by the CM Agent. If the primary CM Server is faulty, leader election is triggered for the CM Server cluster based on the Quorum protocol.
  - Rectifies faults caused by storage usage reaching the threshold or disk faults.
    - The CM Server receiver thread receives the usage of disks where the DN data and log paths are located, which is periodically reported by the CM Agent, and detects faults at an interval specified by **datastorage_threshold_check_interval**, which is 10 seconds by default. The detection logs are generated every minute.
    - When the disk usage exceeds 85% of the read-only threshold, a warning is generated; if the disk usage does not exceed 85% of the read-only threshold, the warning is cleared. The read-only threshold is controlled by **datastorage_threshold_value_check** and is 85% by default. When the read-only threshold is exceeded, a read-only command is sent to all DNs through CM Agent and a read-only alarm is sent. When the read-only threshold is not exceeded, the read-only commands sent to all DNs are cleared through CM Agent and the read-only alarm is also cleared.
    - Note that the read-only status is set at the cluster level. Therefore, if the disk usage of any node in the cluster reaches the threshold, the cluster is set to read-only. As a result, failure messages will be returned for write requests.

HUAWEI

Enhanced CM Features

- CM supports external status query and push.
  - The HTTP/HTTPS service is used to remotely query the cluster status, helping management personnel and O&M platforms monitor the cluster status.
  - When a switchover occurs in the database cluster, the latest primary/standby information of the cluster is pushed to the receiving address registered by the application through the HTTP/HTTPS service in time. In this way, the application can detect the primary/standby change of the cluster in time and quickly connect to the new primary and standby nodes.
- CM supports the two-node deployment mode.
  - The minimum number of nodes in a CM cluster is reduced from 3 to 2, which brings significant cost advantages.
  - The third-party gateway IP address is introduced to effectively solve the self-arbitration problem in the two-node CM cluster deployment mode. In addition, the CM cluster failover policy and database cluster split-brain fault recovery policy can be dynamically configured to ensure the integrity and consistency of cluster data.
- CM deployment can be decoupled from database deployment.
  - If an openGauss database cluster has been deployed but no CM has been deployed, you can deploy the CM cluster directly without upgrading the database.
- The CM service can be suspended or resumed in one-click mode.
  - The CM automatic fault handling service can be suspended by one click to prevent O&M personnel's operations during O&M from being affected by the CM. After the O&M is complete, the CM service can be resumed by one click.
- User-defined scripts can be called by event.

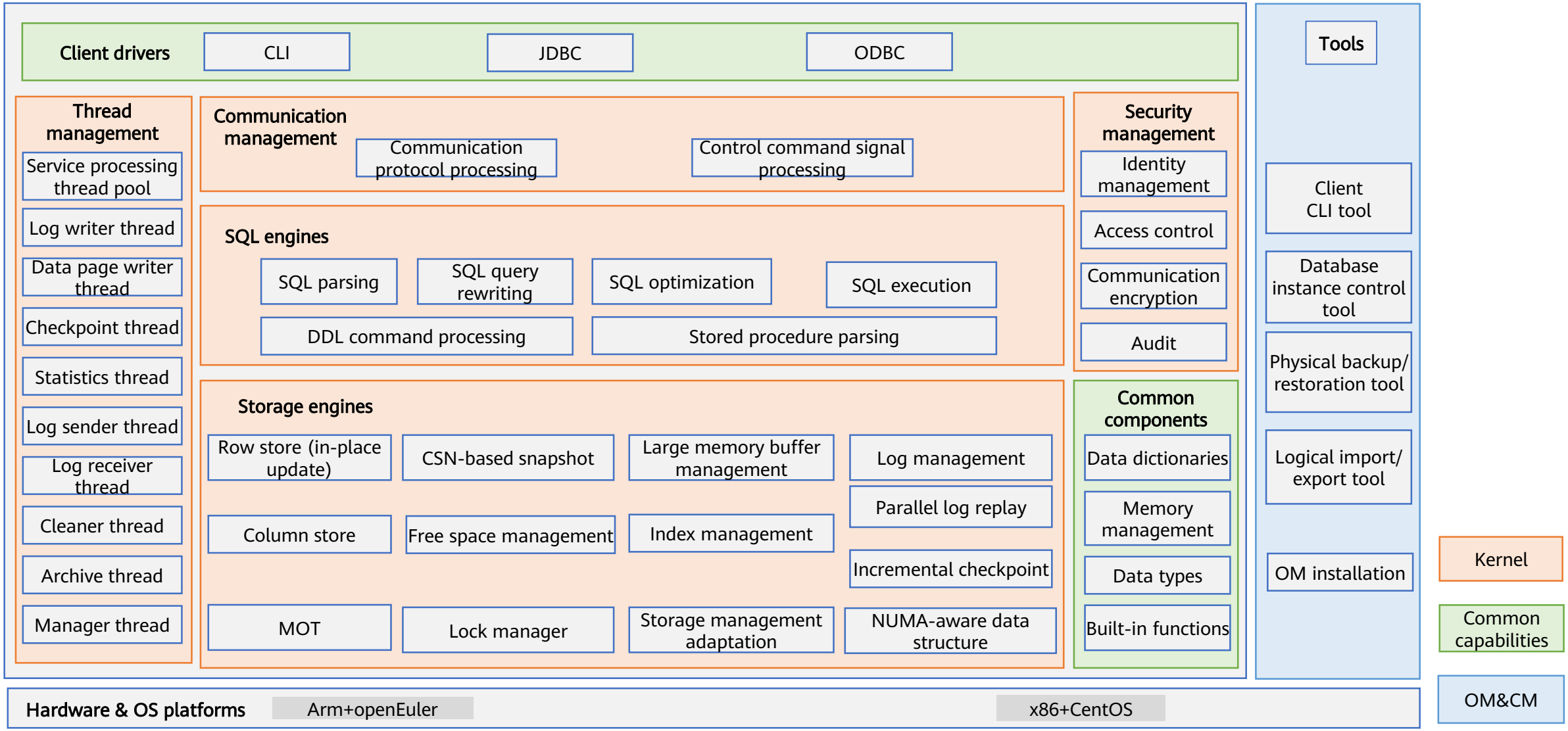
HUAWEI

Contents

1. System Architecture
2. Software Architecture
3. **Logical Architecture**
4. Thread Architecture

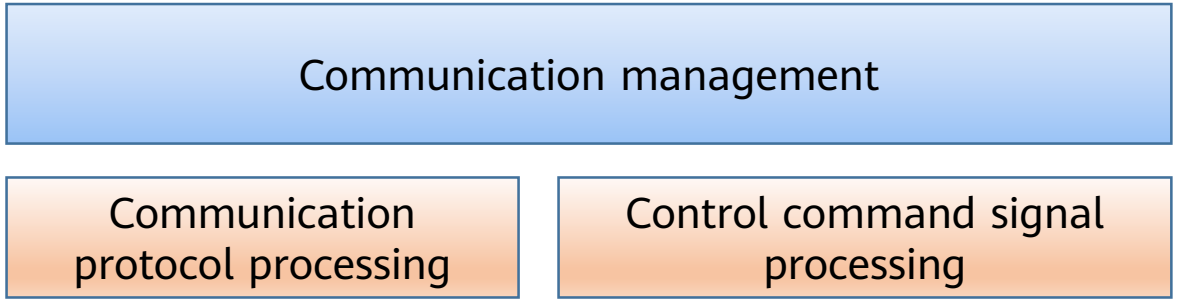
HUAWEI

openGauss Logical Architecture



HUAWEI

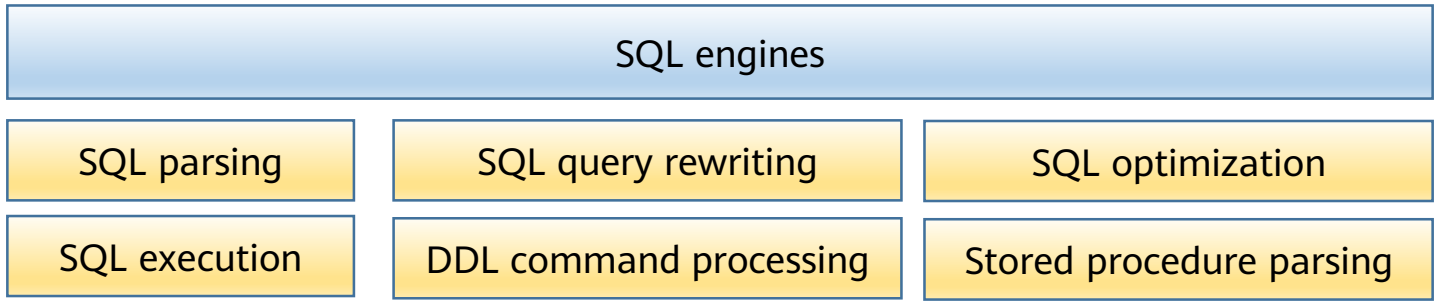
Kernel - Communication Management



- Communication protocol processing
  - This part mainly involves the front-end and back-end protocols used by the openGauss database. There are several sub-protocols based on the connection status, such as startup, query, function calling, COPY, and termination.
- Control command signal processing
  - Signals are a software interrupt mechanism. Communication between openGauss database threads depends on these signals, such as SIGTERM, SIGQUIT, SIGCHILD, SIGUSR1 and SIGUSR2.

HUAWEI

Kernel - SQL Engines (1)



- SQL parsing. After receiving SQL statements from clients, the server parses the SQL statements as follows:
  - Lexical parsing. SQL statements entered by users are segmented into tokens (that is, word sequences) to identify keywords, identifiers, and constants and determine the inherent part-of-speech of each word.
  - Syntax parsing. Syntax rules are defined based on SQL standards. Words generated in lexical analysis are used to match syntax rules. If an SQL statement matches a syntax rule, an abstract syntax tree is generated.
  - Semantic parsing. The syntax tree is checked to check whether the corresponding tables, columns, functions, and expressions in the syntax tree have corresponding metadata, and convert the abstract syntax tree into a logical execution plan.
- SQL query rewriting.
  - After a logical execution plan is generated for an SQL statement, that is, in the SQL query rewriting phase, a more efficient equivalent statement is generated by using existing statement features and relational algebra operations (for example, commutative law, associative law, and distributive law).

HUAWEI

Kernel - SQL Engines (2)

- SQL optimization.
  - SQL optimization depends on SQL query rewriting. It enumerates different candidate execution paths based on the generated efficient equivalent SQL statements. These execution paths are equivalent to each other but vary in execution efficiency. After the execution cost is calculated, an optimal execution path is obtained.
  - The execution cost is, for example, the CPU cost of processing a tuple, the I/O cost of loading a data page, or the transmission cost of a data tuple in a distributed database.
  - Based on the following statistics, the quantity of output tuples and the execution cost of different execution methods of each step are calculated: table feature values, such as the number of tuples, column width, NULL record ratio, distinct value, most common values in each column of the table, and HB values in a histogram (excluding MCVs), and a specific cost calculation model.
- SQL execution. It is implemented by an SQL executor, which connects to the upstream optimizer and downstream storage in the entire database architecture.
  - After receiving the execution plan tree returned by the optimizer, the SQL executor traverses the entire execution tree and performs initialization based on features of different operators. For example, the HashJoin operator initializes a hash table in the initialization phase. The initialization mainly involves memory allocation.
  - After the initialization is complete, the execution phase starts. The executor iteratively traverses the execution tree, reads data from disks, and completes the semantic query based on the specific logic of the execution tree.
  - The last step is cleanup. Here, the memory and resources allocated to the executor are cleared.

HUAWEI

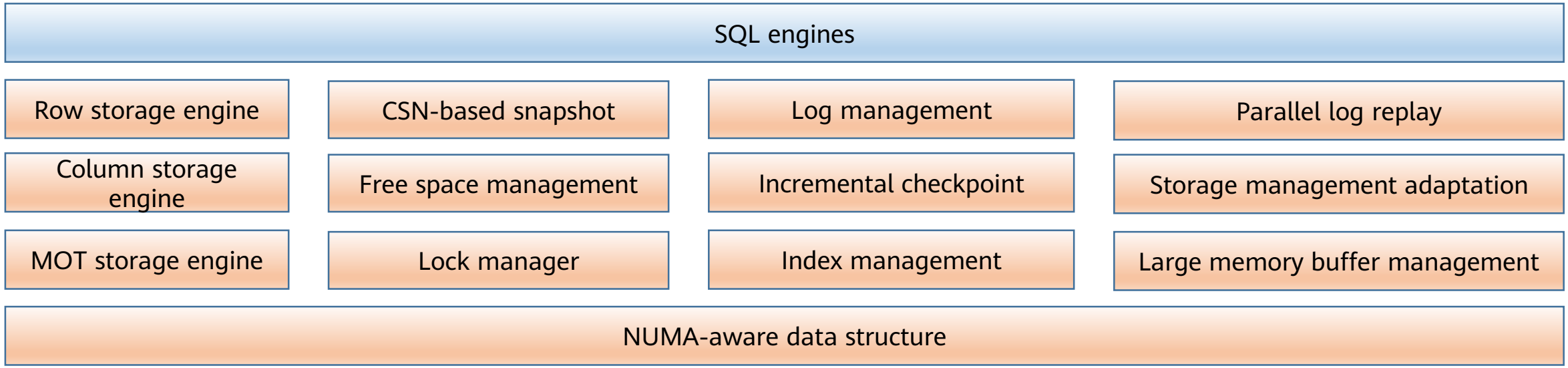
Kernel - SQL Engines (3)

- DDL command processing.
  - DDL commands also need lexical parsing, syntax parsing, and semantic parsing, but barely need optimization.
  - After DDL commands are parsed, you can query the corresponding data dictionary and perform the update operation.
- Stored procedure parsing.
  - A stored procedure is a set of SQL statements that can implement specific functions. After being compiled, a stored procedure is stored in the database.
  - The execution efficiency of a stored procedure is much higher than that of SQL statements that are executed one by one. This is because during the execution of a common SQL statement, it is parsed, compiled, and executed each time, while a stored procedure parses, compiles, and executes SQL statements only once when they are executed for the first time. For later execution of the stored procedure, you can call the execution result directly.
  - Stored procedure parsing is completed in the SQL engine module.

HUAWEI

Kernel - Storage Engines (1)

- The storage engine solves the following problems:
  - Atomicity (A), consistency (C), isolation (I), and durability (D), that is, the ACID property of stored data is ensured.
  - Supports high-concurrency and high-performance read/write.
  - Makes full use of hardware performance to improve efficient data storage and retrieval capabilities.



Kernel - Storage Engines (2)

- Row storage engine. Tables are stored to hard disk partitions by row, supporting high-concurrent and low-latency read/write. It is designed for OLTP scenarios, featuring frequent random small I/O operations and big data changes.
  - The row storage engine uses multi-version concurrency control (MVCC) which does not update data in place. Instead, the previous version of data is retained on the page and a new version of data is created on the current page (or a new page if the space is insufficient) for accumulation and update of data by version.
  - In this case, the same row of data of different versions is stored on a page. When transactions of different snapshot versions are obtained, the read and write operations on the data do not conflict with each other, and the concurrent performance is good. Historical data can be retrieved on the page or adjacent pages; in addition, no extra CPU or I/O overhead is required, and the efficiency is very high.
- Column storage engine. It is designed for OLAP scenarios.
  - Data in a column of a table is compressed as a data block, which is the basic unit for data compression, also know as a compression unit (CU).
  - The row storage engine manages data by row, whereas the column storage engine divides a table into multiple CUs based on columns and manages them by CU.
  - Advantages of column store: high compression ratio, low I/O consumption, and better query performance. In addition, together with a vectorized executor, the CPU cache hit ratio can be relatively high. Basic DML operations and MVCC are also supported.
- MOT storage engine. The MOT storage engine is based on full-memory data storage. Under different service loads, the transaction processing capability of the MOT storage engine can reach 3 to 10 times that of other engines.
  - Its index structure and overall data organization are implemented based on Masstree, hence providing strong transaction processing capabilities.
  - Full-memory data storage does not mean that data processed by the MOT storage engine will be lost during system faults.

HUAWEI

Kernel - Storage Engines (3)

- Commit sequence number(CSN)-based snapshots. Each CSN, a 64-bit unsigned auto-increment integer, identifies a transaction to be committed. It is used for multi-version visibility judgement and MVCC.
  - In openGauss, the CSN is used as the logical timestamp to simulate the time sequence inside a database. A CSN-based snapshot is created when an openGauss transaction is started.
  - In the MVCC mechanism, each transaction has an independent transaction status storage area, which records the status and CSN information of the transaction.
- Free space management. The MVCC mechanism is used, and UPDATE and DELETE operations do not delete data on a page. As a result, after the database runs for a long time, a large number of data of different versions is stored in the storage space, causing space bloat. Therefore, the storage engine needs to periodically delete historical data.
  - Data in the storage space can be cleared at the page, table, or database level.
  - The page-level clearing mechanism is also called heap_page_prune, which is a lightweight clearing mode. This clearing is triggered by the idle page space threshold. Only the page itself is modified, and the corresponding index page is not modified, which is a good solution for managing historical versions of the same data record.
  - The table-level and database-level clearing mechanism is also called VACUUM,which can be manually triggered by users or automatically triggered in the background based on threshold parameter settings. In addition to clearing obsolete tuple data, the VACUUM operation also clears indexes to release storage space and updates corresponding statistics.
- Lock manager. The lock manager manages locks on a database object for concurrent transactions and determines which lock can be placed when two transactions access the same object.
  - The lock manager records, tracks, and manages locks used during concurrent transactions.

HUAWEI

Kernel - Storage Engines (4)

- Large memory buffer management. The large memory buffer is located between the data storage engine and external file system. It is used as a buffer for exchanging pages with the external file system. The LRU algorithm is used to eliminate dirty pages in the shared memory and flush the dirty pages to disks to ensure efficient memory usage and reduce disk access.
- Storage management adaptation. It refers to managing (for example, managing the use of HDDs and SSDs), adapting, and packaging various storage media at the storage media layer, shielding the differences among various underlying storage systems for upper-layer data page access.
- Log management.
  - Traditional databases generally adopt the serial logging design because logs have sequential dependencies. In this way, multiple redo/undo records generated by a transaction have strong sequential dependencies.
  - openGauss adopts the parallel design and uses multiple parallel log writer threads to break the lock control bottleneck of serial logging and give full play to the multi-channel I/O processing capability of SSDs.
- Parallel log replay. It refers to the process of reapplying data file changes recorded in redo logs to the system or pages.
  - openGauss uses the multi-thread parallel replay technology. By default, there are four parallel replayer threads. You can also set the **recovery_max_workers** parameter to adjust the number of parallel replayer threads.
  - It consists of the allocation module and replay module. The allocation module parses logs and allocates logs to the replay module. The replay module replays the parsed logs.
  - Because the sequence of transaction commits and operations is critical to data consistency, each record in the redo log has a log sequence number (LSN), which is a 64-bit unsigned integer.
  - To improve the reliability of the overall parallel replay mechanism, openGauss verifies the LSN in the transaction log and **Last_LSN** in the page structure before replaying each page to ensure database system consistency during replay.

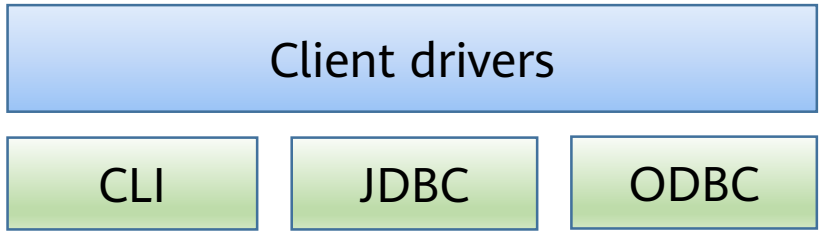
HUAWEI

Kernel - Storage Engines (5)

- Incremental checkpoint. It enables the storage engine to write dirty pages in the data buffer to disks and record information to log files and control files.
  - When a database instance recovers from a crash, you can replay logs from the checkpoint identified by an LSN. This reduces the cost of replaying database logs and shortens the fault recovery time.
  - According to the checkpoint mechanism, all dirty pages in the buffer are flushed to disks, causing high physical I/O pressure. To address this problem, the incremental checkpoint mechanism is introduced. By default, the checkpoint is executed every minute.
  - In the incremental checkpoint mechanism, a dirty page queue arranged in ascending order of LSNs is maintained. The pagewriter thread periodically flushes dirty pages in the queue and records the recLSN of the dirty pages that have been flushed to disks. When an incremental checkpoint is triggered, the recLSN of the current dirty page queue is used as the checkpoint LSN instead of waiting for all dirty pages to be flushed to disks. Such an incremental checkpoint mechanism makes the I/O performance of the entire database system smoother and shortens the system fault recovery time.
  - When the incremental checkpoint mechanism is enabled, **full_page_writes** is no longer used to prevent half-page write problems. Instead, doublewriter protection is used. The pagewriter thread copies dirty data pages to the doublewrite area and flushes the pages to disks. Then, the pagewriter thread forwards the dirty pages to the bgwriter subthread for flushing data to disks. This prevents data page breakage.
- NUMA-aware data structure.
  - Based on the multi-core NUMA architecture of Kunpeng processors, openGauss has made a series of optimizations. For example, binding cores to NUMA nodes is used to minimize the latency of cross-core memory access. To give full play to the multi-core advantages of the Kunpeng processors, the parallel log system design and Clog partitioning are used.
  - Based on the Armv8.1 architecture used by Kunpeng processors, openGauss uses the large system extensions (LSEs) to replace traditional compare-and-swap (CAS) instructions to implement efficient atomic operations, improving the synchronization performance between multiple threads and Xlog write performance.

HUAWEI

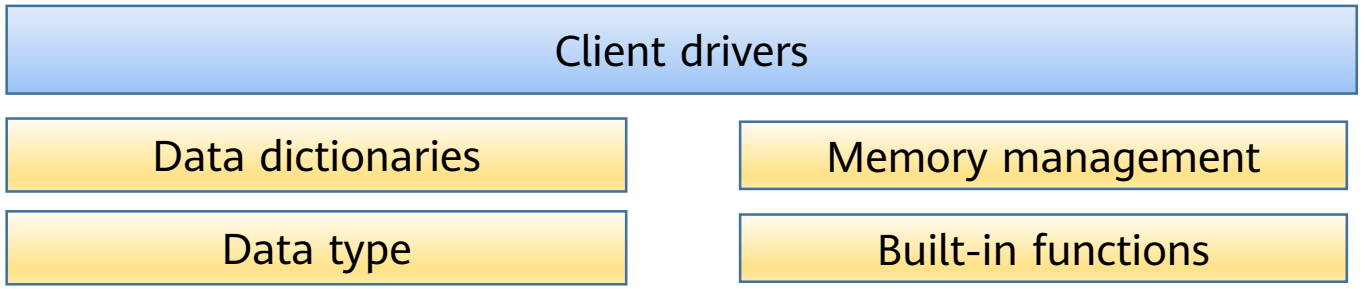
Common Capabilities - Client Driver



- CLI refers to the gsql client provided by openGauss.
  - The basic functions of gsql include connecting to a database, executing SQL statements, and executing meta-commands.
  - gsql also has some advanced features, such as variable setting, customized prompt setting, and client operation history setting.
- Java Database Connectivity (JDBC) is a Java API used to execute SQL statements.
  - It provides a unified access API for multiple relational databases based on which applications can operate data.
  - openGauss supports JDBC 4.0 and requires JDK 1.8 for code compiling. It does not support JDBC-ODBC bridging.
- Open Database Connectivity (ODBC) is a Microsoft API for accessing databases based on the X/Open CLI.
  - Applications interact with databases through the ODBC APIs, which enhances their portability, scalability, and maintainability.
  - Currently, openGauss supports ODBC 3.5.

HUAWEI

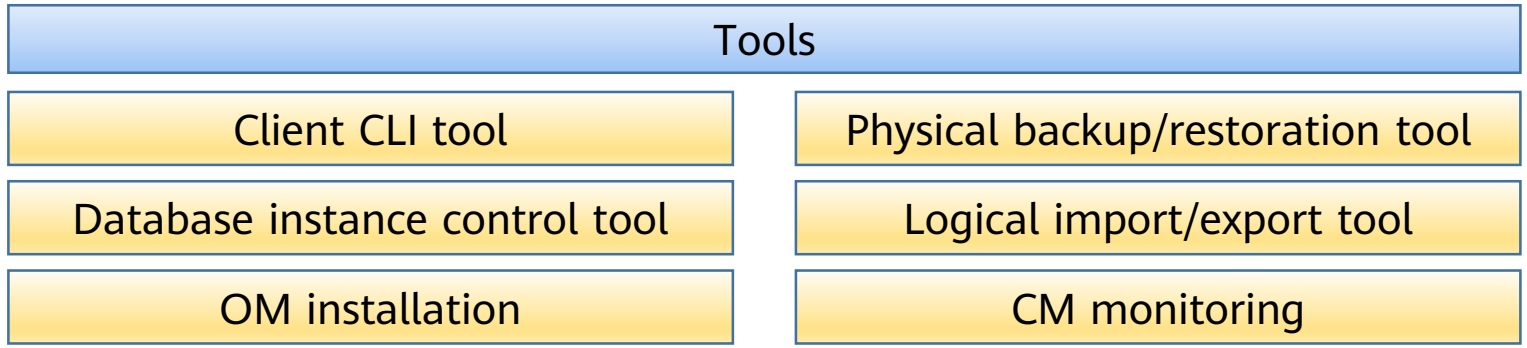
Common Capabilities - Common Components



- Data dictionaries. They are the metadata of a database, which are used to describe data attributes.
  - You can query system catalogs to obtain database information, such as table attributes, database attributes, and data type attributes.
  - They also assist in semantic parsing during SQL execution.
- Memory management. You can plan the memory allocation of the openGauss database based on the database parameter settings.
  - For example, you can use the **max_process_memory** parameter to control the maximum memory of a database node, and use the **work_mem** parameter to control the size of memory that can be used for in-memory ORDER BY in SQL and the size of memory that can be used before a hash table is written to temporary storage.
- Data type. It defines the attributes of data in the database. Each column in a table belongs to a data type. The database uses the corresponding function to perform operations on the data content based on the data type.
- Built-in functions. A function is a subprogram defined in a database and can implement specific functions. For example, mathematical calculation, database management, and database information query.

HUAWEI

OM&CM (1)



- Database instance control tool.
  - Such as, gaussdb (for starting a database), gs_ctl (for starting and stopping a database, loading the configuration file, switching between the primary and standby roles, and querying the status), and gs_initdb (for initializing a database instance, generating database directories and system catalogs, and creating the default database and template database)
- Physical backup/restoration tool. openGauss provides gs_basebackup to perform basic physical backup.
  - gs_basebackup is used to copy database files in binary mode. To remotely run gs_basebackup, you need to use the system administrator account.
  - Currently, this tool supports hot backup and compressed backup.
  - openGauss also provides gs_backup to help you back up binary applications, parameter files (**postgresql.conf**), and client authentication files (**pg_hba.conf**) of each node in a cluster.

OM&CM (2)

- Logical import/export tool. openGauss provides the gs_dump and gs_dumpall tools.
  - gs_dump helps you export database objects, such as schemas, tables, and views, in plain-text, directory, tar, or user-defined format.
  - gs_dumpall can export all data of the openGauss database in document format, including data in the default **postgres** database, data in a customized database, and common global objects of all openGauss databases.
  - In terms of database restoration, you can use gsql to import plain-text backup data and use the **-f** parameter to specify the backup text. You can use the gs_restore tool to restore backup sets in other formats.
- OM installation. openGauss provides the gs_om tool to help you maintain openGauss database instances, including starting and stoping a database, querying the database status, generating static configuration files, refreshing dynamic configuration files, and displaying help and version number information.

HUAWEI

Contents

1. System Architecture
2. Software Architecture
3. Logical Architecture
4. **Thread Architecture**

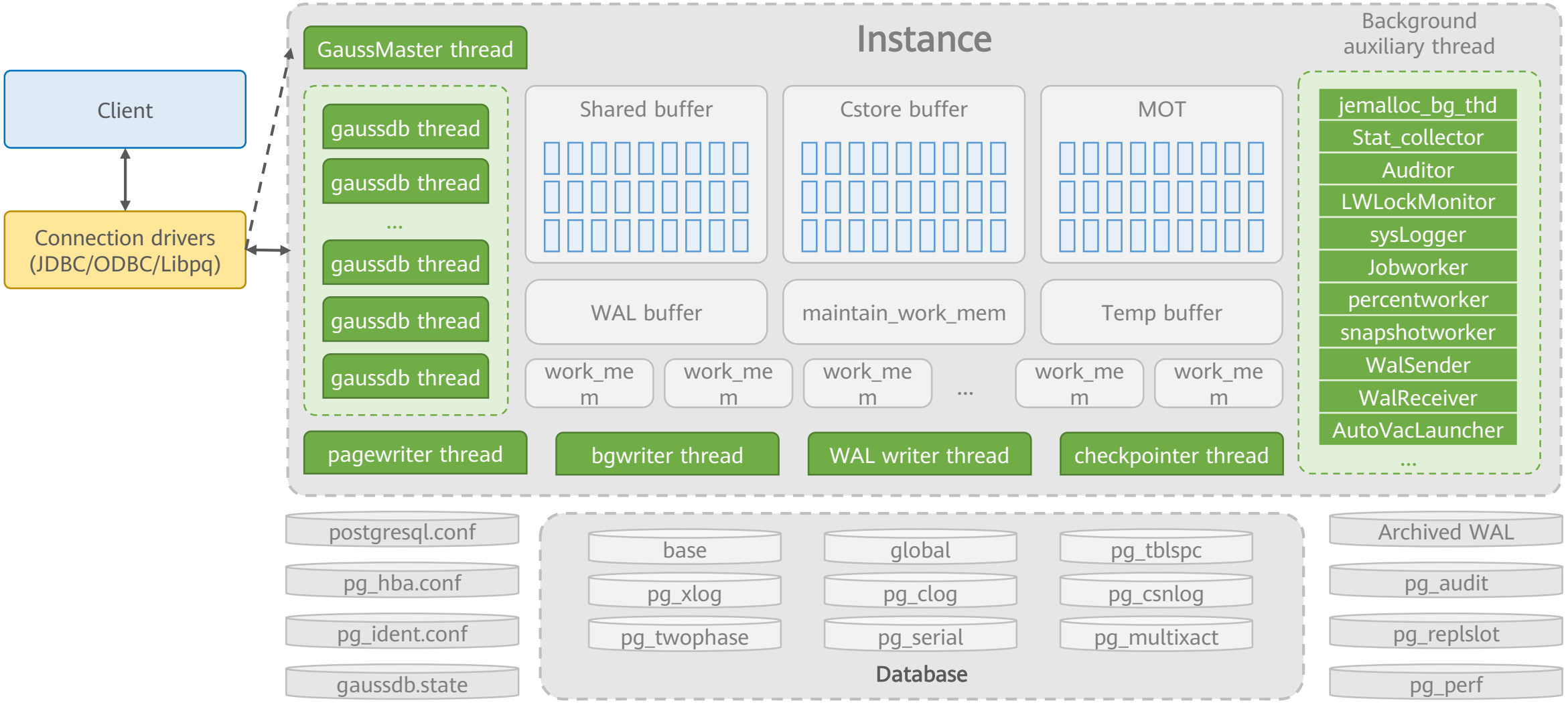
HUAWEI

Advantages of the openGauss Multi-Thread Architecture

- Because a large amount of data is shared and synchronized between database server service processes or threads, multithreading can make full use of multiple CPUs to execute multiple closely related tasks in parallel. For example, an execution engine may fully use concurrent threads to improve execution performance. Multithreading has many advantages:
- Advantage 1: The thread startup overhead is far less than the process startup overhead.
  - Compared with processes, it is a very "frugal" way of multitasking. In Linux, to start a new process, you must allocate an independent address space to it and create many data tables to maintain its code segments, stack segments, and data segments. This is an expensive multi-task working mode.
  - Multiple threads running in a process use the same address space and share most data. The space required for starting a thread is far less than that required for starting a process.
- Advantage 2: Convenient communication mechanism between threads.
  - Each process has an independent data space. Data can be transmitted only through communication, which is time-consuming and inconvenient.
  - Threads in the same process share the data space. Therefore, data of a thread can be directly used by other threads, which is quick and convenient.
- Advantage 3: The thread switching overhead is less than the process switching overhead.
  - For the Linux system, the process switchover is divided into two steps: 1. Switch the page directory to use the new address space. 2. Switch the kernel stack and hardware context.
  - For thread switching, the first step is not required, and the second step needs to be performed for both processes and threads. Therefore, the thread switching overhead is low.

HUAWEI

openGauss Thread Architecture



HUAWEI

Instance

- The openGauss database is a single-process multi-thread database. The instance part refers to the memory part during database running. The service process is as follows:
  - A client uses a driver such as JDBC, ODBC, Libpq, or Psycopg to send a connection request to the openGauss backend manager thread GaussMaster.
  - After receiving a service request from a client program, the GaussMaster thread immediately forks a subthread based on the received information. After authenticating the request, the subthread becomes the corresponding backend service processing subthread (gaussdb).
  - Subsequent requests sent by the client are processed by the service processing subthread (gaussdb). After receiving the SQL query requests sent by the client, the service processing subthread (gaussdb) calls the SQL engine of openGauss.
  - The SQL engine performs operations such as lexical parsing, syntax parsing, semantic parsing, and query rewriting on SQL statements, and sends the execution result to the client.
- During execution, the SQL executor usually accesses the shared buffer first. Operations on data pages in the shared buffer are recorded in the WAL buffer. When the client initiates a commit request of a transaction, the contents of the WAL buffer are flushed to disks by the WalWriter thread and stored in WAL files. This ensures that committed transactions are permanently recorded and will not be lost.
  - The shared buffer is the default buffer used by the row storage engine.
  - The Cstore buffer is the default buffer used by the column storage engine.
  - MOT is the default buffer used by the MOT storage engine.

HUAWEI

Background Auxiliary Threads

Thread	Description
jemalloc_bg_thd	Manages and implements dynamic memory allocation.
StatCollector	Collects statistics on openGauss databases, including the physical hardware resource usage, object attributes and usage, SQL running information, session information, lock information, and thread information, and saves the collected statistics in the **gaussdb.stat** file.
Auditor	Audit thread, which obtains audit data from the manager thread, backend thread, and other subthreads in redirection mode and saves the data in audit files.
LWLockMonitor	Lightweight lock monitoring thread, which detects deadlocks generated by lightweight locks (LWLocks). Lightweight locks provide mutually exclusive access control on shared memory, such as Clog buffer (storing transaction commit status), shared buffer (storing data pages), and subtran buffer (storing subtransactions).
sysLogger	System log thread, which catches all stderr output from the manager threads, backend threads, and other subthreads by redirecting to a pipe, and writes it to a set of log files.
Jobworker	Job threads are classified as scheduler threads (JobScheduler) and worker threads (Jobworker). JobScheduler calls expired jobs based on the job period defined in the pg_job table. Jobworker executes the jobs.
snapshotworker	Collects snapshot information. In the openGauss database, WDRs depend on the snapshots.
WalSender	Runs on the primary node in an openGauss primary/standby deployment environment and sends WALs to standby nodes.
WalReceiver	Runs on a standby node in an openGauss primary/standby deployment environment and receives WALs.
ashworker	Collects statistics on historical active sessions.
AlarmChecker	Alarm checker thread in the openGauss database.
PercentileJob	Percentage statistics thread, which calculates the percentage of responding SQL statements based on the percentile parameter settings. Currently, the percentile parameter value can only be **80** or **95**.
pagewriter	Copies dirty page data to the doublewrite area and flushes the data to disks. Then, the dirty pages are forwarded to the bgwriter subthread to flush the data to disks. If a data page is "broken", the complete data page is found in the doublewrite area for recovery.

HUAWEI

Database-related Files

Directory	Description
base	By default, openGauss database objects are stored in this directory, such as the default database **postgres**, user-created databases, and associated tables.
global	Stores system catalogs or data dictionary tables shared within openGauss.
pg_tblspc	openGauss tablespace directory, which stores soft links of tablespace directories defined by openGauss. These soft links point to the actual storage directory of tablespace files in the openGauss database.
pg_xlog	Stores WAL files of the openGauss database.
pg_clog	Stores transaction commit status information of the openGauss database.
pg_csnlog	Stores snapshot information of the openGauss database. When an openGauss transaction is started, a CSN-based snapshot is created. In the MVCC mechanism, the CSN is used as the logical timestamp to simulate the time sequence in the openGauss database and determine whether other transactions are visible to the current transaction.
pg_twophase	Stores two-phase transaction commit information to ensure data consistency.
pg_serial	Stores information about committed serializable transactions.
pg_multixact	Stores multi-transaction status information, which is generally used for shared row locks.

HUAWEI

openGauss Configuration File and Other Important Files

File/Directory	Description
postgresql.conf	openGauss configuration file. When the GaussMaster thread is started, this file is read to obtain configuration information such as the listening address, service port, memory allocation, and function settings. Based on this file, the shared memory and semaphore pool are created when openGauss is started.
pg_hba.conf	Host-based access authentication configuration file, which stores authentication information, such as databases that can be accessed, users, IP address segments, and encryption modes.
pg_ident.conf	Configuration file for client authentication, which stores user mapping information and maps host operating system users to openGauss database users.
gaussdb.state	Stores the current database status information, such as the primary and standby roles, rebuild progress and cause, synchronization status, and LSN information.
Archived WAL	Archive directory of WALs in the openGauss database, which stores historical WALs of the openGauss database.
pg_audit	Stores audit log files of the openGauss database.
pg_replslot	Stores replication transaction slot data of the openGauss database.
pg_llog	Stores the status data during logical replication.

HUAWEI

Thread Management (1)

- The WalWriter thread updates the WAL page data in the memory to the WAL file.
  - The log writer thread is started along with the Postmaster thread. When the disk flushing frequency of this thread cannot meet the system requirements, other common backend threads still have the permission to flush the WAL pages.
  - If the log writer thread crashes unexpectedly, the Postmaster thread considers that the entire openGauss backend thread crashed. In this case, SIGTERM is called to forcibly stop all backend threads and reset shared memory to restore backend threads.
  - The **wal_writer_delay** parameter specifies the write interval of the WalWriter process. By default, the WalWriter process calls the log write operation every 200 ms.
- Data page write thread. openGauss supports two threads: pagewriter and bgwriter.
  - pagewriter copies dirty page data to the doublewrite area, writes the data to disks, and forwards the dirty pages to the bgwriter subthread to write the data to disks.
  - If the bgwriter flushes data to disks but the database does not have sufficient available shared buffers, other common backend threads still have the right to flush data to disks.
  - The bgwriter thread is also started along with the Postmaster thread. This thread is also started when the recovery operation is performed.
  - The Postmaster thread can be normally closed by calling SIGTERM or be forcibly closed by calling SIGQUIT.
  - If the bgwriter thread crashes unexpectedly, the Postmaster thread considers that the entire openGauss backend thread crashed. In this case, SIGTERM is called to forcibly stop all backend threads and reset shared memory to restore backend threads.

HUAWEI

Thread Management (2)

- The autovacuum thread recycles the storage space occupied by deleted rows in tables or B-Tree indexes.
  - It consists of an initiation thread and an execution thread, which are named AutoVacLauncher and AutoVacWorker respectively in openGauss.
  - When vacuum needs to be initiated, the AutoVacLauncher thread sets the corresponding flag in the shared memory and sends a signal to the Gaussmaster thread. After receiving the signal, the Gaussmaster thread only needs to start an AutoVacWorker subthread to connect to the shared memory. The AutoVacWorker subthread obtains the information about the tasks to be cleared from the shared memory and recycles space of the objects.
- Manager thread. Usually, it refers to the Postmaster thread, which is named GaussMaster in openGauss.
  - The manager thread also participates in operating system operations, such as starting and stopping the database. However, it does not participate in basic operations in the openGauss database. It only starts the corresponding subthread to complete the operation when necessary. When some backend threads crash, the manager thread resets the threads.
  - The manager thread creates shared memory and semaphore pools at startup, but in principle does not interfere with them. In addition, it is not a member of the backend PGPROC array. Therefore, it does not participate in lock manager operations or share memory operations, which makes it simpler and more reliable.
  - Almost all manager threads restore a single backend thread from a crash by resetting the shared memory. If it also performs a large number of operations on the shared memory, it is prone to crash with the backend thread.

HUAWEI

Quiz

1. (Single-choice) Which of the following components is an independent process that can directly manage raw disk arrays and provide functions similar to the distributed file system?

   A. DSS

   B. DMS

   C. RDMA

   D. SCM

2. (True or False) SQL optimization depends on SQL query rewriting and functions as a link in the entire database architecture. It connects to the upstream executor and downstream storage.

   A. True

   B. False

HUAWEI

Summary

- After learning this chapter, you have a comprehensive understanding of the overall framework of the database and master the functions of each database component, including the logical database architecture, kernel components, common components, functions of threads in each module, and their working principles.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/zh/
- openGauss Help (5.0.0)
  - https://docs.opengauss.org/zh/docs/5.0.0/docs/GettingStarted/GettingStarted.html

HUAWEI

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that
could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Chapter 2 Advanced Management of Database Objects



HUAWEI

Foreword

- This chapter describes common system catalogs and system views, permission management and permission types of various database objects, common database object functions, information system functions, definitions of scheduled jobs, syntaxes and related advanced packages and functions, and management and examples of scheduled jobs.

HUAWEI

Objectives

- Upon completion of this course, you will be able to:
  - Master the usage of common system catalogs and system views.
  - Understand the permissions of various database objects and master their usage.
  - Be familiar with common database object functions and be able to use them skillfully.
  - Master syntaxes of scheduled jobs and be proficient in managing scheduled jobs.

HUAWEI

Contents

1. **System Catalogs and System Views**
2. System Information Functions and Database Object Functions
3. Database Object Permission Management
4. Scheduled Jobs

HUAWEI

Overview of System Catalogs and System Views

- System catalogs store structured metadata of openGauss. They are the source of information used by openGauss to control system running and are a core component of the database system.
- System views provide ways to query the system catalogs and internal database status.
- System catalogs and views are visible to either system administrators or all users.
- You can delete and re-create system catalogs, add columns to them, and insert and update values in them, but doing so may cause system information inconsistency and system faults. Generally, you should avoid manually modifying system catalogs or system views, or renaming their schemas. They are automatically maintained by the system.
  - You are advised not to modify the permissions on system catalogs and system views.
  - You should not add, delete, or modify system catalogs because doing so will result in exceptions or even openGauss unavailability.
- System catalogs and views inherited from PostgreSQL have the prefix “PG”.
- System catalogs and views added in openGauss have the prefix “GS”.

HUAWEI

Viewing System Catalogs and System Views

- Run the following meta-commands to view all system catalogs:

```
openGauss=# \dt pg_*
openGauss=# \dt gs_*
```

- Run the following meta-commands to view all views:

```
openGauss=# \dv pg_*
openGauss=# \dv gs_*
```

- View all tables in the public schema in the PG_TABLES system catalog.

```
openGauss=# SELECT distinct(tablename) FROM pg_tables WHERE SCHEMANAME = 'public';
```

- View the list of all users in the database and view the user ID (**USESYSID**) and permissions using PG_USER.

```
openGauss=# SELECT * FROM pg_user;
```


HUAWEI

Viewing and Stopping the Running Query Statements

- You can view the running query statements in the PG_STAT_ACTIVITY view.
  - Set **track_activities** to **on**. In this case, the database system collects the running information about activity queries only.

```
openGauss=# SET track_activities = on;
```

  - View the running query statements. View the database names, users performing queries, query status, and the corresponding PIDs which are connected to the running query statements. If the **state** column is **idle**, the connection is idle and requires a user to enter a command.

```
openGauss=# SELECT datname, usename, state,pid FROM pg_stat_activity;
```

  - To cancel queries that have been running for a long time, use the PG_TERMINATE_BACKEND function to end sessions based on the thread ID.

```
openGauss=# SELECT PG_TERMINATE_BACKEND(139834759993104);
```

```
# If information similar to the following is displayed, the session is successfully terminated:
PG_TERMINATE_BACKEND
----------------------
 t
(1 row)
```

Obtaining Object Information (1)

- Record tables and information that contains columns or tables, including index primary keys.

```
openGauss=# select * from pg_class;
```

- Database information.

```
openGauss=#select * from pg_database;
```

- Table information.

```
openGauss=#select * from pg_tables where tablename = 'test';
```

- Table column information.

```
openGauss=#select * from information_schema.columns where table_name = 'test_user_p';
```

- CHECK, PRIMARY KEY, UNIQUE, FOREIGN KEY, and EXCLUSION constraints.

```
openGauss=#select * from pg_constraint;
```

- Table inheritance relationship (partitioned table), that is, **inhparent** corresponds to the parent table OID and **inhrelid** corresponds to the child table OID.

```
openGauss=#select * from pg_inherits;
```


HUAWEI

Obtaining Object Information (2)

- Specifies the namespace, that is, schema.

```
openGauss=# select * from pg_namespace;
```

- Specifies the database role information, including the maximum number of connections (**rolconnlimit**).

```
openGauss=# select * from pg_roles;
```

- Specifies a database user who has limited permissions to access databases.

```
openGauss=# select * from pg_shadow;
```

- Specifies a database user, which is the publicly readable view on pg_shadow that blanks out the password field.

```
openGauss=# select * from pg_user;
```

- Specifies the view information, which provides detailed information about each view in the database.

```
openGauss=# select * from pg_views;
```


HUAWEI

Viewing Information About the Number of Connections

- View expired connections.

```
openGauss=# select * from pg_stat_activity where state = 'idle';
```

- Delete the connection. The value in the brackets is the PID.

```
openGauss=# select pg_terminate_backend(25837);
```

- View the maximum number of connections.

```
openGauss=# show max_connections;
```

- Change the maximum number of connections, which requires the superuser permission.

```
openGauss=# alter system set max_connections= 1000;
```


HUAWEI

Resource Monitoring View - Real-Time Top SQLs

- In the DBE_PERF schema, views are used to diagnose performance issues and are also the data source of WDR snapshots.
- In openGauss, you can query top SQLs in real-time resource monitoring views at query and operator levels. The real-time resource monitoring view records the resource usage (including memory, disks, CPU time, and I/Os) and performance alarm information during job running.

View Level	Query View
Query	DBE_PERF.STATEMENT_COMPLEX_RUNTIME
Operator	DBE_PERF.OPERATOR_RUNTIME

  - View load management information about jobs being executed by the current user on the primary database node.

```
openGauss=# select * from dbe_perf.STATEMENT_COMPLEX_RUNTIME;
```

  - View operators of the jobs that are being executed by the current user.

```
openGauss=# select * from dbe_perf.OPERATOR_RUNTIME;
```


HUAWEI

Resource Monitoring View - Historical Top SQLs (1)

- In openGauss, you can query historical top SQLs in historical resource monitoring views at query and operator levels.
  - A historical resource monitoring view records the resource usage (of memory, disks, CPU time, and I/Os), status (including errors, termination, and exceptions), and performance alarm information when the query is finished. If queries are abnormally terminated due to **FATAL** or **PANIC** errors, their status will be displayed as **aborted** and no detailed information will be recorded. By default, gs_wlm_session_history data is dumped to gs_wlm_session_info every 3 minutes.
  - To use the view monitoring function of Workload Manager, you need to enable the **use_workload_manager** and **enable_resource_track** parameters. In addition, set the following threshold control parameters as required.

GUC Parameter	Function	Default Value
use_workload_manager	Specifies whether to enable resource management.	on
enable_resource_track	Specifies whether to enable the real-time resource monitoring function. **on** indicates that the resource monitoring function is enabled, and **off** indicates that the resource monitoring function is disabled.	on
resource_track_level	Specifies the resource monitoring level. **none** indicates that the resource recording function is disabled. **query** indicates that the resource recording function at the query level is enabled. **operator** indicates that the resource recording function at the query and operator levels is enabled.	query
resource_track_cost	Specifies the minimum execution cost of statements under resource monitoring. The value ranges from –1 to *INT_MAX*. **–1** indicates that resources are not monitored for all statements. A value greater than or equal to 0 indicates that resources are monitored for statements whose execution cost exceeds the value of this parameter.	100000
resource_track_duration	Specifies the minimum statement execution time that determines whether information about jobs of a statement recorded in the real-time view will be dumped to a historical view after the statement is executed.	1min
enable_resource_record	Specifies whether the resource recording function is enabled. **on** indicates that resource recording is enabled, and **off** indicates that resource recording is disabled.	off

Resource Monitoring View - Historical Top SQL Statements (2)

- DBE_PERF.STATEMENT_COMPLEX_HISTORY displays load management information about completed jobs executed on the primary database node.
  - To collect GUC parameters, set **use_workload_manager** to **on** and **enable_resource_track** to **on**.
  - Filter parameter settings, for example, set **resource_track_duration** to **1min**, **resource_track_cost** to **0**, and **resource_track_level** to **'query'**.
  - View the load management records after the current job is complete.

```
openGauss=# select * from DBE_PERF.STATEMENT_COMPLEX_HISTORY;
```

- STATEMENT_COMPLEX_HISTORY_TABLE displays load management records after jobs executed by the primary node of the database are complete. The data dumped from the kernel to the system catalog is displayed.
  - If the GUC parameter **enable_resource_record** is set to **on**, the record **DBE_PERF.STATEMENT_COMPLEX_HISTORY** in the kernel is dumped to the system catalog every 3 minutes.
  - View archived load management records.

```
openGauss=# select * from DBE_PERF.STATEMENT_COMPLEX_HISTORY_TABLE;
```


HUAWEI

Contents

1. System Catalogs and System Views
2. **System Information Functions and Database Object Functions**
   - System Information Functions
   - Database Object Functions
3. Database Object Permission Management
4. Scheduled Jobs

HUAWEI

Session Information Functions (1)

Name	Description	Example
current_catalog	Name of the current database.	SELECT current_catalog;
current_database()	Name of the current database.	SELECT current_database();
current_query()	Current execution statement committed by the client.	SELECT current_query();
current_schema[()]	Name of the current schema. current_schema returns the first valid schema name in the search path. It is synonymous with schema().	SELECT current_schema();
current_schemas(Boolean)	Returns an array of the names of all schemas in the search path. The Boolean option specifies whether implicitly included system schemas such as pg_catalog are included in the returned search path.	SELECT current_schemas(true);
current_user	Username in the current operating environment. current_user is the user identifier that is applicable for permission checking. Normally, it is equal to the session user, but it can be changed by using SET ROLE. It also changes during the execution of functions with the SECURITY DEFINER attribute. It is synonymous with definer_current_user().	SELECT current_user;
pg_current_sessionid()	pg_current_sessionid() is used to obtain the session ID in the current operating environment. The structure of the value is *Timestamp.Session ID*. When **enable_thread_pool** is set to **on**, the actual session ID is used. When **enable_thread_pool** is set to **off**, the session ID is the thread ID.	SELECT pg_current_sessionid();
pg_current_sessid	In thread pool mode, the ID of the current session is obtained. In non-thread pool mode, the backend thread ID of the current session is obtained.	select pg_current_sessid();
pg_current_userid	ID of the current user.	SELECT pg_current_userid();
working_version_num()	Version number. It returns a version number related to system compatibility.	SELECT working_version_num();

HUAWEI

Session Information Functions (2)

Name	Description	Example
tablespace_oid_name()	Queries the tablespace name based on the tablespace OID.	select tablespace_oid_name(1663);
inet_client_addr()	Specifies the address of the remote connection. The IP address of the current client is returned. It is valid only in remote connection mode.	SELECT inet_client_addr();
inet_client_port()	Specifies the port of the remote connection. The port number of the current client is returned. It is valid only in remote connection mode.	SELECT inet_client_port();
inet_server_addr()	Specifies the address of the local connection. The IP address on which the server accepts the current connection is returned. It is valid only in remote connection mode.	SELECT inet_server_addr();
inet_server_port()	Specifies the port of the local connection. The number of the port receiving the current connection is returned. All these functions return **NULL** if the current connection is via a UDS. It is valid only in remote connection mode.	SELECT inet_server_port();
pg_backend_pid()	Specifies the process ID of the server process attached to the current session.	SELECT pg_backend_pid();
pg_conf_load_time()	Configures load time. pg_conf_load_time returns the timestamp when the server configuration files were last loaded.	SELECT pg_conf_load_time();
pg_my_temp_schema()	Returns the OID of the current session's temporary schema, or zero if it has none (because it has not created any temporary tables). pg_is_other_temp_schema returns **true** if the given OID is the OID of another session's temporary schema.	SELECT pg_my_temp_schema();
pg_is_other_temp_schema(oid)	Specifies whether the schema is the temporary schema of another session.	SELECT pg_is_other_temp_schema(25356);
pg_postmaster_start_time()	Specifies the server start time. pg_postmaster_start_time returns the timestamp with time zone when the server is started.	SELECT pg_postmaster_start_time();

HUAWEI

Session Information Functions (3)

Name	Description	Example
pg_get_ruledef(rule_oid)	Obtains the **CREATE RULE** command for rule.	select * from pg_get_ruledef(24828);
pg_trigger_depth()	Specifies the nesting level of triggers.	SELECT pg_trigger_depth();
session_user[()]	Specifies the session username. **session_user** is usually the user who initiated the current database connection, but administrators can change this setting with **SET SESSION AUTHORIZATION**. It is equivalent to user[()].	SELECT session_user();
connection_id()	Returns the ID of the connection thread. The returned result is the same as the ID column in the output of SHOW PROCESSLIST.	SELECT CONNECTION_ID();
getpgusername()	Obtains the database username.	select getpgusername();
getdatabaseencoding()	Obtains the database encoding mode.	select getdatabaseencoding();
version()	Specifies the version information. version returns a string describing a server's version.	select version();
opengauss_version()	Specifies the openGauss version information.	select opengauss_version();
gs_deployment()	Specifies information about the deployment mode of the current system.	select gs_deployment();
get_hostname()	Returns the host name of the current node.	SELECT get_hostname();
get_nodename()	Returns the name of the current node.	SELECT get_nodename();
get_schema_oid(cstring)	Returns the OID of the queried schema.	SELECT get_schema_oid('public');

HUAWEI

System Catalog Information Functions

Name	Description
getdistributekey(table_name)	Obtains a distribution key for a hash table. Distribution is not supported in a standalone system and the return value of this function is empty.
pg_get_constraintdef(constraint_oid)	Obtains the definition of a constraint. The return type is text.
pg_get_functiondef(func_oid)	Obtains the definition of a function. The return type is text.
pg_get_indexdef(index_oid)	Obtains the **CREATE INDEX** command for an index. The return type is text.
pg_get_userbyid(role_oid)	Obtains the role name with a given OID. The return type is name.
pg_get_tabledef(table_name)	Obtains the definition of a table based on **table_name**, for example, **select * from pg_get_tabledef('t1');**.
pg_get_tabledef(table_oid)	Obtains the definition of a table based on **table_oid**.
pg_get_viewdef(view_oid)	Obtains the underlying SELECT command for view. The return type is text.
pg_check_authid(role_id)	Checks whether a user exists based on **role_id**.
pg_tablespace_databases(tablespace_oid)	Obtains the set of database OIDs that have objects in the tablespace. The return type is setof oid pg_tablespace_databases allows a tablespace to be examined. It returns the set of OIDs of databases that have objects stored in the tablespace. If this function returns any rows of data, the tablespace is not empty and cannot be dropped. To display the specific objects in the tablespace, you need to connect to the databases identified by pg_tablespace_databases and query their pg_class catalogs.
pg_get_serial_sequence(tablename, colname)	Obtains the sequence of the corresponding table name and column name. The return type is text.
pg_sequence_parameters(sequence_oid)	Obtains the parameters of a specified sequence, including the start value, minimum value, maximum value, and incremental value.

HUAWEI

Contents

1. System Catalogs and System Views
2. **System Information Functions and Database Object Functions**
   - System Information Functions
   - Database Object Functions
3. Database Object Permission Management
4. Scheduled Jobs

HUAWEI

Database Object Size Functions (1)

- Database object size functions calculate the actual disk space used by database objects.

Name	Description
pg_column_size(any)	Specifies the number of bytes used to store a particular value (possibly compressed). It is used to display the space used to store an independent data value. For example, you can run **SELECT pg_column_size(1);**.
pg_database_size(oid)	Specifies the disk space used by the database with the specified OID.
pg_database_size(name)	Specifies the disk space used by the database with the specified name. The OID or name of a database is received and the disk space used by the corresponding object is returned. For example, you can run **SELECT pg_database_size('postgres');**.
pg_relation_size(oid)	Specifies the disk space used by the table with a specified OID or index.
get_db_source_datasize()	Estimates the total size of non-compressed data in the current database. (1) You need to perform an analysis before this function is called. (2) You need to calculate the total data capacity in the non-compressed state by estimating the compression rate of the column-store tables. For example, you can run the **analyze;** and **select get_db_source_datasize();** commands.
pg_relation_size(text)	Specifies the disk space used by the table with a specified name or index. The table name can be schema-qualified.
pg_partition_size(oid,oid)	Specifies the disk space used by the partition with a specified OID. The first **oid** is the OID of the table and the second **oid** is the OID of the partition.
pg_partition_size(text, text)	Specifies the disk space used by the partition with a specified name. The first **text** is the table name and the second **text** is the partition name.
pg_partition_indexes_size(oid,oid)	Specifies the disk space used by the index of the partition with a specified OID. The first **oid** is the OID of the table and the second **oid** is the OID of the partition.
pg_partition_indexes_size(text,text)	Specifies the disk space used by the index of the partition with a specified name. The first **text** is the table name and the second **text** is the partition name.
pg_indexes_size(regclass)	Specifies the total disk space used by the index appended to the specified table.

Database Object Size Functions (2)

Name	Description
pg_size_pretty(bigint)	Converts a size in bytes expressed as a 64-bit integer into a human-readable format with size units.
pg_size_pretty(numeric)	Converts a size in bytes expressed as a numeric value into a human-readable format with size units. The function is used format the result of other functions into a human-readable format. KB, MB, GB, and TB can be used.
pg_table_size(regclass)	Specifies the disk space used by a specified table, excluding indexes (but including TOAST, free space map, and visibility map).
pg_tablespace_size(oid)	Specifies the disk space used by the tablespace with the specified OID.
pg_tablespace_size(name)	Specifies the disk space used by the tablespace with the specified name. The OID or name of a database is received and the disk space used by the corresponding object is returned.
pg_total_relation_size(oid)	Specifies the disk space used by a table with a specified OID, including indexes and the compressed data.
pg_total_relation_size(regclass)	Specifies the total disk space used by a specified table, including all indexes and TOAST data.
pg_total_relation_size(text)	Specifies the disk space used by a table with a specified name, including indexes and the compressed data. The table name can be schema-qualified. The OID or name of a table or a compressed table is received, and the size of the data, related indexes, and the compressed tables are returned.
datalength(any)	Calculates the number of bytes used by an expression of a specified data type (data management space, data compression, or data type conversion is not considered). It is used to calculate the space of an independent data value. For example, you can run **SELECT datalength(1)**.

HUAWEI

Database Object Position Functions

- pg_relation_filenode(relation regclass)
  - Description: Specifies the number of file nodes in a specified relationship. The return type is OID.
  - pg_relation_filenode receives the OID or name of a table, index, sequence, or compressed table, and returns the number of file nodes allocated to it. A file node is the basic component of the file name used by the relationship. For most tables, the result is the same as that of pg_class.relfilenode. For a specified system directory, **relfilenode** is set to **0** and this function must be used to obtain the correct value. If a relationship that is not stored is transmitted, such as a view, this function returns **NULL**.
- pg_relation_filepath(relation regclass)
  - Description: Specifies the name of a file path with the specified relationship. The return type is text.
  - pg_relation_filepath is similar to pg_relation_filenode, except that pg_relation_filepath returns the whole file path name for the relationship (relative to the data directory **PGDATA** of the openGauss).
- pg_filenode_relation(tablespace oid, filenode oid) is used to obtain the table names corresponding to the tablespace and **relfilenode**. The return type is regclass.
- pg_partition_filenode(partition_oid) is used to obtain **filenode** corresponding to the OID lock of a specified partitioned table. The return type is OID.
- pg_partition_filepath(partition_oid) is used to specify the file path name of the partition. The return type is text.
- gs_is_recycle_object(classid, objid, objname) is used to specify whether an object is in the recycle bin. The return type is Boolean.

HUAWEI

Contents

1. System Catalogs and System Views
2. System Information Functions and Database Object Functions
3. **Database Object Permission Management**
4. Scheduled Jobs

HUAWEI

Overview

- A user who creates an object is the owner of this object. By default, separation of duties is disabled after database installation. A database system administrator has the same permissions as object owners. After an object is created, only the object owner or system administrators can query, modify, and delete the object, and grant permissions on the object to other users through GRANT by default.
- To enable another user to use the object, you must grant required permissions to the user or the role that contains the user.
- To revoke permissions, you can use REVOKE.
- Object owners have implicit permissions (such as ALTER, DROP, COMMENT, INDEX, VACUUM, GRANT, and REVOKE) on objects. That is, once becoming the owner of an object, the owner is immediately granted the implicit permissions on the object. **Object owners can revoke their own common permissions**, for example, making tables read-only to themselves or others, except system administrators.

HUAWEI

Permission Types (1)

- openGauss supports the following permissions, which are associated with different object types:
  - SELECT: allows **SELECT** on a specified table, view, or sequence. The SELECT permission on the corresponding column is also required for UPDATE or DELETE.
  - INSERT: allows **INSERT** on a specified table.
  - UPDATE: allows **UPDATE** on any column of a specified table.
    - Generally, **UPDATE** also requires the SELECT permission to query which rows need to be updated.
    - **SELECT... FOR UPDATE**, **SELECT... FOR NO KEY UPDATE**, **SELECT... FOR SHARE**, and **SELECT... FOR KEY SHARE** require the UPDATE permission in addition to the SELECT permission.
  - DELETE: allows **DELETE** on a row of a specified table. Generally, **DELETE** also requires the SELECT permission to query which rows need to be deleted.
  - TRUNCATE: allows **TRUNCATE** on a specified table.
  - REFERENCES: allows creation of a FOREIGN KEY constraint. This permission is required on both referencing and referenced tables.

HUAWEI

Permission Types (2)

- Permission types are associated with object types.
  - CREATE:
    - For databases, allows new schemas to be created within the database.
    - For schemas, allows new objects to be created within the schema. To rename an existing object, you must own the object and have the CREATE permission on the schema of the object.
    - For tablespaces, allows tables to be created within the tablespace, and allows databases and schemas to be created that have the tablespace as their default tablespace.
  - CONNECT: allows access to specified databases.
  - EXECUTE: allows use of specified functions and the operators that are realized by the functions.
  - USAGE:
    - For procedural languages, allows use of the language for the creation of functions in that language.
    - For schemas, allows access to objects contained in the schema. Without this permission, it is still possible to see the object names.
    - For sequences, allows use of the nextval function.
    - For data sources, specifies access permissions or is used as ALL PRIVILEGES.

Permission Types (3)

- Permission types are associated with object types.
  - ALTER: allows users to modify the attributes of a specified object, excluding the owner and schema of the object.
  - DROP: allows users to delete a specified object.
  - COMMENT: allows users to define or modify comments of a specified object.
  - INDEX: allows users to create indexes on specified tables, manage indexes on the tables, and perform REINDEX and CLUSTER operations on the tables.
  - VACUUM: allows users to perform ANALYZE and VACUUM operations on specified tables.
  - ALL PRIVILEGES: grants all available permissions to specified users or roles at a time. Only a system administrator has the GRANT ALL PRIVILEGES permission.

HUAWEI

Examples (1)

- Create the **test** schema and the **customer** table.

```
openGauss=# CREATE SCHEMA test;
CREATE SCHEMA
openGauss=# CREATE TABLE customer
(
    c_customer_sk             integer,
    c_customer_id             char(5),
    c_first_name              char(6),
    c_last_name               char(8),
    Amount                    integer
);
CREATE TABLE
```

- Revoke user **joe**'s sysadmin permission, and grant it the usage permission on the **test** schema and all permissions on the **customer** table.

```
openGauss=# REVOKE ALL PRIVILEGES FROM joe;
ALTER ROLE
openGauss=# GRANT USAGE ON SCHEMA test TO joe;
GRANT
openGauss=# GRANT ALL PRIVILEGES ON customer TO joe;
GRANT
```


HUAWEI

Examples (2)

- Grant the query permission on the **c_customer_sk**, **c_customer_id**, and **c_first_name** columns and the update permission on the **c_last_name** column in the **customer** table to user **joe**.

```
openGauss=# GRANT select (c_customer_sk,c_customer_id,c_first_name),update (c_last_name) ON
customer TO joe;
GRANT
```

- After the permissions are successfully granted, user **joe** immediately has the query permission on the **c_customer_sk**, **c_customer_id**, **c_first_name** columns in the **customer** table. To enable user **joe** to grant these permissions to other users, execute the following statement:

```
openGauss=# GRANT select (c_customer_sk,c_customer_id,c_first_name) ON customer TO joe WITH
GRANT OPTION;
GRANT
```

- Grant the **postgres** database connection permission and schema creation permission to user **joe** and enable this user to grant these permissions to other users.

```
openGauss=# GRANT create,connect on database postgres TO joe WITH GRANT OPTION;
GRANT
```


HUAWEI

Examples (3)

- Create the **test_manager** role, grant the access and object creation permissions on the **test** schema to the **test_manager** role, but do not allow this role to grant these permissions to others.

```
openGauss=# CREATE ROLE test_manager PASSWORD 'xxxxxxxx';
CREATE ROLE
openGauss=# GRANT USAGE,CREATE ON SCHEMA test TO test_manager;
GRANT
```

- Create the **tpcds_tbspc** tablespace and grant all permissions on the tablespace to user **joe**. However, user **joe** cannot grant these permissions to other users.

```
openGauss=# CREATE TABLESPACE tpcds_tbspc RELATIVE LOCATION 'tablespace/tablespace_1';
CREATE TABLESPACE
openGauss=# GRANT ALL ON TABLESPACE tpcds_tbspc TO joe;
GRANT
```


HUAWEI

Access Permission Query Functions (1)

- The access permission query functions only specify whether a user has a certain permission on an object (that is, a permission on an object recorded in the **relacl** column of a system catalog is returned).

Function	Description	Return Type
has_any_column_privilege(user, table, privilege)	Queries whether a specified user has permissions on any column of tables.	Boolean
has_any_column_privilege(table, privilege)	Queries whether the current user has permissions on any column of tables.	Boolean
has_column_privilege(user, table, column, privilege)	Queries whether a specified user has permissions on columns.	Boolean
has_column_privilege(table, column, privilege)	Queries whether the current user has permissions on columns.	Boolean
has_cek_privilege(user, cek, privilege)	Queries whether a specified user has permissions on CEKs. The permissions include USAGE and DROP.	Boolean
has_cmk_privilege(user, cmk, privilege)	Queries whether a specified user has permissions on CMKs. The permissions include USAGE and DROP.	Boolean
has_database_privilege(user, database, privilege)	Queries whether a specified user has permissions on databases.	Boolean

- Notes:
  - has_any_column_privilege checks whether a user can access any column of a table in a particular way. Its parameter possibilities are analogous to those of has_table_privilege, except that the access permission types must be some combinations of SELECT, INSERT, UPDATE, COMMENT, and REFERENCES.
  - Note that having any of these permissions at the table level indicates that the permission is implicitly granted for each column of the table. Therefore, has_any_column_privilege always returns **true** if it has the same parameters as has_table_privilege. A success message is also returned if a column-level permission is granted for at least one column.
  - has_column_privilege checks whether a user can access a column in a particular way. Its parameter possibilities are analogous to those of has_table_privilege, with the addition that the column can be specified either by name or attribute number. The desired access permission types must be some combinations of SELECT, INSERT, UPDATE, COMMENT, and REFERENCES.

Access Permission Query Functions (2)

Function	Description	Return Type
has_database_privilege(database, privilege)	Queries whether the current user has permissions on databases.	Boolean
has_foreign_data_wrapper_privilege(fdw, privilege)	Queries whether the current user has permissions on foreign-data wrappers. The permission type must be USAGE.	Boolean
has_function_privilege(user, function, privilege)	Queries whether a specified user has permissions on functions.	Boolean
has_function_privilege(function, privilege)	Queries whether the current user has permissions on functions.	Boolean
has_language_privilege(user, language, privilege)	Queries whether a specified user has permissions on languages.	Boolean
has_language_privilege(language, privilege)	Queries whether the current user has permissions on languages.	Boolean
has_nodegroup_privilege(user, nodegroup, privilege)	Checks whether a user has permissions on database nodes.	Boolean
has_nodegroup_privilege(nodegroup, privilege)	Queries whether the current user has permissions on database nodes. The access permission must be USAGE, CREATE, COMPUTE, ALTER, or DROP.	Boolean

- Notes:
  - has_database_privilege checks whether a user can access a database in a particular way. Its parameter possibilities are analogous to those of has_table_privilege. The access permission types must be some combinations of CREATE, CONNECT, TEMPORARY, ALTER, DROP, COMMENT, and TEMP (which is equivalent to TEMPORARY).
  - has_function_privilege checks whether a user can access a function in a particular way. The access permission type must be EXECUTE, ALTER, DROP, or COMMENT.
  - has_language_privilege checks whether a user can access a procedural language in a particular way. The access permission type must be USAGE.

Access Permission Query Functions (3)

Function	Description	Return Type
has_schema_privilege(user, schema, privilege)	Queries whether a specified user has permissions on schemas.	Boolean
has_schema_privilege(schema, privilege)	Queries whether the current user has permissions on schemas.	Boolean
has_server_privilege(user, server, privilege)	Queries whether a specified user has permissions on foreign servers.	Boolean
has_server_privilege(server, privilege)	Queries whether the current user has permissions on foreign servers.	Boolean
has_table_privilege(user, table, privilege)	Queries whether a specified user has permissions on tables.	Boolean
has_tablespace_privilege(user, tablespace, privilege)	Specifies whether a specified user has permissions on tablespaces.	Boolean
has_tablespace_privilege(tablespace, privilege)	Specifies whether the current user has permissions on tablespaces.	Boolean
pg_has_role(user, role, privilege)	Specifies whether a specified user has permissions on roles.	
pg_has_role(role, privilege)	Specifies whether the current user has permissions on roles. The permission types must be some combinations of MEMBER and USAGE.	Boolean
has_any_privilege(user, privilege)	Queries whether a specified user has certain ANY permission. If multiple permissions are queried at the same time, **true** is returned as long as one permission is obtained.	Boolean

- has_schema_privilege checks whether a user can access a schema in a particular way. It is equivalent to has_table_privilege. The permission types must be some combinations of CREATE, USAGE, ALTER, DROP, or COMMENT.
- has_server_privilege checks whether a user can access a foreign server in a particular way. It is equivalent to has_table_privilege. The permission type must be USAGE, ALTER, DROP, or COMMENT.
- has_tablespace_privilege checks whether a user can access a tablespace in a particular way. It is equivalent to has_table_privilege. The permission type must be CREATE, ALTER, DROP, or COMMENT.

HUAWEI

Access Permission Query Functions - Example

- has_table_privilege(table, privilege) queries whether the current user has permissions on tables. The return type is Boolean.
  - The user can be specified by name or by OID (**pg_authid.oid**), or be set to **public**, which indicates public pseudo roles. If this parameter is omitted, **current_user** is used.
  - The table can be specified by name or by OID. When it is specified by name, the name can be schema-qualified if necessary. The desired access permission type is specified by a text string, which must be one of the values **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **TRUNCATE**, **REFERENCES**, **TRIGGER**, **ALTER**, **DROP**, **COMMENT**, **INDEX**, or **VACUUM**.
  - Optionally, WITH GRANT OPTION can be added to a permission type to test whether the permission is held with the grant option. Also, multiple permission types can be separated by commas, in which case the result will be **true** if any of the listed permissions is held.

```
openGauss=# SELECT has_table_privilege('tpcds.web_site', 'select');
 has_table_privilege
---------------------
 t
(1 row)
openGauss=# SELECT has_table_privilege('omm', 'tpcds.web_site', 'select,INSERT WITH GRANT OPTION ');
 has_table_privilege
---------------------
 t
(1 row)
```

Forbidding Granting All Permissions of an Object to the PUBLIC Role

- All users are attached to the PUBLIC role. Therefore, this role should have the fewest permissions for database security purposes.
  - The keyword PUBLIC indicates that the permissions are to be granted to all roles, including those that might be created later. PUBLIC can be regarded as an implicitly defined group that always includes all roles.
  - Any particular role will have the sum of permissions granted directly to it, permissions granted to any role it is presently a member of, and permissions granted to PUBLIC.
  - By default, permissions on tables, columns, sequences, foreign data sources, foreign servers, schemas, and tablespaces are not granted to PUBLIC.
  - Permissions on the following objects are granted to PUBLIC: CONNECT and CREATE TEMP TABLE permissions on databases, EXECUTE permission on functions, and USAGE permission on languages and data types (including domains).
- Revoke permissions on objects from the PUBLIC role.

```
openGauss=# REVOKE ALL ON <OBJECT_NAME> FROM public;
```

- Check method:

```
openGauss=# SELECT relname,relacl FROM pg_class WHERE CAST(relacl AS TEXT) LIKE '%,=%}' OR
CAST(relacl AS TEXT) LIKE '{=%}';
```

Batch Object Permission Management

- Manage permissions on a type of objects in a schema, for example, query permissions on all tables.

```
grant USAGE on SCHEMA xxx to user;
grant SELECT on ALL TABLES in schema xxx to user;
alter DEFAULT PRIVILEGES in schema xxx grant SELECT on TABLES to user;
```

  - The first statement grants the use permission on schemas.
  - The second statement grants the query permission on all current tables.
  - The third statement adds the query permission to the default permissions on the new table.

HUAWEI

ANY Permission Management

- It is not allowed to grant the ANY permissions to PUBLIC or revoke the ANY permissions from PUBLIC.
- The ANY permissions are database permissions and are valid only for database objects that are granted with the permissions. For example, SELECT ANY TABLE only allows a user to view all user table data in the current database, but the user does not have the permission to view user tables in other databases.
- Even if a user is granted with the ANY permissions, the user cannot perform INSERT, DELETE, UPDATE, and SELECT operations on the objects of private users.
- The ANY permissions and the original permissions do not affect each other.
- If a user is granted with the CREATE ANY TABLE permission, the owner of a table created in a schema with the same name as the user is the owner of the schema. When the user performs other operations on the table, the user needs to be granted with the corresponding operation permission. Similarly, if a user is granted with the CREATE ANY FUNCTION, CREATE ANY PACKAGE, CREATE ANY TYPE, CREATE ANY SEQUENCE, or CREATE ANY INDEX permission, the owner of an object created in a schema with the same name is the owner of the schema. If a user is granted with the CREATE ANY TRIGGER or CREATE ANY SYNONYM permission, the owner of an object created in a schema with the same name is the creator.
- Exercise caution when granting the CREATE ANY FUNCTION permission to users to prevent other users from using the SECURITY DEFINER functions for permission escalation.
- **WITH ADMIN OPTION**:
  - If **WITH ADMIN OPTION** is specified for a role, the grantee can grant the role to or revoke the role from other roles or users.
  - If **WITH ADMIN OPTION** is specified for the ANY permissions, the grantee can grant the ANY permissions to or revoke them from other roles or users.

HUAWEI

Schema Permission Control

- By default, a user can only access database objects in their own schema. Only after a user is granted with the usage permission on a schema by the schema owner, the user can access the objects in the schema.
- By granting the CREATE permission on a schema to a user, the user can create objects in this schema. By default, all roles have the USAGE permission on the public schema, but common users do not have the CREATE permission on the public schema. It is insecure for a common user to connect to a specified database and create objects in its public schema. If the common user has the CREATE permission on the public schema, you are advised to:
  - Revoke **PUBLIC**'s permission to create objects in the public schema. In the following command, **public** indicates the schema and **PUBLIC** indicates all roles:

```
openGauss=# REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

  - Run the following command to view the current schema:

```
openGauss=# select current_schema();
 current_schema
----------------
 myschema
(1 row)
```

Contents

1. System Catalogs and System Views
2. System Information Functions and Database Object Functions
3. Database Object Permission Management
4. **Scheduled Jobs**
   - Definition and Syntax Format
   - Scheduled Job Management

HUAWEI

Overview of Scheduled Jobs

- Context
  - When a user executes some time-consuming jobs during the day time, (for example, statistics summary job or other database synchronization jobs), the service performance will be influenced. So users execute jobs on database during night time, increasing the workload. The scheduled job function of the database provides the scheduled job function that users can create scheduled jobs. When the scheduled job time arrives, the job will be triggered. Therefore, the workload of O&M has been reduced.
  - openGauss provides the functions to create scheduled jobs, execute jobs automatically, delete jobs, and modify job attributes (including job ID, the enabled/disabled status of a job, job triggering time, triggering interval, and job contents).
- System catalogs (views) and advanced packages related to scheduled jobs in openGauss:
  - The advanced package **DBE_SCHEDULER** supports more flexible creation of scheduled jobs through scheduling and programing.
  - ADM_SCHEDULER_JOBS displays information about all DBE_SCHEDULER scheduler jobs in the database. By default, only a system administrator can access this view. Common users can access the view only after being authorized. This view exists in both the PG_CATALOG and SYS schemas.
  - PG_JOB records detailed information about jobs created by users. Dedicated threads poll the pg_job system catalog and trigger jobs based on scheduled job execution time, and update job status in the pg_job system catalog. This system catalog belongs to Shared Relation. All job records are visible to all databases. Common users can access it only after being authorized.
  - The advanced function package **DBE_TASK** provides APIs for managing scheduled jobs.
  - The advanced function package **PKG_SERVICE** provides APIs for managing scheduled jobs.

HUAWEI

Precautions for Scheduled Jobs (1)

- Operations related to scheduled jobs are supported only when **sql_compatibility** is set to **'B'**.
- A user without the sysadmin permission must obtain the permission from the user who has the sysadmin permission to create, modify, or delete a scheduled job. The operation permissions of a scheduled job are the same as those of creating scheduled jobs for the advanced package **DBE_SCHEDULER**.
- Currently, the interval expression of a scheduled job is compatible with the syntax of floating-point number, for example, **interval 0.5 minute**. However, the floating-point number is rounded up during calculation. Therefore, you are advised not to use the floating-point number for the interval.
- Scheduled jobs with the same name are not supported in the same database.
- The security-related operations are as follows:
  - Use encryption functions.
  - Create and set users and groups.
  - Connect to a database.
  - Encrypt a function.

HUAWEI

Precautions for Scheduled Jobs (2)

- The statements to be executed in a scheduled event are any SQL statements except security-related operations. However, some statements with restrictions fail to be executed. For example, a database cannot be created by using composite statements.
- The definer fails to be specified for a scheduled job in the following scenarios:
  - The user who operates the scheduled job does not have the sysadmin permission.
  - If the current user is inconsistent with the specified definer:
    - An initial user is specified as the definer.
    - A private user, O&M administrator, or monitor administrator is specified as the definer.
    - The parameter **enableSeparationOfDuty** is set to **on** to enable the separation of duties.
- Only the owner has the permission to modify the scheduled job to be modified. By default, the system administrator has the permission to modify all scheduled jobs.
- You can execute SHOW EVENTS or view the **log_user** column in the PG_JOB system catalog to obtain the job owner information.
- Each time a scheduled job is modified, the owner of the modified job is changed to the current user. If a definer is specified during modification, the owner is changed to the specified definer.

HUAWEI

Creating Scheduled Jobs (1)

```
CREATE
    [DEFINER = user]
EVENT
    [IF NOT EXISTS]
    event_name
    ON SCHEDULE schedule
    [ON COMPLETION [NOT] PRESERVE]
    [ENABLE | DISABLE | DISABLE ON SLAVE]
    [COMMENT 'string']
    DO event_body;
```

- **DEFINER**
  - Specifies the permission on a scheduled job statement to be executed during execution. By default, the permission of the user who creates the scheduled job is used. When a definer is specified, the permission of the specified user is used.
  - Only users with the sysadmin permission can specify the definer.
- **ON SCHEDULE**: specifies the time when a scheduled job is executed.
- **INTERVAL**: specifies the interval. The value of **INTERVAL** consists of a number and a time unit specified by **quantity**, for example, **1 YEAR**.
- **ON COMPLETION [NOT] PRESERVE**: Once a transaction is complete, the scheduled job is deleted from the system catalog immediately by default. You can overwrite the default behavior by setting **ON COMPLETION PRESERVE**.

HUAWEI

Creating Scheduled Jobs (2)

```
schedule: {
    AT timestamp [+ INTERVAL interval] ...
  | EVERY interval
    [STARTS timestamp [+ INTERVAL interval] ...]
    [ENDS timestamp [+ INTERVAL interval] ...]
}
```

- **ON SCHEDULE**: specifies the time when a scheduled job is executed. A scheduled job can be set to be executed once or multiple times through SCHEDULE.
  - **AT timestamp [+ INTERVAL interval]** indicates that the scheduled job is executed only once at **timestamp [+ INTERVAL interval]**.
  - **EVERY interval** indicates that the scheduled job is executed at the interval specified by **interval**.
    - **STARTS timestamp [+ INTERVAL interval]** indicates that user can specify the start time for a scheduled job that can be executed repeatedly. That is, the scheduled job is executed since **timestamp [+ INTERVAL interval]**. If this parameter is left empty, the job is executed from the current time by default.
    - **ENDS timestamp [+ INTERVAL interval]** indicates that user can specify the end time for a scheduled job that can be executed repeatedly. That is, the scheduled job ends at **timestamp [+ INTERVAL interval]**. If this parameter is left empty, the default value **3999-12-31 16:00:00** is used.

HUAWEI

Creating Scheduled Jobs - Examples

```
openGauss=# CREATE TABLE t_ev(num int);

-- Create a scheduled job that is executed once.
openGauss=# CREATE EVENT IF NOT EXISTS event_e1 ON SCHEDULE AT current_timestamp + interval 5 second +
interval 33 minute DISABLE DO insert into t_ev values(0);

-- Create a scheduled job that is executed every minute.
openGauss=# CREATE EVENT IF NOT EXISTS event_e1 ON SCHEDULE EVERY 1 minute DO insert into t_ev values(1);
```

- Notes:
  - If a scheduled job fails to be executed after being created, you can view the failure cause use SHOW EVENTS or in the PG_JOB table.
  - When operations related to user passwords (such as creating weak passwords) are performed in the statements to be executed for a scheduled job, system catalog records the password in plaintext. Therefore, you are advised not to perform operations related to user passwords in the statements to be executed for the scheduled job.

HUAWEI

Modifying Scheduled Jobs - Syntax Format

```
ALTER
    [DEFINER = user]
EVENT event_name
    [ON SCHEDULE schedule]
    [ON COMPLETION [NOT] PRESERVE]
    [RENAME TO new_event_name]
    [ENABLE | DISABLE | DISABLE ON SLAVE]
    [COMMENT 'string']
    [DO event_body]
```

- Parameters:
  - The parameter description is the same as that in the syntax for creating a scheduled job.
  - **RENAME TO** indicates the name of the scheduled job to be updated.
- Note: If a system administrator modifies a scheduled job created by another user, the owner of the modified job is changed to the system administrator, who will execute the statements to be executed.

Modifying Scheduled Jobs - Examples

- Create a scheduled job.

```
openGauss=# CREATE TABLE t_ev(num int);

openGauss=# CREATE EVENT IF NOT EXISTS event_e1 ON SCHEDULE AT current_timestamp + interval 5 second + interval 33 minute DISABLE DO insert into t_ev values(0);
```

- Modify the scheduled job.
  - Modify the scheduled job status and statements to be executed.

```
openGauss=# ALTER EVENT event_e1 ENABLE DO select 1;
```

  - Change the name of a scheduled job.

```
openGauss=# ALTER EVENT event_e1 RENAME TO event_ee;
```


HUAWEI

SHOW EVENTS

- Function: Displays basic information about all scheduled jobs in a specified schema.

```
SHOW EVENTS
    [{FROM | IN} schema_name]
    [LIKE 'pattern' | WHERE
condition]
```

- Parameters:
  - **{FROM | IN}** specifies the schema to be queried. By default, the current schema is queried.
  - **LIKE** matches a scheduled job by name. If this parameter is not specified, all scheduled jobs in the current schema are printed.
  - The WHERE clause forms an expression for row selection to narrow down the query range of SHOW EVENTS. **condition** indicates any expression that returns a value of Boolean type. Rows that do not meet this condition will not be retrieved.
- Example:
  - View information about all scheduled jobs queried through pattern matching **'_e'** in the **event_a** schema.

```
openGauss=# SHOW EVENTS IN event_a LIKE '_e';
```


HUAWEI

Scheduled Jobs - Related Parameters

- **job_queue_processes** specifies the number of jobs that can be concurrently executed.
  - This parameter is a POSTMASTER parameter. You can set it using gs_guc, and you need to restart GaussDB to make the setting take effect.
  - Value range: 0 to 1000
    - **0** indicates that the scheduled job function is disabled and that no job will be executed. (Enabling scheduled jobs may affect the system performance. At sites where this function is not required, you are advised to disable it.)
    - A value that is greater than 0 indicates that the scheduled job function is enabled and this value is the maximum number of jobs that can be concurrently processed.
  - After the scheduled job function is enabled, the job_scheduler thread polls the pg_job system catalog at a scheduled interval. The scheduled job check is performed every second by default.
  - Too many concurrent jobs consume many system resources, so you need to set the number of concurrent jobs to be processed. If the current number of concurrent jobs reaches the value of **job_queue_processes** and some of them expire, these jobs will be postponed to the next polling period.
- **enable_prevent_job_task_startup** specifies whether to start the job thread. This parameter is a SIGHUP parameter.
  - Value range: Boolean.
  - **on** indicates that the job thread cannot be started. **off** (default) indicates that the job thread can be started.

HUAWEI

Contents

1. System Catalogs and System Views
2. System Information Functions and Database Object Functions
3. Database Object Permission Management
4. **Scheduled Jobs**
   - Definition and Syntax Format
   - Scheduled Job Management

HUAWEI

Scheduled Job Management - PKG_SERVICE Advanced Package - Related API Functions

- A scheduled job involves the following functions:
  - Delete a specified scheduled job.

```
PKG_SERVICE.JOB_CANCEL(
job  IN  INTEGER);  // Specified job number.
```

  - Disable or enable a scheduled job.

```
PKG_SERVICE.JOB_FINISH(
id          IN   INTEGER,
broken      IN   BOOLEAN,  // true indicates disabled and false indicates enabled.
next_time   IN   TIMESTAMP  DEFAULT  sysdate);  // Next running time.
```

  - Commit a scheduled job provided by the system.

```
PKG_SERVICE.JOB_SUBMIT(
id            IN   BIGINT DEFAULT,
content       IN   TEXT,
next_date     IN   TIMESTAMP DEFAULT sysdate,
interval_time IN   TEXT  DEFAULT 'null',  // Execute only once by default.
job           OUT  INTEGER);
```

  - Modify the attributes of a scheduled job.

```
# Includes the task content, next execution time, and execution interval.
PKG_SERVICE.JOB_UPDATE(
id            IN   BIGINT,
next_time     IN   TIMESTAMP,
interval_time IN   TEXT,
content       IN   TEXT);
```

Scheduled Job Management - PKG_SERVICE Advanced Package - Case (1)

```
// Step 1: Create a table.
openGauss=# create table test3(id int);
CREATE TABLE

// Step 2: Create a sequence to insert data into the test3 table.
openGauss=# create sequence seq2 start with 1 increment 1
nomaxvalue;
CREATE SEQUENCE

// Step 3: Create a stored procedure as the content of the
scheduled job.
openGauss=# create or replace procedure job_test
as
begin
insert into test3 values(seq2.nextval);
end;
/
openGauss$# /
CREATE PROCEDURE
```

```
// Step 4: Obtain the definition of the function or stored
procedure from the pg_proc system catalog or the
pg_get_functiondef(oid) function.
openGauss=# select oid from pg_proc where proname='job_test';
  oid
-------
 16919
(1 row)
openGauss=# select * from pg_get_functiondef(16919);
 headerlines |                  definition
-------------+-----------------------------------------------
           1 | CREATE OR REPLACE PROCEDURE public.job_test()+
             | AS  DECLARE                                  +
             | begin                                        +
             | insert into test3 values(seq2.nextval);      +
             | end;                                         +
             | /                                            +
             |
(1 row)
```


HUAWEI

Scheduled Job Management - PKG_SERVICE Advanced Package - Case (2)

```
// Step 5: Create a scheduled job.
openGauss=# select pkg_service.job_submit(1,'select
job_test()',sysdate,'sysdate+1/(24*60)');
 job_submit
------------
          1
(1 row)
# The first parameter specifies the number of the scheduled job.
If the value of first parameter is null, the system automatically
generates a number for the scheduled job.
# The preceding scheduled job indicates that the job is executed
every minute.

// Step 6: View the current scheduled jobs in the pg_job system
catalog.

select * from pg_job;
```

```
// Step 7: Check whether the scheduled job is successful.
# 1. Check whether data is inserted into the test3 table.
openGauss=# select * from test3;
 id
----
  1
  2
  3
  4
(4 rows)
# 2. View the log file.
/var/log/omm/omm/pg_log/dn_6001/postgresql-2023-08-
17_086739.log
As the scheduled job is enabled, the size of the log file increases.
You need to delete logs periodically.
```


HUAWEI

Scheduled Job Management - PKG_SERVICE Advanced Package - Case (3)

```
//Step 8: Invalidate the scheduled job. The third parameter can be omitted, indicating the next execution time of the job.
openGauss=# select pkg_service.job_finish(1,true);
 job_finish
------------
(1 row)
// Step 9: Check that the value of job_status for the scheduled job changes to d, and the value of next_run_date changes to 4000-01-01 00:00:00.
openGauss=# select * from pg_job;
-[ RECORD 1 ]--------+-------------------------
job_id               | 1
current_postgres_pid | -1
log_user             | omm
priv_user            | omm
dbname               | postgres
node_name            | dn_6001
job_status           | d
start_date           | 2023-08-17 16:41:50.469569
next_run_date        | 4000-01-01 00:00:00
failure_count        | 0
```

```
// Step 10: Change the scheduled job to be executed once a day. (Note: Before the modification, start the scheduled job pkg_service.job_finish.)

openGauss=# select pkg_service.job_update(1,sysdate,'sysdate+1',null);
-[ RECORD 1 ]
job_update |


// Step 11: Delete the scheduled job. (The following information indicates that the \x meta-command is executed on the gsql client in advance.)
openGauss=# select pkg_service.job_cancel(1);
-[ RECORD 1 ]
job_cancel |
```


HUAWEI

Quiz

1. (Multiple-choice) If openGauss uses the view monitoring function of Workload Manager, which of the following parameters need to be enabled?

   A. use_workload_manager

   B. enable_resource_record

   C. resource_track_duration

   D. enable_resource_track

2. (True or False) By default, only a system administrator can access the ADM_SCHEDULER_JOBS view. Common users can access the view only after being authorized. This view exists only in the PG_CATALOG schema.

   A. True

   B. False

HUAWEI

Summary

- After learning this chapter, you will be proficient in using the functions, system catalogs, and system views related to database objects, master the permission types and operations of database objects, have a comprehensive understanding of the background, functions, and syntaxes of scheduled tasks, and be proficient in managing scheduled jobs through the functions of advanced packages.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/en/
- openGauss Help (5.0.0)
  - https://docs-opengauss.osinfra.cn/en/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 3 Database System Management



HUAWEI

Foreword

- This chapter describes routine O&M management of the database system, including checking the openGauss database status and lock information, checking the OS health, overview of backup and restoration management, checking database performance, checking and clearing OS logs and database run logs, checking the number of application connections, maintaining tables and rebuilding indexes, and exporting and viewing workload diagnosis reports (WDRs).

HUAWEI

Objectives

- Upon completion of this course, you will be able to:
  - Check and maintain the database system status.
  - Check the OS health status.
  - Master the basic principles of backup, restoration, and disaster recovery (DR).
  - Check and clear logs.
  - Export and view WDRs.

HUAWEI

Contents

1. **Routinely Checking Databases**
2. Managing Backup and Restoration
3. Introduction to HA and DR
4. Checking Database Performance
5. Checking and Clearing Logs
6. Checking the Number of Application Connections
7. Maintaining Tables and Rebuilding Indexes
8. Exporting and Viewing WDRs

HUAWEI

Checking the Database Cluster Status

- Run the following commands as the database installation user **omm** to check the database status:

```
[omm@Run861 ~]$ gs_om -t status --detail
[  CMServer State   ]

node      node_ip         instance                                    state
------------------------------------------------------------------------------
1  Run861 7.213.197.105   1    /opt/huawei/data/cmserver/cm_server Primary
2  Run862 7.213.202.227   2    /opt/huawei/data/cmserver/cm_server Standby
3  Run863 7.213.208.176   3    /opt/huawei/data/cmserver/cm_server Standby

[   Cluster State   ]

cluster_state   : Normal
redistributing  : No
balanced        : No
current_az      : AZ_ALL

[  Datanode State   ]

node      node_ip         instance                        state
------------------------------------------------------------------------------
1  Run861 7.213.197.105   6001 /opt/huawei/install/data/dn P Standby Normal
2  Run862 7.213.202.227   6002 /opt/huawei/install/data/dn S Primary Normal
3  Run863 7.213.208.176   6003 /opt/huawei/install/data/dn S Standby Normal
```

CMServer state: CM Server status in the cluster.

Datanode state: openGauss database status.

node: node ID.

node_ip: host name and IP address of the node.

port: database port.

instance: data directory.

state: status of the current database node.

HUAWEI

Running the **gs_guc** Command to Modify Database Parameters

- Modify a parameter: **gs_guc reload -N all -I all -c "enable_cbm_tracking=on"**
- Check whether the parameter setting takes effect: **gsql -d postgres -p +** ***port*** **-c "show enable_cbm_tracking"**

```
[omm@Run861 ~]$ gsql -d postgres -p 15400 -c "show enable_cbm_tracking";
 enable_cbm_tracking
---------------------
 off
(1 row)

[omm@Run861 ~]$ gs_guc reload -N all -I all -c "enable_cbm_tracking=on"
The gs_guc run with the following arguments: [gs_guc -N all -I all -c enable_cbm_tracking=on reload ].
NOTICE: Turn on cbm tracking function.
Begin to perform the total nodes: 3.
Popen count is 3, Popen success count is 3, Popen failure count is 0.
Begin to perform gs_guc for datanodes.
Command count is 3, Command success count is 3, Command failure count is 0.

Total instances: 3. Failed instances: 0.
ALL: Success to perform gs_guc!


[omm@Run861 ~]$ gsql -d postgres -p 15400 -c "show enable_cbm_tracking";
 enable_cbm_tracking
---------------------
 on
(1 row)
```


HUAWEI

Checking the Database System Configuration

- Check the initial configuration of the database according to the **clusterconfig.xml** configuration file.

```
[root@Run861 openGauss]# more clusterconfig.xml
<?xml version="1.0" encoding="UTF-8"?>
<ROOT>
    <!-- openGauss整体信息 -->
    <CLUSTER>
        <PARAM name="clusterName" value="Cluster_OG" />
        <PARAM name="nodeNames" value="Run861,Run862,Run863" />

        <PARAM name="gaussdbAppPath" value="/opt/huawei/install/app" />
        <PARAM name="gaussdbLogPath" value="/var/log/omm" />
        <PARAM name="tmpMppdbPath" value="/opt/huawei/tmp"/>
        <PARAM name="gaussdbToolPath" value="/opt/huawei/install/om" />
        <PARAM name="corePath" value="/opt/huawei/corefile"/>
        <PARAM name="backIp1s" value="7.213.197.105,7.213.202.227,7.213.208.176"/>

    </CLUSTER>
    <!-- 每台服务器上的节点部署信息 -->
    <DEVICELIST>
        <!-- node1上的节点部署信息 -->
        <DEVICE sn="Run861">
            <PARAM name="name" value="Run861"/>
            <PARAM name="azName" value="AZ1"/>
            <PARAM name="azPriority" value="1"/>
            <!-- 如果服务器只有一个网卡可用，将backIP1和sshIP1配置成同一个IP -->
            <PARAM name="backIp1" value="7.213.197.105"/>
            <PARAM name="sshIp1" value="7.213.197.105"/>
            <!--CM节点部署信息-->
            <PARAM name="cmsNum" value="1"/>
            <PARAM name="cmServerPortBase" value="15000"/>
            <PARAM name="cmServerListenIp1" value="7.213.197.105,7.213.202.227,7.213.208.176"/>
            <PARAM name="cmServerHaIp1" value="7.213.197.105,7.213.202.227,7.213.208.176"/>
            <PARAM name="cmServerlevel" value="1"/>
            <PARAM name="cmServerRelation" value="Run861,Run862,Run863"/>
            <PARAM name="cmDir" value="/opt/huawei/data/cmserver"/>
            <!--dn-->
            <PARAM name="dataNum" value="1"/>
            <PARAM name="dataPortBase" value="15400"/>
            <PARAM name="dataNode1" value="/opt/huawei/install/data/dn,Run862,/opt/huawei/install/data/dn,Run863,/opt/huawei/install/data/dn"/>
            <PARAM name="dataNode1_syncNum" value="0"/>
        </DEVICE>
```


HUAWEI

Description of Items in the Installation Configuration File

Item	Description
clusterName	openGauss database cluster name.
nodeNames	Host name in openGauss.
backIp	Intranet IP address of the host in the backend storage network. All the openGauss hosts communicate with each other on this network.
gaussdbAppPath	Installation directory of the openGauss program.
gaussdbLogPath	Directory that stores run logs and operation logs of openGauss. This directory is optional. If the directory is not specified, *$GAUSSLOG/Installation user account* will be specified as the log directory by default during openGauss installation.
tmpMppdbPath	Directory for storing temporary database files. If **tmpMppdbPath** is not set, files are stored in the **/opt/huawei/wisequery/***Installation user account*_**mppdb** directory by default. /**opt/huawei/wisequery** is the default database system tool directory.
gaussdbToolPath	Directory for storing openGauss system tools. This directory is used to store tools for mutual trust. This directory is optional. If this parameter is not specified, **/opt/huawei/wisequery** is specified as the database system tool directory by default during openGauss installation.
corePath	Directory for storing the openGauss core file.
dataPortBase	Database port.
datanode	Data directory.

HUAWEI

Checking the openGauss Status

- openGauss provides tools to check database and instance status, ensuring that databases and instances are running properly to provide data services.
- Check instance status.

```
[omm@Run861 ~]$  gs_check -U omm -i CheckClusterState
```

- Check a parameter.

```
openGauss=# SHOW parameter_name;
```

- Modify a parameter.

```
[omm@Run861 ~]$  gs_guc reload  -D /opt/huawei/install/data/dn -c "paraname=value"
```


HUAWEI

Checking Lock Information

- The lock mechanism is an important method to ensure data consistency. Checking lock information helps learn about database transactions and running status.
- Query lock information in the database.

```
openGauss=# SELECT * FROM pg_locks;
```

- Query the status of threads waiting to acquire locks.

```
openGauss=# SELECT * FROM pg_thread_wait_status WHERE wait_status = 'acquire lock';
```

- End the system process, search for a system process that is running, and run the **kill**

```
ps ux
kill -9 pid
```


HUAWEI

Collecting Event Statistics

- Long-time running of SQL statements will occupy a lot of system resources. Event occurrence time and occupied memory need to be checked to learn about database running status.
- Query the time points about an event.
  - Query the thread start time, transaction start time, SQL statement start time, and status change time of an event.

```
openGauss=# SELECT backend_start,xact_start,query_start,state_change FROM pg_stat_activity;
```

- Query the number of sessions on the current server.

```
openGauss=# SELECT count(*) FROM pg_stat_activity;
```

- Query system-level statistics.

```
openGauss=# SELECT * FROM pv_session_memory_detail() ORDER BY usedsize desc limit 10;
```


HUAWEI

Checking Objects (1)

- Tables, indexes, partitions, and constraints are key storage objects of a database. A database administrator needs to routinely maintain key information and these objects.
- View table details.

```
openGauss=#\d+ table_name;
```

- Query table statistics.

```
openGauss=# SELECT * FROM pg_statistic;
```

- View index details.

```
openGauss=# \d+ index_name;
```

- Query partitioned table information.

```
openGauss=#select  *  from pg_partition;
```


HUAWEI

Checking Objects (2)

- Collect statistics.
  - Run the **ANALYZE** statement to collect related statistics on the database.
  - Run the **VACUUM** statement to recycle space and update statistics.
- Query constraint information.

```
openGauss=# SELECT * FROM pg_constraint;
```

- Check an SQL report.
- Run the **EXPLAIN** SQL statement to view an execution plan.

HUAWEI

Checking Basic Information

- Basic information includes versions, components, and patches. Periodically checking and recording database information are important for database lifecycle management.
- Check version information.

```
openGauss=# SELECT version();
```

- Check table size and database size.

```
openGauss=# SELECT pg_table_size('table_name');

openGauss=# SELECT pg_database_size('database_name');
```


HUAWEI

Checking OS Parameters

- Use the gs_checkos tool provided by openGauss to check the OS status.
- Prerequisites
  - The hardware and network are working properly.
  - The trust relationship of user **root** among the hosts is normal.
  - Only user **root** is authorized to run the **gs_checkos** command.
- Procedure
  - Log in to a server as user **root**.
  - Run the following command to check OS parameters of servers where the openGauss nodes are deployed:

```
# gs_checkos -i A
```


HUAWEI

Checking openGauss Health Status

- Use the gs_check tool provided by openGauss to check the openGauss health status.
- Notes
  - Only user **root** is authorized to check new nodes. Other checks must be performed by user **omm**.
  - The parameter **-i** or **-e** must be set. **-i** is used to specify a single item to be checked and **-e** is used to specify an inspection scenario where multiple items will be checked.
  - If **-i** is not set to a root item or no such items are contained in the check item list of the scenario specified by **-e**, you do not need to enter the name or password of user **root**.
  - You can run **--skip-root-items** to skip checking root items so that you do not need to enter the name or password of user **root**.
  - When checking the consistency between new nodes and existing nodes, run the **gs_check** command on an existing node and specify the **--hosts** parameter. The IP address of the new node needs to be written into the **hosts** file.

HUAWEI

Contents

1. Routinely Checking Databases
2. **Managing Backup and Restoration**
3. Introduction to HA and DR
4. Checking Database Performance
5. Checking and Clearing Logs
6. Checking the Number of Application Connections
7. Maintaining Tables and Rebuilding Indexes
8. Exporting and Viewing WDRs

HUAWEI

Introduction to Backup (1)

- The openGauss database supports three backup and restoration types, multiple backup and restoration solutions, and data reliability assurance mechanisms during backup and restoration.
- Backup and restoration can be classified into logical backup and restoration, physical backup and restoration, and flashback.
  - Logical backup and restoration: backs up data by logically exporting data. This method can dump data that is backed up at a certain time point, and restore data only to this backup point.
  - Physical backup and restoration: copies physical files in the unit of disk blocks from the primary node to the standby node to back up a database.
  - Flashback: restores dropped tables from the recycle bin. Like in a Window OS, dropped table information is stored in the recycle bin of databases. The MVCC mechanism is used to restore data to a specified point in time or commit sequence number (CSN).

HUAWEI

Introduction to Backup (2)

Backup Type	Scenario	Tool	Recovery Time	Advantages and Disadvantages
Logical backup and restoration	Small volume of data needs to be processed.	gs_dump	It takes a long time to restore data in plain-text format. It takes a long time to restore data in archive format.	This tool is used to export database information. Users can export a database or its objects (such as schemas, tables, and views). The database can be the default **postgres** database or a user-specified database. The exported file can be in plain-text format or archive format. Data in plain-text format can be restored only by using gsql, which takes a long time. Data in archive format can be restored only by using gs_restore. The restoration time is shorter than that for the plain-text format.
	You can back up a single table, multiple tables, a single database, or all databases. The backup data needs to be restored using gsql or gs_restore. When the data volume is large, the restoration takes a long time.	gs_dumpall	Long data restoration time.	This tool is used to export all information of the openGauss database, including the data of the default **postgres** database, data of user-specified databases, and global objects of all openGauss databases. Only data in plain-text format can be exported. The exported data can be restored only by using gsql, which takes a long time.
Flashback	This method is applicable to the following scenarios: 1. A table is deleted by mistake. 2. Data in the tables needs to be restored to a specified time point or CSN.	None	You can restore a table to the status at a specified time point or before the table structure is deleted. The restoration takes a short time.	Flashback can selectively and efficiently undo the impact of a committed transaction and recover from a human error. Before the flashback technology is used, the committed database modification can be retrieved only by means of backup and restoration or PITR. The restoration takes several minutes or even hours. After the flashback technology is used, it takes only seconds to restore the committed data before the database is modified. The restoration time is irrelevant to the database size. Flashback supports two restoration modes: Data restoration based on MVCC: applicable to the query and restoration of data that is deleted, updated, or inserted by mistake. You can configure the retention period of historical data and run the corresponding command to query or restore data to a specified time point or CSN. Recovery based on the recycle bin (similar to that on Windows OS): This method is applicable to the restoration of tables that are dropped or truncated by mistake. You can configure the recycle bin function and run the corresponding command to restore the tables that are dropped or truncated by mistake.

HUAWEI

Introduction to Backup (3)

Backup Type	Scenario	Tool	Recovery Time	Advantages and Disadvantages
Physical backup and restoration	Huge volume of data needs to be processed. It is mainly used for full backup and restoration as well as the backup of all archived WALs and run logs in the database.	gs_backup	Small data volume and fast data restoration.	The OM tool for exporting database information can be used to export database parameter files and binary files. It helps openGauss to back up and restore important data, and display help and version information. During the backup, you can select the type of the content to be backed up. During the restoration, ensure that the backup files exist in the backup directory of each node. Database instances are restored based on the database instance information in the static configuration file. Because only the parameter files are restored, the restoration time can be greatly reduced.
		gs_basebackup	During the restoration, you can directly copy and replace the original files, or directly start the database on the backup database. The restoration takes a short time.	This tool is used to fully copy the binary files of the server database. Only the database at a certain time point can be backed up. With PITR, you can restore data to a time point after a full backup.
		gs_probackup	Data can be directly restored to a backup point and the database can be started on the backup database. The restoration takes a short time.	gs_probackup is a tool used to manage the openGauss database backup and restoration. It periodically backs up openGauss instances. It can be used for physical backup of a standalone database or the primary node of a database instance. It can also be used to back up contents in external directories, such as script files, configuration files, log files, and dump files. Besides, it supports incremental backup, periodic backup, and remote backup. The time required for incremental backup is shorter than that for full backup. You only need to back up the modified files. Currently, the data directory is backed up by default. If the tablespace is not in the data directory, you need to manually specify the tablespace directory to be backed up. Currently, data can be backed up only on the primary node.

HUAWEI

Logical Backup and Restoration (1)

- gs_dump, provided by openGauss, is used to export database information. You can export a database or its objects (such as schemas, tables, and views), excluding objects in the recycle bin. The database can be the default **postgres** database or a user-specified database.
  - gs_dump is executed by OS user **omm**.
  - When gs_dump is used to export data, other users can still access (read and write) the openGauss database.
  - gs_dump can export complete and consistent data. For example, if gs_dump is started to export database A at T1, data of the database at T1 will be exported, and modifications on the database after T1 will not be exported.
  - The columns generated by executing gs_dump are not dumped.

HUAWEI

Logical Backup and Restoration (2)

- **gs_dump**: exports full information about the **postgres** database.

```
gs_dump -U omm -W  password  -f backup/MPPDB_backup.sql -p 15432 postgres -F p
```

- **gs_dump -n**: exports a specified schema.

```
gs_dump -U omm –W password  -f backup/test_use1_backup.sql -p 5432 postgres -n test_use1 -F p
```

- **gs_dump -t**: exports a specified table.

```
gs_dump -U omm –W password  -f backup/test_use1_backup.sql -p 5432 postgres -t test_use1.ora_alter_table -F p
```

- **gs_dump -s**: exports only table definitions but not data.

```
gs_dump -U omm –W password -f backup/test_use1_backup.sql -p 5432 postgres -t test_use1.ora_alter_table -s -F p
```


HUAWEI

Format of Exported Files

Format	Value of -F	Description	Suggestion	Import Tool
Plain-text	p	A plain-text script file that contains SQL statements and commands. The commands can be executed on gsql, a command line terminal, to rebuild database objects and load table data.	You are advised to use the plain-text format for small databases.	Before using gsql to restore database objects, you can use a text editor to edit the exported plain-text file as required.
User-defined archive format	c	A binary file that allows the restoration of all or selected database objects from an exported file.	You are advised to use a user-defined archive format for medium or large databases.	You can use gs_restore to import database objects from a user-defined-, directory-, or TAR-format archive.
Directory archive format	d	A directory that contains directory files and the data files of tables and BLOBs.	-	
TAR archive format	t	A TAR-format archive that allows the restoration of all or selected database objects from an exported file. It cannot be further compressed and has an 8-GB limitation on the size of a single table.	-	

Introduction to gs_dumpall

- gs_dumpall, provided by openGauss, is used to export all openGauss database information, including data of the default database **postgres**, user-defined databases, and common global objects of all openGauss databases.
  - gs_dumpall is executed by the OS user **omm**.
  - When gs_dumpall is used to export data, other users can still access (read and write) databases.
  - gs_dumpall can export complete and consistent data. For example, if gs_dumpall is enabled to export the openGauss database at T1, the openGauss data status at the time point T1 is exported, and modifications on openGauss after T1 will not be exported.
  - The columns generated by executing gs_dumpall are not dumped.
  - Syntax: basically the same as gs_dump.

HUAWEI

Introduction to Data Import Using gs_restore

- gs_restore, provided by openGauss, is used to import data or files that were exported using gs_dump. It can also be used to import files exported by gs_dump.
- gs_restore is executed by OS user **omm**.
- Data import to a database:
  - If a database is specified in the connection configuration, data is imported to the database. For parallel import, the password for connecting to the database is required. During data import, the generated columns are automatically updated and saved as common columns.
- Data import to a script file:
  - If no database is specified, a script containing the SQL statement to rebuild the database is created and written to a file or standard output. This script output is equivalent to the plain-text output format of gs_dump.

HUAWEI

Examples of Importing Data Using gs_restore

- Import **table1** in the **PUBLIC** schema.

```
gs_restore -h host_name -p port_number -d postgres -t table1 backup/MPPDB_backup.tar
```

- Import **test1** in the **test1** schema and **test2** in the **test2** schema.

```
gs_restore -h host_name -p port_number -d postgres -n test1 -t test1 -n test2 -t test2 backup/MPPDB_backup.tar
```

- Import **table1** in the **PUBLIC** schema and **table1** in the **test1** schema.

```
gs_restore -h host_name -p port_number -d postgres -n PUBLIC -t table1 -n test1 -t table1
backup/MPPDB_backup.tar
```


HUAWEI

Physical Backup and Restoration

- gs_backup is used to back up configuration files. It helps openGauss back up and restore important data, and display help information and version number information.
  - The openGauss database can be connected.
  - During the restoration, backup files exist in the backup directory on each node.
  - gs_backup is executed by OS user **omm**.
- Execute the gs_backup script to back up the database host.

```
gs_backup -t backup --backup-dir=/opt/software/gaussdb/backup_dir -h plat1 --parameter
```

- Execute the gs_backup script to restore the database host.

```
gs_backup -t restore --backup-dir=/opt/software/gaussdb/backup_dir -h plat1 --parameter
```


HUAWEI

Full Backup Tool - gs_basebackup

- gs_basebackup copies the binary files of the database on the server using a replication protocol. To remotely execute gs_basebackup, you need to use the system administrator account. gs_basebackup supports hot backup and compressed backup.
  - gs_basebackup supports only full backup of the primary and standby nodes.
  - gs_basebackup supports hot backup and compressed backup.
  - If you use gs_basebackup to back up a tablespace that contains an absolute path on the same server, you can use tablespace mapping to redirect the tablespace path or use the archive mode to back up the tablespace.
  - If the incremental checkpoint and doublewrite functions are enabled, gs_basebackup also backs up doublewrite files.
  - If the **pg_xlog** directory is a soft link, no soft link is created during backup. Data is directly backed up to the **pg_xlog** directory in the destination path.
  - If the backup permission is revoked during the backup, the backup may fail or the backup data may be unavailable.
  - If the server does not respond due to a temporary network fault, gs_basebackup will exit after waiting for a maximum of 120 seconds.

HUAWEI

Backup Tool - gs_basebackup

- Prerequisites
  - The openGauss database can be connected.
  - User permissions are not revoked during the backup.
  - In the **pg_hba.conf** file, the replication connection is allowed and the connection is established by a system administrator.
  - If the Xlog transmission mode is **stream**, the value of **max_wal_senders** must be configured to at least **1**.
  - If the Xlog transmission mode is **fetch**, the **wal_keep_segments** parameter must be set to a large value so that logs are not removed before the backup ends.
  - During the restoration, backup files exist in the backup directory on all the nodes. If backup files are lost on any node, copy them to it from another node.
- Example:

```
gs_basebackup -D /home/test/trunk/install/data/backup -h 127.0.0.1 -p 21233
INFO:  The starting position of the xlog copy of the full build is: 0/1B800000. The slot minimum LSN is: 0/1B800000.
```


HUAWEI

Restoring Data from Backup Files

- When the database is faulty, you need to restore data from backup files. gs_basebackup backs up the database in binary mode. Therefore, you can directly copy and replace the original files or start the database on the backup database.
  - If the current database instance is running, a port conflict may occur when you start the database from the backup file. In this case, you need to modify the **port** parameter in the configuration file or specify a port when starting the database.
  - If the current backup file is used for the DR scenario deployed in the primary/standby mode, you may need to modify the replication connections between the primary and standby nodes, such as **replconninfo1** and **replconninfo2** in the **postgresql.conf** file.
  - If the **data_directory** parameter in the **postgresql.conf** configuration file is enabled and configured and the backup directory is used to start the database, the database fails to be started because the value of **data_directory** is different from the backup directory. You can change the value of **data_directory** to a new data directory or comment out this parameter.

HUAWEI

Restoring the Original Database

- To restore the original database, perform the following steps:
  - Disable the database server.
  - Copy the original database and all tablespaces to another location for future use.
  - Delete all or part of the files from the original database.
  - Restore the required database files from the backup with the database OS user permission.
  - If a link file exists in the database, modify the link file so that it can be linked to the correct file.
  - Restart the database server and check the database content to ensure that the database is restored to the required status.

HUAWEI

Introduction to the gs_probackup

- gs_probackup is a tool used to manage the openGauss database backup and restoration. It periodically backs up the openGauss instances so that the database server can be restored when a database fault occurs.
  - It supports the physical backup for databases deployed in standalone or primary/standby mode.
  - It supports the backup of contents in external directories, such as script files, configuration files, log files, and dump files.
  - It supports incremental backup, periodic backup, and remote backup.
  - It supports settings on the backup retention policy.
- Prerequisites
  - The openGauss database can be connected.
  - To use PTRACK incremental backup, manually add **enable_cbm_tracking = on** to **postgresql.conf**.
  - To prevent Xlogs from being cleared before the transmission is complete, increase the value of **wal_keep_segments** in the **postgresql.conf** file.

HUAWEI

Backup Tool - Restrictions on gs_probackup (1)

- The backup must be performed by the user who runs the database server.
- The major version number of the backup database server must be the same as that of the database server to be restored.
- To back up a database in remote mode using SSH, install the database of the same major version on the local and remote hosts, and run the **ssh-copy-id remote_user@remote_host** command to set a password-free SSH connection between the local backup database and the remote database.
- In remote mode, only the subcommands **add-instance**, **backup**, and **restore** can be executed.
- Before running the **restore** subcommand, stop the gaussdb process.
- If a user-defined tablespace exists, add the **--external-mapping** parameter when backing up the tablespace. Otherwise, the tablespace will not be backed up.

HUAWEI

Backup Tool - Restrictions on gs_probackup (2)

- If a large amount of data needs to be backed up, adjust the values of **session_timeout** and **wal_sender_timeout** in the **postgresql.conf** file to prevent backup timeout. In addition, adjust the value of **–rw-timeout** in the backup command line parameters.
- When using the **-T** option to redirect the external directory in the backup to a new directory during restoration, specify the **–external-mapping** parameter.
- After an incremental backup is used for restoration, the created logical replication slot is unavailable and needs to be deleted and rebuilt.
- When remote backup is used, ensure that the clock of the remote server is synchronized with that of the backup server. Otherwise, gaussdb may fail to be started when **--recovery-target-time** is used for restoration.
- When remote backup is valid (**remote-proto** is set to **ssh**), ensure that **-h** and **–remote-host** specify the same server. When remote backup is invalid, if the **-h** option is specified, ensure that **-h** specifies the local address or local host name.
- Currently, logical replication slots cannot be used for backup.

HUAWEI

Backup Tool - Backup Process of gs_probackup

- Initialize the backup directory. Create the **backups/** and **wal/** subdirectories in the specified directory to store backup files and WAL files respectively.

```
gs_probackup init -B backup_dir
```

- Add a backup instance. gs_probackup can store backups of multiple database instances in the same backup directory.

```
gs_probackup add-instance -B backup_dir -D data_dir --instance instance_name
```

- Create a backup for a specified database instance. Before performing an incremental backup, you must create at least one full backup.

```
gs_probackup backup -B backup_dir --instance instance_name -b backup_mode
```

- Restore data from the backup of a specified instance.

```
gs_probackup restore -B backup_dir --instance instance_name -D pgdata-path -i backup_id
```


HUAWEI

Flashback

- Flashback is a part of the database recovery technology. It can be used to selectively cancel the impact of a committed transaction and restore data from incorrect manual operations.
- Before the flashback technology is used, the committed database modification can be retrieved only by means of backup and restoration or PITR. The restoration takes several minutes or even hours.
- After the flashback technology is used, it takes only seconds to restore the committed data before the database is modified. The restoration time is irrelevant to the database size.
- Flashback is applicable to the following scenarios:
  - Flashback query
  - FLASHBACK TABLE
  - Flashback DROP/TRUNCATE
  - Note: Astore does not support flashback DROP/TRUNCATE. You must use Ustore.

HUAWEI

Flashback Query

- Flashback query enables you to query a snapshot of a table at a certain time point in the past. This feature can be used to view and logically rebuild damaged data that is deleted or modified by mistake.
- The flashback query is based on the MVCC mechanism. You can retrieve and query data of an earlier version.
- Parameters:
  - **TIMESTAMP**: specifies a historical time point of the table data to be queried.
  - **CSN**: specifies a logical commit time point of the data in the entire database to be queried. Each CSN in the database represents a consistency point of the entire database. To query the data under a CSN means to query the data related to the consistency point in the database through SQL statements.
- The **undo_retention_time** parameter has been set for specifying the retention period of undo logs.

HUAWEI

Examples of Flashback Query

- Example 1

```
SELECT * FROM t1 TIMECAPSULE TIMESTAMP to_timestamp ('2020-02-11 10:13:22.724718', 'YYYY-MM-DD HH24:MI:SS.FF');
```

- Example 2

```
SELECT * FROM t1 TIMECAPSULE CSN 9617;
```

- Example 3

```
SELECT * FROM t1 AS t TIMECAPSULE TIMESTAMP to_timestamp ('2020-02-11 10:13:22.724718', 'YYYY-MM-DD HH24:MI:SS.FF');
```

- Example 4

```
SELECT * FROM t1 AS t TIMECAPSULE CSN 9617;
```

FLASHBACK TABLE

- FLASHBACK TABLE enables you to restore a table to a specific point in time. When only one table or a group of tables are logically damaged instead of the entire database, this feature can be used to quickly restore the table data.
- Based on the MVCC mechanism, FLASHBACK TABLE deletes incremental data at a specified time point and after the specified time point and retrieves the data deleted at the specified time point and the current time point to achieve table-level data restoration.
- The **undo_retention_time** parameter has been set for specifying the retention period of undo logs.
- Examples

```
TIMECAPSULE TABLE t1 TO TIMESTAMP to_timestamp ('2020-02-11 10:13:22.724718', 'YYYY-MM-DD HH24:MI:SS.FF');

TIMECAPSULE TABLE t1 TO CSN 9617;
```


HUAWEI

Flashback DROP/TRUNCATE

- Flashback DROP
  - Flashback DROP enables you to restore tables that are dropped by mistake and their auxiliary structures, such as indexes and table constraints, from the recycle bin.
  - Flashback DROP is based on the recycle bin mechanism. You can restore physical table files from the recycle bin to restore the dropped tables.
- Flashback TRUNCATE
  - Flashback TRUNCATE enables you to restore tables that are truncated by mistake and restore the physical data of the truncated tables and indexes from the recycle bin.
  - Flashback TRUNCATE is based on the recycle bin mechanism. You can restore physical table files from the recycle bin to restore the truncated tables.
- Prerequisites
  - The **enable_recyclebin** parameter has been set for enabling the recycle bin.
  - The **recyclebin_retention_time** parameter has been set for specifying the retention period of objects in the recycle bin. The objects will be automatically deleted after the retention period expires.

HUAWEI

Syntax Example

```
DROP TABLE t1 PURGE;

PURGE TABLE t1;
PURGE TABLE "BIN$04LhcpndanfgMAAAAAANPw==$0";
PURGE INDEX i1;
PURGE INDEX "BIN$04LhcpndanfgMAAAAAANPw==$0";
PURGE RECYCLEBIN;

TIMECAPSULE TABLE t1 TO BEFORE DROP;
TIMECAPSULE TABLE t1 TO BEFORE DROP RENAME TO new_t1;
TIMECAPSULE TABLE "BIN$04LhcpndanfgMAAAAAANPw==$0" TO BEFORE DROP;
TIMECAPSULE TABLE "BIN$04LhcpndanfgMAAAAAANPw==$0" TO BEFORE DROP RENAME TO new_t1;
```


HUAWEI

Contents

1. Routinely Checking Databases
2. Managing Backup and Restoration
3. **Introduction to HA and DR**
4. Checking Database Performance
5. Checking and Clearing Logs
6. Checking the Number of Application Connections
7. Maintaining Tables and Rebuilding Indexes
8. Exporting and Viewing WDRs

HUAWEI

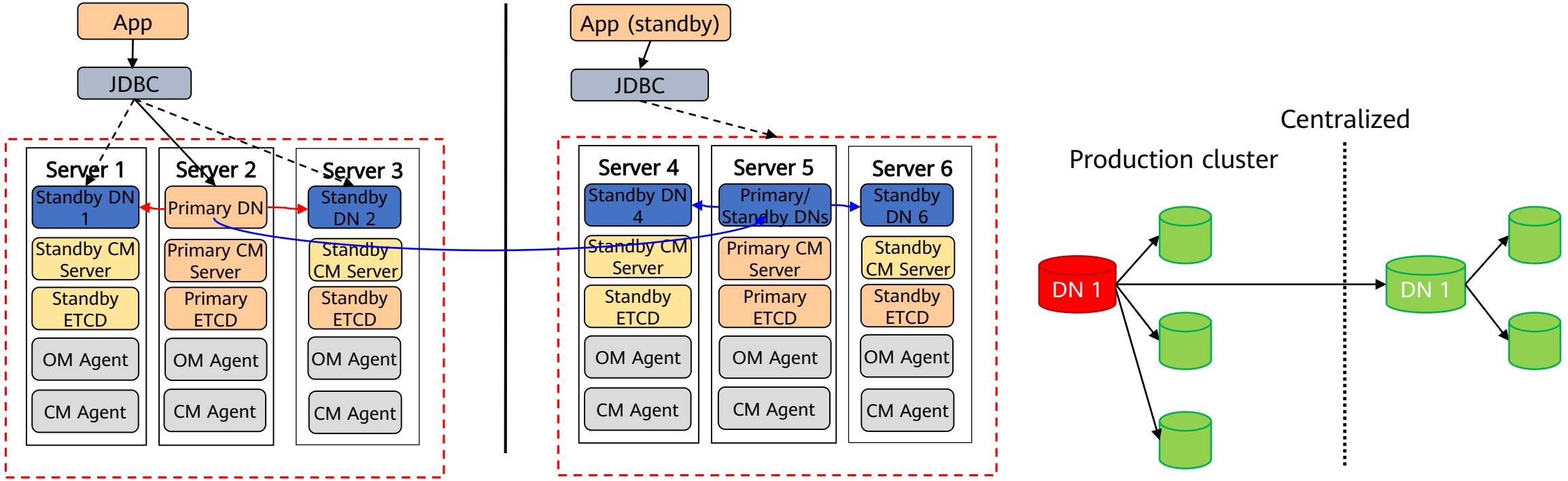
Introduction to HA and DR (1)

- HA aims to reduce the downtime (planned downtime and unplanned downtime) considering the following aspects:
  - Data availability: Ensure that data can be accessed to prevent service interruption.
  - Data protection: Prevent data loss that affects service feasibility.
  - Performance: Provide sufficient response time for efficient service operations.
  - Cost: Reduce deployment, management, and support costs to save enterprise resources.
  - Risk: Consistently achieve the required service level over a long period of time.
- The main HA methods are as follows:
  - Redundancy deployment, and backup and restoration
  - Log synchronization and switchover
  - Data reconstruction and cluster management
  - DR is an intra-AZ and cross-city cross-region HA solution. It ensures service continuity when manual operations cause a disaster in the data center, leading to faults in the production cluster.

Introduction to HA and DR (2)

- The remote DR solution based on streaming replication supports the two-city three-DC cross-region DR feature. Two-city three-DC indicates that the three DCs (production center, intra-city DR center, and remote DR center) are deployed in two cities.
- A remote DR DC is deployed in a different city and is used to back up data of the two DCs in the same city. When faults occur in the two DCs, the remote DR DC can recover services from backup data.
- The primary cluster is a complete cluster, so does the standby cluster. The latter also has the cluster management component in addition to the backup of DNs. After the standby cluster is promoted to primary, it is a complete and independent cluster.
- The standby database instance for DR can be promoted to primary. Certain data loss is allowed (RPO ≤ 10s). The standby database instance for DR is in the **normal** state. The RTO is less than or equal to 10 minutes.
- DR switchover: The primary and standby database instances are switched over as planned, and no data is lost (RPO = 0).

Streaming-based Remote DR Solution - Networking



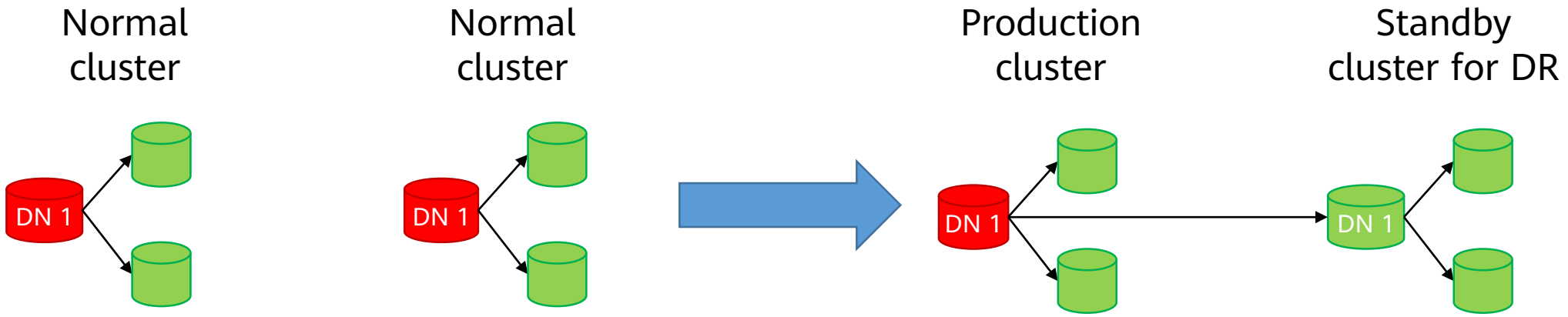
- The standby cluster for DR has the same cluster components as the primary cluster.
- The number of copies in the primary cluster can be different from that in the standby cluster for DR. The latter has at least two copies.

Processes and Functions in the DR Solution (1)

- Establishing a DR relationship: Two normal clusters become the primary cluster and standby cluster in a DR relationship.
- Promoting the standby cluster to primary (failover): No matter whether the primary cluster is abnormal, the standby cluster for DR can be promoted to primary to provide services and exit the DR state.
- Removing the DR relationship of the primary cluster: After the standby cluster becomes the primary cluster, the primary cluster deletes the DR information and exits the DR state.
- Cluster switchover: A switchover is performed when both the primary and standby clusters are normal. In this case, the primary cluster is demoted to standby, and the standby cluster is promoted to primary.
- DR status query: Monitor the DR status and report the cluster DR status, progress of DR relationship establishment, failover, and switchover, and the RTO and RPO in the cluster.

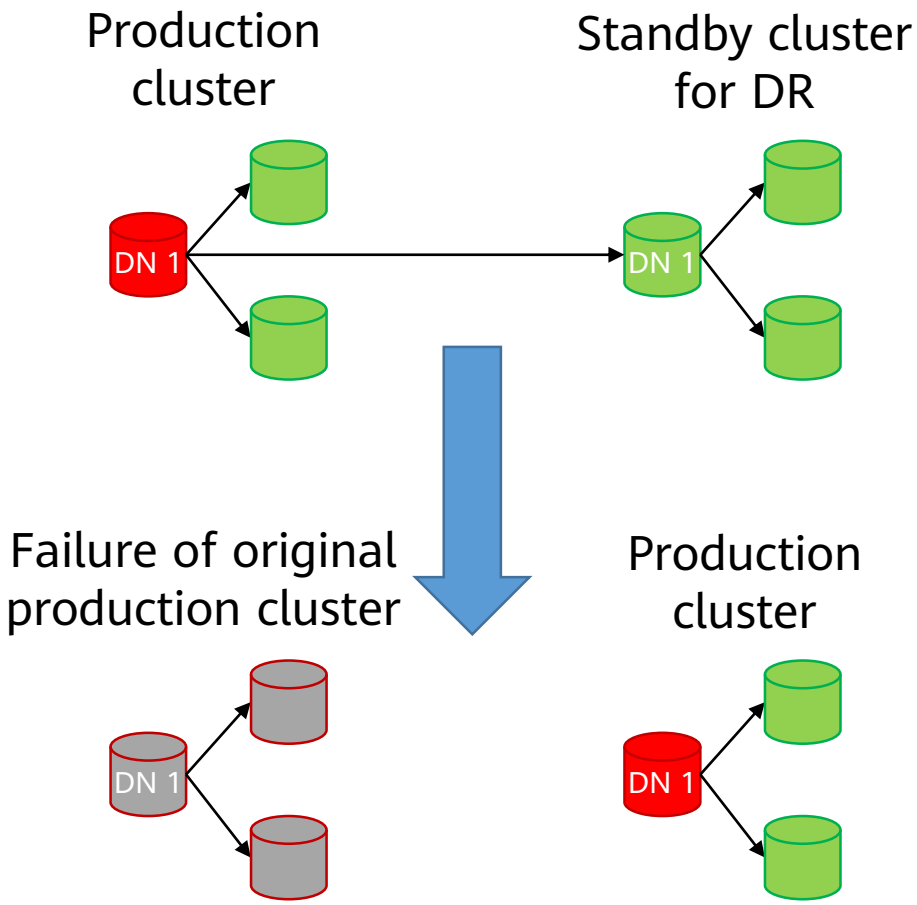
HUAWEI

Processes and Functions in the DR Solution (2)

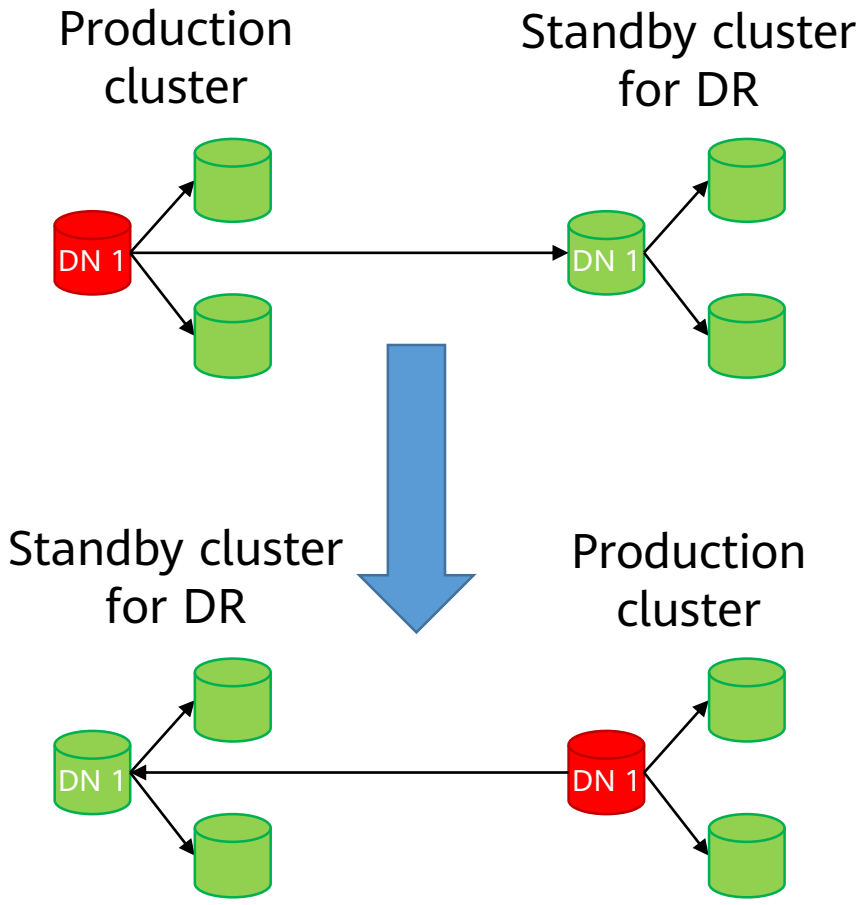


- Establishing a DR relationship is a process of establishing a DR relationship between two independent clusters.
- The number of copies in the primary cluster can be different from that in the standby cluster.
- The standby cluster consists of the first standby DN and cascaded standby DN. Only the first standby DN copies all data from the primary DN in the primary cluster and establishes a remote streaming replication relationship.
- The cascaded standby node in the standby cluster for DR copies data from the first standby node and establishes a streaming replication relationship with the first standby node.

DR Failover and Switchover



DR failover RPO ≤ 10s; RTO ≤ 10 min



DR switchover

RPO = 0s

HUAWEI

Processes and Functions in the DR Solution (1)

- Flow control: You can set the target value (of GUC parameters) to control the log generation speed of the primary cluster to ensure the RPO and RTO of remote DR.
  - In streaming DR mode, you can set **hadr_recovery_time_target** to enable the standby database instance to write and replay logs.
  - **hadr_recovery_point_target** specifies the RPO time allowed for the standby database instance to flush logs to disks in streaming DR mode.
- Compression: The compression (GUC parameter) can be enabled for log transmission between the primary and standby clusters to save bandwidth.
  - You can set **enable_wal_shipping_compression** to enable cross-database instance log compression in streaming DR mode.

HUAWEI

Processes and Functions in the DR Solution (2)

Status	Definition
normal	The cluster does not participate in streaming DR.
full_backup	Full data replication in the primary cluster is in progress.
archive	Streaming log replication in the primary cluster is in progress.
backup_fail	Full data replication in the primary cluster fails.
archive_fail	Streaming log replication in the primary cluster fails.
switchover	Planned switchover is in progress.
restore	Full data restoration in the standby cluster for DR is in progress.
restore_fail	Full data restoration in the standby cluster for DR fails.
recovery	Streaming log replication in the standby cluster for DR is in progress.
recovery_fail	Streaming log replication in the standby cluster for DR fails.
promote	The standby cluster for DR is being promoted to primary.
promote_fail	The standby cluster for DR fails to be promoted to primary.

HUAWEI

Demonstrating the Management Tool gs_sdr

- Setup:
  - Primary: `gs_sdr -t start -m primary -X /sshmode/ins/3node_install.xml -U disaster_user -W "huawei@123"`
  - Standby for DR: `gs_sdr -t start -m disaster_standby -X /sshmode/ins/3node_install.xml -U disaster_user -W "huawei@123"`
- Switchover:
  - Standby-to-primary: `gs_sdr -t switchover -m primary`
  - Primary-to-standby: `gs_sdr -t switchover -m disaster_standby`
- DR failover:

```
gs_sdr -t failover
```

- DR query:

```
gs_sdr -t query
```

- DR relationship removal:

```
gs_sdr -t stop -X /sshmode/ins/3node_install.xml
```


HUAWEI

Contents

1. Routinely Checking Databases
2. Managing Backup and Restoration
3. Introduction to HA and DR
4. **Checking Database Performance**
5. Checking and Clearing Logs
6. Checking the Number of Application Connections
7. Maintaining Tables and Rebuilding Indexes
8. Exporting and Viewing WDRs

HUAWEI

Check Scheme

- Use the gs_checkperf tool provided by openGauss to check hardware performance.
- Prerequisites
  - The openGauss database is running properly.
  - Services are running properly on the database.
- Procedure
  - Log in as the OS user **omm** to the primary node of the database.
  - Run the **gs_checkperf** command to check the openGauss database performance.

HUAWEI

Example - Execution Result

```
[omm@Run861 ~]$ gs_checkperf -i pmk -U omm
Cluster statistics information:
    Host CPU busy time ratio                 :    8.08       %
    MPPDB CPU time % in busy time            :    2.22       %
    Shared Buffer Hit ratio                  :    99.99      %
    In-memory sort ratio                     :    0
    Physical Reads                           :    12677
    Physical Writes                          :    1529476
    DB size                                  :    2262       MB
    Total Physical writes                    :    1529476
    Active SQL count                         :    1
    Session count                            :    5
```


HUAWEI

Abnormal Results and Solutions

Abnormal Status	Solution
High CPU usage of hosts	1. Add high-performance CPUs, or replace current CPUs with them.
	2. Run the **top** command to check which system processes cause high CPU usage, and run the **kill** command to stop unused processes.
High CPU usage of the openGauss database	1. Add high-performance CPUs, or replace current CPUs with them.
	2. Run the **top** command to check which database processes cause high CPU usage, and run the **kill** command to stop unused processes.
	3. Use the gs_expand tool to add new hosts to lower the CPU usage.
Low hit ratio of the shared memory	1. Expand the memory.
	2. Run the following command to check the OS configuration file **/etc/sysctl.conf** and increase the value of **kernel.shmmax**.
	vim /etc/sysctl.conf
Low in-memory sort ratio	Expand the memory.
High I/O and disk usage	1. Replace existing disks with high-performance ones.
	2. Adjust the data layout to evenly distribute I/O requests to all the physical disks.
	3. Run **VACUUM FULL** for the entire database.
	4. Clean up the disk. Run **VACUUM FULL** for the entire database or run **VACUUM** or **VACUUM FULL** for a single table.
	5. Reduce the number of concurrent connections.
Transaction statistics	Query the pg_stat_activity system catalog to break unnecessary connections. (Log in to the database and run **openGauss=# \d+ pg_stat_activity;**.)

HUAWEI

Contents

1. Routinely Checking Databases
2. Managing Backup and Restoration
3. Introduction to HA and DR
4. Checking Database Performance
5. **Checking and Deleting Logs**
6. Checking the Number of Application Connections
7. Maintaining Tables and Rebuilding Indexes
8. Exporting and Viewing WDRs

HUAWEI

Checking OS Logs

- You are advised to monthly check OS logs to detect and prevent potential OS problems.
- Run the following command to check OS log files:

```
vim /var/log/messages
```

- Pay attention to words like "kernel", "error", and "fatal" in the logs generated within the last month and handle the problems based on the alarm information.
- You are advised to check OS logs and database run logs monthly for monitoring system status and troubleshooting.
- You are also advised to monthly clean run logs, because they occupy much disk space if the database has been running for a long time.

HUAWEI

Checking the openGauss Run Logs

- A database can still run when some operation errors occur. However, data may be inconsistent before and after the error occurrences. Therefore, you are advised to monthly check the openGauss run logs to detect potential problems in time.
- Prerequisites
  - The host used for collecting logs is running properly, and the network connection is normal. Database installation users trust each other.
  - An OS tool (for example, gstack) that the log collection tool depends has been installed. If it is not installed, an error message is displayed, and this collection item is skipped.

HUAWEI

Demonstrating Log Collection

- Log in as the OS user **omm** to the primary node of the database.
- Run the following command to collect database logs:

```
gs_collector --begin-time="20230816 11:01" --end-time="20230816 11:30"
```

  - In the command, **20230816 11:01** indicates the start time of the log and **20230816 11:30** indicates the end time of the log.

- As suggested in the result, access the related log collection directory, decompress collected database logs, and check these logs.
  - Here, the **/opt/gaussdb/tmp/gaussdba_mppdb/collector_20230816_103145.tar.gz** log path is used as an example.

```
tar -xvzf /opt/gaussdb/tmp/gaussdba_mppdb/collector_20230816_103145.tar.gz
cd /opt/gaussdb/tmp/gaussdba_mppdb/collector_20230816_103145
```


HUAWEI

Deleting Run Logs

- A large number of run logs will be generated during database running and occupy huge disk space. You are advised to delete expired run logs and retain logs generated within one month.
- Procedure
  - Log in as the OS user **omm** to the primary node of the database.
  - Clean logs.
    - Back up logs generated over one month ago to other disks.
    - Access the directory where logs are stored.
  - Access the corresponding sub-directory and run the following command to delete logs generated one month ago.

```
cd $GAUSSLOG
```

  - The naming convention of the log file is **postgresql-***year-month-day_HHMMSS*.

```
rm log_name
```


HUAWEI

Contents

1. Routinely Checking Databases
2. Managing Backup and Restoration
3. Introduction to HA and DR
4. Checking Database Performance
5. Checking and Clearing Logs
6. **Checking the Number of Application Connections**
7. Maintaining Tables and Rebuilding Indexes
8. Exporting and Viewing WDRs

HUAWEI

Checking the Number of Application Connections

- If the number of connections between applications and the database exceeds the maximum value, new connections cannot be established. You are advised to daily check the number of connections, release idle connections in a timely manner, or increase the allowed maximum number of connections.
- Procedure
  - Log in as the OS user **omm** to the primary node of the database.
  - Connect to the database.

```
gsql -d postgres -p 8000
```

  - Run the following SQL statement to check the number of connections:

```
SELECT count(*) FROM (SELECT pg_stat_get_backend_idset() AS backendid) AS s;
```

  - View the allowed maximum number of connections.

```
SHOW max_connections;
```


HUAWEI

Contents

1. Routinely Checking Databases
2. Managing Backup and Restoration
3. Introduction to HA and DR
4. Checking Database Performance
5. Checking and Clearing Logs
6. Checking the Number of Application Connections
7. **Maintaining Tables and Rebuilding Indexes**
8. Exporting and Viewing WDRs

HUAWEI

Routinely Maintaining Tables

- To ensure proper database running, after insert and delete operations, you need to routinely run **VACUUM FULL** and **ANALYZE** as appropriate for customer scenarios and update statistics to obtain better performance.
- You need to routinely run **VACUUM**, **VACUUM FULL**, and **ANALYZE** to maintain tables. The reasons are as follows:
- VACUUM FULL can be used to recycle disk space occupied by updated or deleted data and combine small-sized data files.
- VACUUM can be used to maintain a visualized mapping for each table to track pages that contain arrays visible to other active transactions. A common index scan uses the mapping to obtain the corresponding arrays and check whether the arrays are visible to the current transaction. If the arrays cannot be obtained, capture a batch of arrays to check the visibility. Therefore, updating the visualized mapping of a table can accelerate unique index scans.
- Running **VACUUM** can avoid data loss caused by duplicate transaction IDs when the number of executed transactions exceeds the database threshold.
- ANALYZE can be used to collect statistics on tables in databases. The statistics are stored in the system catalog PG_STATISTIC. Then, the query optimizer uses the statistics to work out the most efficient execution plan.

HUAWEI

Routinely Maintaining Tables - Example

- You can run the **ANALYZE** *table name*; statement to update statistics.
- During the execution of **VACUUM** *table name*;, the SELECT, INSERT, UPDATE, and DELETE statements can be used, excluding ALTER TABLE.
- During the execution of **VACUUM FULL** *table name*;, you need to add an access exclusive lock to the table and stop all other database operations.
- Maintenance suggestions:
  - Routinely run **VACUUM FULL** for large tables. If the database performance deteriorates, run **VACUUM FULL** for the entire database. If the database performance is stable, you are advised to run **VACUUM FULL** monthly.
  - Routinely run **VACUUM FULL** on system catalogs, especially PG_ATTRIBUTE.
  - Enable automatic vacuum threads (**AUTOVACUUM**) in the system. The processes automatically run the **VACUUM** and **ANALYZE** statements to recycle the record space marked as the deleted state and update statistics in the table.

HUAWEI

Routinely Rebuilding Indexes

- When data deletion is repeatedly performed in the database, index keys will be deleted from the index pages, resulting in index bloat. Rebuilding indexes routinely improves query efficiency.
- The database supports B-tree indexes. Routinely rebuilding indexes improves query efficiency.
  - If a large amount of data is deleted, index keys on the index pages will be deleted. As a result, the number of index pages reduces and index bloat occurs. Rebuilding an index helps recycle wasted space.
  - In a newly created index, pages with adjacent logical structures tend to have adjacent physical structures. Therefore, a new index supports faster access than an index that has been updated for multiple times.

HUAWEI

Main Methods of Routine Index Rebuilding

- Run the **DROP INDEX** statement to delete the index, and then run the **CREATE INDEX** statement to create an index.
  - When you delete an index, a temporary exclusive lock is added in the parent table to block related read/write operations. When you create an index, the write operation is locked but the read operation is not. The data is read and scanned by order.
- Run **REINDEX** to rebuild an index.
  - When you run the **REINDEX TABLE** statement to rebuild an index, an exclusive lock is added to block related read/write operations.
  - When you run the **REINDEX INTERNAL TABLE** statement to rebuild an index for a **desc** table (such as column-store **cudesc** table), an exclusive lock is added to block related read/write operations on the table.

HUAWEI

Contents

1. Routinely Checking Databases
2. Managing Backup and Restoration
3. Introduction to HA and DR
4. Checking Database Performance
5. Checking and Clearing Logs
6. Checking the Number of Application Connections
7. Maintaining Tables and Rebuilding Indexes
8. **Exporting and Viewing WDRs**

HUAWEI

Generating a WDR (1)

- To generate snapshot data, set **enable_wdr_snapshot** to **on** and **wdr_snapshot_interval** to a certain value.
- Accessing WDR snapshot data requires the sysadmin or monadmin permission. Therefore, to generate a WDR, you must use the **root** account or other accounts with the aforementioned permissions.
  - Run the following command to create the report file:

```
touch  /home/omm/wdrTestNode.html
```

  - Connect to the default **postgres** database.

```
gsql -d postgres -p [port number] -r
```

  - Select two different snapshots in the **snapshot.snapshot** table. If no service restart occurs between the two snapshots, use the two snapshots to generate a report.

```
openGauss=#  select * from snapshot.snapshot order by start_ts desc limit 10;
```


HUAWEI

Generating a WDR (2)

- Run the following command to generate a WDR in HTML format on the local PC:
  - Run the following command to set the report format: **\a** indicates that table row and column symbols are not displayed. **\t** indicates that column names are not displayed. **\o** specifies an output file.

```
openGauss=# \a \t \o /home/omm/wdrTestNode.html
```

  - Run the following command to generate a WDR in HTML format.

```
openGauss=# select generate_wdr_report(begin_snap_id Oid, end_snap_id Oid, int report_type, int
report_scope, int node_name );
```

  - Example 1: Generate a cluster-level report.

```
openGauss=# select generate_wdr_report(1, 2, 'all', 'cluster',null);
```


HUAWEI

Generating a WDR (3)

- Example 2: Generate a report for a node.

```
openGauss=# select generate_wdr_report(1, 2, 'all', 'node', pgxc_node_str()::cstring);
```

- Run the following command to set the report format: **\a** indicates that table row and column symbols are not displayed. **\t** indicates that column names are not displayed. **\o** specifies an output file.

```
openGauss=# \a \t \o /home/omm/wdrTestNode.html
```

- View the WDR in **/home/omm/** as required.

HUAWEI

Meanings of WDR Metrics

Item	Description
Database Stat (cluster-level)	Database performance statistics: transactions, read and write operations, row activities, write conflicts, and deadlocks.
Load Profile (cluster-level)	Cluster performance statistics: CPU time, DB time, logical or physical read operation, I/O performance, login and logout, load strength, and load performance.
Instance Efficiency Percentages (cluster- or node-level)	Cache hit ratio in the cluster or node.
I/O Profile (cluster- or node-level)	I/O usage in the cluster or node.
Top 10 Events by Total Wait Time (node-level)	Events that consume the most time.
Wait Classes by Total Wait Time (node-level)	Category of the wait time that is most time-consuming.
Host CPU (node-level)	CPU usage of the host.
Memory Statistics (node-level)	Kernel memory usage distribution.
Time Model (node-level)	Time distribution information about statements at the node level.
Wait Events (node-level)	Statistics on wait events at the node level.
Cache I/O Stats (cluster- or node-level)	I/O statistics of user tables and indexes.
Utility Status (node-level)	Status information about the replication slot and background checkpoint.
Object Stats (cluster- or node-level)	Performance statistics in the index and table dimensions.
Configuration Settings (node-level)	Hosts configuration file
SQL Statistics (cluster- or node-level)	SQL statement performance statistics: end-to-end time, row activities, cache hit, CPU consumption, and time consumption.
SQL Detail (cluster- or node-level)	Details of the SQL statement text.

HUAWEI

Quiz

1. (Multi-choice) gs_dump is a tool used by openGauss to export database information. Which of the following statements about gs_dump are correct?
   A. gs_dump is executed by the database administrator.
   B. When gs_dump is used to export data, other users can still access the openGauss database.
   C. gs_dump can export complete and consistent data.
   D. gs_dump supports the export of text files that are compatible with the V1 database.
2. (True or False) If a large amount of data is deleted from a table, the index key on the index page will be deleted. As a result, the number of index pages increases and index bloat occurs.
   A. True
   B. False

HUAWEI

Summary

- After learning this chapter, you are proficient in using tools to perform routine check and health check on databases or OSs, and master concepts and tools related to backup, restoration, and DR. To be more specific, you know how to check database performance, clear logs (including OS logs and database run logs), check the number of application connections, maintain tables, rebuild indexes, and export and view WDRs.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/en/
- openGauss Help (5.0.0)
  - https://docs-opengauss.osinfra.cn/en/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

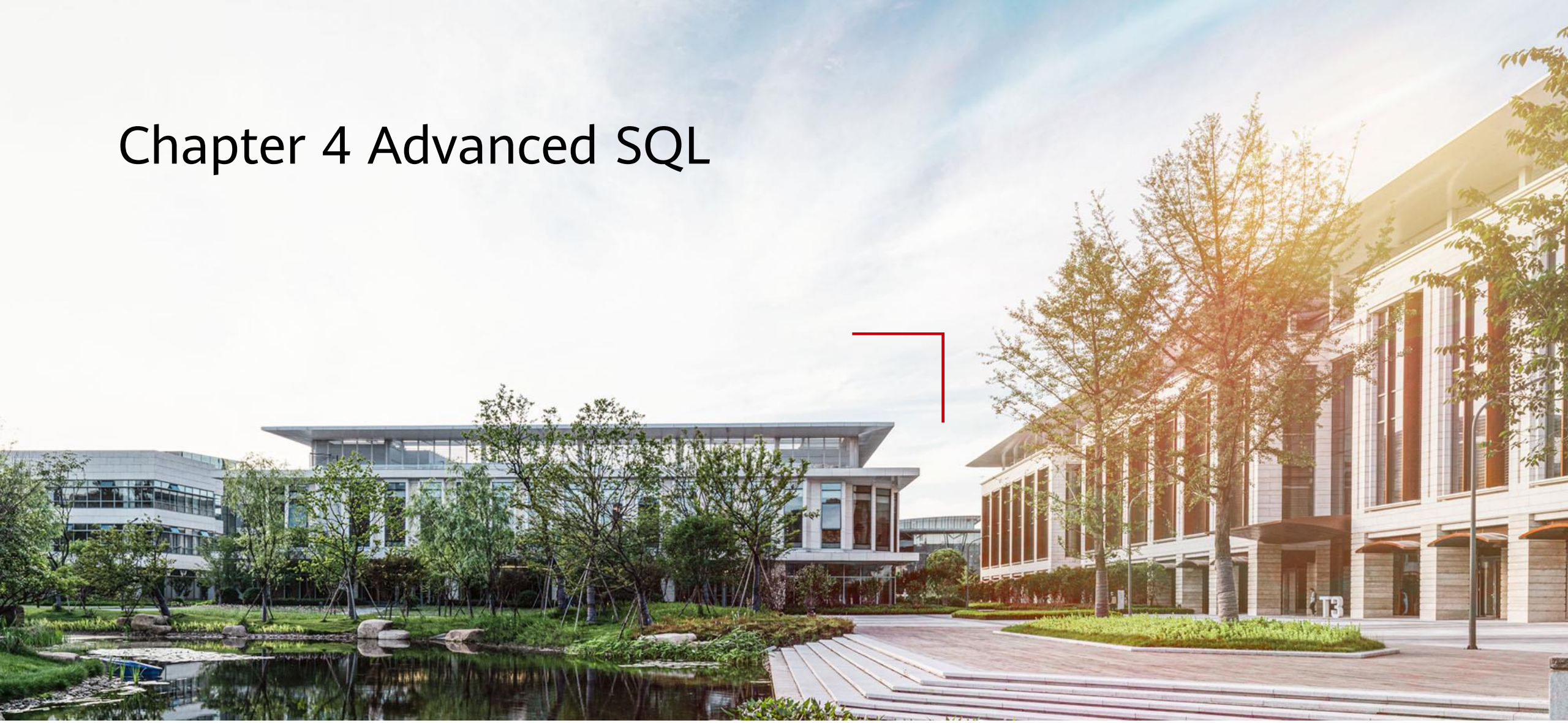
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 4 Advanced SQL



HUAWEI

Foreword

- This chapter describes the principles and usage of advanced SQL syntax such as constraints, subqueries, JOIN, and VACUUM, the concept of stored procedure, various types of statements, cursors, and anonymous blocks, the concept of trigger and its types, and development based on application programming interfaces (APIs), including JDBC-based development, ODBC-based development, and Psycopg-based development.

HUAWEI

Objectives

- Upon completion of this course, you will be able to:
  - Create and use stored procedures and triggers;
  - Use multi-table JOIN to design efficient SQL statements;
  - Create and use NOT NULL, UNIQUE, PRIMARY KEY, and FOREIGN KEY constraints;
  - Distinguish the functions of the TRUNCATE, DROP, and DELETE commands;
  - Distinguish between VACUUM and VACUUM FULL;
  - Use different types of APIs for development.

HUAWEI

Contents

1. **Advanced SQL Syntax**
   - Constraints and Table Joins
   - TRUNCATE TABLE
   - Subquery
   - WHERE Clause
   - ORDER BY
   - System Functions
   - VACUUM

HUAWEI

Constraints (1)

- A constraint clause specifies the conditions that new rows or rows to be updated must satisfy for an INSERT or UPDATE operation to succeed. If there is any data behavior that violates the constraint, the behavior is terminated by the constraint.
- Constraints can be specified when a table is created (by using the CREATE TABLE statement) or after a table is created (by using the ALTER TABLE statement).
- Constraints can be column-level or table-level. Column-level constraints apply only to columns, and table-level constraints apply to the entire table.

HUAWEI

Constraints (2)

- Common constraints of openGauss are as follows:
  - NOT NULL: specifies that a column cannot store NULL values.
  - UNIQUE: ensures that the value of a column is unique.
  - PRIMARY KEY: the combination of NOT NULL and UNIQUE, ensures that a column (or the combination of two or more columns) has a unique identifier to help quickly locate a specific record in a table.
  - FOREIGN KEY: ensures the referential integrity for data in one table to match values in another table.
  - CHECK: ensures that values in a column meet specified conditions.

HUAWEI

NOT NULL Constraint

- If no constraint is specified during table creation, the default value is **NULL**, indicating that **NULL** values can be inserted into columns. If you do not want a column to be set to **NULL**, you need to define the NOT NULL constraint on the column to specify that NULL values are not allowed in the column. When you insert data, if **NULL** is to be inserted to the column, an error is reported and the data fails to be inserted.
- NULL does not mean that there is no data. It specifies unknown data.
  - Create a table **staff** containing five columns, where the **NAME** and **ID** columns cannot be set to **NULL**.

```
openGauss=# CREATE TABLE staff(
 ID          INT          NOT NULL,
NAME        char(8)      NOT NULL,
AGE         INT ,
ADDRESS   CHAR(50), SALARY REAL);
```

  - Insert data into the **staff** table. When a **NULL** value is inserted into the **ID** column, the database returns an error.

```
openGauss=# INSERT INTO staff VALUES (1,'lily',28);
INSERT 0 1
openGauss=# INSERT INTO staff (NAME,AGE) VALUES ('JUCE',28);
ERROR: null value in column "id" violates not-null constraint DETAIL: Failing row contains (null, JUCE , 28, null, null).
```


HUAWEI

UNIQUE Constraint

- The UNIQUE constraint specifies that a group of one or more columns of a table can contain only unique values.
- For the UNIQUE constraint, **NULL** is not considered equal.
  - Create a table **staff1** containing five columns, where **AGE** is set to **UNIQUE**. In this case, you cannot add two records with the same age.

```
openGauss=# CREATE TABLE staff1(
 ID          INT          NOT NULL,
 NAME        char(8)      NOT NULL,
AGE          INT          NOT NULL              UNIQUE ,
ADDRESS      CHAR(50),
SALARY       REAL
);
```

  - Insert data into the **staff1** table. When two identical data records are inserted into the **AGE** column, the database returns an error.

```
openGauss=# INSERT INTO staff1 VALUES (1,'lily',28);
INSERT 0 1
openGauss=# INSERT INTO staff1 VALUES (2, 'JUCE',28);
ERROR: duplicate key value violates unique constraint "staff1_age_key" DETAIL: Key (age)=(28) already exists.
```


HUAWEI

PRIMARY KEY Constraint

- PRIMARY KEY is the unique identifier of each record in a data table. It specifies that a column or multiple columns in a table can contain only unique and non-NULL values.
- PRIMARY KEY is the combination of NOT NULL and UNIQUE. Only one primary key can be specified for a table.
  - For example, create a table **staff2**, where the **ID** column is the primary key.

```
openGauss=# CREATE TABLE staff2(
 ID          INT          PRIMARY KEY ,
 NAME        TEXT         NOT NULL,
AGE          INT          NOT NULL,
ADDRESS   CHAR(50),
SALARY       REAL
 );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index"staff2_pkey" for table "staff2"

CREATE TABLE
```

FOREIGN KEY Constraint

- The FOREIGN KEY constraint specifies that the value of a column (or a group of columns) must match the value in a row of another table. Generally, the foreign key in one table points to the unique key in another table. That is, the referential integrity between two related tables is maintained.
  - Create a table **staff3** containing five columns.

```
openGauss=# CREATE TABLE staff3(
   ID            INT     PRIMARY KEY  NOT NULL,
   NAME            TEXT    NOT NULL,
   AGE           INT     NOT NULL,
   ADDRESS         CHAR(50),
   SALARY        REAL
);
```

  - Create a table **DEPARTMENT** containing three columns, where the **EMP_ID** column specifies the foreign key and it maps to the **ID** column of the **staff3** table.

```
openGauss=# CREATE TABLE DEPARTMENT(
   ID INT PRIMARY KEY      NOT NULL,
   DEPT           CHAR(50) NOT NULL,
   EMP_ID         INT      references staff3(ID)
);
```

CHECK Constraint

- The CHECK constraint specifies an expression producing a Boolean result where the INSERT or UPDATE operation on new rows or rows to be updated can succeed only when the expression result is **TRUE** or **UNKNOWN**; otherwise, an error is thrown and the database is not altered.
  - Create a table **staff4** and add a CHECK constraint to the **SALARY** column to ensure that the inserted value is greater than **0**.

```
openGauss=# CREATE TABLE staff4(
ID          INT          PRIMARY KEY          NOT NULL,
NAME        TEXT         NOT NULL,
AGE         INT          NOT NULL,
ADDRESS     CHAR(50),                                  TRUNCATE
SALARY      REAL         CHECK(SALARY > 0)
);
```

  - Insert data into the **staff4** table. When the value to be inserted to the **SALARY** column is not greater than **0**, the database reports an error.

```
openGauss=# INSERT INTO staff4(ID,NAME,AGE,SALARY) VALUES (2, 'JUCE',16,0);
ERROR: new row for relation "staff4" violates check constraint "staff4_salary_check"
DETAIL: N/A
```

Table Join

- A JOIN clause combine rows in two or more tables based on the common columns in these tables.
- In openGauss, there are five JOIN types:
  - CROSS JOIN
  - INNER JOIN
  - LEFT OUTER JOIN
  - RIGHT OUTER JOIN
  - FULL OUTER JOIN

HUAWEI

CROSS JOIN

- When a JOIN query does not specify any JOIN conditions, it is called a CROSS JOIN, also known as Cartesian product.
- Example: CROSS JOIN on **table1** and **table2**

```
openGauss=# SELECT * FROM table1 CROSS JOIN table2;
```

table1

name	city
Joes	BJ
Lily	BJ
James	SH

table2

id	Name	dept
1	Tommy	IT
2	Lily	IT
3	Li	sales



name	city	id	name	dept
Joes	BJ	1	Tommy	IT
Joes	BJ	2	Lily	IT
Joes	BJ	3	Li	sales
Lily	BJ	1	Tommy	IT
Lily	BJ	2	Lily	IT
Lily	BJ	3	Li	sales
James	SH	1	Tommy	IT
James	SH	2	Lily	IT
James	SH	3	Li	sales

HUAWEI

INNER JOIN

- When INNER JOIN is used, if the related columns in two tables meet the JOIN conditions, data is extracted from the two tables and combined to create a result table. That is, in an INNER JOIN query, only the tuples that meet the conditions can be displayed in the result table.
- Example: Query the records with the same value of **name** in **table1** and **table2**.

```
openGauss=# SELECT * FROM table1 a INNER JOIN table2 b ON a.name = b.name;
```

table1

name	city
Joes	BJ
Lily	BJ
James	SH

table2

id	Name	dept
1	Tommy	IT
2	Lily	IT
3	Li	sales



name	city	id	name	dept
Lily	BJ	2	Lily	IT

HUAWEI

LEFT OUTER JOIN

- LEFT OUTER JOIN returns all tuples in the left table and matching tuples in the right table.
- If no matching tuples is found in the right table, the corresponding tuples are NULL.
- Example:

```
openGauss=# SELECT * FROM table1 a LEFT OUTER JOIN table2 b ON a.name = b.name;
```

table1

name	city
Joes	BJ
Lily	BJ
James	SH

table2

id	Name	dept
1	Tommy	IT
2	Lily	IT
3	Li	sales



name	city	id	name	dept
Joes	BJ			
Lily	BJ	2	Lily	IT
James	SH			

HUAWEI

RIGHT OUTER JOIN

- Similar to LEFT OUTER JOIN, RIGHT OUTER JOIN returns all tuples in the right table and matching tuples in the left table. That is, all contents of the right table are returned anyhow.
- Example:

```
openGauss=# SELECT * FROM table1 a RIGHT OUTER JOIN table2 b ON a.name = b.name;
```

table1

name	city
Joes	BJ
Lily	BJ
James	SH

table2

id	Name	dept
1	Tommy	IT
2	Lily	IT
3	Li	sales



name	city	id	name	dept
Lily	BJ	2	Lily	IT
		3	Li	sales
		1	Tommy	IT

HUAWEI

FULL OUTER JOIN

- FULL OUTER JOIN returns all tuples in both the left and right tables. If no matching tuple is found, the corresponding tuple is NULL.
- Example:

```
openGauss=# SELECT * FROM table1 a FULL OUTER JOIN table2 b ON a.name = b.name;
```

table1

name	city
Joes	BJ
Lily	BJ
James	SH

table2

id	Name	dept
1	Tommy	IT
2	Lily	IT
3	Li	sales



name	city	id	name	dept
Joes	BJ			
Lily	BJ	2	Lily	IT
James	SH			
		3	Li	sales
		1	Tommy	IT

HUAWEI

NULL Value

- NULL specifies unknown data. NULL and 0 cannot be compared because they are not equivalent to each other.
- When creating a table, you can specify whether **NULL** can be stored in a column. For details, see the NOT NULL constraint.
- Example: Query all data whose **id** is NULL in the **customer** table.

```
openGauss=# SELECT * FROM customer WHERE  id  IS NULL;
```

customer

customer_sk	id	first_name	amount
3869	hello	Grace	1000
3869		Grace	
6985	maps	Joes	2200
9976	James	3	5000



customer_sk	id	first_name	amount
3869		Grace	

If **id** is NULL, the record is displayed in the query result.

HUAWEI

UNION Clause

- UNION computes the union of rows returned by multiple SELECT statements. Each SELECT statement inside the UNION clause must have the same number of columns, and the values of the columns must be of similar data types. The sequence of columns in each SELECT statement must be the same.

```
SELECT name FROM table1 UNION SELECT name FROM table2;
```

name	City
Joes	BJ
Lily	BJ
James	SH

Id	name	dept
1	Tommy	IT
2	Lily	IT
3	Li	sales



name
Joes
Lily
Li
James
Tommy

UNION: retains only one value if duplicate values exist in the results.

```
SELECT name FROM table1 UNION ALL SELECT name FROM table2;
```

name	City
Joes	BJ
Lily	BJ
James	SH

Id	name	dept
1	Tommy	IT
2	Lily	IT
3	Li	sales



name
Joes
Lily
Li
James
Lily
Tommy

UNION ALL: returns all results, including duplicate values.

HUAWEI

Contents

1. **Advanced SQL Syntax**
   - Constraints and Table Joins
   - TRUNCATE TABLE
   - Subquery
   - WHERE Clause
   - ORDER BY
   - System Functions
   - VACUUM

HUAWEI

Differences Between TRUNCATE, DELETE, and DROP

Difference	DROP	DELETE	TRUNCATE
Command	DDL	DML	DDL
Object	Both content and definition are deleted to release space.	Only the content is deleted. The definition is not deleted, and space release is not involved.	Only the content is deleted and space is released. The definition is not deleted.
Condition	WHERE is not supported.	WHERE is supported.	WHERE is not supported.
Speed	Quick	Slow	Quicker

HUAWEI

TRUNCATE Syntax

- Truncate data from a table.

```
TRUNCATE [ TABLE ] [ ONLY ] {table_name [ * ]} [, ... ]
    [ CONTINUE IDENTITY ]
    [ CASCADE | RESTRICT][PURGE]};
```

- Truncate data from a partition.

```
ALTER TABLE [ IF EXISTS  ] { [ ONLY  ] table_name
                        | table_name *
                        | ONLY ( table_name )  }
TRUNCATE PARTITION { partition_name
                    | FOR (  partition_value  [, ...] )  } [ UPDATE GLOBAL INDEX ];
```

Note: When the PARTITION FOR clause is used, the entire partition where **partition_value** is located is cleared.

HUAWEI

TRUNCATE Example (1)

- Create a table **staff** and initialize data.

```
openGauss=# CREATE TABLE staff(
    ID      INT      PRIMARY KEY NOT NULL,
    NAME    TEXT     NOT NULL,
    AGE     INT      NOT NULL
);
openGauss=# INSERT INTO staff VALUES (2, 'JUCE',16);
```


HUAWEI

TRUNCATE Example (2)

Roll back TRUNCATE in a transaction.

```
openGauss=# BEGIN;
BEGIN
openGauss=# TRUNCATE TABLE
staff;
TRUNCATE TABLE
openGauss=# ROLLBACK;
ROLLBACK
openGauss=# SELECT * FROM
staff;
 id | name | age
----+------+-----
  2 | JUCE |  16
```

Commit TRUNCATE in a transaction.

```
openGauss=# BEGIN;
BEGIN
openGauss=# TRUNCATE TABLE staff;
TRUNCATE TABLE
openGauss=# COMMIT;
COMMIT
openGauss=# SELECT * FROM staff;
 id | name | age
----+------+-----
(0 rows)
```


HUAWEI

Contents

1. **Advanced SQL Syntax**
   - Constraints and Table Joins
   - TRUNCATE TABLE
   - Subquery
   - WHERE Clause
   - ORDER BY
   - System Functions
   - VACUUM

HUAWEI

Subquery (1)

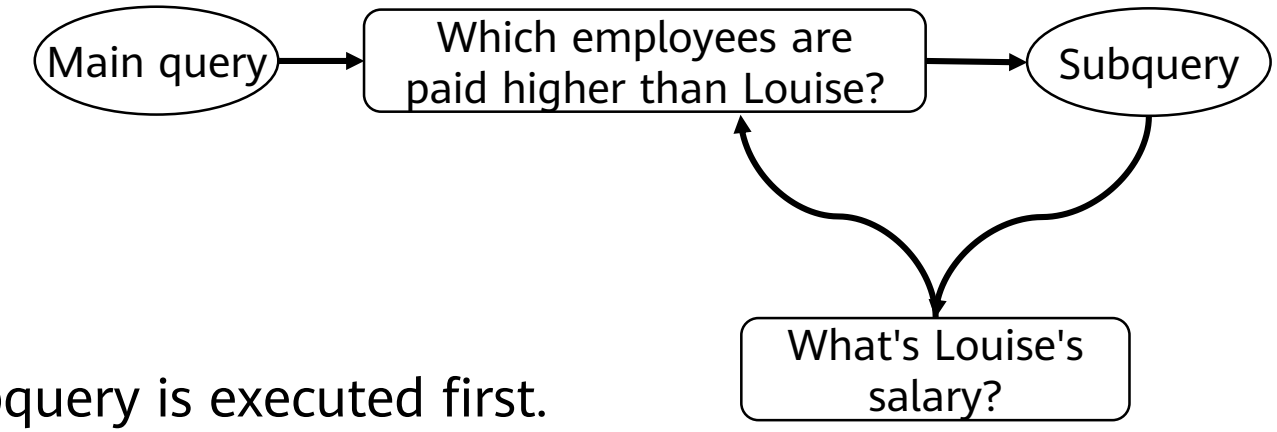
- A subquery, also called an internal query, is a nested query. The subquery embeds a query statement in the WHERE clause of a database query and it is equivalent to a temporary table. The query result of a SELECT statement can be used as the input value of another statement.
- A subquery must comply with the following rules:
  - A subquery must be enclosed in parentheses.
  - A subquery can have only one column in the SELECT clause, unless there are multiple columns in the main query to compare with the columns selected by the subquery.
  - A subquery returns more than one row. It can be used only with multi-value operators, such as IN.
  - The BETWEEN operator cannot be used with a subquery, but it can be used inside the subquery.

HUAWEI

Subquery (2)

- Basic subquery syntax:

```
SELECT    select_list
FROM      table
WHERE     expr operator
(SELECT   select_list FROM table);
```



- Notes:
  - When a query is executed, its subquery is executed first.
  - The main query statement calls the subquery result.

Subquery (3)

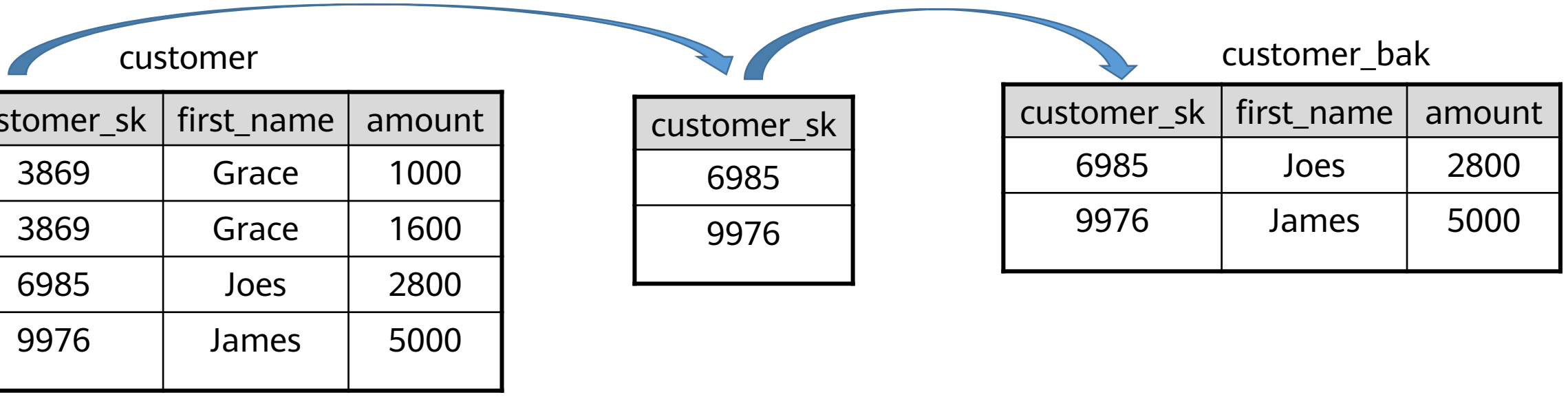
- Usage suggestions:
  - A subquery is usually included in parentheses.
  - A subquery is usually placed on the right of comparison operators.
  - Do not include the ORDER BY clause in a subquery.
  - Use a single-row comparison operator for a single-row subquery.
  - Use a multi-row comparison operator for a multi-row subquery.

HUAWEI

Subquery in the SELECT Statement

- The SELECT statement queries data returned by its subquery.

```
SELECT * FROM customer WHERE customer_sk IN (SELECT  customer_sk FROM customer_bak WHERE amount > 2500) ;
```



Subquery Example

Operator	Description
IN	Equal to a value in the list.
ANY	Compare with any value in the list.
ALL	Compare with all values in the list.

- Query information about employees whose **employee_id** is in the range 11, 101, 192, and 201.

```
test=> select employee_id,first_name,hire_date,salary from employees where employee_id in(11,101,192,201);
 employee_id | first_name |      hire_date      |  salary
-------------+------------+---------------------+----------
         101 | Neena      | 2005-09-21 00:00:00 | 17000.00
         192 | Sarah      | 2004-02-04 00:00:00 |  4000.00
         201 | Michael    | 2004-02-17 00:00:00 | 13000.00
```

- Query information about any employee whose salary is greater than 80 in the department.

```
test=> select employee_id,first_name,hire_date,salary from employees
where salary > any (select salary from employees where  department_id=80);
 employee_id | first_name |      hire_date      |  salary
-------------+------------+---------------------+----------
         100 | Steven     | 2003-06-17 00:00:00 | 24000.00
         101 | Neena      | 2005-09-21 00:00:00 | 17000.00
         102 | Lex        | 2001-01-13 00:00:00 | 17000.00
         103 | Alexander  | 2006-01-03 00:00:00 |  9000.00
         108 | Nancy      | 2002-08-17 00:00:00 | 12008.00
         109 | Daniel     | 2002-08-16 00:00:00 |  9000.00
         110 | John       | 2005-09-28 00:00:00 |  8200.00
```

- Query information about all employees whose salaries are greater than 80 in the department.

```
test=> select employee_id,first_name,hire_date,salary from employees
where salary > all (select salary from employees where  department_id=80);
 employee_id | first_name |      hire_date      |  salary
-------------+------------+---------------------+----------
         100 | Steven     | 2003-06-17 00:00:00 | 24000.00
         101 | Neena      | 2005-09-21 00:00:00 | 17000.00
         102 | Lex        | 2001-01-13 00:00:00 | 17000.00
```


HUAWEI

Subquery in the INSERT Statement

- The INSERT statement inserts the data returned by the subquery into another table.
  - The basic syntax is as follows:

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
SELECT [ *|column1 [, column2 ] ]
FROM table1 [, table2 ]
[ WHERE VALUE OPERATOR ]
```

  - Create a table **customer_bak** with the same structure as table **customer_t1**.

```
openGauss=# CREATE TABLE customer_bak (
customer_sk             integer,     Id              char(5),
 first_name              char(6),    Amount     integer
);
```

  - Insert data in the **customer** table into the **customer_bak** table.

```
openGauss=# INSERT INTO customer_bak SELECT * FROM customer  WHERE  customer_sk IN (SELECT customer_sk FROM
customer) ;
```


HUAWEI

Subquery in the UPDATE Statement (1)

- The UPDATE statement updates multiple columns in a table by using the result returned by the subquery.

```
UPDATE table    SET column_name = new_value
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME
[ WHERE ])
```

  - Change the value of **amount** of all customers whose **c_customer_sk** is set to a value greater than **4000** in the **customer** table to 0.5 times of the original value.

```
UPDATE customer  SET amount = amount * 0.5 WHERE customer_sk IN (SELECT customer_sk FROM
customer_bak  WHERE c_customer_sk > 5000 );
```


HUAWEI

Subquery in the UPDATE Statement (2)

- The UPDATE operation affects two rows. After the update, the data in the **customer** table is as follows:

```
customer_sk      | id                  | first_name     | amount
3869               | hello               | Grace          | 1000
3869               |                     |Grace           |
6985               | maps                | Joes           | 1100   (Because the value of customer_sk is greater than 5000, the
corresponding value of amount has been halved.)
9976               | world               | James          | 2500   (Because the value of customer_sk is greater than 5000, the
corresponding value of amount has been halved.)
```


HUAWEI

Contents

1. **Advanced SQL Syntax**
   - Constraints and Table Joins
   - TRUNCATE TABLE
   - Subquery
   - WHERE Clause
   - ORDER BY
   - System Functions
   - VACUUM

HUAWEI

WHERE Clause

- In a query qualified with a WHERE clause, the WHERE clause forms a row selection expression, which is used to filter the data to be queried and narrow down the SELECT query scope.
  - Basic query syntax of the WHERE clause:

```
SELECT [DISTINCT] {*|column [alias],...}
  FROM    table
  [WHERE  contition(s)];
  -- condition is any expression that returns a result of the Boolean type. Rows that do not satisfy this condition will be eliminated from the returned result.
```

  - Basic usage example:

```
test=> select  employee_id,first_name,last_name,hire_date,salary,manager_id from employees where department_id=100;
 employee_id | first_name  | last_name |      hire_date      |  salary  | manager_id
-------------+-------------+-----------+---------------------+----------+------------
         108 | Nancy       | Greenberg | 2002-08-17 00:00:00 | 12008.00 |        101
         109 | Daniel      | Faviet    | 2002-08-16 00:00:00 |  9000.00 |        108
         110 | John        | Chen      | 2005-09-28 00:00:00 |  8200.00 |        108
         111 | Ismael      | Sciarra   | 2005-09-30 00:00:00 |  7700.00 |        108
         112 | Jose Manuel | Urman     | 2006-03-07 00:00:00 |  7800.00 |        108
         113 | Luis        | Popp      | 2007-12-07 00:00:00 |  6900.00 |        108
(6 rows)
```


HUAWEI

WHERE Clause Example 1 (Comparison Operators)

- Basic numeric comparison statement:

```
test=> select  employee_id,first_name,last_name,hire_date,salary,manager_id from employees where
department_id =100;
test=> select  emptest=> select  employee_id,first_name,last_name,hire_date,salary,manager_id from employees
where department_id <=100;
loyee_id,first_name,last_name,hire_date,salary,manager_id from employees where department_id !=100;
```

- Use BETWEEN AND to restrict the query of values.

```
test=>  select  employee_id,first_name,last_name,hire_date,salary,manager_id from employees where salary between 7500 and 8000;
 employee_id | first_name  | last_name |      hire_date      | salary  | manager_id
-------------+-------------+-----------+---------------------+---------+------------
         111 | Ismael      | Sciarra   | 2005-09-30 00:00:00 | 7700.00 |        108
         112 | Jose Manuel | Urman     | 2006-03-07 00:00:00 | 7800.00 |        108
         120 | Matthew     | Weiss     | 2004-07-18 00:00:00 | 8000.00 |        100
         122 | Payam       | Kaufling  | 2003-05-01 00:00:00 | 7900.00 |        100
         153 | Christopher | Olsen     | 2006-03-30 00:00:00 | 8000.00 |        145
         154 | Nanette     | Cambrault | 2006-12-09 00:00:00 | 7500.00 |        145
         159 | Lindsey     | Smith     | 2005-03-10 00:00:00 | 8000.00 |        146
         160 | Louise      | Doran     | 2005-12-15 00:00:00 | 7500.00 |        146
(8 rows)
```


HUAWEI

WHERE Clause Example 2 (Comparison Operators)

- Use BETWEEN AND to specify a time range for query.

```
test=>  select  employee_id,first_name,last_name,hire_date,salary,manager_id from employees where hire_date  between '2005-09-01'  and '2005-10-01';
 employee_id | first_name | last_name |      hire_date      |  salary  | manager_id
-------------+------------+-----------+---------------------+----------+------------
         101 | Neena      | Kochhar   | 2005-09-21 00:00:00 | 17000.00 |        100
         110 | John       | Chen      | 2005-09-28 00:00:00 |  8200.00 |        108
         111 | Ismael     | Sciarra   | 2005-09-30 00:00:00 |  7700.00 |        108
(3 rows)
```

- IN query:

```
test=> select employee_id,first_name,last_name,hire_date,salary,manager_id,department_id from employees where manager_id in(101,149,201);
 employee_id | first_name | last_name  |      hire_date      |  salary  | manager_id | department_id
-------------+------------+------------+---------------------+----------+------------+---------------
         108 | Nancy      | Greenberg  | 2002-08-17 00:00:00 | 12008.00 |        101 |           100
         174 | Ellen      | Abel       | 2004-05-11 00:00:00 | 11000.00 |        149 |            80
         175 | Alyssa     | Hutton     | 2005-03-19 00:00:00 |  8800.00 |        149 |            80
         176 | Jonathon   | Taylor     | 2006-03-24 00:00:00 |  8600.00 |        149 |            80
         177 | Jack       | Livingston | 2006-04-23 00:00:00 |  8400.00 |        149 |            80
         178 | Kimberely  | Grant      | 2007-05-24 00:00:00 |  7000.00 |        149 |
         179 | Charles    | Johnson    | 2008-01-04 00:00:00 |  6200.00 |        149 |            80
         200 | Jennifer   | Whalen     | 2003-09-17 00:00:00 |  4400.00 |        101 |            10
         202 | Pat        | Fay        | 2005-08-17 00:00:00 |  6000.00 |        201 |            20
         203 | Susan      | Mavris     | 2002-06-07 00:00:00 |  6500.00 |        101 |            40
         204 | Hermann    | Baer       | 2002-06-07 00:00:00 | 10000.00 |        101 |            70
         205 | Shelley    | Higgins    | 2002-06-07 00:00:00 | 12008.00 |        101 |           110
(12 rows)
```


HUAWEI

WHERE Clause Example 3 (Comparison Operators)

- LIKE fuzzy match query: Use LIKE to match a character string value (containing a wildcard) and select a specific row.

```
test=> select employee_id,first_name,last_name,hire_date,salary,manager_id,department_id from employees where first_name like 'S%';
 employee_id | first_name | last_name |      hire_date      |  salary  | manager_id | department_id
-------------+------------+-----------+---------------------+----------+------------+---------------
         100 | Steven     | King      | 2003-06-17 00:00:00 | 24000.00 |            |            90
         116 | Shelli     | Baida     | 2005-12-24 00:00:00 |  2900.00 |        114 |            30
         117 | Sigal      | Tobias    | 2005-07-24 00:00:00 |  2800.00 |        114 |            30
         123 | Shanta     | Vollman   | 2005-10-10 00:00:00 |  6500.00 |        100 |            50
         128 | Steven     | Markle    | 2008-03-08 00:00:00 |  2200.00 |        120 |            50
         138 | Stephen    | Stiles    | 2005-10-26 00:00:00 |  3200.00 |        123 |            50
         161 | Sarath     | Sewall    | 2006-11-03 00:00:00 |  7000.00 |        146 |            80
         166 | Sundar     | Ande      | 2008-03-24 00:00:00 |  6400.00 |        147 |            80
         173 | Sundita    | Kumar     | 2008-04-21 00:00:00 |  6100.00 |        148 |            80
         192 | Sarah      | Bell      | 2004-02-04 00:00:00 |  4000.00 |        123 |            50
         194 | Samuel     | McCain    | 2006-07-01 00:00:00 |  3200.00 |        123 |            50
         203 | Susan      | Mavris    | 2002-06-07 00:00:00 |  6500.00 |        101 |            40
         205 | Shelley    | Higgins   | 2002-06-07 00:00:00 | 12008.00 |        101 |           110
(13 rows)
```

- IS NULL/IS NOT NULL query:

```
test=> select employee_id,first_name,last_name,hire_date,salary,manager_id,department_id from employees where manager_id is null;
 employee_id | first_name | last_name |      hire_date      |  salary  | manager_id | department_id
-------------+------------+-----------+---------------------+----------+------------+---------------
         100 | Steven     | King      | 2003-06-17 00:00:00 | 24000.00 |            |            90
(1 row)
```


HUAWEI

WHERE Clause Example 4 (Logical Operators) (1)

- Common logical operators include AND, OR, and NOT. Their operation result can be TRUE, FALSE, or NULL.
- The priorities of logical operations are as follows: NOT > AND > OR. The operators AND and OR are commutative, that is, you can switch the left and right operand without affecting the result.
  - Use the AND logical operator to query data that meets the following conditions: manager_id = 144 and salary < 2800.

```
test=> select employee_id,first_name,salary,manager_id from employees where manager_id=114 and salary<2800 ;
 employee_id | first_name | salary  | manager_id
-------------+------------+---------+------------
         118 | Guy        | 2600.00 |        114
         119 | Karen      | 2500.00 |        114
```


HUAWEI

WHERE Clause Example 4 (Logical Operators) (2)

- Use the OR logical operator to query data that meets either of the following conditions: manager_id = 144 or salary < 2800.

```
test=> select employee_id,first_name,salary,manager_id from employees where manager_id=114 or  salary<2800 ;
 employee_id | first_name | salary  | manager_id
-------------+------------+---------+------------
         115 | Alexander  | 3100.00 |        114
         116 | Shelli     | 2900.00 |        114
         117 | Sigal      | 2800.00 |        114
         118 | Guy        | 2600.00 |        114
         119 | Karen      | 2500.00 |        114
         126 | Irene      | 2700.00 |        120
         127 | James      | 2400.00 |        120
         128 | Steven     | 2200.00 |        120
```


HUAWEI

WHERE Clause Example 5 (Logical Operators)

- The NOT logical operator is used for logical negation. Commonly, it is used as follows:
  - !=

```
test=> select employee_id,first_name,salary,manager_id from employees where manager_id=120 and  salary != 2800 ;
 employee_id | first_name | salary  | manager_id
-------------+------------+---------+------------
         125 | Julia      | 3200.00 |        120
         126 | Irene      | 2700.00 |        120
         127 | James      | 2400.00 |        120
         128 | Steven     | 2200.00 |        120
```

  - <>

```
test=> select employee_id,first_name,salary,manager_id from employees where manager_id=120 and  salary <> 2800 ;
 employee_id | first_name | salary  | manager_id
-------------+------------+---------+------------
         125 | Julia      | 3200.00 |        120
         126 | Irene      | 2700.00 |        120
         127 | James      | 2400.00 |        120
```

  - not in

```
test=> select employee_id,first_name,salary,manager_id from employees where manager_id not in (120,201) and  salary =  2800 ;
 employee_id | first_name | salary  | manager_id
-------------+------------+---------+------------
         117 | Sigal      | 2800.00 |        114
         130 | Mozhe      | 2800.00 |        121
         195 | Vance      | 2800.00 |        123
```


HUAWEI

Alias (1)

- When querying data in the openGauss database, you can use aliases for table columns or tables to enhance the readability of query results.
- Column alias syntax

```
SELECT { * | [column [ AS ] output_name, ...] }
[ FROM from_item [, ...] ]
[ WHERE condition ];
```

```
test=> select employee_id as "工号",first_name,hire_date as "雇佣时间",salary as "薪资",manager_id,department_id from employees limit 2;
 工号 | first_name |      雇佣时间       |   薪资   | manager_id | department_id
------+------------+---------------------+----------+------------+---------------
  100 | Steven     | 2003-06-17 00:00:00 | 24000.00 |            |            90
  101 | Neena      | 2005-09-21 00:00:00 | 17000.00 |        100 |            90
```


HUAWEI

Alias (2)

- Table alias syntax

```
SELECT column1, column2....
FROM table_name AS output_name
WHERE [condition];
```

```
test=> select employee_id as "工号",first_name,hire_date as "雇佣时间",salary as "薪资",manager_id,department_id from employees as "雇员表" limit 2;
 工号 | first_name |      雇佣时间       |   薪资   | manager_id | department_id
------+------------+---------------------+----------+------------+---------------
  100 | Steven     | 2003-06-17 00:00:00 | 24000.00 |            |            90
  101 | Neena      | 2005-09-21 00:00:00 | 17000.00 |        100 |            90
```


HUAWEI

Contents

1. **Advanced SQL Syntax**
   - Constraints and Table Joins
   - TRUNCATE TABLE
   - Subquery
   - WHERE Clause
   - ORDER BY
   - System Functions
   - VACUUM

HUAWEI

ORDER BY

- The ORDER BY clause follows the SELECT statement. You can use the column name, alias, column position number, or expression to call ORDER BY for result sorting.
- If there is a WHERE clause, the ORDER BY clause follows the WHERE clause, and the ORDER BY clause is the last clause in the SELECT statement.

```
test=> select employee_id,first_name,hire_date,salary,manager_id,department_id from employees where department_id=80 order by salary;
 employee_id | first_name  |      hire_date      | salary  | manager_id | department_id
-------------+-------------+---------------------+---------+------------+---------------
         173 | Sundita     | 2008-04-21 00:00:00 | 6100.00 |        148 |            80
         179 | Charles     | 2008-01-04 00:00:00 | 6200.00 |        149 |            80
         167 | Amit        | 2008-04-21 00:00:00 | 6200.00 |        147 |            80
         166 | Sundar      | 2008-03-24 00:00:00 | 6400.00 |        147 |            80
         165 | David       | 2008-02-23 00:00:00 | 6800.00 |        147 |            80
         155 | Oliver      | 2007-11-23 00:00:00 | 7000.00 |        145 |            80
         161 | Sarath      | 2006-11-03 00:00:00 | 7000.00 |        146 |            80
         164 | Mattea      | 2008-01-24 00:00:00 | 7200.00 |        147 |            80
         172 | Elizabeth   | 2007-03-24 00:00:00 | 7300.00 |        148 |            80
         171 | William     | 2007-02-23 00:00:00 | 7400.00 |        148 |            80
         154 | Nanette     | 2006-12-09 00:00:00 | 7500.00 |        145 |            80
         160 | Louise      | 2005-12-15 00:00:00 | 7500.00 |        146 |            80
         153 | Christopher | 2006-03-30 00:00:00 | 8000.00 |        145 |            80
         159 | Lindsey     | 2005-03-10 00:00:00 | 8000.00 |        146 |            80
         177 | Jack        | 2006-04-23 00:00:00 | 8400.00 |        149 |            80
```


HUAWEI

ORDER BY in Descending Order

- In the SELECT statement, you can use the ORDER BY clause to sort the output rows. **desc** specifies the descending order, and **asc** specifies the ascending order.
  - asc (default)

```
test=> select employee_id,first_name,salary  from employees where department_id=80 order by salary;
 employee_id | first_name  |  salary
-------------+-------------+----------
         173 | Sundita     |  6100.00
         179 | Charles     |  6200.00
         167 | Amit        |  6200.00
         166 | Sundar      |  6400.00
         165 | David       |  6800.00
         155 | Oliver      |  7000.00
         161 | Sarath      |  7000.00
         164 | Mattea      |  7200.00
         172 | Elizabeth   |  7300.00
         171 | William     |  7400.00
```

  - desc

```
test=> select employee_id,first_name,salary  from employees where department_id=80 order by salary desc;
 employee_id | first_name  |  salary
-------------+-------------+----------
         145 | John        | 14000.00
         146 | Karen       | 13500.00
         147 | Alberto     | 12000.00
         168 | Lisa        | 11500.00
         148 | Gerald      | 11000.00
         174 | Ellen       | 11000.00
         162 | Clara       | 10500.00
```


HUAWEI

ORDER BY (Multi-Column Sorting) (1)

- If two columns are equal according to the leftmost expression, they are compared according to the next expression and so on.

```
test=> select employee_id,first_name,manager_id,salary,department_id from employees where department_id=80  order by manager_id, salary;
 employee_id | first_name  | manager_id |  salary  | department_id
-------------+-------------+------------+----------+---------------
         149 | Eleni       |        100 | 10500.00 |            80
         148 | Gerald      |        100 | 11000.00 |            80
         147 | Alberto     |        100 | 12000.00 |            80
         146 | Karen       |        100 | 13500.00 |            80
         145 | John        |        100 | 14000.00 |            80
         155 | Oliver      |        145 |  7000.00 |            80
         154 | Nanette     |        145 |  7500.00 |            80
         153 | Christopher |        145 |  8000.00 |            80
         152 | Peter       |        145 |  9000.00 |            80
         151 | David       |        145 |  9500.00 |            80
         150 | Peter       |        145 | 10000.00 |            80
         161 | Sarath      |        146 |  7000.00 |            80
         160 | Louise      |        146 |  7500.00 |            80
         159 | Lindsey     |        146 |  8000.00 |            80
```

Sort data by the first column **manager_id** after ORDER BY.

Sort data by the second column **salary** after ORDER BY.

HUAWEI

ORDER BY (Multi-Column Sorting) (2)

- In multi-column sorting, you can sort a column in ascending order and other columns in descending order.

```
test=> select employee_id,first_name,manager_id,salary,department_id from employees where department_id=80  order by manager_id , salary desc ;
 employee_id | first_name  | manager_id |  salary  | department_id
-------------+-------------+------------+----------+---------------
         145 | John        |        100 | 14000.00 |            80
         146 | Karen       |        100 | 13500.00 |            80
         147 | Alberto     |        100 | 12000.00 |            80
         148 | Gerald      |        100 | 11000.00 |            80
         149 | Eleni       |        100 | 10500.00 |            80
         150 | Peter       |        145 | 10000.00 |            80
         151 | David       |        145 |  9500.00 |            80
         152 | Peter       |        145 |  9000.00 |            80
         153 | Christopher |        145 |  8000.00 |            80
         154 | Nanette     |        145 |  7500.00 |            80
         155 | Oliver      |        145 |  7000.00 |            80
         156 | Janette     |        146 | 10000.00 |            80
         157 | Patrick     |        146 |  9500.00 |            80
         158 | Allan       |        146 |  9000.00 |            80
         159 | Lindsey     |        146 |  8000.00 |            80
         160 | Louise      |        146 |  7500.00 |            80
         161 | Sarath      |        146 |  7000.00 |            80
         162 | Clara       |        147 | 10500.00 |            80
         163 | Danielle    |        147 |  9500.00 |            80
         164 | Mattea      |        147 |  7200.00 |            80
         165 | David       |        147 |  6800.00 |            80
         166 | Sundar      |        147 |  6400.00 |            80
         167 | Amit        |        147 |  6200.00 |            80
```

Sort data in ascending order by the first column **manager_id** after ORDER BY.

Sort data in descending order by the second column **salary** after ORDER BY.

HUAWEI

Removing Duplicate Values (1)

- DISTINCT keyword
  - Remove all duplicate rows from the result set of the SELECT statement so that each row in the result set is unique. Value range: existing column names or expression
- Syntax
  - If there is only one column after the DISTINCT keyword, this column is used for deduplication. If there are two or more columns, the combination of these columns is used for duplication check.

```
SELECT DISTINCT [ , ... ] FROM table_reference [ , ... ]
```

Removing Duplicate Values (2)

- Use the DISTINCT keyword to query the position and bonus of an employee and remove duplicate records with the same position and bonus.

staff_id	name	job	bonus
30	wangxin	developer	9000
31	xufeng	tester	7000
34	denggui	tester	7000
35	caoming	developer	10000
37	lixue	developer	9000

```
select distinct job, bonus from sections;

JOB                  BONUS
-------------------- ------------
developer              9000
tester                 7000
developer              10000

3 rows fetched.
```


HUAWEI

GROUP BY (1)

- GROUP BY is very important for database queries. It collects data across multiple records and groups the results by one or more columns.
- Syntax

```
GROUP BY { column_name } [ , ... ]
```

- Usage
  - Expressions in the GROUP BY clause can be any columns from the table or view in the FROM clause, regardless of whether the columns appear in the SELECT list.
  - The GROUP BY clause groups rows, but does not guarantee the order of result sets. To sort groups, use the ORDER BY clause.
  - The expressions following GROUP BY can use parentheses, for example, GROUP BY (*expr1*, *expr2*), or GROUP BY (*expr1*), (*expr2*). However, the GROUP BY (*expr1*, *expr2*), *expr3* format is not supported.

HUAWEI

GROUP BY (2)

- Use the DISTINCT keyword to query the position and bonus of an employee and remove duplicate records with the same position and bonus.

staff_id	name	job	bonus
30	wangxin	developer	9000
31	xufeng	tester	7000
34	denggui	tester	7000
35	caoming	developer	10000
37	lixue	developer	9000
39	chenjing	developer	9000

```
select job, bonus, count(staff_id) sum from bonuses_depa
group by(job,bonus) order by sum;

JOB                  BONUS       SUM
-------------------- ----------- --------------------
developer              10000       1
tester                  7000       2
developer              9000        3

3 rows fetched
```


HUAWEI

GROUP BY (3)

- HAVING clause

  It is used with the GROUP BY clause to select special groups. The HAVING clause compares some attributes of groups with a constant. Only groups that match the conditions in the HAVING clause are extracted.

- Syntax

```
HAVING condition [ , ... ]
```

- Example: Query the total number of employees in each position whose number of employees is greater than 3 in the **sections** table.

```
select job, count(staff_id) from bonuses_depa group by job having count(staff_id) >3;

JOB                 COUNT(STAFF_ID)
------------------- -------------------
developer             4
```


HUAWEI

LIMIT/OFFSET (1)

- The data limit function involves two independent clauses: LIMIT and OFFSET.
  - The LIMIT clause allows users to limit the rows returned by a query. You can specify an offset and the number or percentage of rows to be returned. This clause can be used to generate top-N reports. To obtain consistent results, specify the ORDER BY clause to ensure a deterministic sort order.

    ```
    LIMIT { count | ALL }
    ```

  - The OFFSET clause sets the position from which the return begins.

    ```
    OFFSET start
    ```

- Usage
  - **start**: specifies the number of rows to skip before returning rows.
  - **count**: specifies the maximum number of rows to be returned.
  - When both **start** and **count** are specified, the rows specified by **start** are skipped before the rows specified by **count** are calculated.
  - LIMIT 5,20 is equivalent to LIMIT 20 OFFSET 5 and OFFSET 5 LIMIT 20.

LIMIT/OFFSET (2)

- Query employee information in the **bonuses_depa** table. Add **LIMIT 2 OFFSET 1** to skip the first row and query a total of two rows.

staff_id	**name**	job	bonus
30	wangxin	developer	9000
31	xufeng	document developer	6000
34	denggui	quality control	9000
35	caoming	tester	10000

```
SELECT name, job, bonus FROM bonuses_depa LIMIT 2 OFFSET 1;

NAME                                     JOB                          BONUS
---------------------------------------- ---------------------------- --------------
xufeng                                   document developer           6000
denggui                                  quality control              9000
```


HUAWEI

Aggregate Functions (1)

- Use sum(expression) to calculate the sum.
  - For BIGINT input, the output type is NUMBER.
  - For SMALLINT or INT input, the output type is BIGINT.
  - For floating-point input, the output type is DOUBLE PRECISION.

```
test=>  select sum(salary) from employees;
    sum
-----------
 691416.00
(1 row)
```


HUAWEI

Aggregate Functions (2)

- Use avg(expression) to calculate the average value.
  - For any integer input, the output type is NUMBER.
  - For any floating-point input, the output type is DOUBLE PRECISION.

```
test=>  select avg(salary) from employees;
          avg
-----------------------
 6461.8317757009345794
(1 row)
```


HUAWEI

Aggregate Function Examples

- Calculate the maximum and minimum values.
  - Argument types: any array, numeric, string, or date/time
  - Return type: same as the argument data type

```
test=>  select max(salary),min(salary) from employees;
   max     |   min
-----------+---------
 24000.00 | 2100.00
```

- Calculate the number of rows.
  - count(*) --> Return the number of rows in the table, including empty rows.
  - count(1) --> Return the number of rows in the table, including empty rows.
  - count(expression) --> Return the rows whose **expression** is not NULL.

```
test=> select count(*),count(1),count(commission_pct) from employees;
 count | count | count
-------+-------+-------
   107 |   107 |    35
```


HUAWEI

Batch Processing (1)

- openGauss can execute SQL statements from text files. openGauss provides the gsql tool to process SQL statements in batches. Batch processing is recommended in the following scenarios:
  - If you run a query repeatedly (for example, daily or weekly), you can set it as a script to avoid repeated input.
  - You can generate a new query from an existing similar query by copying and editing the script file.
  - For multi-line statements or multi-statement sequences, if an error occurs, you do not need to input all the contents again. You only need to edit the script to correct the error, and then execute it again.
  - When interactive use is not allowed in some cases, batch processing is necessary.
- Syntax

```
gsql -d dbname -p port -f filename
```

  - **dbname**: specifies the name of the database to be connected.
  - **port**: specifies the port number of the database server.
  - **-f filename**: uses a file as the command source instead of interactive input. This parameter specifies the path and name of the text file to be read.

Batch Processing (2)

- Create a local **sql.txt** file. The content looks like:

```
CREATE TABLE customer (
 c_customer_sk integer, c_customer_id char(5), c_first_name char(6), c_last_name char(8),
Amount integer );
INSERT INTO customer(c_customer_sk, c_customer_id, c_first_name,Amount) VALUES (3769, 'hello', 'Grace', 1000);
INSERT INTO customer (c_customer_sk, c_first_name) VALUES (3769, 'Grace');
INSERT INTO customer (c_customer_sk, c_customer_id, c_first_name) VALUES (3769, 'hello', DEFAULT);
INSERT INTO customer (c_customer_sk, c_customer_id, c_first_name,Amount) VALUES (6885, 'maps', 'Joes',2200),
(4321, 'tpcds', 'Lily',3000), (9527, 'world', 'James',5000);
```

- Execute the SQL statements in the **sql.txt** file.

```
gsql -d postgres -p 21013 -f /home/user/sql.txt
```


HUAWEI

Contents

1. **Advanced SQL Syntax**
   - Constraints and Table Joins
   - TRUNCATE TABLE
   - Subquery
   - WHERE Clause
   - ORDER BY
   - System Functions
   - VACUUM

HUAWEI

Main System Functions

- System functions encapsulate service logic to implement specific functions. A system function may or may not have parameters. After a system function is executed, its results are returned.
- Main system functions are as follows:
  - Numeric functions
  - Character processing functions
  - Time/Date functions
  - Interval functions
  - Type conversion functions

HUAWEI

Numeric Functions (1)

- abs(exp), cos(exp), sin(exp), acos(exp), and asin(exp): return the absolute value, cosine value, sine value, arc cosine value, and arc sine value of an expression.
  - The return value type of abs(exp) is the same as that of **exp**. The return values of the other parameters are numbers.
  - The input parameter *X* of acos(*X*) and asin(*X*) is an expression that can be converted into a numeric value. The value range is [–1,1].

```
openGauss=# select abs(-10),cos(0),sin(0),acos(1),asin(0) from sys_dummy;

ABS(-10)        COS(0)        SIN(0)        ACOS(1)        ASIN(0)
---------- --------------- ------------- ---------------- ----------------
10              1             0              0               0

1 rows fetched.
```


HUAWEI

Numeric Functions (2)

- bitand(exp1, exp2), bitor(exp1, exp2), and bitxor(exp1, exp2): bitwise AND, bitwise OR, and bitwise XOR.

```
openGauss=# select bitand(29,15),bitxor(29,15),bitor(29,15) from sys_dummy;

BITAND(29,15)       BITXOR(29,15)       BITOR(29,15)
------------------- ------------------- -------------------
13                  18                  31

1 rows fetched.
```

- round(*number*[, *decimals*]): truncates a numeric value to the left or right of the decimal point specified by *decimals*.

```
openGauss=# select round(1234.5678,-2),round(1234.5678,2) from sys_dummy;

ROUND(1234.5678,-2)                 ROUND(1234.5678,2)
----------------------------------- ---------------------------------------
1200                                1234.57

1 rows fetched.
```


HUAWEI

Numeric Functions (3)

Syntax	Function	Example
ceil(exp)	Returns the smallest integer greater than or equal to *n*.	ceil(15.3) 16
rawtohex(exp)	Converts the value *exp* of the raw class into a hexadecimal string.	rawtohex('012') 303132
sign(exp)	Returns the sign of *exp*. If *exp* > 0, **1** is returned; if *exp* < 0, **–1** is returned; if *exp* = 0, **0** is returned.	sign(2*3) 1
sort(n)	Returns the square root of a non-negative real number. The input parameter is an expression that can be converted to a non-negative numeric value.	sort(49) 7
trunc(number, scale)	Truncates the entered value in the specified format. The input parameter *number* specifies the data to be truncated and *scale* specifies the truncation precision.	trunk(15.79,1) 15.7;trunk(15.79,-1) 10
floor(exp)	Calculates the largest integer equal to or less than the value of an expression. The input parameter is an expression that can be converted to a numeric value. The return type is NUMBER.	floor(12.8) 12
mod(exp1, exp2)	Performs the Modulo operation. The input parameter is an expression that can be converted to a NUMBER value. The return type is NUMBER.	mod(29,3) 2
power(base, expn)	Returns the result of multiplying a number by a power. The input parameter *base* specifies the base number and *expn* specifies the exponent. The return type is NUMBER.	power(5,3) 125

HUAWEI

Character Processing Functions (1)

- concat(str[, ...]),concat_ws(separator, str1, str2, ...): concatenates one or more character strings. There is no delimiter in the first function. Delimiters can be specified in the second function.

```
openGauss=# select concat('11','null','22'),concat_ws('-','11',null,'22') from sys_dummy;

CONCAT('11','NULL','22')    CONCAT_WS('-','11',NULL,'22')
--------------------------- ----------------------------
11null22                    11-22
```

- hextoraw(str): converts a hexadecimal character string into a binary character string.

```
openGauss=# select hex('ABC'),hex2bin('0x28'),hextoraw('abc') from sys_dummy;

HEXTORAW('ABC')
-----------------------------------
0ABC
```


HUAWEI

Character Processing Functions (2)

- replace(str,src,dst): string insertion and replacement function.

```
openGauss=# select replace('123456','45','abds') from sys_dummy;
REPLACE('123456','45','ABDS')
-----------------------------
123abds6
```

- instr(str1,str2[,pos[,n]])): character string search function. Returns the position of the string to be searched for in the source string. Returns the position matching string 2 in string 1 for the *int2*th time from the position specified by *int1*. *int1* specifies the start position for matching, and *int2* specifies the number of matching times.

```
openGauss=# SELECT INSTR('abcdabcdabcd', 'bcd'), INSTR('abcdabcdabcd', 'bcd', 2, 2 );
INSTR('abcdabcdabcd', 'bcd')  INSTR( 'abcdabcdabcd', 'bcd', 2, 2 )
----------------------------  ------------------------------------
2                             6
```


HUAWEI

Character Processing Functions (3)

Syntax	Function	Example
left(str, length)	Returns a specified number of leftmost characters of a specified string.	left('abcdef',3) abc left('abcdef',0) or left('abcdef',0) - NULL string
length(str)	Obtains the length of a character string.	length('1234a') - 5
lower(str)	Returns a string in which all characters are converted into lowercase.	lower('ABCD') abcd lower('1234') 1234
upper(str)	Returns a string in which all characters are converted into uppercase.	upper('abcd') ABCD upper('1234') 1234
right(str,len)	Returns a specified number of rightmost characters of a specified string.	right('abcdef',3) def right ('abcdef',0) or right ('abcdef',0) - NULL string
reverse(str)	Returns a string in a reverse order. Only the string type is supported.	reverse('abcd') dcba
substr(str, start[, len])	Truncates a substring.	substr('abcdefg',3,4) cdef Truncate four characters starting from the third character of the 'abcdefg' string.

Time/Date Functions

Syntax	Function	Example
current_timestamp	Obtains the current system time and time zone.	current_timestamp 2023-08-23 23:59:07.498876+08
now(fractional_second_precision)	Obtains the current system time and time zone.	Select now() 2023-08-23 23:59:07.498876+08
age(timestamp,timestamp)	Subtracts arguments, producing a result in YYYY-MM-DD format. If the result is negative, the returned result is also negative. Return type: interval	postgres=# SELECT age(timestamp '2001-04-10', timestamp '1957-06-13'); -> 43 years 9mons 27 days (1 row)
current_date	Obtains the current date. Return type: date	SELECT current_date; 2023-08-23
trunc(timestamp)	Truncates to day by default.	SELECT trunc(timestamp '2001-02-16 20:38:40');-> 2023-08-23 00:00:00
numtodsinterval(num,'interval_unit')	Obtains the day specified by **interval** and a numeric value to second.	sysdate 2019-08-23 12:52:43 Return the time one hour later than the current time. : sysdate+numtodsinterval(1,'hour') 2023-08-23 13:52:43

Conditional Expression Functions

- coalesce(expr1, expr2, ..., exprn): returns the first non-NULL parameter value in the parameter list.
- nullif(expr1, expr2): returns NULL only when *expr1* and *expr2* are equal. Otherwise, *expr1* is returned.
- nvl(expr1, expr2): If *expr1* is NULL, *expr2* is returned. If *expr1* is not NULL, *expr1* is returned.

```
openGauss=# SELECT coalesce(NULL,'hello');
 coalesce
----------
 hello
(1 row)

openGauss=# SELECT nullif(10,12),nvl(null,12);
nullif  | nvl
--------+-----
10      | 12
(1 row)
```

Type Conversion Functions (1)

- to_char (datetime/interval [, fmt]): converts the input into a character string.
- to_clob(char/nchar/varchar/nvarchar/varchar2/nvarchar2/text/raw): converts the input into CLOB.
- to_date(text) and to_number(text, text): convert a value of the string type to a number in the specified format.

```
openGauss=# select to_char(35),to_clob('excellent staff of 2018'),to_date('2019-08-
23','YYYY-MM-DD'),to_number('12E','xxxx');
to_char  | to_clob                | to_date     | to_number
---------+------------------------+-------------+-----------
35       | excellent staff of 2018 | 2019-08-23 | 302
(1 row)
```


HUAWEI

Type Conversion Functions (2)

Syntax	Function	Example
ascii(str)	Returns the ASCII value of the first character in *str*.	ascii('hello') 104
cast(expr as datatype)	Converts the column name/value to the data type specified by **datatype**. An expression can be converted to the same type as itself.	cast('10' as int) 10
chr(n)	Returns the character whose ASCII value is *n*. The value range of *n* is [0,127]. The input parameter is an expression that can be converted to a numeric value.	chr(67) C
convert(expr, data_type)	Converts *expr* to *data_type*. The value range of *data_type* is all data types except CLOB, BLOB, and IMAGE.	convert('2018-06-28 13:14:15', timestamp) 2018-06-28 13:14:15.000000
to_hex(number int or bigint)	Converts *number* to the hexadecimal format.	to_hex(2147483647) 7fffffff

HUAWEI

System Information Function Examples (1)

- Query the database instance startup time, current database name (called catalog in standard SQL), and host name.

```
test=> select pg_postmaster_start_time(),current_catalog,current_database(),get_hostname();
   pg_postmaster_start_time    | current_database | current_database | get_hostname
-------------------------------+------------------+------------------+--------------
 2023-09-27 10:25:27.886419+08 | test             | test             | Run861
```

- Query the current schema and schemas in the search path.

```
test=> select current_schema,current_schemas(true);
 current_schema |     current_schemas
----------------+-------------------------
 test           | {pg_catalog,test,public}
```

- Query the user and user OID in the current environment.

```
test=> select user,current_user,current_role,session_user,definer_current_user(),pg_current_userid();
 current_user | current_user | current_user | session_user | definer_current_user | pg_current_userid
--------------+--------------+--------------+--------------+----------------------+-------------------
 test         | test         | test         | test         | test                 |             18761
```


HUAWEI

System Information Function Examples (2)

- Query the current session ID and thread ID.

```
test=> select pg_current_sessid(),pg_current_sessionid(),pg_backend_pid();
 pg_current_sessid |    pg_current_sessionid    | pg_backend_pid
-------------------+----------------------------+----------------
   140127492503232 | 1696525196.140127492503232 | 140127492503232
```

Timestamp, session ID

- Query the access permission.

```
test=> select has_database_privilege('lyc1','test','create');
 has_database_privilege
------------------------
 f
(1 row)

test=> select has_table_privilege('restore_user','employees','select');
 has_table_privilege
---------------------
 t
(1 row)
```

Check whether user **lyc1** has the CREATE permission in the **test** database.

Check whether user **restore_user** has the query permission on the **employees** table.

HUAWEI

Contents

1. **Advanced SQL Syntax**
   - Constraints and Table Joins
   - TRUNCATE TABLE
   - Subquery
   - WHERE Clause
   - ORDER BY
   - System Functions
   - VACUUM

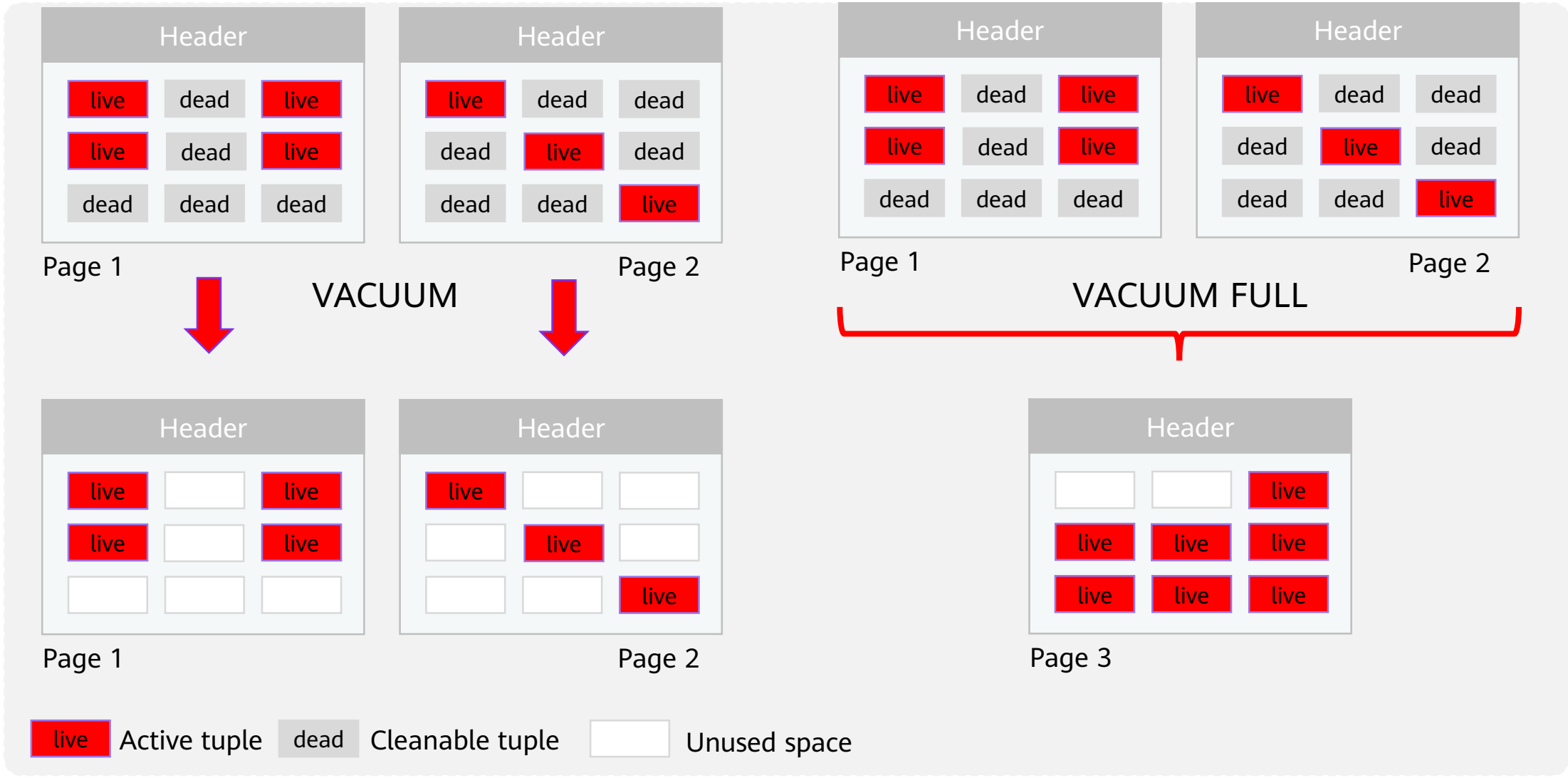
HUAWEI

Table Cleanup with VACUUM (1)

- Astore table bloat
  - By default, tables in openGauss use the append-only storage engine (Astore). Astore supports the multi-version tuple mechanism. That is, different versions of historical tuples are reserved for the same record on the data page to resolve read/write conflicts between concurrent transactions. This method of retaining historical tuples avoids extra undo log write consumption, but introduces table bloat.
- VACUUM
  - VACUUM recycles the storage space occupied by deleted rows in a table or B-tree indexes. That is, historical tuples are deleted.
  - In normal database operation, rows that have been deleted are not physically removed from their table; instead, they remain present until a VACUUM operation is done. Therefore, it is necessary to perform VACUUM periodically, especially on frequently-updated tables.

HUAWEI

Table Cleanup with VACUUM (2)



HUAWEI

Table Cleanup with VACUUM - Syntax

- Recycle space and update statistics. The keyword order does not matter.

```
VACUUM [ ( { FULL | FREEZE | VERBOSE | {ANALYZE | ANALYSE }} [,...] ) ]
    [ table_name [ (column_name [, ...] ) ] [ PARTITION ( partition_name ) | SUBPARTITION (
subpartition_name ) ] ] ;
```

- Recycle space without updating statistics.

```
VACUUM [ FULL [COMPACT] ] [ FREEZE ] [ VERBOSE ] [ table_name
[ PARTITION ( partition_name )  | SUBPARTITION ( subpartition_name ) ] ] ;
```

- Recycle space and update statistics. The keywords need to be in order.

```
VACUUM [ FULL ] [ FREEZE ] [ VERBOSE ] { ANALYZE | ANALYSE } [ VERBOSE ]
    [ table_name [ (column_name [, ...] ) ] ] [ PARTITION ( partition_name ) ];
```


HUAWEI

Example of Table Cleanup with VACUUM

- VACUUM cannot be executed inside a transaction block. VACUUM causes a significant increase in I/O traffic, which may affect the performance of other active sessions. Therefore, it is sometimes advisable to use the cost-based VACUUM delay feature. It is recommended that production databases be vacuumed frequently (at least nightly) ensure that dead rows are removed. It is strongly recommended that you perform VACUUM ANALYZE on corresponding tables after adding or deleting a large number of records. **FULL** is recommended only in special scenarios. For example, you wish to physically downsize a table to decrease the occupied disk space after deleting most rows of the table. Before performing the VACUUM FULL operation, you are advised to delete all indexes in related tables, run **VACUUM FULL**, and then rebuild the index.

HUAWEI

Contents

1. Advanced SQL Syntax
2. **Stored Procedure and Trigger**
   - Stored Procedure
   - Trigger
3. Application Development

HUAWEI

Stored Procedure

- A stored procedure is a set of SQL statements that can implement specific functions in a database for external programs to call. It can be regarded as a function or subprogram of a database.
- Stored procedure:
  - Stored procedures can be encapsulated to hide complex business logic.
  - SQL statements are compiled once and called repeatedly. This reduces the number of repeated SQL statements and improves work efficiency.
  - More SQL statements are sent during one interaction, reducing the number of interactions between the client and server.
  - For complex SQL statements that are run for multiple times, the stored procedure can cache SQL plans, reducing the time for generating plans.

HUAWEI

Creating a Stored Procedure

- A stored procedure is a combination of SQL and PL/SQL.
- Syntax:

```
CREATE [ OR REPLACE ]  [DEFINER = user] PROCEDURE procedure_name
    [ ( {[ argmode ] [ argname ] argtype [ { DEFAULT | := | = } expression ]}[,...]) ]
  { IS | AS }
  plsql_body
/
```

  - The creation and execution of a stored procedure are similar to those of a function.
  - The difference is that the stored procedure does not have a return value but the function has a return value.
  - plsql_body: PL/SQL stored procedure body (The syntax structure is the same as that of an anonymous block.)

HUAWEI

Calling or Deleting a Stored Procedure

- A stored procedure is a combination of SQL and PL/SQL.
- Calling syntax:

```
CALL procedure_name ( param_expr );
```

- Deletion syntax:

```
DROP PROCEDURE [ IF EXISTS ] procedure_name [( param_expr )] ;
```

  - When deleting a stored procedure, you do not need to specify the parameter list and its external parentheses.
  - If no parameter list is specified and a reloaded stored procedure with the same name exists, they are deleted together.

HUAWEI

Stored Procedure Example

```
-- Create a stored procedure.
CREATE OR REPLACE PROCEDURE prc_add ( param1 IN INTEGER, param2 IN OUT INTEGER )
AS
BEGIN
    param2:= param1 + param2;
    RAISE INFO 'result is: %', param2;
END;
/

--Call a stored procedure.
CALL prc_add(param1 := 1,param2 := 4);
CALL prc_add(1,4);

-- Delete a stored procedure.
DROP PROCEDURE prc_add;
DROP PROCEDURE IF EXISTS prc_add;
```


HUAWEI

Declaring a PL/SQL Block

- PL/SQL blocks are classified into anonymous blocks and subprograms, which are also called command blocks. A PL/SQL block can contain a subblock which can be placed in any section. The structure of the PL/SQL block is as follows:
  - DECLARE: declares variables, types, cursors, and regional stored procedures and functions used in the PL/SQL block.

```
DECLARE
```

  - BEGIN: procedures and SQL statements, which are the main part of the program.

```
BEGIN
```

  - EXCEPTION: processes errors.

```
EXCEPTION
```

  - END: The end character "/" must occupy a line.

```
END;
/
```


HUAWEI

Declaring an Anonymous Block

- An anonymous block is generally used for scripts that are not frequently executed or activities that are not repeatedly executed. An anonymous block is executed in a session and is not stored.
- Syntax:

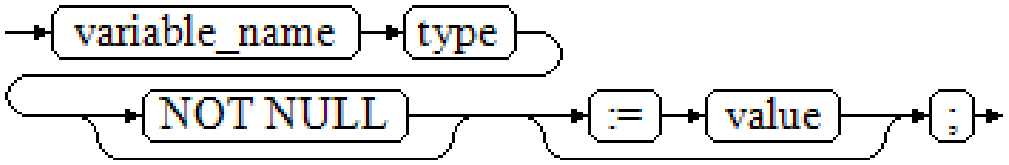
```
[DECLARE [declare_statements]]
BEGIN
execution_statements
END;
/
```

  - The executable part of an anonymous block starts with a BEGIN statement, has a break with an END statement, and ends with a semicolon (;).
  - The declaration section includes the variable, type, and cursor definition.
  - A simplest anonymous block does not execute any commands. But there is at least one statement, even a NULL statement, in any implementation block.

HUAWEI

Basic Statements - Defining a Variable

- Declare a variable.



  - **variable_name** specifies the name of a variable.
  - **type** specifies the type of a variable.
  - **value** specifies the initial value of a variable (if no initial value is given, the initial value is **NULL**). **value** can also be an expression.

- In addition to basic types, variable types %TYPE and %ROWTYPE are supported.
  - my_name employee.firstname%TYPE
  - my_employee employee%ROWTYPE

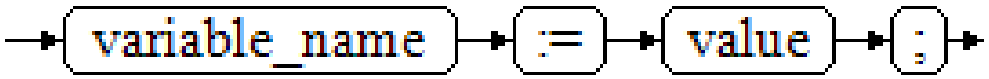
Basic Statements - Scope of a Variable

- The scope of a variable specifies the accessibility and availability of the variable in a code block. In other words, a variable takes effect only within its scope.
  - To define a scope, a variable must be first declared in the DECLARE section, that is, the BEGIN-END block. The necessity of such declaration is also determined by the block structure, which requires that a variable has different scopes and lifetime in a stored procedure.
  - A variable can be defined multiple times in different scopes, and the inner definition can cover the outer one.
  - A variable defined in an outer block can also be used in its nested block. However, the outer block cannot access variables in nested blocks.

HUAWEI

Basic Statements — Assignment

- Assign values to variables.



  - **variable_name** specifies the name of a variable.
  - **value** can be a value or an expression. The type of **value** must be compatible with the type of **variable_name**.

```
openGauss=# DECLARE
    emp_id  INTEGER := 7788; -- Assignment
BEGIN
    emp_id := 5; -- Assignment
    emp_id := 5*7784;
END;
/
```

Control Statements - RETURN

- RETURN: returns results from a stored procedure of function to the caller. openGauss supports RETURN or RETURN NEXT and RETURN QUERY. RETURN NEXT and RETURN QUERY are used only for functions and cannot be used for stored procedures.
- RETURN controls return from a stored procedure or function to the caller.




```
CREATE OR REPLACE PROCEDURE proc_return AS
v_num NUMBER(8,2);
v_sum INTEGER;
BEGIN
    proc_staffs(30, v_sum, v_num); -- Call statement
    RAISE INFO 'v_sum is: %', v_num;
    RETURN; -- Return statement
END;
/
```


HUAWEI

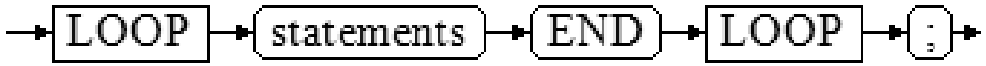
Control Statements - Condition

- Conditional statements are used to decide whether given conditions are met. Operations are executed based on the decisions made.
- openGauss supports five usages of IF:

IF	Concept	Example
IF_THEN::=	IF_THEN is the simplest form of IF statements. If the condition is true, the statement is executed. If the condition is false, the statement is skipped and IF_THEN ends.	IF v_user_id <> 0 THEN UPDATE users SET email = v_email WHERE user_id = v_user_id; END IF;
IF_THEN_ELSE	IF_THEN_ELSE has an ELSE branch and can be executed if the condition is false.	IF parentid IS NULL OR parentid = '' THEN RETURN; ELSE hp_true_filename(parentid); -- Call a stored procedure. END IF;
IF_THEN_ELSE IF	IF statements can be nested.	IF sex = 'm' THEN pretty_sex := 'man'; ELSE IF sex = 'f' THEN pretty_sex := 'woman'; END IF; END IF;
IF_THEN_ELSIF_ELSE	IF_THEN_ELSE has an ELSE branch and can be executed if the condition is false.	IF number_tmp = 0 THEN result := 'zero'; ELSIF number_tmp > 0 THEN result := 'positive'; ELSIF number_tmp < 0 THEN result := 'negative'; ELSE result := 'NULL'; END IF;
IF_THEN_ELSEIF_ELSE	ELSEIF is an alias of ELSIF.	

Control Statements - LOOP

- LOOP: LOOP must be used together with EXIT. Otherwise, an infinite loop occurs.

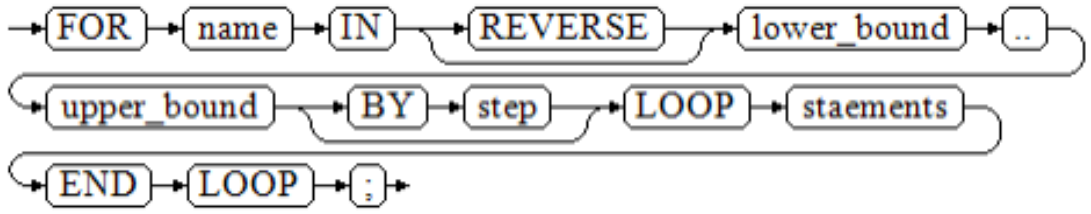



```
CREATE OR REPLACE PROCEDURE proc_loop(i in integer, count out integer)  AS
   BEGIN
      count:=0;
      LOOP
      IF count > i THEN
         raise info 'count is %. ', count;
         EXIT;
      ELSE
         count:=count+1;
      END IF;
      END LOOP;
   END;
/
CALL proc_loop(10,5);
```


HUAWEI

Control Statements - FOR_LOOP (1)

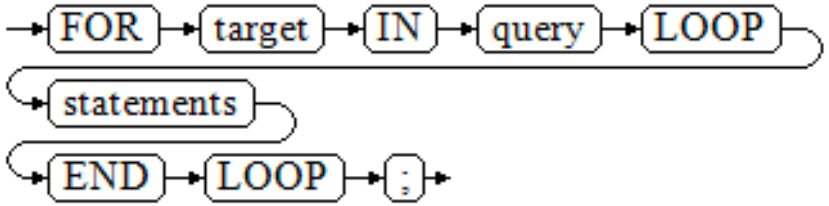
- FOR_LOOP (*Integer variable*):




```
CREATE OR REPLACE PROCEDURE proc_for_iloop() AS
BEGIN
   FOR i IN 0..5 LOOP
     RAISE NOTICE '% time', i;
   END LOOP;
END;
/
CALL proc_for_iloop();
```

Control Statements - FOR_LOOP (2)

- FOR_LOOP:

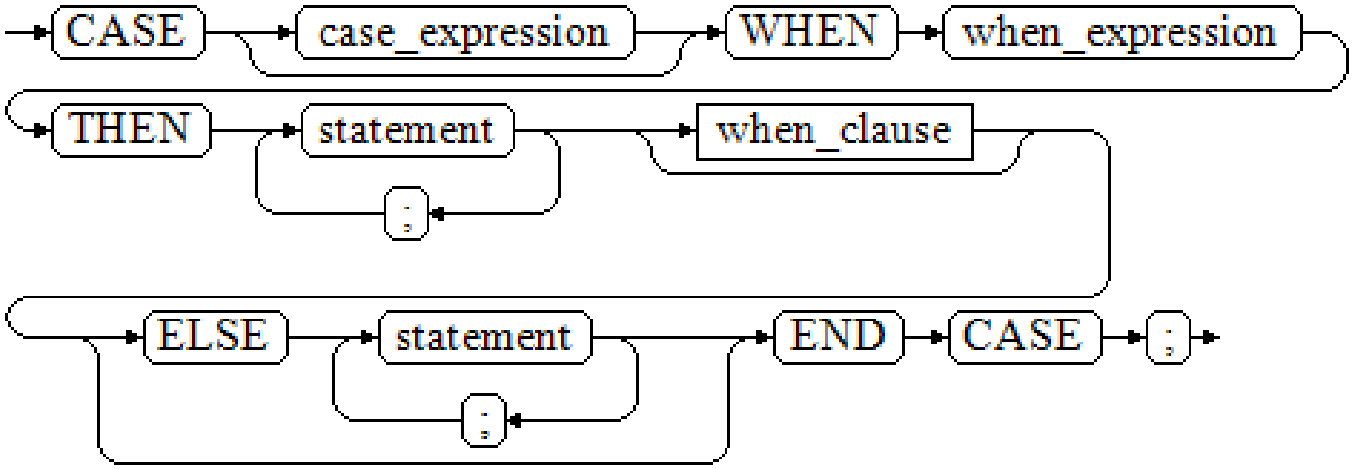



```
CREATE OR REPLACE PROCEDURE proc_for_loop()
AS
  BEGIN
    FOR emp IN SELECT * FROM t_emp1
    LOOP
      RAISE NOTICE '%', emp.name; -- Print the employee name.
      END LOOP;
    END;
/
CALL proc_for_loop();
```

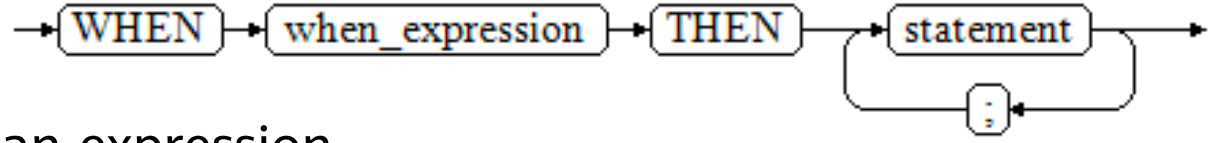

HUAWEI

Control Statements - CASE WHEN (1)

- CASE WHEN:



- The when_clause clause is as follows:



  - **case_expression**: specifies a variable or an expression.
  - **when_expression**: specifies a constant or conditional expression.
  - **statement**: specifies the statement to be executed.

Control Statements - CASE WHEN (2)

```
CREATE OR REPLACE PROCEDURE proc_case_branch(pi_result in integer, pi_return out integer)
AS
    BEGIN
        CASE pi_result
            WHEN 1 THEN
                pi_return := 111;
            WHEN 2 THEN
                pi_return := 222;
            WHEN 3 THEN
                pi_return := 333;
            WHEN 6 THEN
                pi_return := 444;
        END CASE;
        raise info 'pi_return : %',pi_return ;
    END;
/
CALL proc_case_branch(3,0);
```


HUAWEI

Control Statements - NULL

- In PL/SQL programs, the NULL statement does nothing. It serves as a placeholder and can improve program readability by making the meaning and action of some statements clear.
- The usage of NULL statements is as follows:

```
DECLARE
    ...
BEGIN
    ...
    IF v_num IS NULL THEN
        NULL; -- No data needs to be processed.
    END IF;
END;
/
```


HUAWEI

Control Statements - EXCEPTION (1)

- By default, any error occurring in a PL/SQL function aborts execution of the function, and rolls back the surrounding transaction as well. You can trap errors and restore from them by using a BEGIN block with an EXCEPTION clause. The syntax is an extension of the syntax for a BEGIN block.

```
[<<label>>]
[DECLARE
    declarations]
BEGIN
    statements
EXCEPTION
    WHEN condition [OR condition ...] THEN
        handler_statements
    [WHEN condition [OR condition ...] THEN
        handler_statements
    ...]
END;
```

Control Statements - EXCEPTION (2)

```
CREATE OR REPLACE PROCEDURE proc_exp()
AS
DECLARE
    x INT :=0;
    y INT;
BEGIN
    UPDATE mytab SET firstname = 'Joe' WHERE lastname =
'Jones';
    x := x + 1;
    y := x / 0;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'caught division_by_zero';
        RAISE NOTICE 'x: %',x;
        RETURN ;
END;
/
call fun_exp();
```

```
openGauss=# select * from mytab ;
 id | firstname | lastname
----+-----------+----------
    | Tom       | Jones
(1 row)
openGauss=# call proc_exp();
NOTICE:  caught division_by_zero
NOTICE:  x: 1
 proc_exp
----------


(1 row)
openGauss=# select * from mytab ;
 id | firstname | lastname
----+-----------+----------
    | Tom       | Jones
(1 row)
```


HUAWEI

Control Statements - GOTO (1)

- The GOTO statement unconditionally transfers the control from the current statement to a labeled statement. The GOTO statement changes the execution logic. Therefore, exercise caution when using this statement. Alternatively, you can use the EXCEPTION statement to handle issues in special scenarios. To run a **GOTO** statement, the labeled statement must be unique.
- label declaration ::=



- goto statement ::=



HUAWEI

Control Statements - GOTO (2)

```
CREATE OR REPLACE PROCEDURE GOTO_test()
AS
DECLARE
    v1  int;
BEGIN
    v1  := 0;
        LOOP
        EXIT WHEN v1 > 100;
                v1 := v1 + 2;
                if v1 > 25 THEN
                        GOTO pos1;
                END IF;
        END LOOP;
<<pos1>>
v1 := v1 + 10;
raise info 'v1 is %. ', v1;
END;
/
call GOTO_test(); -- INFO:  v1 is 36.
```


HUAWEI

Control Statements - GOTO (3)

- Using GOTO statements has the following restrictions:
  - A GOTO statement does not allow multiple labeled statements even if the statements are in different blocks.
  - A GOTO statement cannot transfer control to the IF, CASE, or LOOP statement.
  - A GOTO statement cannot transfer control from one IF clause to another, or from one WHEN clause in the CASE statement to another.
  - A GOTO statement cannot transfer control from an outer block to an inner BEGIN-END block.
  - A GOTO statement cannot transfer control from an exception handler to the current BEGIN-END block. However, a GOTO statement can transfer control to the upper-layer BEGIN-END block.
  - To branch to a position that does not have an executable statement, add the NULL statement.

HUAWEI

Contents

1. Advanced SQL Syntax
2. **Stored Procedure and Trigger**
   - Stored Procedure
   - Trigger
3. Application Development

HUAWEI

Trigger

- A trigger is a data object associated with a data table. It automatically executes user-defined functions when a specified database event occurs.
- Key elements of a trigger:
  - Associated object: table associated with the trigger. Triggers cannot be created on column-store tables, temporary tables, or unlogged tables.
  - Trigger event: event for starting a trigger, including INSERT, UPDATE, DELETE, and TRUNCATE.
  - Trigger occasion: time when the trigger function is executed. For example, **BEFORE** specifies that the trigger function is started before a trigger event is executed.
  - Trigger function: a user-defined function that is automatically executed when a trigger event occurs.

HUAWEI

Trigger Syntax

- Creation

```
CREATE TRIGGER trigger_name { BEFORE | AFTER | INSTEAD OF } { event [ OR ... ] }
    ON table_name
    [ FOR [ EACH ] { ROW | STATEMENT } ]
    [ WHEN ( condition ) ]
    EXECUTE PROCEDURE function_name ( arguments );
```

- Modification

```
ALTER TRIGGER trigger_name ON table_name RENAME TO new_trigger_name;
```

- Deletion

```
DROP TRIGGER trigger_name ON table_name [ CASCADE | RESTRICT ];
```


HUAWEI

Trigger Example (1)

```
-- Create a source table and a target table.
CREATE TABLE test_trigger_src_tbl(id1 INT, id2 INT, id3 INT);
CREATE TABLE test_trigger_des_tbl(id1 INT, id2 INT, id3 INT);

-- Create a trigger function.
CREATE OR REPLACE FUNCTION tri_insert_func() RETURNS TRIGGER AS $$
DECLARE
BEGIN
        INSERT INTO test_trigger_des_tbl VALUES(NEW.id1, NEW.id2, NEW.id3);
        RETURN NEW;
END
$$ LANGUAGE PLPGSQL;
```


HUAWEI

Trigger Example (2)

```
-- Create an INSERT trigger.
CREATE TRIGGER insert_trigger BEFORE INSERT ON test_trigger_src_tbl FOR EACH ROW
EXECUTE PROCEDURE tri_insert_func();

-- Execute the INSERT event and check the trigger results.
INSERT INTO test_trigger_src_tbl VALUES(100,200,300);
SELECT * FROM test_trigger_src_tbl;
SELECT * FROM test_trigger_des_tbl;  // Check whether the trigger operation takes effect.

-- Modify a trigger.
ALTER TRIGGER insert_trigger ON test_trigger_src_tbl RENAME TO insert_trigger_renamed;

-- Delete a trigger.
DROP TRIGGER insert_trigger_renamed ON test_trigger_src_tbl;
```


HUAWEI

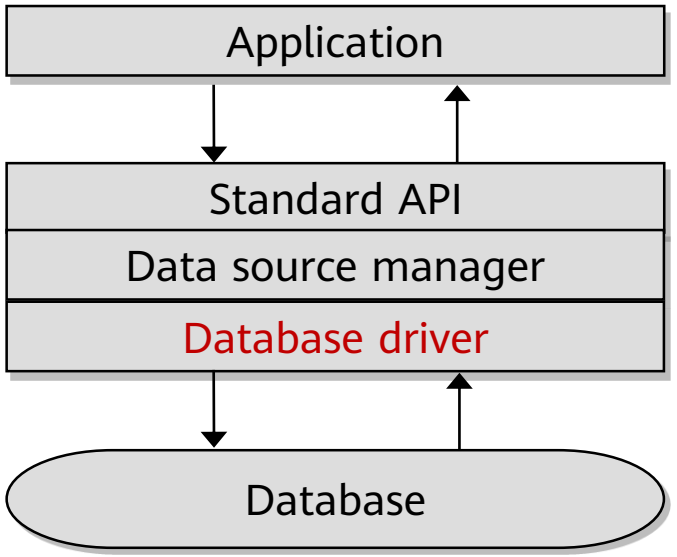
Contents

1. Advanced SQL Syntax
2. Stored Procedure and Trigger
3. **Application Development**
   - JDBC-based Development
   - ODBC-based Development
   - Psycopg-based Development

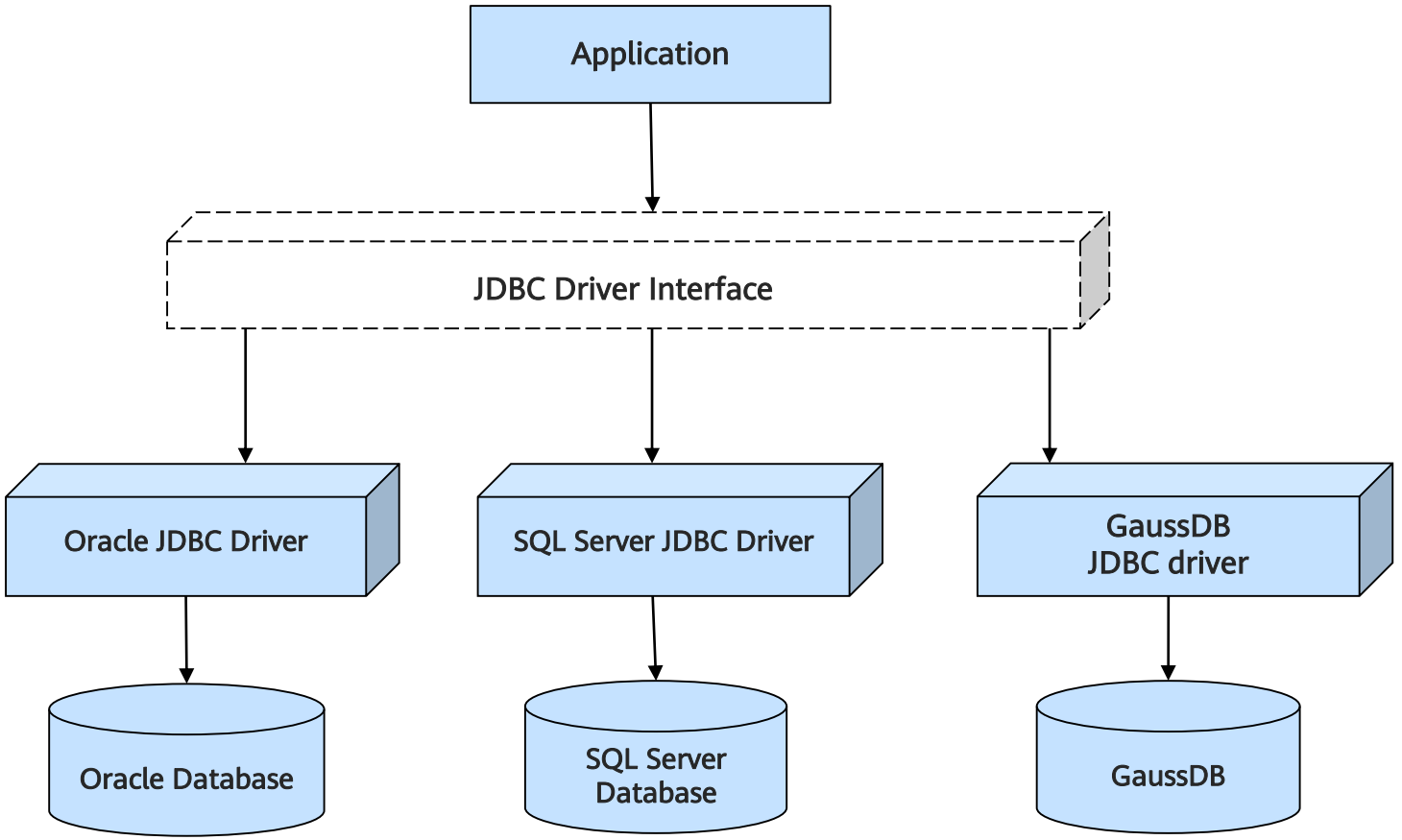
HUAWEI

Introduction to Database Drivers

A database driver provides a generic interface that translates the requests made by applications (such as Java and C applications) into a language that a database understands, so that applications can interact with the database in a standardized way. To meet different needs, openGauss supports both JDBC and ODBC drivers.



JDBC Architecture



Note: Since Java has good portability, there is no platform restrictions on JDBC.

HUAWEI

Common Data Types

Database Data Type	Java Data Type	Database Data Type	Java Data Type
VARCHAR(n) CHAR(n) NCHAR(n) text	java.lang.String	DOUBLE PRECISION, FLOAT8	double
"char"	byte	NUMERIC	java.math.BigDecimal
bytea	byte[]	DATE	java.sql.Timestamp
SMALLINT	short	TIME	java.sql.Time
INTEGER	int	TIMESTAMP	java.sql.Timestamp
BIGINT	long	CLOB	java.sql.Clob
FLOAT4, REAL	float	BLOB	java.sql.Blob
BOOLEAN	boolean		

JDBC Packages, Drivers, and Environments

- **JDBC packages**
  - openGauss provides two types of JDBC JAR packages: **postgresql.jar** and **openGauss-jdbc-*x.x.x*.jar**. The two types of JAR packages have the same functions and are used to resolve the JDBC driver package name conflict between openGauss and PostgreSQL.
  - Run the **build.sh** command in the source code directory on the Linux server to obtain the driver JAR packages **postgresql.jar** and **openGauss-jdbc-*x.x.x*.jar** in the **output** directory of the source code. You can also obtain the **openGauss-*x.x.x*-JDBC.tar.gz** package from the official website.
  - The driver package is compatible with PostgreSQL. The class name and structure of the driver are the same as those of the PostgreSQL driver. All applications running on PostgreSQL can be smoothly migrated to the current system.
- **Drivers**
  - openGauss is compatible with PostgreSQL in terms of JDBC usage. When the JDBC drivers of PostgreSQL and openGauss with the same name are loaded to the same JVM VM, the org.postgresql.Driver conflict may occur.
- **Environments**
  - JDK 1.8 must be configured on the client.

HUAWEI

Using the Username and Password as Parameters to Create a Connection

```
public static Connection getConnect(String username, String passwd)
   {
      String driver = "org.opengauss.Driver"; // Driver class
      String sourceURL = "jdbc:opengauss://10.10.0.13:8000/postgres";  // Database connection descriptor
      Connection conn = null;
      try
      {
         Class.forName(driver); // Load the driver.
      }
      catch( Exception e )
      {
         e.printStackTrace();
         return null;
      }
      try
      {
         conn = DriverManager.getConnection(sourceURL, username, passwd); // Create a connection.
         System.out.println("Connection succeed!");
      }
      catch(Exception e)
      {
         e.printStackTrace();
         return null;
      }
      return conn;
   };
```


HUAWEI

Using the Properties Object as a Parameter to Create a Connection

```
public static Connection getConnectUseProp(String username, String passwd)
   {
      String driver = "org.opengauss.Driver"; // Driver class
      String sourceURL = "jdbc:opengauss://10.10.0.13:8000/postgres?"; // Database connection descriptor
      Connection conn = null;
      Properties info = new Properties();
      try
      {
         Class.forName(driver);     // Load the driver.
      }
      catch( Exception e )
      {
         e.printStackTrace();
         return null;
      }
      try
      {
          info.setProperty("user", username);
          info.setProperty("password", passwd);
          conn = DriverManager.getConnection(sourceURL, info);   // Create a connection.
          System.out.println("Connection succeed!");
      }
      catch(Exception e)
      {
         e.printStackTrace();
         return null;
      }
      return conn;
   };
```


HUAWEI

Running a Common SQL Statement

- Applications operate database data by executing SQL statements (with no arguments).
- Create a statement object by calling the createStatement method in Connection.

```
Connection conn = DriverManager.getConnection("url","user","password");
Statement stmt = conn.createStatement();
```

- Run an SQL statement by calling the executeUpdate method in Statement.

```
int rc = stmt.executeUpdate("CREATE TABLE customer_t1(c_customer_sk INTEGER,
c_customer_name VARCHAR(32));");
```

- Close a statement object.

```
stmt.close();
```


HUAWEI

Running a Prepared SQL Statement

- Prepared statements were complied and optimized once, and can have additional parameters for different usage. Using prepared statements improves execution efficiency. If you want to run a statement for several times, use a prepared statement.
- Create a prepared statement object by calling the prepareStatement method in Connection.

```
PreparedStatement pstmt = con.prepareStatement("UPDATE customer_t1 SET c_customer_name = ? WHERE
c_customer_sk = 1");
```

- Set parameters by calling the setShort method in PreparedStatement.

```
pstmt.setShort(1, (short)2);
```

- Run a prepared statement by calling the executeUpdate method in PreparedStatement.

```
int rowcount = pstmt.executeUpdate();
```

- Close a prepared statement object by calling the close method in PreparedStatement.

```
pstmt.close();
```


HUAWEI

Calling a Stored Procedure

- Create a call statement object by calling the prepareCall method in Connection.

```
Connection myConn = DriverManager.getConnection("url","user","password");
 CallableStatement cstmt = myConn.prepareCall("{? = CALL TESTPROC(?,?,?)}");
```

- Set parameters by calling the setInt method in CallableStatement.

```
cstmt.setInt(2, 50);
cstmt.setInt(1, 20);
```

- Register an output parameter by calling the registerOutParameter method in CallableStatement.

```
cstmt.registerOutParameter(4, Types.INTEGER);  // Register an OUT parameter of the integer type.
```

- Call a stored procedure by calling the execute method in CallableStatement.

```
cstmt.execute();
```

- Obtain the output parameter by calling the getInt method in CallableStatement.

```
int out = cstmt.getInt(4);  // Obtain the OUT parameter.
```

- Close a call statement by calling the close method in CallableStatement.

```
cstmt.close();
```


HUAWEI

Batch Processing

- When a prepared statement batch processes multiple pieces of similar data, the database creates only one execution plan. This improves the compilation and optimization efficiency.
- Create a prepared statement object by calling the prepareStatement method in Connection.

```
Connection conn = DriverManager.getConnection("url","user","password");
 PreparedStatement pstmt = conn.prepareStatement("INSERT INTO customer_t1 VALUES (?)");
```

- Call the **setShort** parameter for each piece of data, and call **addBatch** to confirm that the setting is complete.

```
pstmt.setShort(1, (short)2);
```

- Perform batch processing by calling the executeBatch method in PreparedStatement.

```
int[] rowcount = pstmt.executeBatch();
```

- Close a prepared statement object by calling the close method in PreparedStatement.

```
pstmt.close();
```


HUAWEI

Processing Data in a Result Set

- Different types of result sets are applicable to different application scenarios. Applications select proper types of result sets as required. Before running an SQL statement, you must create a statement object. Some methods of creating statement objects can set the result set type.
- Create a Statement object. This object will generate a ResultSet object of the specified type and concurrency.

```
createStatement(int resultSetType, int resultSetConcurrency);
```

- Create a PreparedStatement object. This object will generate a ResultSet object of the specified type and concurrency.

```
prepareStatement(String sql, int resultSetType, int resultSetConcurrency);
```

- Create a CallableStatement object. This object will generate a ResultSet object of the specified type and concurrency.

```
prepareCall(String sql, int resultSetType, int resultSetConcurrency);
```

- After you complete required data operations in the database, close the database connection.

```
Connection conn = DriverManager.getConnection("url","user","password") ;
conn.close();
```


HUAWEI

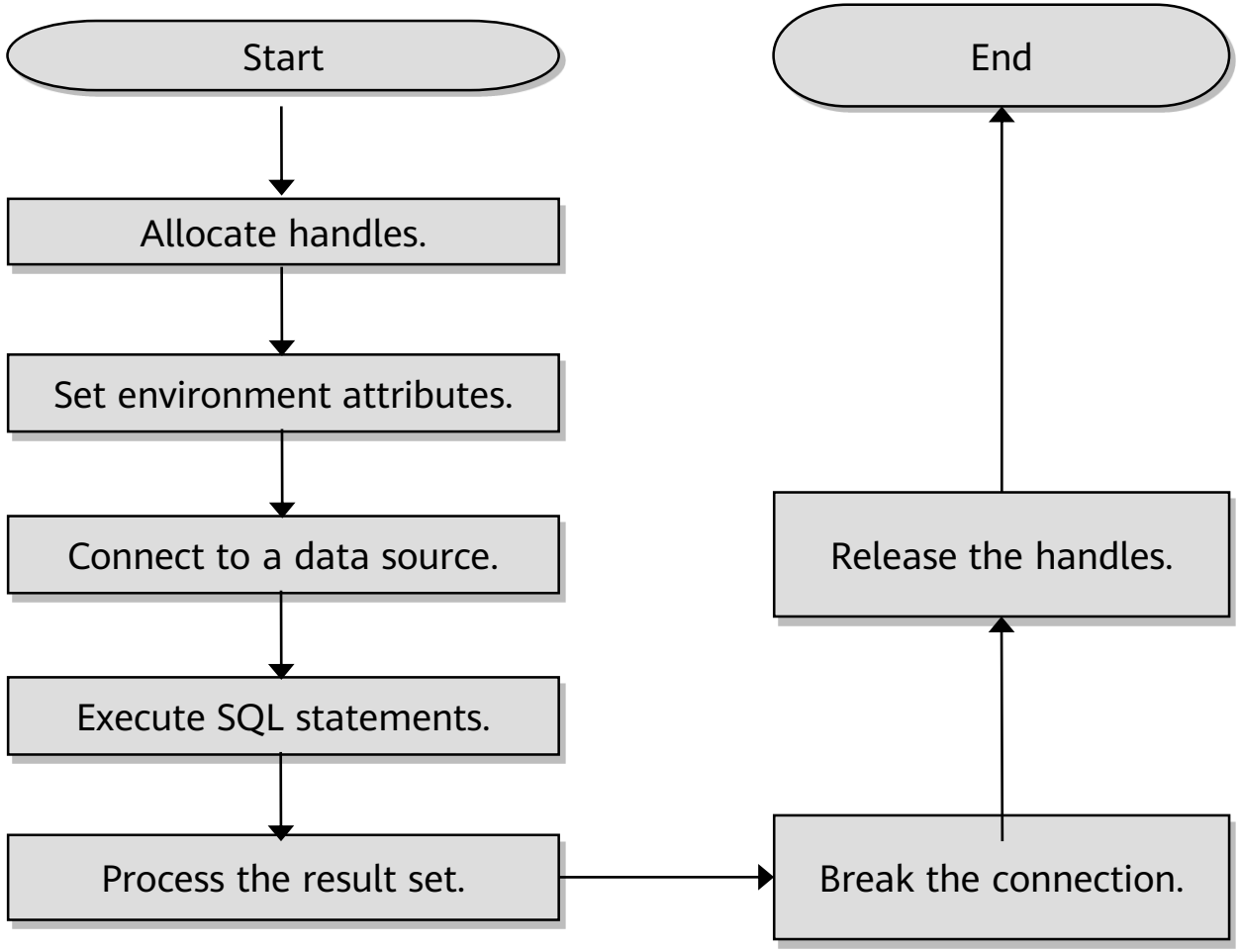
Contents

1. Advanced SQL Syntax
2. Stored Procedure and Trigger
3. **Application Development**
   - JDBC-based Development
   - ODBC-based Development
   - Psycopg-based Development

HUAWEI

Introduction to ODBC

Open Database Connectivity (ODBC) is a Microsoft API developed based on X/Open CLI. The ODBC API is for applications to access databases.



Basic ODBC Implementation Requirements on Linux

- Compilable C/C++ programs: These programs use the ODBC API to access databases.
- ODBC drivers: psqlodbc, Oracle ODBC, DB2 ODBC, and others. A driver is the implementation of the ODBC API. It allows applications to access and exchange data with databases.
- ODBC driver managers: open-source unixODBC and iODBC. A driver manager is the bridge between an ODBC application and the ODBC driver. It provides unified API definitions. During the execution, a driver manager loads a specific driver based on the data source type.

HUAWEI

ODBC Installation and Configuration (1)

- Install the ODBC driver manager.
  - Obtain the unixODBC source code package.
    - Download **unixODBC-2.3.7.tar.gz** or a later version from http://www.unixodbc.org/.
  - Install unixODBC.
    - During the compilation and installation, unixODBC is installed in the **/usr/local** directory by default. The data source file is generated in the **/usr/local/etc** directory, and the library file is generated in **/usr/local/lib**.

```
tar -zxvf openGauss-5.0.0-ODBC.tar.gz
cd unixODBC-2.3.7
./configure --enable-gui=no
make
make install
```


HUAWEI

ODBC Installation and Configuration (2)

- Replace the openGauss client driver.
  - Decompress the **openGauss-*x.x.x*-ODBC.tar.gz** package. After the decompression, the **lib** and **odbc** folders are generated. The **odbc** folder contains another **lib** folder. Copy the **psqlodbca.la**, **psqlodbca.so**, **psqlodbcw.la**, and **psqlodbcw.so** files from **/odbc/lib** to **/usr/local/lib**.
  - Copy the library in the **lib** directory obtained after decompressing **openGauss-*x.x.x*-ODBC.tar.gz** package to the **/usr/local/lib** directory.
- Configure a data source.
  - Add the following content to the **/usr/local/etc/odbcinst.ini** file:

```
[GaussMPP]
Driver64=/usr/local/lib/psqlodbcw.so
setup=/usr/local/lib/psqlodbcw.so
```


HUAWEI

ODBC Installation and Configuration (3)

- Configure the data source file.
  - Add the following content to the /usr/local/etc/odbc.ini file:

```
[MPPODBC]
Driver=GaussMPP
Servername=(Database server IP address)
Database =(Database name)
Username=(Database username)
Password=(User password for logging in to the database)
Port=(Database listening port)
Sslmode=allow
```

ODBC Installation and Configuration (4)

- Log in as the OS user **omm** to the primary node of the database.
- Add NIC IP addresses or host names which are separated by commas (,). The NICs and hosts are used to provide external services. In the following command, *NodeName* specifies the name of the current node.

```
gs_guc reload -N NodeName -I all -c "listen_addresses='*'"
```

- Add an authentication rule to the configuration file of the primary database node.

```
gs_guc reload -N all -I all -h "host all all  0.0.0.0/0 sha256"
```

- Restart openGauss.

```
gs_om -t stop
gs_om -t start
```


HUAWEI

ODBC Installation and Configuration (5)

- Configure the environment variables on the client.

```
#vim ~/.bashrc
Add the following information to the configuration file:
export LD_LIBRARY_PATH=/usr/local/lib/:$LD_LIBRARY_PATH
export ODBCSYSINI=/usr/local/etc
export ODBCINI=/usr/local/etc/odbc.ini
Run the following command to validate the setting:
#source ~/.bashrc
```

- Verify the data source configuration.
  - After the installation, the generated binary file is stored in the **/usr/bin** directory. You can run the **isql -v** *MPPODBC* command (*MPPODBC* is the data source name).
  - If the following information is displayed, the configuration is correct and the connection succeeds.

```
+---------------------------------------+
| Connected!                            |
|                                       |
| sql-statement                         |
| help [tablename]                      |
| quit                                  |
+---------------------------------------+
SQL>
```


HUAWEI

Contents

1. Advanced SQL Syntax
2. Stored Procedure and Trigger
3. **Application Development**
   - JDBC-based Development
   - ODBC-based Development
   - Psycopg-based Development

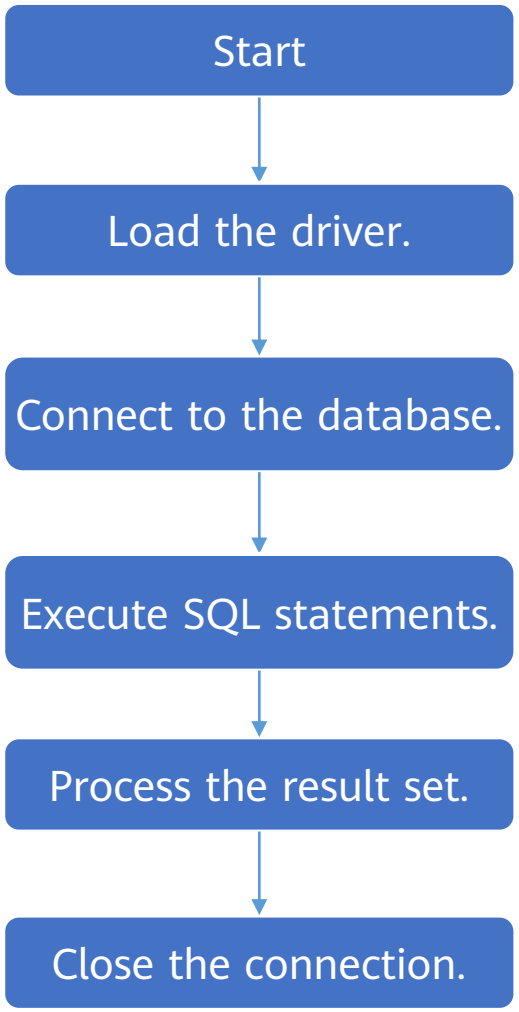
HUAWEI

Python

- Python is an interpretative language that saves a lot of program development time because it does not require compilation and connection. Interpreters can be used interactively so that you can easily experiment with the various features of the language, write programs that are used only once, or test functions in bottom-up program development. It is also a convenient calculator.
- Python is an object-oriented and straightforward computer programming language. It is also a powerful and comprehensive general-purpose language. It has been developed for more than 10 years and is mature and stable. This language features simple and clear syntax, is suitable for completing various high-level tasks, and can run in almost all operating systems. At present, related technologies based on this language are developing rapidly, the number of users increases sharply, and there are a large number of related resources.
- Mainstream operating systems that Python can be used include: Windows, Linux/Unix, and MacOS.

HUAWEI

Introduction to the Psycopg Driver Package



- Psycopg is a Python API used to execute SQL statements and provides a unified access API for PostgreSQL and openGauss. Applications can perform data operations based on psycopg.
- Psycopg2 is the encapsulation of libpq and is implemented using the C language, which is efficient and secure. It provides client-side and server-side cursors, asynchronous communication and notification, and COPY TO and COPY FROM functions.
- Psycopg2 adapts to PostgreSQL data types. Through the flexible object adaptation system, you can extend and customize APIs.
- openGauss supports the psycopg2 feature and allows psycopg2 to be connected in SSL mode.

Loading the Driver (1)

- Download the compiled psycopg2 package from the openGauss official website (https://opengauss.org/zh/download/). Pay attention to the corresponding operating system and openGauss database version.



HUAWEI

Loading the Driver (2)

- Decompress the driver package of the corresponding version and copy psycopg2 to the **site-packages** folder in the Python installation directory as user **root**.
- Change the **psycopg2** directory permission to **755**.
- Add the **psycopg2** directory to the environment variable *$PYTHONPATH* and validate it.
- For non-database users, configure the **lib** directory in *LD_LIBRARY_PATH* after decompression.
- Load the database driver before creating a database connection.

```
import psycopg2
```


HUAWEI

Connecting to the Database

- Use the psycopg2.connect function to obtain a connection object.
- Use the connection object to create a cursor object.
- The following Python code shows how to connect to an existing database. If the database does not exist, it will be automatically created and a database object will be returned.

```
#!/usr/bin/python
import psycopg2
conn = psycopg2.connect(database="testdb", user="openGauss", password="xxxxxxxx",
host="127.0.0.1", port="26000")
```


HUAWEI

Executing SQL Statements and Processing the Result Set

- Execute SQL statements.
  - Construct an operation statement and use %s as a placeholder. During execution, psycopg2 will replace the placeholder with the parameter value. You can add the RETURNING clause to obtain the automatically generated column values.
  - The cursor.execute method is used to perform operations on one row, and the cursor.executemany method is used to perform operations on multiple rows.
- Process the result set.
  - cursor.fetchone(): fetches the next row in a query result set and returns a sequence. If no data is available, NULL is returned.
  - cursor.fetchall(): fetches all (remaining) rows in the query result and returns a list. An empty list is returned when no rows are available.
- Close the connection.
  - After you complete required data operations in the database, close the database connection. Call a close method such as connection.close() to close the connection.

HUAWEI

Example

```
import psycopg2
# Common connection modes of psycopg2
1. conn = psycopg2.connect(dbname="postgres", user="user", password="password", host="localhost", port=port)
2. conn = psycopg2.connect("dbname=postgres user=user password=password  host=localhost port=port")
# Create a connection object.
conn=psycopg2.connect(database="postgres",user="user",password="password",host="localhost",port=port)
cur=conn.cursor() # Create a pointer object.
# Create a table.
cur.execute("CREATE TABLE student(id integer,name varchar,sex varchar);")
# Insert data.
cur.execute("INSERT INTO student(id,name,sex) VALUES(%s,%s,%s)",(1,'Aspirin','M'))
cur.execute("INSERT INTO student(id,name,sex) VALUES(%s,%s,%s)",(2,'Taxol','F'))
cur.execute("INSERT INTO student(id,name,sex) VALUES(%s,%s,%s)",(3,'Dixheral','M'))
# Obtain the result.
cur.execute('SELECT * FROM student')
results=cur.fetchall()
print(results)
# Close the connection.
conn.commit()
cur.close()
conn.close()
```


HUAWEI

Quiz

1. Which of the following are numerical calculation functions? (Multi-choice)
   A. length(str)
   B. sin(n)
   C. trunc(number, scale)
   D. md5(string)
2. Which of the following values is returned by the nvl(*expr1*, *expr2*) function when *expr1* is NULL?
   A. exp1
   B. exp2
   C. exp3
   D. exp1 + exp2

HUAWEI

Summary

- This chapter further describes the use of SQL-related advanced syntax, API-based development, stored procedures, and triggers. Advanced syntax is introduced in terms of constraints, JOIN, batch processing, TRUNCATE, subqueries, and VACUUM operations. In addition, the concepts, syntax, and applications of stored procedures and triggers are provided. Application development mainly introduces common development based on JDBC, ODBC, and Psycopy.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/zh/
- openGauss Help (5.0.0)
  - https://docs.opengauss.org/zh/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

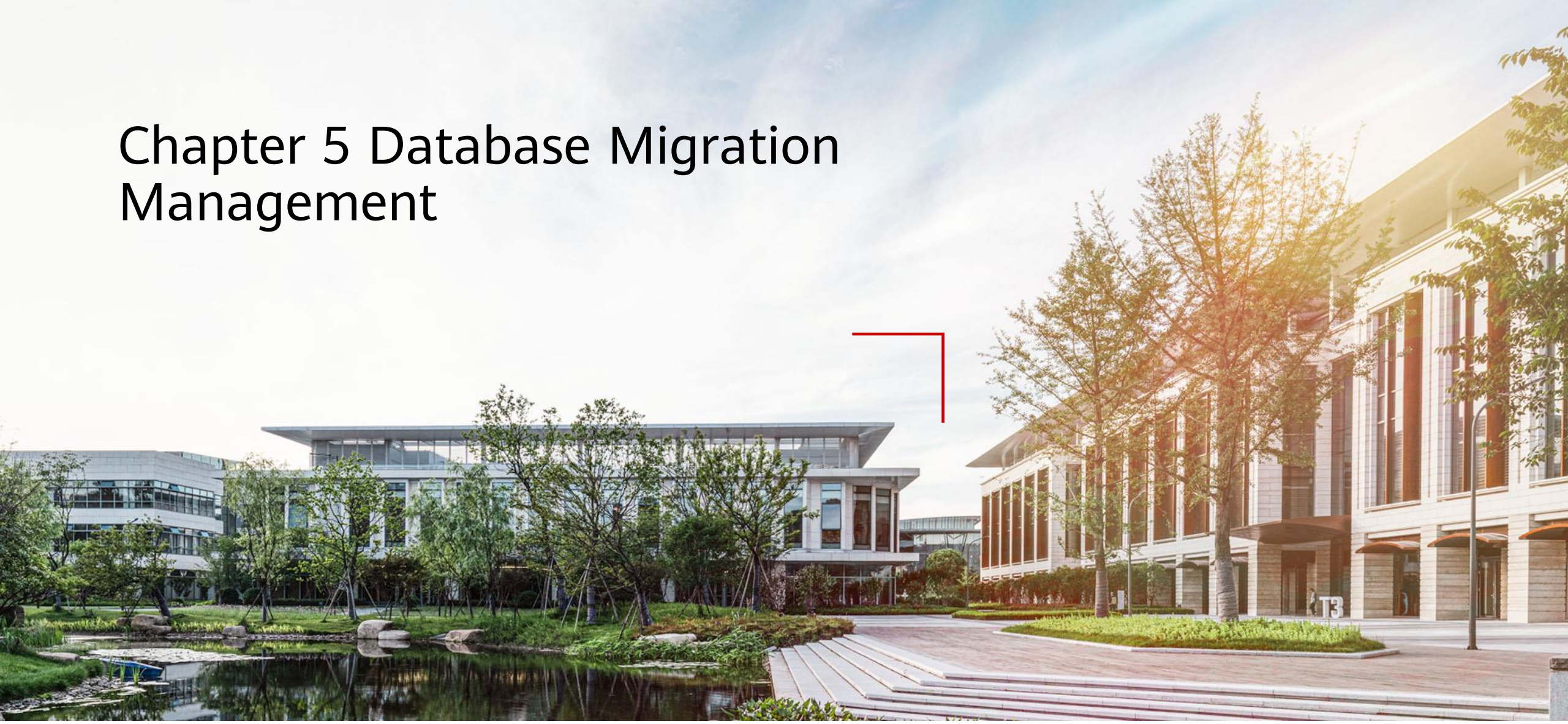
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 5 Database Migration Management



HUAWEI

Foreword

- This chapter provides an overview of data migration. Then it describes the differences between the heterogeneous databases openGauss and MySQL in terms of data types and syntaxes. Moreover, it focuses on the implementation principles and usage of the openGauss migration tools, including the compatibility evaluation, full migration, incremental migration, full verification, incremental verification, reverse incremental migration, and reverse full migration. Finally, it describes the openGauss one-stop migration platform DataKit.

HUAWEI

Objectives

- Upon completion of this course, you will be able to:
  - Understand the basic procedure of data migration.
  - Understand the syntax differences between openGauss and MySQL.
  - Learn the implementation principles of the openGauss migration tools.
  - Master how to use the openGauss migration tools.
  - Be familiar with the openGauss integrated migration platform DataKit.

HUAWEI

Contents

1. **Data Migration Overview**
2. Syntax Differences Between openGauss and MySQL
3. openGauss Migration Tool Set
   - Compatibility Evaluation Tools
   - Full Migration and Incremental Migration
   - Full Verification and Incremental Verification
   - Reverse Incremental Migration and Reverse Full Migration
   - Migration Portal and One-Stop Migration Platform DataKit

HUAWEI

Data Migration Overview

- Data migration refers to migrating data from one database to another, which may be migration between homogeneous databases or between heterogeneous databases.
- Based on the data flow direction, data migration is classified into data export and data import. Generally, there are intermediate files, such as SQL files or CSV files. The intermediate files can be stored on a disk and imported to the target database as required, so that data export and import can be decoupled.
- Data migration is classified into cold migration and hot migration based on whether applications remain running. Cold migration is easy to perform, but services need to be stopped. Therefore, cold migration is not applicable to real-time online services. Hot migration has little impact on user services and is unaware of service applications. However, the migration process is complex.
- Homogeneous database migration is relatively simple. Backup and restoration technologies (logical backup and restoration and physical backup and restoration) can be used to implement migration.
- Heterogeneous database migration is complex. You need to consider the compatibility, models, application reconstruction costs, migration costs, and performance of heterogeneous databases.
- This course describes hot migration between heterogeneous databases (MySQL and openGauss).

HUAWEI

Five Steps of Data Migration



Current capabilities

Migration evaluation	Application adaptation	Data migration	Trial run	Production cutover
Compatibility evaluation tools • Use a tool to evaluate the compatibility of the kernel syntaxes. • Use database survey tables to survey the deployment architecture, typical services, data capacity, functions, and performance.	**Compatibility** • Be compatible with 475+ syntaxes in total (100% of common syntaxes and 80% of full syntaxes), including common syntaxes such as unsigned integers, ENUM and SET data types, time data types, auto-increment columns, JSON functions, and REPLACE INTO.	**Migration tools** • gs_mysync: fully migrates data and objects such as tables, indexes, and stored procedures. • gs_replicate: supports DDL incremental migration for table creation and modification and DML incremental migration. • gs_datacheck: verifies data in real time based on the Merkle tree and supports full and incremental data verification.	**Monitoring and diagnosis** • Fault diagnosis: log analysis and slow SQL diagnosis. • Database monitoring: database metric monitoring in more than 100 views.	**Reverse migration and monitoring tools** • Reverse migration tools support the migration from openGauss to MySQL, meeting the escape requirements after the migration. • Monitoring tools are used to check whether database metrics are normal.

Planning enhancements

Migration evaluation	Application adaptation	Data migration	Trial run	Production cutover
Migration evaluation • Use tools to evaluate MySQL capacity, deployment architecture, and load and generate evaluation reports.	**Compatibility** • Improve compatibility to migrate applications without modification. • Be fully compatible with common MySQL protocols and seamlessly replace the JDBC.	**Migration tools** • Support resumable data transfer of incremental migration, improving migration reliability. • Enhance the usability and monitoring capability of the tools and support the integrated tool platform DataKit.	**Traffic mirroring** • Fault diagnosis: openGauss functions as the shadow library, and the MySQL traffic is equivalently distributed to openGauss.	**Reverse migration and monitoring tools** • Reverse migration supports resumable data transfer and full migration, improving migration reliability.

HUAWEI

Migration Evaluation (1)

- Migration evaluation tools provide suggestions on selecting and deploying the target database based on the source database profiling for evaluating the compatibility of the target database and determining the application redevelopment workload and migration duration.
- Source database profiling contributes to comprehensive understanding of source databases and provides basis and guidance for subsequent model selection evaluation and data migration.
  - **System-level profiling:** collects system-level information, such as hardware (CPUs, memory, network, and disks), OS (kernel, parameters, and security policies), and performance (system load in 24 hours during peak hours).
  - **Instance-level profiling:** collects instance-level information, such as architecture (standalone system/cluster [primary and standby, primary and secondary, cross-AZ deployment, and remote deployment] and version), parameters (database parameters), data scale (database quantity and size, and space usage of tables and indexes), running status information (concurrency of database instances, QPS, TPS, sessions, transactions, slow SQL statements, and wait events), and resource usage (system CPUs, memory, network I/Os, and disk I/Os)
  - **Object-level profiling:** collects object-level information, such as structure information (tables, partitions, shards, indexes, views, sequences, stored procedures, and user-defined functions), statistics, access characteristics (read/write ratio and frequency), warnings (large tables, complex structures, DB Links, and foreign tables), and special types (BLOB, array, and user-defined types)
  - **Statement-level profiling:** includes SQL text (full), execution characteristics (such as number of times and response time), execution plans, and hotspots (such as frequently accessed objects and frequently executed SQL statements/stored procedures).
  - **Application-level profiling:** includes computes text (such as stored procedures, triggers, and functions) and execution characteristics (such as number of times and response time).

HUAWEI

Migration Evaluation (2)

- Use the application profiling to collect application architectures, relationships between applications and databases, and application access characteristics to lay a solid foundation for application redevelopment.
- Based on the source database profiling, determine the target database model and deployment suggestions, including hardware resources, deployment modes, and GUC parameters.
- Evaluate the compatibility of the target database, including object compatibility and SQL compatibility, provide application redevelopment suggestions, and evaluate the redevelopment workload.
- Evaluate the migration duration and costs based on the source database profiling and target database compatibility.

HUAWEI

Application Adaptation

- Based on the migration evaluation result, redevelop and adapt to incompatible SQL statements in applications.
- Focuses of redevelopment and adaptation:
  - Check whether the precisions and maximum lengths of data types are consistent.
  - Check whether the built-in functions contained in the service exist in the target database and whether the functions are consistent.
  - Check whether the database SQL dialects are included.
  - Check whether keywords and reserved words are consistent, and whether identifiers, such as the names of tables, columns, indexes, views, and stored procedures, contain keywords and reserved words.
  - Check whether the case sensitivity is consistent, whether the names of tables, columns, and indexes are case-sensitive, and whether index names must be unique.
- After application redevelopment, manual or automatic test cases need to be added for full self-check to ensure that functions are available.

HUAWEI

Data Migration

- **Network deployment:** Deploy the database network based on the hardware resources, deployment modes, and GUC parameters confirmed in the migration evaluation.
- **Pre-check:** Check the read and write permissions of the database, network connection of the database, and pre-installation parameter settings of the source and target databases.
- **Full data migration:** Migrate all objects (tables, data, indexes, views, functions, stored procedures, and triggers) to the target database.
- **Full data verification:** Verify table data and object definitions, generate verification reports, and provide rectification suggestions for inconsistent objects.
- **Incremental data migration:** Migrate incremental data (DDL and DML) during migration.
- **Incremental data verification:** Periodically verify incremental data (DDL and DML) and provide rectification suggestions for inconsistent records.

HUAWEI

Trial Run

- Migrate traffic to the target database, test services on the target database in actual environment, rectify faults, and optimize performance.
- Check before trial run.
  - Check the synchronization delay and incremental logs on the target database.
  - Synchronize incremental data between the source and target databases.
  - Stop forward incremental data migration.
  - Delete temporary database objects.
- Traffic mirroring: Mirror the service traffic attached with service process to the target database.
- Fault rectification: Monitor the running status of the target database and rectify faults.
- Performance optimization: Monitor the running status of the target database, improve performance, and optimize slow SQL statements.

HUAWEI

Production Cutover

- Start reverse migration, deploy services on the target database, and start applications.
- Use reverse migration as a data escape channel to ensure data consistency between the source and target databases. If the target database is faulty, applications can be switched back to the source database in a timely manner to ensure normal data rollback and avoid impact on services.
- Use monitoring tools to monitor database metrics and detect service running exceptions in a timely manner.
- After services run stably on the target database for a period of time, take the source database offline so that services can be completely applied to the target database.

HUAWEI

Contents

1. Data Migration Overview
2. **Syntax Differences Between openGauss and MySQL**
3. openGauss Migration Tool Set
   - Compatibility Evaluation Tools
   - Full Migration and Incremental Migration
   - Full Verification and Incremental Verification
   - Reverse Incremental Migration and Reverse Full Migration
   - Migration Portal and One-Stop Migration Platform DataKit

HUAWEI

Comparison Between openGauss and MySQL (1)

Dimension	Feature	MySQL	openGauss
High performance	TPCC	2P Kunpeng 540,000 tpmC and 4P Kunpeng 720,000 tpmC	2P Kunpeng 1,500,000 tpmC and 4P Kunpeng 2,300,000 tpmC
	Sysbench	In the environment with 4-core CPUs and 16 GB memory, the read-only test is performed at 43,000 qps and the read/write test is performed at 32,000 qps under 16 concurrent connections.	In the environment with 4-core CPUs and 16 GB memory, the read-only test is performed at 50,000 qps and the read/write test is performed at 44,000 qps under 16 concurrent connections.
High availability	Primary-standby synchronization	Binlog-based logical replication (asynchronous/semi-synchronous) is flexible but prone to delay and data inconsistency.	Physical replication and logical replication are implemented based on the same WAL.
	Primary/Standby failover/switchover RTO	RPO > 0	RPO = 0
		Cross-AZ RTO > 30s	Intra-AZ RTO < 10s; cross-AZ RTO < 30s
Enterprise-level capabilities	SQL standards	Many syntaxes do not meet SQL standards and behaviors are inconsistent, such as the processing of single quotation marks and UPDATE with expressions.	SQL92, SQL99, SQL2003, and SQL2011 specifications are supported.
	Parallel query	Versions earlier than 8.0 do not support parallel query. MySQL 8.0 only supports parallel clustered index read (such as count(*)) and parallel partition scans.	Parallel queries (sequential scans, nested loop, hash join, hash agg, append, etc.) are supported.
	Multiple storage engines	Multiple storage engines are supported. In a MySQL process, tables of multiple engines can be created. In versions later than 5.5, InnoDB engine uses index-organized table. Data in the table is organized by index. Querying data by primary key is fast, and the capacity of a single table is limited, full table scans are slow.	Multiple storage engines can coexist. Data is stored in heap tables, and the capacity of a single table is not limited.
	OLAP	HeatWave is used, which is a high-performance Oracle cloud service that analyzes computing architecture.	Column store is supported. In addition, the TPCH performance of row-store tables is better than that in MySQL.

HUAWEI

Comparison Between openGauss and MySQL (2)

Dimension	Feature	MySQL	openGauss
High security	SM algorithms	Not supported. **caching_sha2_password** is used for authentication. Common encryption and decryption algorithms are supported.	Supports SM3 authentication and SM4 encryption and decryption algorithms, MD5 and SHA-256 authentication algorithms, and MD5 and AES-128 encryption and decryption algorithms.
	Full encryption	Not supported. Only specified fields can be encrypted using the AES_ENCRYPT encryption function and transferred to the database.	Supports full encryption. Data is encrypted and stored on the database server. Ciphertext data can be retrieved and calculated.
Ecosystem migration	Tool ecosystem	Supports various ecosystems, including graphical management tools, multi-language connection drivers, HA components, middleware, and backup and restoration components.	Supports ODBC, JDBC, and Python drivers, native HA component CM, and backup and restoration tools.
	Compatibility with MySQL	1100+ syntaxes such as data types, operators, functions, DDL, and DML.	The overall compatibility of openGauss is 80%, and the compatibility of common syntaxes is 100%.
	Migration tool suite		Supports complete full, incremental, and reverse data migration tools and real-time data comparison.

HUAWEI

Syntax Differences Between openGauss and MySQL - Compatibility Mode (1)

- When creating a database on openGauss, you can specify the compatibility mode.

```
create database database_name with dbcompatibility = compatibility_type;
```

- The value of **compatibility_type** can be **A**, **B**, **C**, or **PG**, indicating O-, MY-, TD-, and POSTGRES databases, respectively. The default value is **A**.
- B-compatible databases can be connected only after Dolphin plug-ins are loaded.
  - A-compatible databases treat empty strings as NULL, and the data type is DATE.
- The data type will be replaced by TIMESTAMP(0) WITHOUT TIME ZONE.
  - When a character string is converted to an integer, if the input is invalid, the input will be converted to **0** due to B compatibility, and an error will be reported due to other compatibility issues.

HUAWEI

Syntax Differences Between openGauss and MySQL - Compatibility Mode (2)

```
openGauss=# create database database_a;
CREATE DATABASE
openGauss=# \q
[douxin@ecs-6ac8 ~]$ gsql -d database_a -p 35432 -r
gsql ((openGauss 5.1.0 build db2a3a72) compiled at 2023-09-12 00:10:08 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

database_a=# show sql_compatibility ;
 sql_compatibility
-------------------
 A
(1 row)
```


HUAWEI

Syntax Differences Between openGauss and MySQL - Compatibility Mode (3)

```
openGauss=# create database database_b with dbcompatibility = 'B';
CREATE DATABASE
openGauss=# \q
[douxin@ecs-6ac8 ~]$ gsql -d database_b -p 35432 -r
gsql ((openGauss 5.1.0 build db2a3a72) compiled at 2023-09-12 00:10:08 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

database_b=# show sql_compatibility ;
 sql_compatibility
-------------------
 B
(1 row)

database_b=# \dx
                                  List of installed extensions
      Name       | Version |   Schema   |                            Description
-----------------+---------+------------+-------------------------------------------------------------------
 dblink          | 1.0     | public     | connect to other PostgreSQL databases from within a database
 dist_fdw        | 1.0     | pg_catalog | foreign-data wrapper for distfs access
 dolphin         | 2.0     | public     | sql engine
 file_fdw        | 1.0     | pg_catalog | foreign-data wrapper for flat file access
 hstore          | 1.1     | pg_catalog | data type for storing sets of (key, value) pairs
 log_fdw         | 1.0     | pg_catalog | Foreign Data Wrapper for accessing logging data
 mot_fdw         | 1.0     | pg_catalog | foreign-data wrapper for MOT access
 plpgsql         | 1.0     | pg_catalog | PL/pgSQL procedural language
 security_plugin | 1.0     | pg_catalog | provides security functionality
(9 rows)
```


HUAWEI

Syntax Differences Between openGauss and MySQL - Compatibility Mode (4)

- openGauss provides Dolphin extensions. Dolphin extensions are MySQL-compatible database extensions of openGauss (**dbcompatibility** is set to **'B'**), compatible with MySQL in terms of keywords, data types, constants and macros, functions and operators, expressions, type conversion, DDL/DML/DCL syntax, stored procedures/user-defined functions, and system views.
- The database installation package downloaded from the openGauss official website automatically loads the Dolphin plug-ins when a B-compatible database is connected for the first time. You do not need to manually install and load the plug-ins.
- If you need to manually compile and load the plug-ins, you can use the compilation and installation method or OM installation method.
  - Compilation and installation
    - Compile and install openGauss.
    - Copy the Dolphin source code to the **contrib** directory of the openGauss-server source code.
    - Go to the **dolphin** directory and run **make install**.
    - Create a B-compatible database and connect to the B-compatible database as the initial user.
  - OM installation
    - Install openGauss by using OM method.
    - Copy the files required by the plug-in.

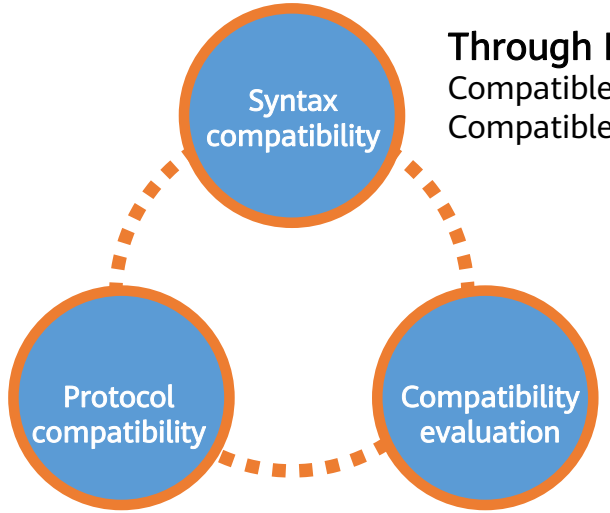
      dolphin.so (path: **app/lib/postgresql**)

      dolphin.control and **dolphin–1.0.sql** (path: **app/share/postgresql/Extension**)
    - Create a B-compatible database and connect to the B-compatible database as the initial user.

HUAWEI

Syntax Differences Between openGauss and MySQL - Dolphin Plug-ins

- In B-compatible mode, openGauss supports most MySQL syntaxes and functions through the kernel and Dolphin plug-ins, achieving 100% compatible with common syntaxes and 80% compatible with full syntaxes in MySQL.
- openGauss implements MySQL protocol compatibility in the Dolphin plug-ins through abstract protocol layer APIs. After setting related parameters, you can directly connect to openGauss through the JDBC driver or the command line client of MySQL.
- The kernel and plug-ins are jointly used to make openGauss syntaxes compatible with those of MySQL. In addition, a complete set of migration tools are provided to ensure a complete migration path from MySQL to openGauss and almost zero modification of applications.



Through Dolphin plug-ins and kernel
Compatible with 100% common syntaxes.
Compatible with 80% full syntaxes.

Through Dolphin plug-ins
The MySQL client and JDBC driver can directly access openGauss, and no application modification is required.

Compatibility evaluation of heterogeneous databases in openGauss
Reduce evaluation labor costs and improve compatibility evaluation efficiency and accuracy.

Syntax Differences Between openGauss and MySQL - Syntax Compatibility Overview (1)

Type	MySQL	Compatible in openGauss	Incompatible in openGauss	Remarks
Data type	61	56	5	Geographic and geometric data types
Operator	49	49	-	All compatible
System function	397	290	107	Geographic functions, some encryption and decryption functions (for security reasons), and primary- and standby-related functions
Expression	72	69	3	The stored procedure that contains only SELECT cannot be called by running **call**. Specifying the collation of expressions is not supported. The {identifier expr} expression is not supported.
Keyword	Reserved: 237 Non-reserved: 385	Reserved: 140 Non-reserved: 588	-	openGauss: distinguishes non-reserved keywords, non-reserved keywords (cannot be functions or types), reserved keywords (can be functions or types), and reserved keywords MySQL: only distinguishes reserved keywords and non-reserved keywords. Reserved keywords must not be used as other identifiers. Non-reserved keywords have special meanings only in a specific environment and can be used as identifiers in other environments.

HUAWEI

Syntax Differences Between openGauss and MySQL - Syntax Compatibility Overview (2)

Type		MySQL	Compatible in openGauss	Incompatible in openGauss	Remarks
User-defined variable		3	3	-	All compatible
SQL	DDL	205	182	23	ALTER TABLE (that specifies options such as **ALGORITHM** and **LOCK**), DISCARD, COALESCE, and UPGRADE (used for partitioned tables), some full-text index syntaxes, CHECK TABLE, and REPAIR TABLE.
	DML	49	45	4	DELETE IGNORE function, LOW_PRIORITY function in the IUD scenario, and HANDLER syntax.
	DCL	241	124	117	PURGE, RESET MASTER, CREATE/ALTER/DROP USER and GRANT/REVOKE that involve SSL certificates (for security reasons), SHOW ENGINE/PROFILE/PROFILES, most FLUSH syntaxes (for storage), and LOAD INDEX.
PL/SQL		115	65	50	HANDLER
SQL MODE		-	-	-	STRICT_TRANS_TABLES (Only the strict mode is supported), ONLY_FULL_GROUP_BY (The column of GROUP BY is not in the target list), pipes_as_concat (pipe function control), ansi_quotes (double quotation mark function control), no_zero_date (date can be all 0s), and pad_char_to_full_length (0s are automatically padded for the CHAR type).
Protocol compatibility		22	12	10	Protocol types such as auth method change, SSL Handshake, COM_STATISTICS, COM_RESET_CONNECTION, COM_DEBUG, COM_CHANGE_USER, COM_SET_OPTION, COM_BINLOG_DUMP, and Multi-Resultset are not supported.
Others		-	-	-	Fields are case-sensitive, delimiters and identifiers are encapsulated using back quotations, and character strings are escaped using backslashes (\).

HUAWEI

Syntax Differences Between openGauss and MySQL - Data Types

No.	Item	openGauss	MySQL	Compatible	Remarks
1	Set the data type to **0**, that is, ZEROFILL.	The integer type supports ZEROFILL, which is equivalent to the UNSIGNED attribute, but does not support the function of padding 0s.	The integer and decimal types support ZEROFILL, which is equivalent to the UNSIGNED attribute. Whether 0s can be padded depends on the client implementation.	Yes	In openGauss, the integer type supports ZEROFILL, the decimal type does not support ZEROFILL, and 0s cannot be padded.
2	Data types that support UNSIGNED	The integer type supports UNSIGNED.	The integer and decimal types support UNSIGNED.	Yes	In openGauss, the integer type supports UNSIGNED, but the decimal type does not support UNSIGNED.
3	BIT[(M)]	The value of *M* ranges from 1 to 83886080. If *M* is not specified, the default value **1** is used.	The value of *M* ranges from 1 to 64. If *M* is not specified, the default value **1** is used.	Yes	
4	TINYINT[(M)] [UNSIGNED] [ZEROFILL]	The modifier (*M*) has no actual function. ZEROFILL is equivalent to UNSIGNED and does not have the function of padding 0s. The value of **TINYINT** ranges from –128 to +127. The value of **TINYINT UNSIGNED** ranges from 0 to 255.	The modifier (*M*) is used together with ZEROFILL to display a specified number of digits (*M*) on the MySQL client (not effective on the JDBC driver). ZEROFILL is equivalent to UNSIGNED. Whether 0s can be padded depends on the client implementation. The value of **TINYINT** ranges from –128 to +127. The value of **TINYINT UNSIGNED** ranges from 0 to 255.	Yes	In openGauss: 1. ZEROFILL does not support zero padding. 2. The (*M*) modifier has no actual effect.
5	BOOL and BOOLEAN	The syntax supports **false** and **true**. If a non-zero value is entered, the value **1** is saved as **true**. Otherwise, the value **false** is saved.	It is equivalent to tinyint(1). The syntax supports **false** and **true**. Actually, **false** indicates **0**, and **true** indicates **1**. The value ranges from –128 to 127. The actual number is saved.	Yes	The BOOL value of MySQL actually refers to a number, and the BOOL value of openGauss refers to only **t** or **f**. In the JDBC scenario, when openGauss returns the BOOL value, **t** is converted to **1** and **f** is converted to **0** before the BOOL value is returned.
6	linestring	Not supported.	Curve with linear interpolation between points. Format: LINESTRING(0 0, 10 10, 20 25, 50 60).	No	Redevelopment suggestion: Use postgis+geometry instead.
7	SPATIAL INDEX()	Not supported.	Creates a spatial index using the MBR of a geometric object.	No	Redevelopment suggestion: Use the GiST index instead.
8	multipoint	Not supported.	Set of multiple points. The format is MULTIPOINT(0 0, 20 20, 60 60).	No	Redevelopment suggestion: Use postgis+geometry instead.

HUAWEI

Syntax Differences Between openGauss and MySQL - Operators

No.	Item	openGauss	MySQL	Compatible	Remarks
1	/	Division	Division	Yes	In some MySQL scenarios, a value can be divided by 0. (This feature is controlled by the **ERROR_FOR_DIVISION_BY_ZERO** and **strict** modes in **sql_mode** and is related to the operator to be executed.) openGauss does not support division by 0. If a value is divided by 0, an error is reported.
2	^	XOR	XOR	Yes	The function is the same as that in MySQL only after **dolphin.b_compatibility_mode** is set to **on**. If this parameter is disabled, exponentiation is used. If you want to use exponentiation after enabling this parameter, use the power function.
3	and, &&	AND logic	AND logic	Yes	The function of && is the same as that in MySQL only after **dolphin.b_compatibility_mode** is set to **on**.
4	div	Division operator	Division operator	Yes	In some MySQL scenarios, a value can be divided by 0. (This feature is controlled by the **ERROR_FOR_DIVISION_BY_ZERO** and **strict** modes in **sql_mode** and is related to the operator to be executed.) openGauss does not support division by 0. If a value is divided by 0, an error is reported.
5	expr LIKE pat [ESCAPE 'escape_char']	String matching operator	String matching operator	Yes	The LIKE operator of openGauss is case-sensitive by default. If **dolphin.b_compatibility_mode** is set to **on**, LIKE is case-insensitive. You can also directly use the ILIKE operator to ignore case.
6	NOT, !	Negation logic	Negation logic	Yes	In openGauss, the exclamation mark (!) refers to factorial default, with the format as "*integer*!". If **dolphin.b_compatibility_mode** is set to **on**, an exclamation mark (!) indicates negation.
7	expr NOT LIKE pat [ESCAPE 'escape_char']	LIKE negation	LIKE negation	Yes	The NOT LIKE operator of openGauss is case-sensitive by default. If **dolphin.b_compatibility_mode** is set to **on**, LIKE is case-insensitive. You can also directly use the NOT ILIKE operator to ignore case.
8	OR, ||	OR logic	OR logic	Yes	In openGauss, || is a string connector by default. When **dolphin.sql_mode** contains **pipes_as_concat**, || indicates a string connector. If **pipes_as_concat** is not contained, || indicates the OR operator. || is widely used as a string connector in openGauss kernel and tools. You are advised to use OR to represent the OR logic. If **pipes_as_concat** is not included, errors may be reported for some tool commands (such as the meta-command **\d** of gsql).

Syntax Differences Between openGauss and MySQL - System Functions (1)

No.	Item	openGauss	MySQL	Compatible	Remarks
1	Coalesce (value,...)	Returns the first value of its parameters that is not **NULL**. If all options in the parameter list are equal to **NULL**, **NULL** is returned. All parameter types must be the same or the ones that can be implicitly converted. For example, all parameters must be of the numeric or string type. Otherwise, an error is reported. At least one parameter is required.	Returns the first value of its parameters that is not **NULL**. If all options in the parameter list are equal to **NULL**, **NULL** is returned. The parameter can be of any type. At least one parameter is required.	Yes	In openGauss, an error is reported when parameter types are inconsistent.
2	date_format (date,format)	date_format(expr, format) Description: The **expr** parameter indicates the input date or datetime format. This function formats the **expr** parameter based on the **format** parameter. Return type: text	Formats the value of **date** based on the format string. If any parameter is set to **NULL**, **NULL** is returned.	Yes	This function is compatible with the strict and non-strict modes during MySQL table insertion.
3	Interval (N,N1,N2,N3, ...)	Compares **N** with the rest of these parameters one by one until **N***k* is greater than **N** and returns the previous subscript, that is, *k* minus 1. If the parameters following **N** are all smaller than **N**, the ordinal number *n* of the last parameter is returned. If **N** is **NULL**, **–1** is returned. If the parameter is not a number: 1. BOOL: **true** is converted to **1**, and **false** is converted to **0**. 2. If the value of the parameter can be truncated in the format of float8, truncate it. Otherwise, it is set to **0**.	Compares **N** with the rest of these parameters one by one until **N***k* is greater than **N** and returns the previous subscript, that is, *k* minus 1. If the parameters following **N** are all smaller than **N**, the ordinal number *n* of the last parameter is returned. If **N** is **NULL**, **–1** is returned. All parameters are considered as integers. (According to the official website, decimals also take effect.) The numbers after **N** need to be sorted in ascending order. Otherwise, the system cannot work properly (the result is incorrect).	Yes	If the parameter following **N** is not sorted in ascending order, the returned results may be inconsistent (MySQL returns an incorrect value because it cannot work correctly).
4	abs(X)	Returns the absolute value of **X**. If **NULL** is input, **NULL** is returned. The input value must be a number or the value that can be implicitly converted to a number. Otherwise, an error is reported. The return value type is the same as the input parameter type.	Returns the absolute value of **X**. If **NULL** is input, **NULL** is returned. The input value must be a number or the value that can be implicitly converted to a number. Otherwise, **0** is returned. The return value is of the type with a larger range.	Yes	In openGauss, if the negative value of an input parameter is converted to a positive value, an error is reported, for example, **abs(-32768::smallint)**.

Syntax Differences Between openGauss and MySQL - System Functions (2)

No.	Item	openGauss	MySQL	Compatible	Remarks
5	crc32(expr)	Calculates the CRC32 value of **expr**. The input parameter is processed as a character string. If **NULL** is input, **NULL** is returned. The type of the return value is bigint.	Calculates the CRC32 value of **expr**. The input parameter is processed as a character string. If **NULL** is input, **NULL** is returned. The type of the return value is int unsigned.	Yes	The return value types are different.
6	format(X,D)	Rounds off *X* to *D* decimal places. The type of the return value is the string type. Format: #,###,###.##	Rounds off *X* to *D* decimal places. The type of the return value is the string type. Format: #,###,###.##	Yes	The function is the same as that of MySQL only after **dolphin.b_compatibility_mode** is set to **on**.
7	adddate (expr,days)	ADDDATE(date/datetime/time, interval/days) Description: Performs a date or time addition operation. When the second parameter is interval, the function is the same as the DATE_ADD function. For details, see DATE_ADD. When the second parameter is an integer, the integer is added to the first parameter as a number of days.	Adds a specified interval to a date value. **expr** is an expression of a date, and **days** indicates the interval of days to be added to a given date.	Yes	Currently, openGauss does not support calculation expressions after INTERVAL.
8	date(expr)	Description: Extracts the date part from **expr** when **expr** is recognized as a date or datetime expression. Return type: date	Extracts the date part **expr** from a date or datetime expression. If **expr** is **NULL**, **NULL** is returned.	Yes	This function is compatible with the strict and non-strict modes during MySQL table insertion. The DATE function can receive the **'0000-00-00'** input and return the date **'0000-00-00'** only if **dolphin.sql_mode** does not contain **'no_zero_date'**.

HUAWEI

Syntax Differences Between openGauss and MySQL - Expressions

No.	Item	openGauss	MySQL	Compatible	Remarks
1	expr || expr	By default: || is a operator indicating the OR logic. After **PIPES_AS_CONCAT** is enabled, || is a string concatenation operator and has a priority between that of the ^ and unary operators.	By default: || is a operator indicating the OR logic. After **PIPES_AS_CONCAT** is enabled, || is a string concatenation operator and has a priority between that of the ^ and unary operators.	Yes	In openGauss, || is a string connector by default. When **dolphin.sql_mode** contains **pipes_as_concat**, || indicates a string connector. When **pipes_as_concat** is not contained, || indicates the OR operator. || is widely used as a string connector in openGauss kernel and tools. You are advised to use OR to represent the OR logic. When **pipes_as_concat** is not included, errors may be reported for some tool commands (such as the meta-command **\d** of gsql).
2	expr && expr	AND logic, same as AND	AND logic The && operator is non-standard extension of the original operator in MySQL. This operator has been deprecated since MySQL 8.0.17.	Yes	To be compatible with MySQL, **dolphin.b_compatibility_mode** must be set to **on**. Otherwise, && is used to determine whether two figures overlap (used for geographic types).
3	! Expr	Negation	By default, the priority of **!** is higher than that of NOT. After **HIGH_NOT_PRECEDENCE** is enabled, **!** has the same priority as NOT.	Yes	In openGauss, the exclamation mark (!) refers to factorial default, with the format as "*integer*!". If **dolphin.b_compatibility_mode** is set to **on**, an exclamation mark (!) indicates negation.
4	param_marker	The ? placeholder used in prepared statements. In openGauss, *$1*, *$2*, ..., and *$n* are used to represent placeholders by default.	The ? placeholder used in prepared statements.	Yes	Only when **dolphin. b_compatibility_mode** is **on** can ? be used as a placeholder.
5	{identifier expr}	Not supported.	select {x 1} from {x test1} limit 1	No	
6	function_call	Calls a stored procedure.	Calls a stored procedure.	No	When a stored procedure is executed by running **call**, the stored procedure cannot contain a pure SELECT statement. Redevelopment suggestion: Use SELECT INTO or PERFORM to replace pure SELECT statements.
7	simple_expr COLLATE collation_name	Not supported.	Specifies the collation of expressions.	No	

HUAWEI

Syntax Differences Between openGauss and MySQL - Keywords

Item	openGauss	MySQL	Item	openGauss	MySQL	Item	openGauss	MySQL
ABORT	Non-reserved	-	ALL	Reserved	Reserved	ARRAY	Reserved	-
ABSOLUTE	Non-reserved	-	ALSO	Non-reserved	-	ASC	Reserved	Reserved
ACCESS	Non-reserved	-	ALTER	Non-reserved	Reserved	ASCII	-	Non-reserved
ACCESSIBLE	-	Reserved	ALWAYS	Non-reserved	Non-reserved	ASENSITIVE	-	Reserved
ACCOUNT	Non-reserved	Non-reserved	ANALYSE	Reserved	Non-reserved	ASSERTION	Non-reserved	-
ACTION	Non-reserved	Non-reserved	ANALYZE	Reserved	Reserved	ASSIGNMENT	Non-reserved	-
ADD	Non-reserved	Reserved	AND	Reserved	Reserved	AST	Non-reserved	-
ADMIN	Non-reserved	-	ANY	Reserved	Non-reserved	ASYMMETRIC	Reserved	Non-reserved
AFTER	Non-reserved	Non-reserved	AS	-	Reserved	AT	Non-reserved	Non-reserved
AGAINST	Reserved (functions and types allowed)	Non-reserved	APP	Non-reserved	-	ATTRIBUTE	Non-reserved	-
AGGREGATE	Non-reserved	Non-reserved	APPEND	Non-reserved	-	AUDIT	Non-reserved	-
ALGORITHM	Non-reserved	Non-reserved	ARCHIVE	Non-reserved	-	AUTHID	Non-reserved (cannot be a function or type)	-

HUAWEI

Syntax Differences Between openGauss and MySQL - User-defined Variables

No.	Item	openGauss	MySQL	Compatible	Remarks
1	SET @var_name = expr [, @var_name = expr] ...	Set user-defined variables based on the current session.	Set user-defined variables based on the current session.	Yes	To enable the function of user-defined variables in openGauss, you need to set **b_format_behavior_compat_options** to **'enable_set_variables'**. When @var_name is used for operator calculation, use spaces to separate operators. In the scenario where values are assigned consecutively, the equal sign (=) must be placed at the beginning of the value, and in other positions the sign indicates an operator. In PREPARE statements, character strings stored in user-defined variables support only the SELECT/INSERT/UPDATE/DELETE/MERGE syntax. User-defined variables store only integers, floating-point numbers, strings, bit strings, and **NULL**.
2	SET @a = @a + 1;	Set user-defined variables based on the current session.	Set user-defined variables based on the current session.	Yes	To enable the function of user-defined variables in openGauss, you need to set **b_format_behavior_compat_options** to **'enable_set_variables'**. When @var_name is used for operator calculation, use spaces to separate operators. In the scenario where values are assigned consecutively, the equal sign (=) must be placed at the beginning of the value, and in other positions the sign indicates an operator. In PREPARE statements, character strings stored in user-defined variables support only the SELECT/INSERT/UPDATE/DELETE/MERGE syntax. User-defined variables store only integers, floating-point numbers, strings, bit strings, and **NULL**.
3	SELECT @a, @a:=@a+1, ...;	Assigns values to user-defined variables and returns the results. When values are assigned to the same variables, if the values are obtained in different positions, the result set in openGauss may be different from that in MySQL. This is because the execution plans of the same SQL statement may be different, and the phases of assigning values to variables and obtaining values may be different due to database optimization. For details, see the official document **SET.md**.	Assigns values to user-defined variables and returns the results.	Yes	When values are assigned to the same variables, if the values are obtained in different positions, the result set in openGauss may be different from that in MySQL. This is because the execution plans of the same SQL statement may be different, and the phases of assigning values to variables and obtaining values may be different due to database optimization. For details, see the official document **SET.md**.

Syntax Differences Between openGauss and MySQL - SQL Compatibility - DDL (1)

No.	Item	openGauss	MySQL	Compatible	Remarks
1	ALTER DATABASE [db_name][DEFAULT] CHARACTER SET [=]	Alters the default character set of a namespace. This item is available only when **dolphin.b_compatibility_mode** is enabled.	Alters the default character set of the database.	Yes	The namespace in openGauss corresponds to that in MySQL.
2	ALTER DATABASE [db_name][DEFAULT] COLLATE [=] collation_name	Modifies the default collation of a namespace. This item is available only when **dolphin.b_compatibility_mode** is enabled.	Alters the default collation of the database.	Yes	The namespace in openGauss corresponds to that in MySQL.
3	ALTER SCHEMA [db_name][DEFAULT] CHARACTER SET [=]	Alters the default character set of a namespace.	Alters the default character set of the database.	Yes	In MySQL, DATABASE and SCHEMA are synonyms. In MySQL, DATABASE corresponds to "namespace" in openGauss.
4	ALTER SERVER server_nameOPTIONS (HOST character-literal)	Adds, modifies, or deletes the parameters of an existing server. You can query existing servers from the pg_foreign_server system catalog.	Alters the server information of **server_name** and adjusts any options allowed in the CREATE SERVER statement. The corresponding fields in the **mysql.servers** table are updated accordingly. Character text or numeric text must be specified for each option. **HOST** specifies the host name.	Yes	For details about the differences of server functions, see the openGauss official website.
5	ALTER TABLE tbl_name LOCK [=] {DEFAULT | NONE | SHARED | EXCLUSIVE}	Not supported.	The LOCK clause is used to control the level of concurrent reads and writes to a table when the table is modified.	No	The lock level cannot be specified during DDL execution. Redevelopment suggestion: This function is required by ONLINE DDL. Currently, openGauss does not support this function. You are advised to delete LOCK-related options from SQL statements, which does not affect the original DDL functions.

Syntax Differences Between openGauss and MySQL - SQL Compatibility - DDL (2)

No.	Item	openGauss	MySQL	Compatible	Remarks
6	CREATE [UNIQUE | FULLTEXT | SPATIAL] INDEX index_nameON tbl_name (key_part,...)	Full-text indexes can be created.	Full-text indexes can be created.	Yes	Only ngram tokenizer is supported. Other tokenizers defined will be escaped to ngram tokenizer.
7	CREATE INDEX index_nameON tbl_name (key_part,...)[KEY_BLOCK_SIZE [=] value]	The **KEY_BLOCK_SIZE** option is not supported.	For MyISAM tables, **KEY_BLOCK_SIZE** can specify the size (in bytes) used for index key blocks.	No	No similar function is available.
8	CREATE TABLESPACE tablespace_nameADD DATAFILE 'file_name'	Creates a tablespace. When you run the **ADD DATAFILE** command to create a tablespace, if the entered path ends with **.ibd**, the Dolphin plug-ins change the path name to end with **_ibd**.	Creates a tablespace.	Yes	In openGauss, a tablespace is usually a folder. In MySQL, a tablespace is usually a file.
9	ALTER TABLE tbl_name ORDER BY col_name [, col_name] ...	Not supported.	The ORDER BY syntax allows you to specify one or more column names for sorting. Each column name can be optionally followed by **ASC** or **DESC** to indicate the ascending or descending sort order, respectively. The default value is **ASC**. Only column names can be used as sorting conditions. Arbitrary expressions are not allowed. ORDER BY is meaningless to InnoDB tables because InnoDB always sorts table rows based on clustered indexes.	No	Redevelopment suggestion: You can run the **CLUSTER** command to achieve the similar effect. However, the **CLUSTER** command is used to cluster and sort tables based on indexes and cannot specify columns.
10	ALTER TABLE tbl_name DISCARD PARTITION {partition_names | ALL} TABLESPACE	Not supported.	DISCARD PARTITION ... The **TABLESPACE** command discards the tablespace of a partition and the .ibd file corresponding to the partition.	No	Tablespaces cannot be discarded. Redevelopment suggestion: openGauss does not provide similar functions. To modify a tablespace, run **ALTER TABLE SET TABLESPACE**.

Syntax Differences Between openGauss and MySQL - SQL Compatibility - DML (1)

No.	Item	openGauss	MySQL	Compatible	Remarks
1	DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name	Deletes rows from **tbl_name** and returns the number of deleted rows.	The DELETE statement deletes rows from **tbl_name** and returns the number of deleted rows.	Yes	The **[LOW_PRIORITY]**, **[QUICK]**, and **[IGNORE]** options are not supported.
2	INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE] [INTO] tbl_name [PARTITION (partition_name [, partition_name] ...)] [(col_name [, col_name] ...)] { {VALUES | VALUE} (value_list) [, (value_list)] ...	Inserts one or more rows of data into the **tbl_name** table.	In MySQL, VALUES and VALUE are synonyms. If you insert data into a partitioned table, the PARTITION clause determines the partitions and subpartitions that receive new rows.	Yes	The **[LOW_PRIORITY]**, **[DELAYED]**, and **[HIGH_PRIORITY]** options are not supported. Data cannot be inserted into a specified partition.
3	INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE] [INTO] tbl_nameVALUES row_constructor_list } [AS row_alias[(col_alias [, col_alias] ...)]]	Inserts one or more rows of data into the **tbl_name** table. An alias can be used for a row.	In MySQL 8.0.19 or later, you can use aliases for rows. You can choose to insert one or more columns between the AS keyword and the VALUES or SET clause.	Yes	The **[LOW_PRIORITY]**, **[DELAYED]**, and **[HIGH_PRIORITY]** options are not supported.
4	SELECT DISTINCT [STRAIGHT_JOIN]select_expr [, select_expr] ...	If DISTINCT is used, the repeated rows will be deleted from the result set.	**STRAIGHT_JOIN** forces the optimizer to join the tables in the order they are listed in the FROM clause to speed up the query if the optimizer joins tables in a non-optimal order. **STRAIGHT_JOIN** can also be used in the **table_references** list.	Yes	The **STRAIGHT_JOIN** option is not supported.
5	SELECT DISTINCTROW [SQL_SMALL_RESULT][SQL_BIG_RESULT] [SQL_BUFFER_RESULT]select_expr [, select_expr] ...	DISTINCTROW is synonymous with DISTINCT.	DISTINCTROW is synonymous with DISTINCT. **SQL_BIG_RESULT** or **SQL_SMALL_RESULT** can be used with GROUP BY or DISTINCT to tell the optimizer that the result set has many rows or is small, respectively. **SQL_BUFFER_RESULT** forcibly puts the result into a temporary table.	Yes	The **[SQL_SMALL_RESULT]**, **[SQL_BIG_RESULT]**, and **[SQL_BUFFER_RESULT]** options are not supported.

HUAWEI

Syntax Differences Between openGauss and MySQL - SQL Compatibility - DML (2)

No.	Item	openGauss	MySQL	Compatible	Remarks
6	HANDLER tbl_name READ index_name { = | <= | >= | < | > } (value1,value2,...) [WHERE where_condition] [LIMIT ...]	Not supported.	The HANDLER statement enables direct access to the table storage engine API. The HANDLER... OPEN statement is used to open a table so that the table can be accessed by subsequent HANDLER... READ statements. The opened table is not accessed by other sessions and is closed until the HANDLER... CLOSE statement is called or the session is terminated. The statement extracts rows whose specified indexes meet the given values and the WHERE condition. If there are multiple columns of indexes, separate the values of an index column by commas to convert them to a list so that you can specify values for all columns of the index or for the leftmost prefixes of the index column.	No	No similar function is available.
7	HANDLER tbl_name READ index_name { FIRST | NEXT | PREV | LAST } [WHERE where_condition] [LIMIT ...]	Not supported.	The HANDLER statement enables direct access to the table storage engine API. The HANDLER... OPEN statement is used to open a table so that the table can be accessed by subsequent HANDLER... READ statements. The opened table is not accessed by other sessions and is closed until the HANDLER... CLOSE statement is called or the session is terminated. The statement obtains a row from a table based on the index sequence that matches the WHERE condition.	No	No similar function is available.
8	HANDLER tbl_name READ { FIRST | NEXT } [WHERE where_condition] [LIMIT ...]	Not supported.	The HANDLER statement enables direct access to the table storage engine API. The HANDLER... OPEN statement is used to open a table so that the table can be accessed by subsequent HANDLER... READ statements. The opened table is not accessed by other sessions and is closed until the HANDLER... CLOSE statement is called or the session is terminated. The statement obtains a row from a table based on the natural sequence that matches the WHERE condition.	No	No similar function is available.
9	HANDLER tbl_name CLOSE	Not supported.	The statement closes the table opened by the HANDLER... OPEN statement.	No	No similar function is available.

Syntax Differences Between openGauss and MySQL - SQL Compatibility - DCL

No.	Item	openGauss	MySQL	Compatible	Remarks
1	COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE]	Commits a transaction. The **[AND [NO] CHAIN] [[NO] RELEASE]** syntax tree is not supported.	Commits a transaction. The CHAIN clause starts a new transaction immediately after the current transaction is committed. The RELEASE clause disconnects the transaction from the server after the transaction ends.	Yes	openGauss does not support the **[AND [NO] CHAIN] [[NO] RELEASE]** syntax tree. Currently, there is no alternative syntax.
2	ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE]	Rolls back a transaction. The **[AND [NO] CHAIN] [[NO] RELEASE]** syntax tree is not supported.	Rolls back a transaction. The CHAIN clause starts a new transaction immediately after the current transaction is rolled back. The RELEASE clause disconnects the transaction from the server after the transaction ends.	Yes	openGauss does not support the **[AND [NO] CHAIN] [[NO] RELEASE]** syntax tree. Currently, there is no alternative syntax.
3	SET autocommit = {0 | 1}	Specifies whether to automatically commit a transaction. If this parameter is set to **1**, each statement automatically starts its own transaction. After the statement is executed, the transaction is automatically committed. If this parameter is set to **0**, you need to run the **COMMIT** command to commit the transaction.	Specifies whether to automatically commit a transaction. If this parameter is set to **1**, each statement automatically starts its own transaction. After the statement is executed, the transaction is automatically committed. If this parameter is set to **0**, you need to run the **COMMIT** command to commit the transaction.	Yes	In MySQL, some SQL statements cannot be executed in transactions. If you attempt to execute such SQL statements during a transaction, the transaction is automatically committed (even if **autocommit** is set to **0**) and then the SQL statements can be executed. In this scenario, you are advised to commit the transaction and then execute the corresponding SQL statement.
4	PURGE { BINARY | MASTER } LOGS {TO 'log_name'}	Not supported.	Deletes a specified **binlog** file.	No	The transaction logs of the openGauss are Xlogs. Currently, the API for directly deleting Xlogs is not provided. The openGauss automatically deletes expired Xlogs based on the configuration. You are advised not to manually delete Xlogs.
5	SELECT MASTER_POS_WAIT('source_log_file', source_log_pos [, timeout][, channel])	Not supported.	This command is executed on the database until the specified location of the database and the number of transactions executed is returned.	No	In openGauss, you can use the pg_state_get_wal_senders function on the primary node to obtain the replay status of the standby node.

HUAWEI

Contents

1. Data Migration Overview
2. Syntax Differences Between openGauss and MySQL
3. **openGauss Migration Tool Set**
   - Compatibility Evaluation Tools
   - Full Migration and Incremental Migration
   - Full Verification and Incremental Verification
   - Reverse Incremental Migration and Reverse Full Migration
   - Migration Portal and One-Stop Migration Platform DataKit

HUAWEI

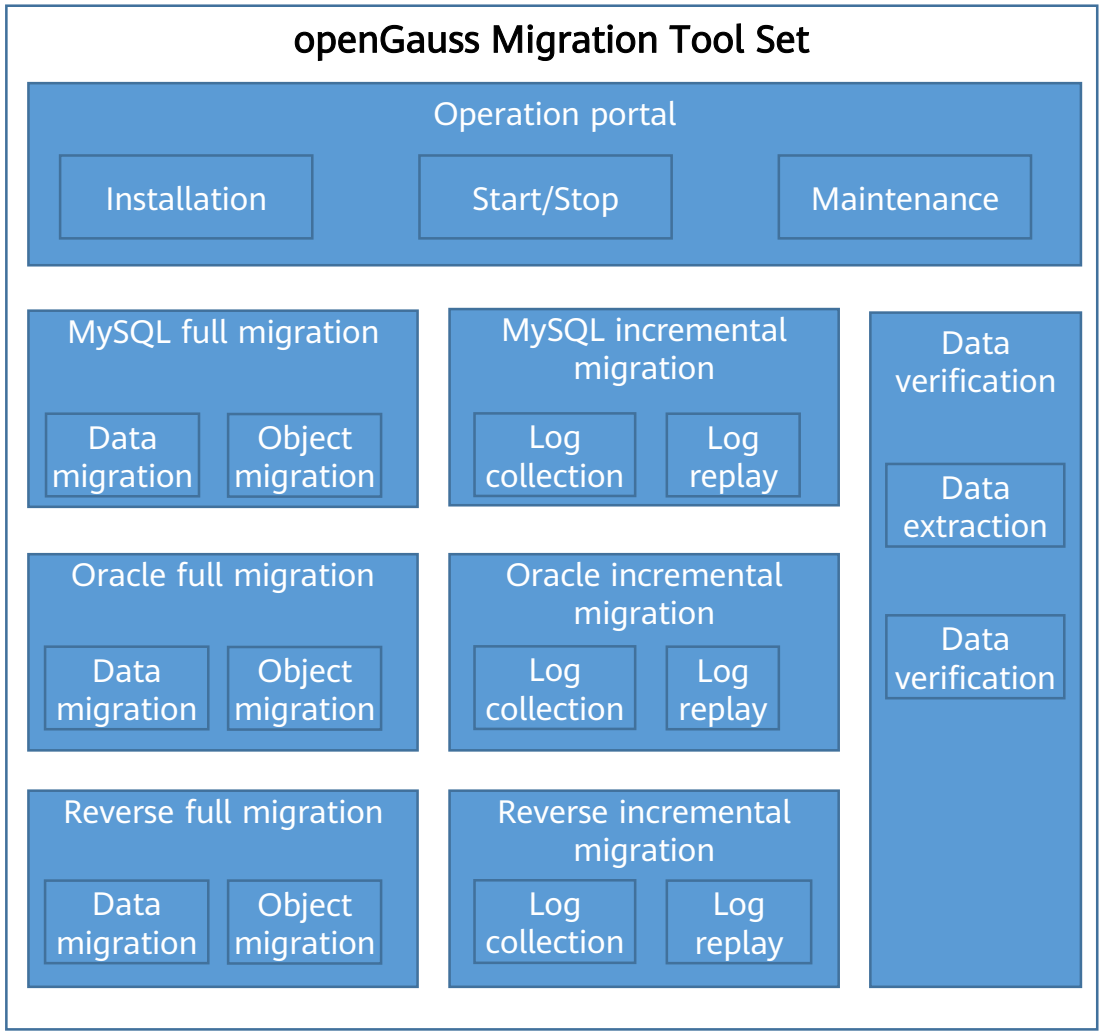
openGauss Migration Toolkit (1)

- The openGauss migration toolkit includes:
  - Full migration tools: support full migration of data and objects based on snapshot points, which can be from MySQL or Oracle to openGauss, or reversely, from openGauss to MySQL.
  - Incremental migration tools: support log-based incremental migration of data and DDL operations, which can be from MySQL or Oracle to openGauss, or reversely, from openGauss to MySQL.
  - Data verification: supports full and incremental data verification between MySQL and openGauss.
  - Operation portal: an independent command line tool developed using Java, which is used to install, start, stop, and maintain the preceding tools.

HUAWEI

openGauss Migration Tool Set (2)

- Installation: You can specify a tool to be installed or set an execution plan. For example, you can use the portal to support the plan of installing the MySQL full migration tool, MySQL incremental migration tool, and data verification tool in sequence.
- Start/Stop: You can specify a tool for running or set an execution plan. For example, you can use the portal to support the plan of running the MySQL full migration, full data verification, MySQL incremental migration, and incremental data verification in sequence.
- Maintenance: Portal monitors the running status of a tool, attempts to restore the tool when detecting that the tool is abnormal, and collects the tool running progress.



MySQL Syntax Compatibility Evaluation Tools - Overview

- The MySQL syntax compatibility evaluation tool is used to evaluate the compatibility of MySQL statements in openGauss.
- SQL files, general log files, slow log files, and mybatis XML mapping files can be parsed, and the compatibility of SQL statements in these files is evaluated.
- The tools can connect to MySQL to collect SQL statements for compatibility evaluation.
- The evaluation report is generated as an HTML file.

HUAWEI

MySQL Syntax Compatibility Evaluation Tools - Principles

- The evaluation tool consists of the SQL parsing module and syntax evaluation module.
- The SQL parsing module extracts SQL statements in multiple scenarios and exports the extracted SQL statements as SQL files.
- The evaluation module reads SQL files, evaluates the compatibility of SQL statements, and generates an evaluation report.
- Evaluation types include object evaluation and SQL statement evaluation. SQL statements can be obtained from files or extracted by connecting to MySQL.

HUAWEI

MySQL Syntax Compatibility Evaluation Tools - Process

- The compatibility evaluation results of SQL statements are as follows:
  - Fully compatible: openGauss fully supports this syntax. The compatibility result may depend on the pre-processing result of the input SQL statement. Therefore, the statements may not be fully compatible when being executed in openGauss.
  - Syntax compatible: openGauss supports this syntax. However, in actual use, issues may occur, for example, the column type is not supported or the function does not exist.
  - Statement incompatible: openGauss does not support this syntax.

HUAWEI

MySQL Syntax Compatibility Evaluation Tools - Examples (1)

- The following is an example of file evaluation. You need to configure the path of the file to be evaluated and information about openGauss in the configuration file. As shown in the following figure, multiple types of files can be evaluated at the same time.

```
[jhb@dev-openeuler-x86 ~]$ ll /data/jhb/testfiles/
total 128K
-rwx------ 1 jhb jhb 116K Jun  6 09:17 dev-centos.general
-rw------- 1 jhb jhb 2.2K Jul  8 17:03 dev-centos-slow.slow
-rw------- 1 jhb jhb  346 Jul 29 15:36 test.sql
-rw------- 1 jhb jhb 2.5K Jun 12 20:47 WebsiteMapper.xml
[jhb@dev-openeuler-x86 ~]$
```


HUAWEI

MySQL Syntax Compatibility Evaluation Tools - Examples (2)

- Run the **sh start.sh –d file –c configure –o report.html** command to start the evaluation.

```
[jhb@dev-openeuler-x86 ~]$ sh start.sh -d file -c configure -o report.html
java -jar -Djavax.xml.accessExternalDTD=all compatibility-assessment-5.1.0.jar compatibility-assessment-5.1.0.jar -d file -c configure -o report.html
log4j: reset attribute= "false".
log4j: Threshold ="null".
log4j: Level value for root is  [info].
log4j: root level set to INFO
log4j: Class name: [org.apache.log4j.ConsoleAppender]
log4j: Parsing layout of class: "org.apache.log4j.PatternLayout"
log4j: Setting property [conversionPattern] to [%d{HH:mm:ss,SS} %-5p (%C{1}:%M) - %m%n].
log4j: Setting property [levelMin] to [INFO].
log4j: Setting property [levelMax] to [ERROR].
log4j: Setting property [acceptOnMatch] to [true].
log4j: Adding filter of type [class org.apache.log4j.varia.LevelRangeFilter] to appender named [log.console].
log4j: Adding appender named [log.console] to category [root].
log4j: Class name: [org.apache.log4j.DailyRollingFileAppender]
log4j: Setting property [file] to [logs/assessment.log].
log4j: Setting property [append] to [true].
log4j: Setting property [datePattern] to [yyyy-MM-dd].
log4j: Parsing layout of class: "org.apache.log4j.PatternLayout"
log4j: Setting property [conversionPattern] to [%d{HH:mm:ss,SS} %-5p (%C{1}:%M) - %m%n].
log4j: Setting property [levelMin] to [INFO].
log4j: Setting property [levelMax] to [ERROR].
log4j: Setting property [acceptOnMatch] to [true].
log4j: Adding filter of type [class org.apache.log4j.varia.LevelRangeFilter] to appender named [log.file].
log4j: setFile called: logs/assessment.log, true
log4j: setFile ended
log4j: Appender [log.file] to be rolled at midnight.
log4j: Adding appender named [log.file] to category [root].
16:12:15,110 INFO  (CompatibilityAssessmenter:checkConditions) - output sqlfile number is : 4
Sep 23, 2023 4:12:15 PM org.opengauss.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: [8c9cfd84-cb68-4641-8dc3-d78292ca9718] Try to connect. IP: 192.168.0.215:11124
Sep 23, 2023 4:12:15 PM org.opengauss.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: [192.168.0.215:38238/192.168.0.215:11124] Connection is established. ID: 8c9cfd84-cb68-4641-8dc3-d78292ca9718
Sep 23, 2023 4:12:15 PM org.opengauss.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: Connect complete. ID: 8c9cfd84-cb68-4641-8dc3-d78292ca9718
16:12:15,596 INFO  (AssessmentEntry:printProcess) - gs_assessment: Analysing[100.00]:6/6
16:12:35,614 INFO  (AssessmentEntry:printProcess) - gs_assessment: Analysing[100.00]:797/797
16:13:19,697 INFO  (AssessmentEntry:printProcess) - gs_assessment: Analysing[100.00]:9/9
16:13:19,733 INFO  (AssessmentEntry:printProcess) - gs_assessment: Analysing[100.00]:3/3
16:13:19,735 INFO  (AssessmentEntry:assessment) - gs_assessment: Create database assessment_1688719386596 automatically, clear it manually!
```


HUAWEI

Contents

1. Data Migration Overview
2. Syntax Differences Between openGauss and MySQL
3. **openGauss Migration Tool Set**
   - Compatibility Evaluation Tools
   - Full Migration and Incremental Migration
   - Full Verification and Incremental Verification
   - Reverse Incremental Migration and Reverse Full Migration
   - Migration Portal and One-Stop Migration Platform DataKit

HUAWEI

Full Migration Using gs_mysync – Principles

- Full migration is to migrate existing data and objects from MySQL to openGauss. Currently, the official website of the openGauss community provides the full migration tool gs_mysync, which is essentially a Chameleon tool developed using Python.
- Full migration supports the following data objects: tables, table data, functions, stored procedures, triggers, and views.
- Full migration performance: Based on the sysbench testing model, 10 tables (without primary keys) containing 5 million data records in each table can be migrated using 20 concurrent read/write threads. The overall performance can reach 300 MB/s.
- Implementation principles of full migration:
  - Multi-process read/write isolation is used. Producers read data from MySQL and write the data to CSV files. Consumers read CSV files and write the data to openGauss. Multiple tables are processed in parallel.
  - A large table is divided into multiple CSV files. By default, the size of a CSV file is 2 MB.
  - Migration sequence: table structure > table data > table indexes

HUAWEI

Full Migration Using gs_mysync – Implementation Logic

- Record the snapshot point when the full migration starts.
- Create the target schema and table structure, excluding indexes.
- Create multiple read and write processes. The main process creates a read task for each table and adds the task to the read task queue.
- Execute the read and write processes in parallel. The read process records the snapshot point of each table, reads table data, and saves the data to multiple CSV files. The write process copies the CSV files to the target database at the same time.
- Create indexes after data migration.
- Stop the read and write processes after all tasks are complete.

HUAWEI

Full Migration Using gs_mysync – Procedure (1)

- Create a configuration file and customize the configuration.

```
chameleon set_configuration_files
```

- Initialize replication and create auxiliary schemas and tables for the replication process.

```
chameleon create_replica_schema --config default
chameleon add_source --config default --source mysql
```

- Replicate the basic table and data.

```
chameleon init_replica --config default --source mysql
```


HUAWEI

Full Migration Using gs_mysync – Procedure (2)

- Replicate functions, stored procedures, triggers, or views.

```
chameleon start_func_replica --config default --source mysql
chameleon start_proc_replica --config default --source mysql
chameleon start_trigger_replica --config default --source mysql
chameleon start_view_replica --config default --source mysql
```

- Replicate online.

```
chameleon start_replica --config default --source mysql
```

- End the replication process and clear resources.

```
chameleon stop_replica --config default --source mysql
chameleon detach_replica --config default --source mysql
chameleon drop_replica_schema --config default
```


HUAWEI

Incremental Migration Using gs_replicate - Principles (1)

- Incremental migration migrates incremental data (IUD operations and DDL) generated on MySQL to openGauss. Currently, the official website of the openGauss community provides the incremental migration tool gs_replicate, which is developed using Java.
- Incremental migration implementation principles: based on Debezium+Kafka
  - The source of the Debezium MySQL connector monitors binlogs of MySQL and writes DDL and DML operations to Kafka in Avro format.
  - The sink of the Debezium MySQL connector reads DDL and DML operations in Avro format from Kafka, assembles the data into transactions, and plays back the transactions in parallel on the openGauss based on the transaction granularity to migrate DDL and DML operations from MySQL to openGauss online.
  - This solution strictly ensures the transaction sequence. Therefore, DDL and DML operations are routed in a topic of Kafka, and the number of partitions of the topic can only be 1 (**num.partitions** is set to **1**). In this way, the data pushed from the source to Kafka and the data obtained by the sink from Kafka are in strict sequence.

HUAWEI

Incremental Migration Using gs_replicate - Principles (2)

- Incremental migration performance: When sysbench is used to perform pressure tests on MySQL in the hybrid IUD scenario, 10 tables are migrated online in 50 threads (including 30 insert threads, 10 update threads, and 10 delete threads) to the openEuler OS with 2-socket Kunpeng 920 CPU-powered server, the performance can reach 30,000 TPS.
- Parallel mechanism for incremental migration: Log parsing on the source and log replay on the sink are performed in parallel.
- Parallelization principle on the source:
  - The open-source component mysql-binlog-connector-java is used to parse binlogs of MySQL as events. The original implementation scheme is serial parsing. The source code is modified to support parallel parsing of events, improving the performance of the Debezium MySQL connector as the source.
- Parallelization principle on the sink:
  - Parallel replay based on transaction granularity: Parallel replay is implemented based on the primary and standby parallel replication mechanism of MySQL.
  - Parallel replay based on table granularity: Parallel replay is implemented based on the table granularity.

HUAWEI

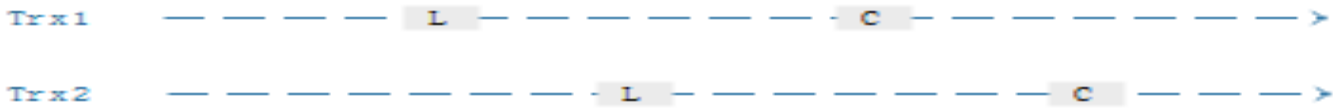
Incremental Migration Using gs_replicate - Parallel Replay Based on Transaction Granularity (1)

- Parallel replay based on transaction granularity: Parallel replay is implemented based on the primary and standby parallel replication mechanism of MySQL.
- The primary and standby replication delay is a pain point. Improving the degree of parallelism is a key factor to reduce the delay. With the evolution of MySQL, the parallelism has been gradually increasing.
- Parallel replay based on transaction granularity: Parallel replay is implemented based on the primary and standby parallel replication mechanism of MySQL.
- If two transactions can obtain their own locks in the same period of time, they can be executed in parallel.
- **lock interval** indicates the interval for a transaction to hold a lock.
  - When the last lock is obtained, the lock interval starts.
  - When the storage engine commits the transaction, the first lock is released and the lock interval ends.

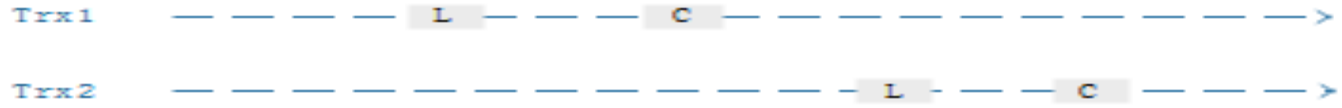
HUAWEI

Incremental Migration Using gs_replicate - Parallel Replay Based on Transaction Granularity (2)

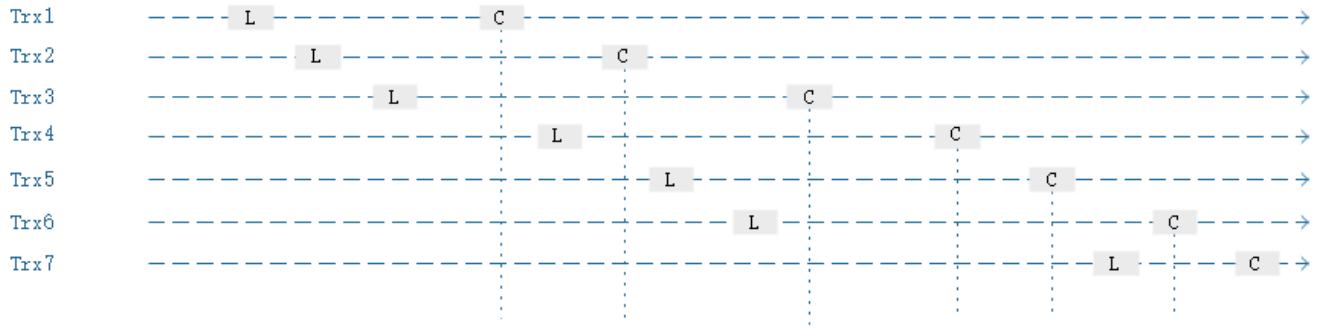
- Transactions that can be executed in parallel



- Transactions that cannot be executed in parallel



- When the lock intervals of Trx1 and Trx2 overlap, they can be executed concurrently.
- The lock interval at the end of Trx1 must be earlier than that at the end of Trx2. The two lock intervals do not overlap and the two transactions cannot be executed concurrently.



- Trx3, Trx4, Trx5, and Trx6 transactions can be executed concurrently.
- After Trx3, Trx4, and Trx5 are executed, Trx6 and Trx7 can be executed concurrently.
- Trx5 and Trx7 cannot be executed concurrently.

HUAWEI

Incremental Migration Using gs_replicate - Parallel Replay Based on Transaction Granularity (3)

- Parallel replay based on transaction granularity: Parallel replay is implemented based on the primary and standby parallel replication mechanism of MySQL.
  - **last_committed** and **sequence_number**
  - Binlogs record the start point (L) and end point (C) of the lock interval of each transaction, which are named **last_committed** and **sequence_number** respectively.

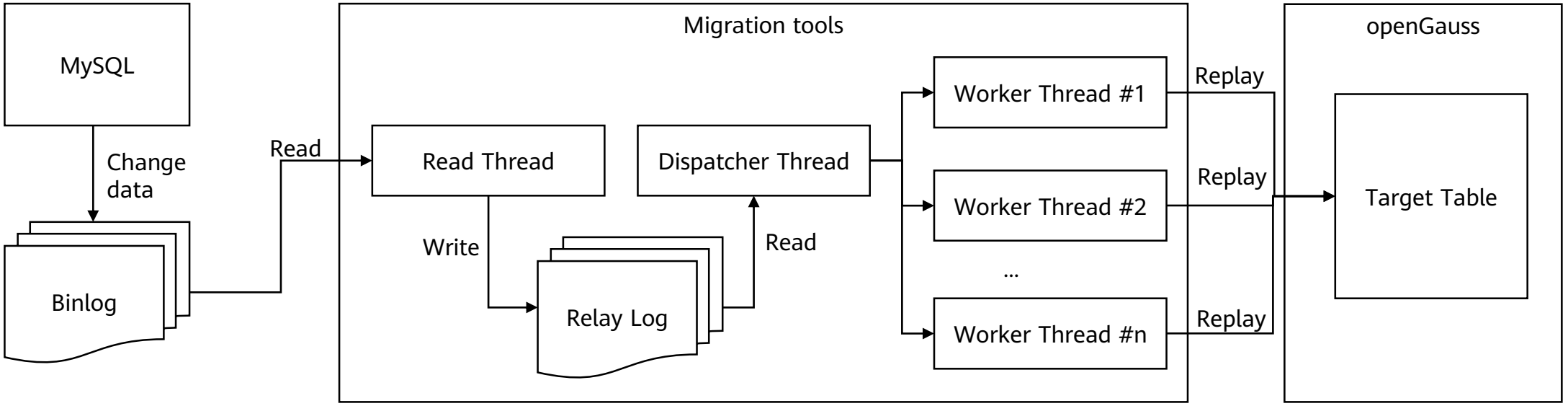
```
mysqlbinlog --no-defaults /home/mysql/data/mysql-bin.000011 --database=mysql_database --base64-output=decode-rows -v | grep last_committed
```

```
[root@localhost logs]# mysqlbinlog --no-defaults /home/mysql/data/mysql-bin.000011  --database=mysql_database --base64-output=decode-rows -v |grep last_committed
WARNING: The option --database has been used. It may filter parts of transactions, but will include the GTIDs in any case. If you want to exclude or include transactions, you should use the options --exclude-gtid
s or --include-gtids, respectively, instead.
#220113 10:17:29 server id 100  end_log_pos 275 CRC32 0x7488ffb5        GTID    last_committed=0        sequence_number=1       rbr_only=yes    original_committed_timestamp=1642040249605077   immediate_commit_tim
estamp=1642040249605077 transaction_length=543
#220113 10:21:04 server id 100  end_log_pos 818 CRC32 0xbf3c35fb        GTID    last_committed=1        sequence_number=2       rbr_only=yes    original_committed_timestamp=1642040464065616   immediate_commit_tim
estamp=1642040464065616 transaction_length=311
#220113 10:21:06 server id 100  end_log_pos 1129 CRC32 0xd8fbc39f       GTID    last_committed=1        sequence_number=3       rbr_only=yes    original_committed_timestamp=1642040466511441   immediate_commit_tim
estamp=1642040466511441 transaction_length=311
#220113 10:22:34 server id 100  end_log_pos 1440 CRC32 0xa8e0ec26       GTID    last_committed=3        sequence_number=4       rbr_only=yes    original_committed_timestamp=1642040554400566   immediate_commit_tim
estamp=1642040554400566 transaction_length=311
#220113 10:22:38 server id 100  end_log_pos 1751 CRC32 0x7c22185c       GTID    last_committed=3        sequence_number=5       rbr_only=yes    original_committed_timestamp=1642040558463906   immediate_commit_tim
estamp=1642040558463906 transaction_length=311
#220113 10:22:42 server id 100  end_log_pos 2062 CRC32 0x81533fb8       GTID    last_committed=3        sequence_number=6       rbr_only=yes    original_committed_timestamp=1642040562161641   immediate_commit_tim
```


HUAWEI

Incremental Migration Using gs_replicate - Parallel Replay Based on Transaction Granularity (4)

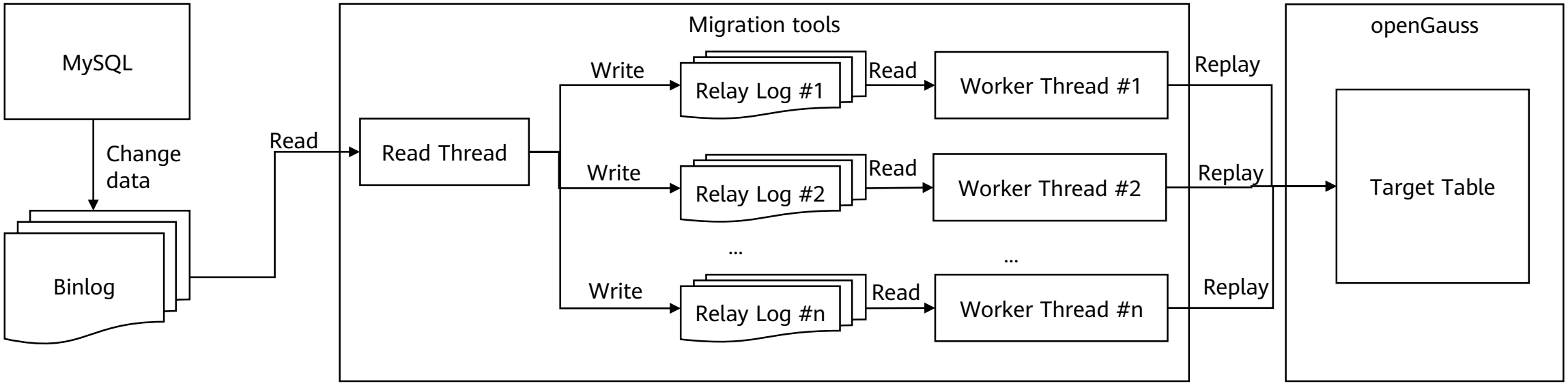
- Parallel replay based on transaction granularity: Parallel replay is implemented based on the primary and standby parallel replication mechanism of MySQL.



- A dispatcher thread is used to distribute logs to be replayed. Each time a log is obtained, it is required to check whether the log can be executed concurrently with the transaction that is being replayed. If the minimum **sequence_number** of all transactions that are being replayed is greater than the value of **last_committed** of the transaction, they can be executed concurrently.
- If they can be executed in parallel, an idle thread is searched for in the worker thread group for execution. Otherwise, the system waits until the conditions are met and then searches for an idle thread in the worker thread group for execution.
- If there is no idle worker thread, wait until a transaction is replayed and the worker thread is released.

Incremental Migration Using gs_replicate - Parallel Replay Based on Table Granularity (5)

- Parallel replay based on table granularity: Parallel replay is implemented based on the table granularity.



- After reading binlogs, the read thread performs hash based on databases and tables and writes the hash values to relay logs.
- A group of worker threads are started to concurrently obtain logs from the corresponding relay log files, parse the logs to construct SQL statements, and replay the statements on openGauss.
- Parallelization is implemented based on databases and tables, which cannot ensure the sequence of transactions. In addition, if the worker thread load is unbalanced, the replay performance deteriorates.

Incremental Migration Using gs_replicate - Comparison Between Two Parallel Solutions

- Comparison between two parallel replay solutions

Solution	Advantage	Disadvantage	Performance Objective
Parallel replay based on databases and tables	1. The implementation is relatively simple. 2. It is not restricted by MySQL versions.	1. Transactions may not be executed in sequence and data may be inconsistent, but eventual data consistency can be ensured. 2. If a table contains foreign keys, the operation may fail because the sequence of tables cannot be ensured. 3. If the data volume and operation frequency of each table in the database differ greatly, the load of the worker thread may be unbalanced. In practice, this situation is common.	The log replay speed is close to the operation speed of the source database.
Parallel replay based on parallel replication mechanism of MySQL	Transactions are executed in sequence and data is consistent.	1. Constraints: This solution can be used only in MySQL 5.7 (released in 2015) or a later version and when GTID is enabled. 2. When a transaction replay is suspended due to an exception, it may block subsequent transaction replays. 3. The implementation is relatively complex.	The log replay speed is close to the operation speed of the source database.

HUAWEI

Incremental Migration Using gs_replicate – Constraints and Limitations

- MySQL 5.7 or later
- MySQL parameter configurations:
  - **log_bin=on**
  - **binlog_format=row**
  - **binglog_row_image=full**
  - **gtid_mode=on** #If this parameter is not enabled, the sink replays transactions in serial, which deteriorates the online migration performance.
- DML and DDL migrations are supported. During online migration, DDL is transparently transmitted. For syntaxes that are incompatible with openGauss or MySQL, an error is reported during DDL migration.

HUAWEI

Incremental Migration Using gs_replicate - Procedure (1)

- Kafka

```
wget -c https://mirrors.tuna.tsinghua.edu.cn/apache/kafka/3.2.3/kafka_2.13-3.2.3.tgz
tar -zxf kafka_2.13-3.2.3.tgz
```

- confluent community

```
wget -c  https://packages.confluent.io/archive/5.5/confluent-community-5.5.1-2.12.zip
unzip confluent-community-5.5.1-2.12.zip
```

- debezium-connector-mysql

```
wget -c https://openGauss.obs.cn-south-1.myhuaweicloud.com/latest/tools/debezium-connector-mysql-1.8.1.Final-plugin.tar.gz
tar -zxvf debezium-connector-mysql-1.8.1.Final-plugin.tar.gz
```


HUAWEI

Incremental Migration Using gs_replicate - Procedure (2)

- ZooKeeper
  - Store configuration files in **/kafka_2.13-3.2.3/config/zookeeper.properties**. The default ZooKeeper port number is 2181, corresponding to the value **2181** of the parameter **clientPort**.
- kafka
  - Store configuration files in **/kafka_2.13-3.2.3/config/server.properties**. Note that the number of partitions of a topic must be 1. Therefore, the value of **num.partitions** must be **1**. However, the default value of this parameter is **1**, which means you do not need to modify it. The default Kafka port number is 9092, corresponding to the value **PLAINTEXT://:9092** of the parameter **listeners**.
- schema-registry
  - Store configuration files in **/confluent-5.5.1/etc/schema-registry/schema-registry.properties**. The default port number of schema-registry is 8081, corresponding to the value **http://0.0.0.0:8081** of the parameter **listeners**.

HUAWEI

Incremental Migration Using gs_replicate - Procedure (3)

- kafka-conenct
  - The configuration files are stored in **/confluent-5.5.1/etc/schema-registry/connect-avro-standalone.properties**. Note: Add the path of **debezium-connector-mysql** to the **plugin.path** configuration item. If **debezium-connector-mysql** is stored in **/data/debezium_kafka/plugin/debezium-connector-mysql**, set the path of the item to the upper-level directory, that is, set **plugin.path** to **share/java,/data/debezium_kafka/plugin**. The default port number of connect-standalone is 8083, corresponding to the value **8083** of the parameter **rest.port**.
- mysql-source.properties
  - The configuration files are stored in **/confluent-5.5.1/etc/kafka/mysql-source.properties**.
- mysql-sink.properties
  - The configuration files are stored in **/confluent-5.5.1/etc/kafka/mysql-sink.properties**.

HUAWEI

Incremental Migration Using gs_replicate - Procedure (4)

- Procedure - startup commands
  - Start the ZooKeeper process.

```
cd kafka_2.13-3.2.3
./bin/zookeeper-server-start.sh ./config/zookeeper.properties
```

  - Start the Kafka process.

```
cd kafka_2.13-3.2.3
./bin/kafka-server-start.sh ./config/server.properties
```

  - Register a schema.

```
cd kafka_2.13-3.2.3
./bin/kafka-server-start.sh ./config/server.properties
```


HUAWEI

Incremental Migration Using gs_replicate - Procedure (5)

- Procedure - startup commands
  - Start the source of kafka-connect.

```
cd confluent-5.5.1
./bin/connect-standalone etc/schema-registry/connect-avro-standalone.properties
etc/kafka/mysql-source.properties
```

  - Start the sink of kafka-connect.

```
cd confluent-5.5.1
./bin/connect-standalone etc/schema-registry/connect-avro-standalone-1.properties
etc/kafka/mysql-sink.properties
```

- Note: The difference between the **connect-avro-standalone.properties** and **connect-avro-standalone-1.properties** configuration files on the source and sink lies in the **rest.port** parameter, whose default value is **8083**. That is, you can set different port numbers in the two files to start multiple kafka-connects so that the sink and source can work independently.

HUAWEI

Incremental Migration Using gs_replicate – Collaboration Between Incremental Migration and Full Migration (1)

- Start full migration.
  - Full migration is performed using Chameleon. After the full migration is started, you can query the snapshot point of the full migration in **sch_chameleon.t_replica_batch** on openGauss.
  - The snapshot point of a single table is stored in **sch_chameleon.t_replica_tables**.
- Start the source database.
  - Before starting the source database, start ZooKeeper and Kafka and register a schema.
  - After querying the snapshot point of the full migration, you can configure the snapshot point of the full migration in the **mysql-source.properties** configuration file on the source database and start the source database without waiting until the full migration is complete. After full migration is started, the source database can be started to minimize the latency of the source database.
  - After the source database is started, if there are DML transactions of a table to be fully migrated before the snapshot point of the table, the corresponding DML operations are skipped. This avoids data duplication or data loss during the migration.

HUAWEI

Incremental Migration Using gs_replicate – Collaboration Between Incremental Migration and Full Migration (2)

- Start the sink after the full migration is complete.
  - After the full migration is complete, start data replay on the sink.
  - If the sink is started before the full migration is complete, data will be out of order. This is an improper operation, which should be avoided during actual operations.

Contents

1. Data Migration Overview
2. Syntax Differences Between openGauss and MySQL
3. **openGauss Migration Tool Set**
   - Compatibility Evaluation Tools
   - Full Migration and Incremental Migration
   - Full Verification and Incremental Verification
   - Reverse Incremental Migration and Reverse Full Migration
   - Migration Portal and One-Stop Migration Platform DataKit

HUAWEI

Verification Tool gs_datacheck – Principles (1)

- Implementation principles of the verification tool
  - The verification tool gs_datacheck extracts data in JDBC mode.
  - Hash calculation is performed on the original table data, and intermediate data is temporarily stored in Kafka topics.
  - The tool uses the intermediate data of the source and target databases to construct two Merkle trees and compares them to check the data consistency between the two databases.
  - The technical framework adopts SpringBoot, Kafka middleware, and Debezium.

HUAWEI

Verification Tool gs_datacheck – Principles (2)

- Restrictions
  - Version: JDK 11 or later
  - The current version 5.1.0 supports data verification only when the source and target databases are MySQL or openGauss.
  - The current version 5.1.0 supports only data verification and does not support table object verification.
  - Version: MySQL 5.7 or later
  - The current version 5.1.0 does not support geographical location geometric data verification.

HUAWEI

Verification Tool gs_datacheck – Principles (3)

- Full data verification
  - Extract data from the MySQL (source) and openGauss (end) by sharding, normalize and hash the data, and push the data to Kafka for temporary storage.
  - The Check service extracts data from Kafka and verifies the extracted data.
  - Export the verification result to a file in a specified path.
- Incremental data verification
  - Use Debezium to monitor the data change records of the source database.

HUAWEI

Verification Tool gs_datacheck – Principles (4)

- The source extraction service processes the monitored change records in a certain frequency (time and quantity) and collects and combines the change records.
- The combination result is sent to the Check service.
- The Check service combines the last verification result with the current incremental data.
- Based on the combination result, the Check service initiates incremental data extraction and verification, and exports the verification result to a file in a specified path.

- Rules-based filtering
  - Three types of filtering rules based on tables, rows, and column are added.

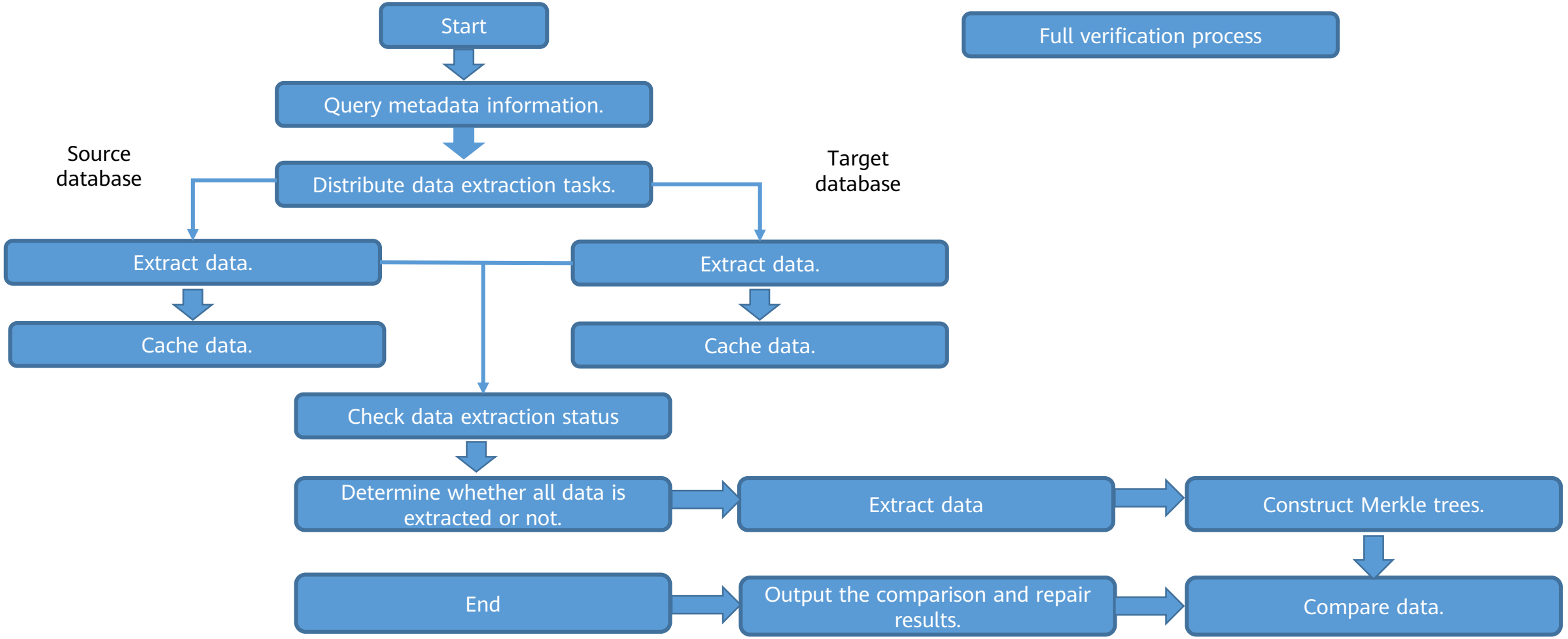
HUAWEI

Full Verification - Process (1)

- Query the metadata from **information_schema.tables** and **information_schema.columns** in the source and target databases to obtain information about all tables to be verified.
- Create extraction and verification tasks based on the tables to be verified.
- For a table with a small number of records, full table query is used. For a large table, the records are automatically extracted for multiple times (multiple shards are divided based on the WHERE condition).
- Extract table data, normalize (standardize) and hash the data, and push the hashed data to Kafka for temporary storage. If the number of records in a table is small, store data in a topic partition. If the number of records in a table is large, store data in multiple topic partitions.
- The extraction service creates a topic for each table, and the source and sink use different topics.
- The Check service extracts data from Kafka by table for verification. It obtains the topic data from the source and target databases of a specified table, constructs two Merkle trees, and then compares the two trees to verify table data.
- Export the verification result to a file in a specified path.

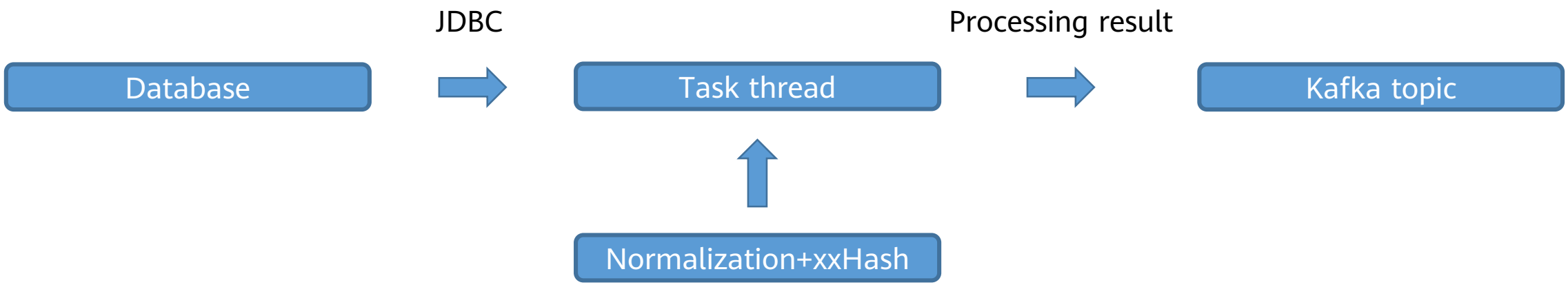
HUAWEI

Full Verification - Process (2)



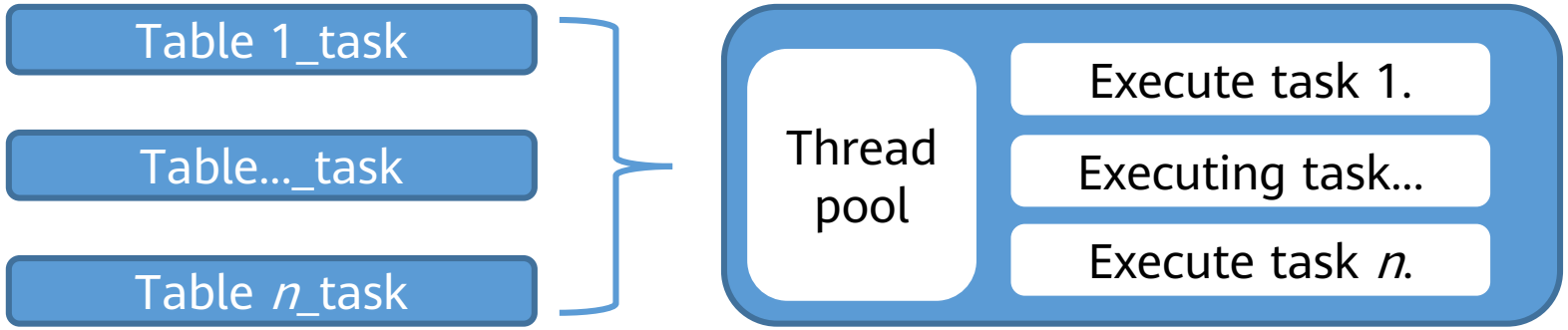
Full Verification - Data Extraction (1)

- The data extraction service creates data extraction tasks based on table metadata. It extracts table data from the databases in JDBC mode, and normalizes and hashes the data.
- The calculation result is stored in Kafka by table. A topic is created for each table.



Full Verification - Data Extraction (2)

- For multiple tables, data is extracted concurrently. For a single table, whether to enable parallel extraction depends on the data volume.
- When the number of table records is greater than the specified threshold, table data of the current task is automatically sharded and parallel extraction is enabled.

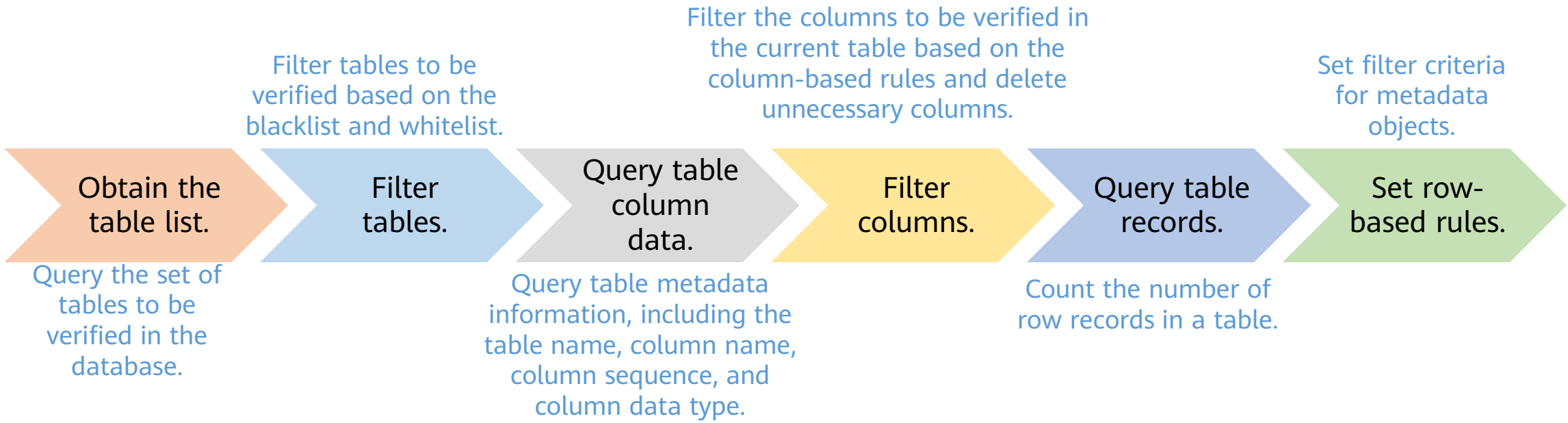


Full Verification - Data Verification (1)

- Data verification hash algorithm: The XXH3 algorithm with the optimal performance among the xxHash algorithms is used.
- xxHash is an extremely fast hash algorithm, working at RAM speed limit. It successfully completes the SMHasher test suite which evaluates collision, dispersion, and randomness qualities of hash functions. Its code is highly portable, and hashes are identical on all platforms. Currently, xxHash is proposed in four flavors: XXH32, XXH64, XXH128, and XXH3, among which XXH3 offers the best performance.
- Filtering rules are classified based on tables, rows, and columns.
  - Filtering based on tables is about adding the table blacklist and whitelist (regular expression) to filter the tables to be verified.
  - Filtering based on rows is about filtering row records in a specified table and obtain some data in the table for data verification.
  - Filtering based on columns is about filtering column fields in a specified table for verification.
  - Table and row filtering rules are matched based on regular expressions. Column filtering rules are matched based on table names. Filtering rules are triggered and executed when the extraction service loads metadata.

HUAWEI

Full Verification - Data Verification (2)



HUAWEI

Full Verification - Merkle Tree (1)

- Merkle tree is a tree. Most of them are binary trees and some are multi-trees. Merkle tree's leaf nodes' value is unit data of dataset, or unit data HASH. The value of a non-leaf node is based on all the leaf node values below it, and then calculated with hash method. You can create a Merkle tree with the complexity of O(n) and the tree height log(n)+1. During data comparison between two nodes, the root node of the Merkle tree is compared. If the root nodes are the same, no processing is performed. If the root nodes are different, the Merkle tree is traversed to locate the inconsistent nodes. The positioning speed is fast, and the positioning time complexity is O(log(n)). In the industry, the number of layers Cassandra uses is fixed at 15, that is, 2^15 (32768) of the maximum number of leaf nodes. In the actual construction process, the final tree height is determined based on the table record scale, but the tree depth cannot exceed 15 layers.

HUAWEI

Full Verification - Merkle Tree Node Comparison (2)

- Compare the two Merkle trees to obtain the leaf nodes of final difference. Then, perform secondary positioning and parsing on the leaf nodes, obtain the final difference records, and return the corresponding row record identifiers.

Added Data				Deleted Data				Updated Data			
Set A		Set B		Set A		Set B		Set A		Set B	
1	Hash A	1	Hash A	1	Hash A	1	Hash A	1	Hash A	1	Hash A
2	Hash B	3	Hash C	3	Hash C	2	Hash B	3	Hash C	3	Hash B
3	Hash C	4	Hash D	4	Hash D	3	Hash C	4	Hash D	4	Hash D
4	Hash D	5	Hash E	5	Hash E	4	Hash D	5	Hash E	5	Hash E
5	Hash E	6	Hash F	6	Hash F	5	Hash E	6	Hash F	6	Hash F
6	Hash F	7	Hash G	7	Hash G	6	Hash F	7	Hash G	7	Hash G
7	Hash G					7	Hash G				

Full Verification - Merkle Tree Node Construction

- The number of table records is uncertain, which may be 100 or 1 million. However, a tree cannot exceed 15 layers and the number of leaf nodes cannot exceed 32,768. In this case, bucketing is introduced, which is to distribute 1 million data records to 32,768 leaf nodes based on hash rules. Each node has a storage container, which is defined as a bucket. The process of distributing data to buckets is defined as bucketing.
- The performance of bucketing is determined by the height of the Merkle tree. When the height does not exceed 15 layers (when the tree height is 15 and the number of leaf nodes of the binary tree is 32,768), the performance is good. When the height exceeds 15, the performances of tree construction and traversal deteriorate.
- Data bucketing: After modulo operation is performed on the obtained data based on the hash value of the primary key, the data is added to different buckets.

HUAWEI

Incremental Verification - Process

- Use Debezium to monitor the data change records of the source database. (Debezium records them in the specified topic.)
- The source extraction service processes the monitored change records (that is, specified topics) in a certain frequency (time and quantity), and collects and combines the change records.
- Collect statistics on the change records (primary key values) in tables by table.
- Send the combined change result to the Check service.
- The Check service combines the last verification result with the current incremental data again.
- Based on the combination result, the Check service initiates incremental data extraction and verification, and exports the verification result to a file in a specified path.

HUAWEI

Contents

1. Data Migration Overview
2. Syntax Differences Between openGauss and MySQL
3. **openGauss Migration Tool Set**
   - Compatibility Evaluation Tools
   - Full Migration and Incremental Migration
   - Full Verification and Incremental Verification
   - Reverse Incremental Migration and Reverse Full Migration
   - Migration Portal and One-Stop Migration Platform DataKit

HUAWEI

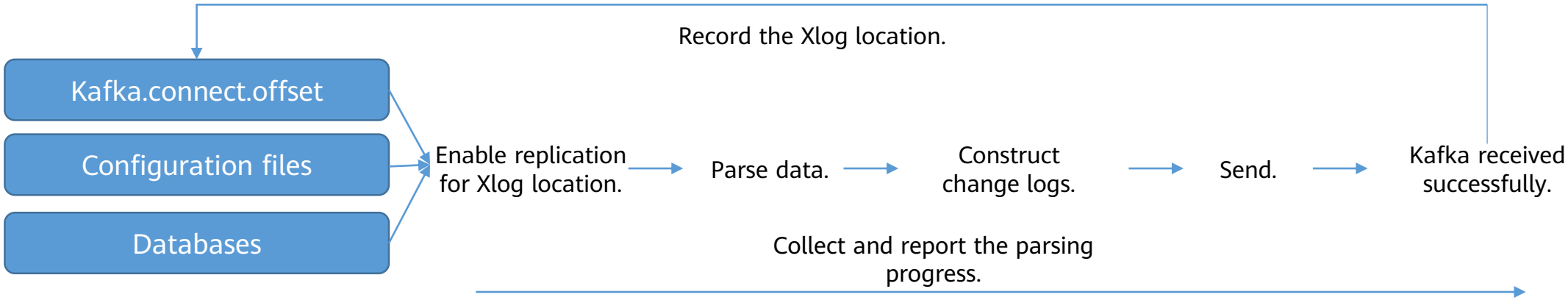
Reverse Incremental Migration Tool gs_replicate

- The Debezium reverse incremental migration tool can synchronize incremental data from openGauss to MySQL in real time.
- Debezium consists of two parts: log parsing (source) and data replay (sink). The two parts can be decoupled through Kafka. The log parsing is about parsing the Xlogs in openGauss to obtain the change records in Avro format. The data replay is about constructing SQL statements compatible with the target database based on the change records and executing the statements through JDBC.

HUAWEI

Reverse Incremental Migration Tool gs_replicate - Implementation Principles on the Source

- During reverse incremental migration, the source database creates a logical replication slot and publishes and subscribes to obtain binary streaming data for parsing.



- Log parsing is executed in serial for various data types of the source database, which are encapsulated together with database metadata into change records and saved to Kafka. The source database supports replication from a user-defined Xlog location. In a migration task, the Xlog location is generated when the forward incremental migration is complete, and the reverse incremental migration starts replication from this location. This ensures that no data is duplicated or lost during the switchover from forward synchronization to reverse synchronization.

Reverse Incremental Migration Tool gs_replicate - Implementation Principles on the Sink

- During reverse incremental migration, the sink extracts change records from Kafka, constructs SQL statements by table in parallel, and connects to the target database for execution.
- When extracting change records from Kafka, the sink monitors the amount of data to be replayed in the memory. If the amount of data is too large, the sink suspends the extraction until the amount of data to be processed decreases to a certain extent. The change records of different tables are distributed to different threads preferentially. If some tables depend on each other, their change records are distributed to the same thread. The tool periodically exports the SQL statements that fail to be executed to a local file.

HUAWEI

Using gs_replicate for Reverse Incremental Migration

- Create configuration files **openGauss-source.properties** and **openGauss-sink.properties**.
- Path: **confluent-5.5.1/etc/kafka/**
- The dependency of the reverse incremental migration tool is the same as that of the forward incremental migration tool.
- The startup method is also the same as that for forward incremental migration.

HUAWEI

Restrictions on gs_replicate for Reverse Incremental Migration

- The logical replication function needs to be enabled in openGauss by setting the GUC parameter **wal_level** to **logical**. Only users with the REPLICATION permission can perform this operation.
- The **pg_hba.cnf** file on the database node needs to be adjusted to allow replication. (The value depends on the actual network configuration and the users who are connected to.)

```
host        replication        repuser        0.0.0.0/0        sha256
```

- A database in openGauss corresponds to a logical replication slot. When the database to be migrated changes, you need to reconfigure the name of the logical replication slot.
- Reverse incremental migration does not support DDL operations.
- The source database version must be openGauss 3.0.0 or later.

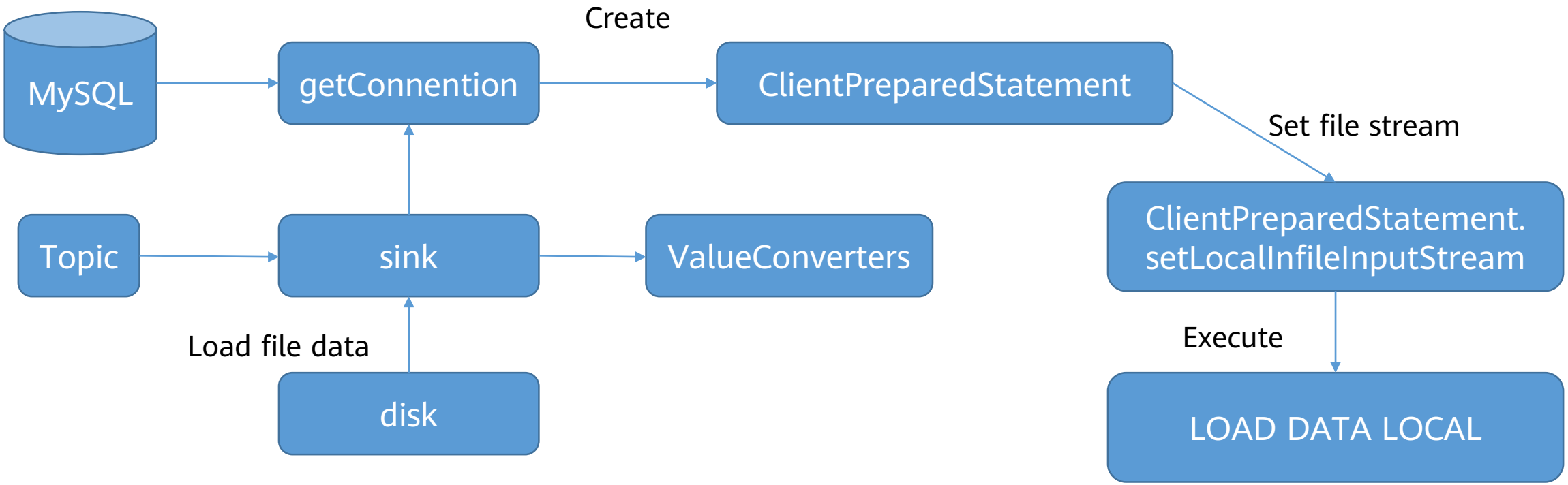
HUAWEI

Reverse Full Data Migration - Principles of the Data Collection Module and Data Replay Module (1)

- Implementation principle of the data collection module
  - The database table metadata is obtained to estimate the amount of data that is divided and distributed to files. When the amount of collected data reaches the estimated amount, an I/O operation is performed. After the data is flushed to disks, full Kafka messages are assembled and the file information is sent to Kafka for interaction with the replay data module.
- Implementation principle of the data replay module
  - The sink consumes Kafka messages, converts the data type of the file data, and uses JDBC to replay the data.

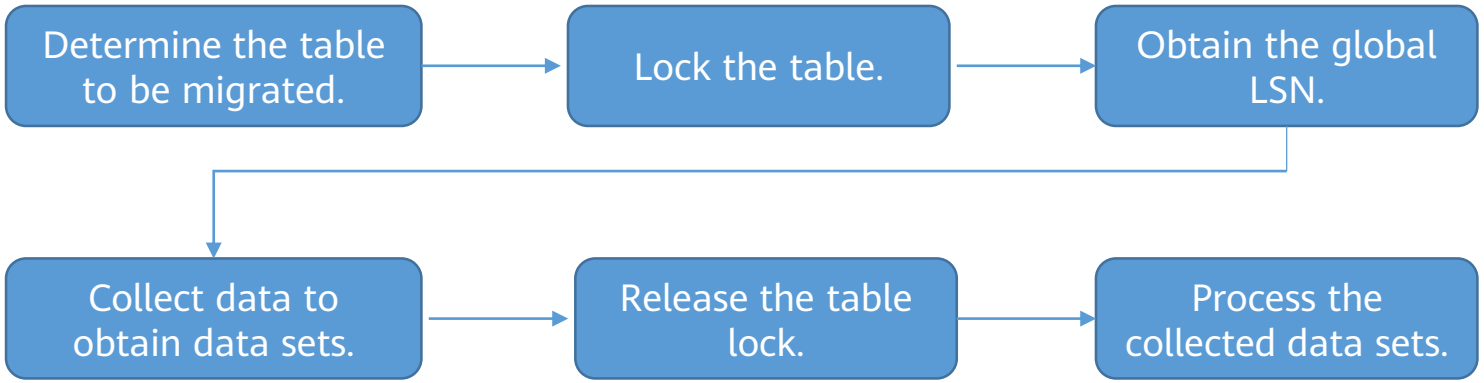
HUAWEI

Reverse Full Data Migration - Principles of the Data Collection Module and Data Replay Module (2)



Reverse Full Data Migration – Interconnecting with Reverse Incremental Migration

- Interconnecting with reverse incremental migration: During incremental migration, Xlogs are parsed to capture data changes. Full and incremental migration needs to be interconnected by the start point of capturing the changes. The following figure shows the steps of full migration.
  - Determine the table to be migrated.
  - Lock the table.
  - Obtain the global LSN (start point of incrementally capturing changes).
  - Collect data to obtain data sets.
  - Release the table lock.
  - Process the collected data sets.



Reverse Full Data Migration - Usage and Restrictions

- The method is the same as that of reverse incremental migration. You need to create the following configuration files on the source and sink:
- **openGauss-source.properties** and **openGauss-sink.properties**
- Setting on the source: **snapshot.mode** is set to **always**.
- The operation is fully adaptable on the sink.
- Restrictions
  - Only full data migration from openGauss to MySQL is supported.
  - MySQL parameter setting: **local_infile** must be set to **1**.
  - Other restrictions of reverse incremental migration are inherited.

Contents

1. Data Migration Overview
2. Syntax Differences Between openGauss and MySQL
3. **openGauss Migration Tool Set**
   - Compatibility Evaluation Tools
   - Full Migration and Incremental Migration
   - Full Verification and Incremental Verification
   - Reverse Incremental Migration and Reverse Full Migration
   - Migration Portal and One-Stop Migration Platform DataKit

HUAWEI

One-Click Migration Tool gs_rep_portal

- The one-click migration tool gs_rep_portal integrates tools for full migration, incremental migration, reverse migration, and data verification. gs_rep_portal allows you to install the preceding tools with one click and set migration tasks. Next, gs_rep_portal calls tools to complete each migration step based on the user-defined execution sequence of migration tasks and display the status, progress, and exception cause of each step.
- By default, there are three types of execution plans on the portal.

Plan	Instruction
Plan 1	Full migration → Full verification
Plan 2	Full migration → Full verification → Incremental migration → Incremental verification
Plan 3	Full migration → Full verification → Incremental migration → Incremental verification → Reverse migration

Starting the One-Click Migration Tool gs_rep_portal

- The command for starting the portal consists of the following parts:

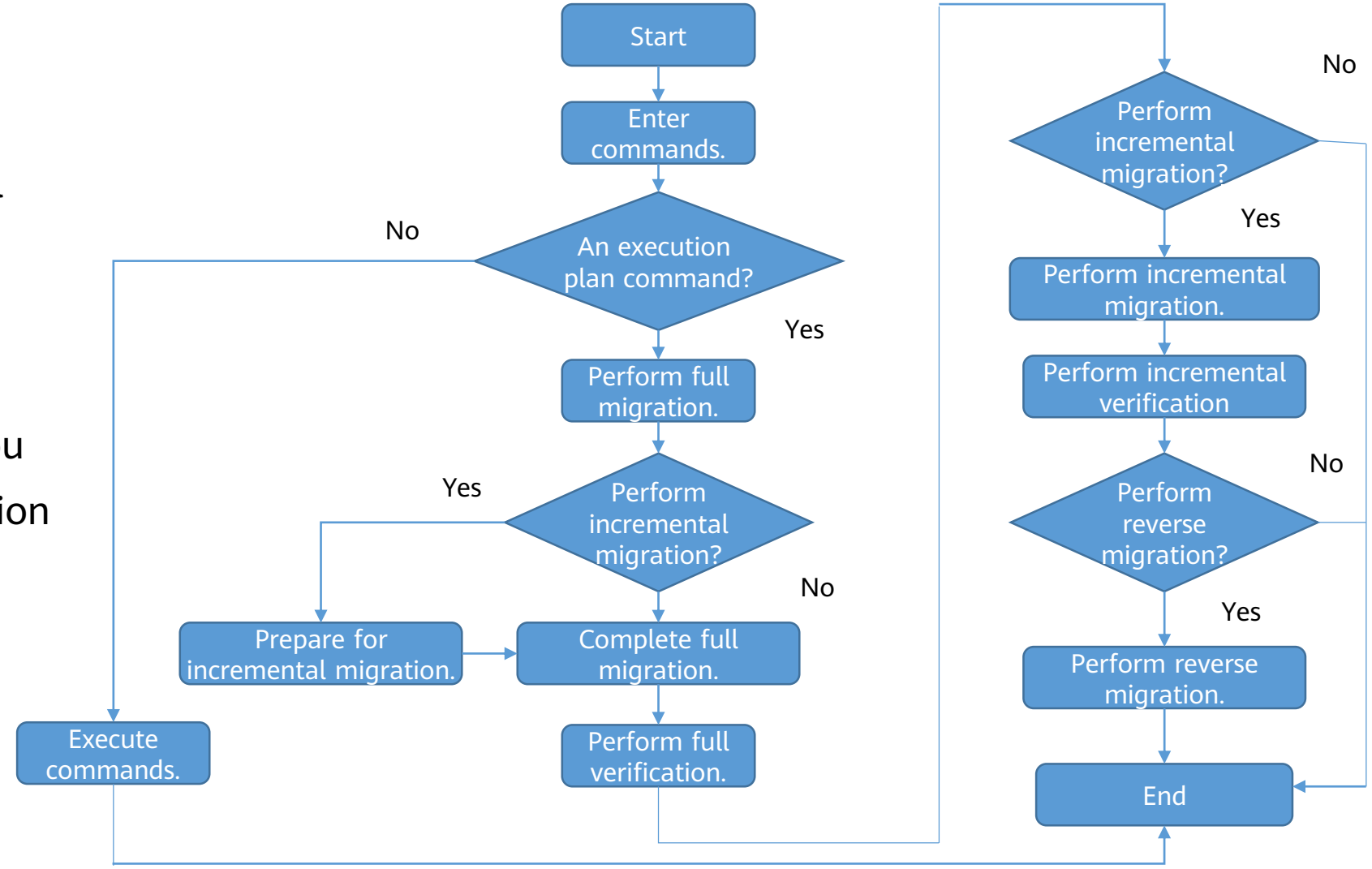
  ```
  sh gs_rep_portal.sh command id
  ```

- The *command* part is the command used for starting Kafka, full migration, a migration plan, or other operations.
- Each time you run the portal, you need to specify an *id* for the command. The command ID can contain lowercase letters and digits.
- The command execution log is stored in **portal/logs** and named **portal_id.log**.
- If the migration command is used, the **workspace/id/logs** folder contains the run logs of each migration tool.
- Example: `sh gs_rep_portal.sh start_mysql_full_migration k1`
- In the preceding command, setting **workspace.id** to **k1** is to start MySQL full migration.

HUAWEI

Workflow of the One-Click Migration Tool gs_rep_portal

- MySQL full-process migration includes five phases: full migration, full verification, incremental migration, incremental verification, and reverse incremental migration. You can determine the migration procedure based on the execution plan.



HUAWEI

Precautions for the One-Click Migration Tool gs_rep_portal

- If the execution plan contains both full migration and incremental migration, the portal needs to obtain the snapshot point from the full migration and start the source database for the incremental migration in advance. This improves the performance of the incremental migration and reduces the waiting time. Therefore, you need to prepare for incremental migration during the full migration.
- Incremental migration is continuous. If you do not run the command to stop incremental migration, it continues.
- Reverse migration refers to reverse incremental migration, which is also continuous. You need to open another command line window for running commands, such as stopping incremental migration, starting reverse migration, and stopping plans.
- Except for full migration, other migration tools depend on Kafka. The portal automatically runs the **start_kafka** command when installing the migration tool. Therefore, you can directly start the migration plan. If Kafka is faulty, you can run the **stop_kafka** command to stop Kafka, delete cache files, and run the **start_kafka** command to restart Kafka.

HUAWEI

One-Click Migration Tool gs_rep_portal – Examples (1)

- Steps to execute a complete migration plan (CentOS 7 is used as an example) are as follows:
- Download the portal installation package from the specified link.

```
wget -c https://openGauss.obs.cn-south-1.myhuaweicloud.com/latest/tools/centos7/PortalControl-5.1.0-x86_64.tar.gz
```

- Decompress the gs_rep_portal installation package.

```
tar -zxvf PortalControl-5.1.0-x86_64.tar.gz
```

- Change the installation path in the **toolspath.properties** file in the **/ops/portal/config** directory and run the installation command.

```
sh gs_rep_portal.sh install_mysql_all_migration_tools 1 &
```

- Modify migration parameters in the **migrationConfig.properties** file in the **/ops/portal/config** directory.

HUAWEI

One-Click Migration Tool gs_rep_portal – Examples (2)

- Steps to execute a complete migration plan (CentOS 7 is used as an example) are as follows:
- Set **workspace.id** to **2** and start the migration plan 3.

```
sh gs_rep_portal.sh start_plan3 2 &
```

- Check that the program automatically runs until both incremental migration and incremental data verification are enabled. Open another window to stop the incremental migration task whose **workspace.id** is **2**. In this case, the program enters the waiting state. Then, you can start reverse migration or directly stop the plan.

```
sh gs_rep_portal.sh stop_incremental_migration 2 &
```

- Start the reverse migration. In this case, the program enters the reverse migration state. Then, you can stop the plan.

```
sh gs_rep_portal.sh run_reverse_migration 2 &
```

- Stop the plan whose **workspace.id** is **2**.

```
sh gs_rep_portal.sh stop_plan 2 &
```


HUAWEI

Integrated Tool Platform DataKit

- openGauss offers DataKit, a toolkit that manages data throughout its lifecycle, covering installation and deployment, data development, intelligent O&M, and data migration. Debezium, a reverse incremental migration tool, can synchronize incremental data from openGauss to MySQL in real time.
- As a toolkit of openGauss, DataKit uses a browser/server (B/S) architecture to manage software and hardware resources such as servers and database instances. In addition, it can be extended through on-demand deployment of function plug-ins, which currently include installation and deployment, data migration, data development, and intelligent O&M.
- DataKit is also a tool development platform that allows users to implement their own functions based on plug-in APIs and specifications. Function plug-ins can be dynamically loaded to and run on DataKit or dynamically uninstalled from DataKit. Plug-ins can call open APIs of the platform or other plug-ins to obtain corresponding data and functions, implementing quick construction of featured functions.

HUAWEI

Main Functions of the Integrated Tool Platform DataKit

1. Plug-in platform:
   - Front-end and back-end integrated framework for developing plug-ins
   - Implementation of plug-ins for MySQL data migration
   - Security center (account management, role permission management, and whitelist management)
2. Plug-in management module:
   - Log center, including operation logs and system logs (levels, splitting rules, and retention period of logs, and log download)
   - Languages (Chinese/English)
   - Themes (black/white)
   - Basic modules (user center, personal password modification, and login/logout)

HUAWEI

Integrated Tool Platform DataKit - Data Development

- DataKit for installation and deployment
  - It supports one-click deployment of the openGauss database, enabling users to quickly install and use the database.
  - The simplified, lite, and enterprise editions of openGauss can be installed and deployed in standalone, primary-standby, or distributed mode.
  - It supports database upgrade and backup.
- DataKit for data development
  - Web Data Studio is a web-based universal integrated development environment. It runs on DataKit as a plug-in to manage openGauss.
  - On the GUIs of Web Data Studio, database development and maintenance personnel can manage databases, including managing database objects, performing operations on data, and developing functions/stored procedures.
  - Web Data Studio also supports function and stored procedure debugging. Database developers can view each step in execution process and variable values through breakpoint operations to quickly and accurately locate exceptions in functions and stored procedures, simplifying troubleshooting.

HUAWEI

Integrated Tool Platform DataKit - Intelligent O&M

- Intelligent O&M provides O&M functions for openGauss. It consists of three function plug-ins.
  - Instance monitoring: The resource consumption of openGauss instances and their running environments are monitored and problems found are analyzed. The main functions include collecting and displaying the running metrics of the database and the system where the database is located in real time, rolling up and drilling down for association analysis of metrics, drilling down top SQL statements to view details, generating WDRs, viewing OS and database configuration parameters, and providing optimization suggestions.
  - Log search: Multiple openGauss instances and OS logs can be collected and displayed. Users can search for these logs by type, level, instance, keyword, and time on a unified GUI. In addition, associated query of log contexts is supported, helping users quickly detect database exceptions from logs.
  - SQL diagnosis: Full-stack trace can be performed for SQL statements that are running to find the root cause of their exceptions. SQL execution plans and database system parameters can be analyzed to provide intelligent diagnosis and optimization suggestions.

HUAWEI

Integrated Tool Platform DataKit - Data Migration (1)

- Data migration plug-ins provide all capabilities required for MySQL migration, including:
  - Object migration: Objects in MySQL, including tables, indexes, constraints, views, stored procedures, functions, and triggers, can be migrated to openGauss.
  - Full data migration: All MySQL data can be concurrently migrated to openGauss at 300 MB/s.
  - Incremental data migration: Binlogs in MySQL are read and converted into SQL statements for replay in openGauss. Parallel replay is supported to ensure transaction consistency. The migration performance reaches 30,000 TPS.
  - Full data verification: Data in the source and target databases is fully verified by table to find differential records, provide rectification suggestions, and generate a verification report.

HUAWEI

Integrated Tool Platform DataKit - Data Migration (2)

- Data migration plug-ins provide all capabilities required for MySQL migration, including:
  - Migration task management: Users can configure migration tasks to migrate data from one or more sources to the target.
  - Migration process monitoring: Users can view the migration progress, logs, and error information in real time and handle the problems accordingly.
  - Concurrent execution: Users can create multiple migration tasks and execute them concurrently to improve migration efficiency.
  - Dynamic resource allocation: The system can properly allocate running machines based on the maximum utilization principle to implement parallel migration.
  - There are two migration modes: offline and online. The offline mode includes two phases: full migration and full verification. The online mode includes five phases: full migration, full verification, incremental migration, incremental verification, and reverse migration.

HUAWEI

Integrated Tool Platform DataKit - Procedure (1)

- Procedure:
  - Create the **config/application-cus.yml** file.

```
keytool -genkey -noprompt -dname "CN=openGauss, OU=openGauss, O=openGauss, L=Beijing, S=Beijing, C=CN" -alias openGauss -storetype PKCS12 -keyalg RSA -keysize 2048 -keystore /ops/ssl/keystore.p12 -validity 3650 -storepass 123456
```

Integrated Tool Platform DataKit - Procedure (2)

- Generate a key.

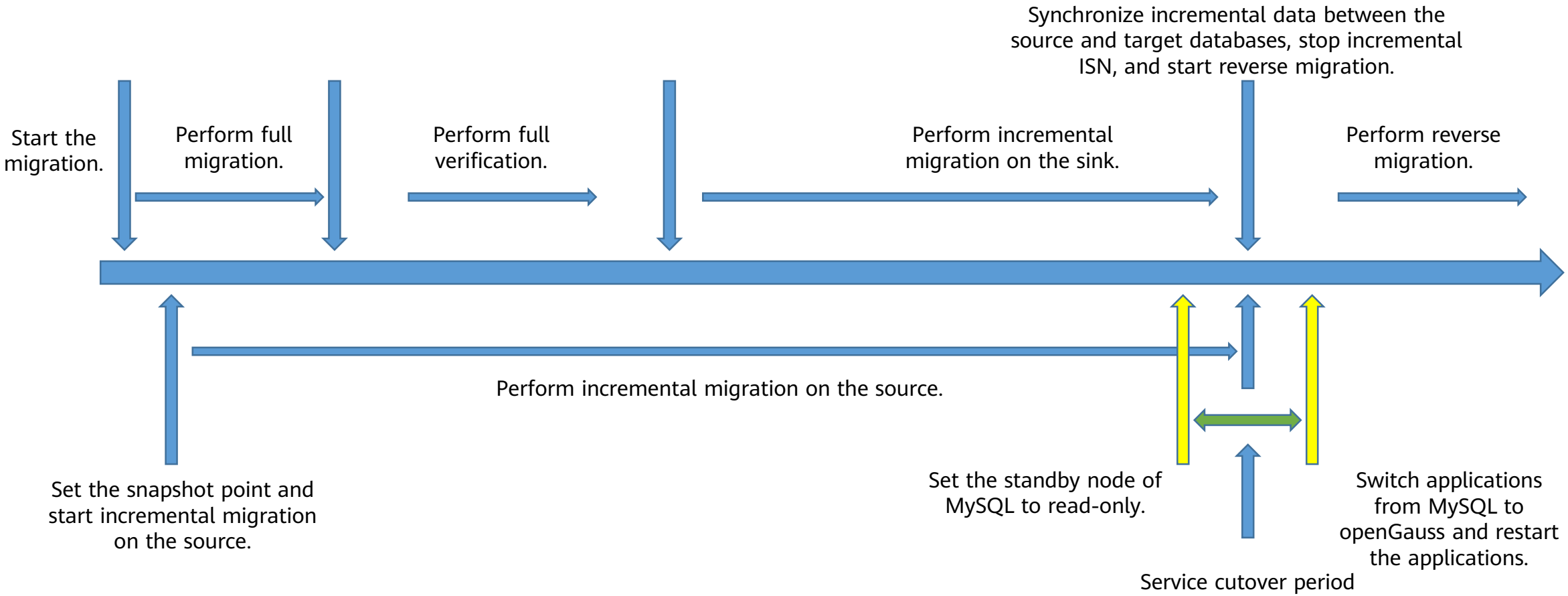
```
nohup java -Xms2048m -Xmx4096m -jar visualtool-main.jar --spring.profiles.active=cus > ./visualtool-main.out 2>&1
```

- Start the main program.
- Install the plug-in on the GUI.

HUAWEI

Integrated Tool Platform DataKit - Workflow

- MySQL full-process migration includes five phases: full migration, full verification, incremental migration, incremental verification, and reverse incremental migration. You can use DataKit to complete one-click migration.



HUAWEI

Section Summary

- This chapter describes the openGauss data migration toolkit, including the compatibility evaluation tools, full migration, incremental migration, full verification, incremental verification, reverse incremental migration, reverse full migration, one-click migration portal, and one-click migration platform DataKit, and summarizes the entire MySQL migration process.

HUAWEI

Quiz

1. (Single-choice) Which of the following methods is used by the verification tool gs_datacheck to extract data?

   A. ODBC

   B. JDBC

   C. Psycopg

   D. libpq

2. (True or False) The Dolphin plug-ins and kernel can achieve full syntax compatibility between MySQL and openGauss. In addition, Dolphin plug-ins enable the MySQL client and JDBC to directly access openGauss without modifying any applications.

   A. True

   B. False

HUAWEI

Summary

- This chapter provides an overview of data migration, describes the syntax differences between MySQL and openGauss, introduces the implementation principles, restrictions, and usage of the openGauss migration toolkit, and illustrates the MySQL migration process.

HUAWEI

Recommendations

- openGauss Migration Guide:

  https://docs-opengauss.osinfra.cn/en/docs/latest/docs/DataMigrationGuide/data-migration-guide.html

- openGauss Dolphin plug-ins:

  https://docs-opengauss.osinfra.cn/en/docs/latest/docs/ExtensionReference/dolphin-extensions.html

HUAWEI

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 6 Database Security Management



HUAWEI

Foreword

- This chapter describes database security from six aspects: client access authentication, users and roles, database audit, encrypted equality query, ledger databases, and TDE.

HUAWEI

Objectives

- Upon completion of this course, you will be able to:
  - Use the parameter configuration of client access authentication to manage certificates.
  - Set the separation-of-duties mode, configure user and role permissions, and set security policies.
  - Manage the database audit log function, and view, delete, and back up audit logs.
  - Use gsql to perform encrypted equality queries.
  - View historical records in the account book database and verify account book data consistency.

HUAWEI

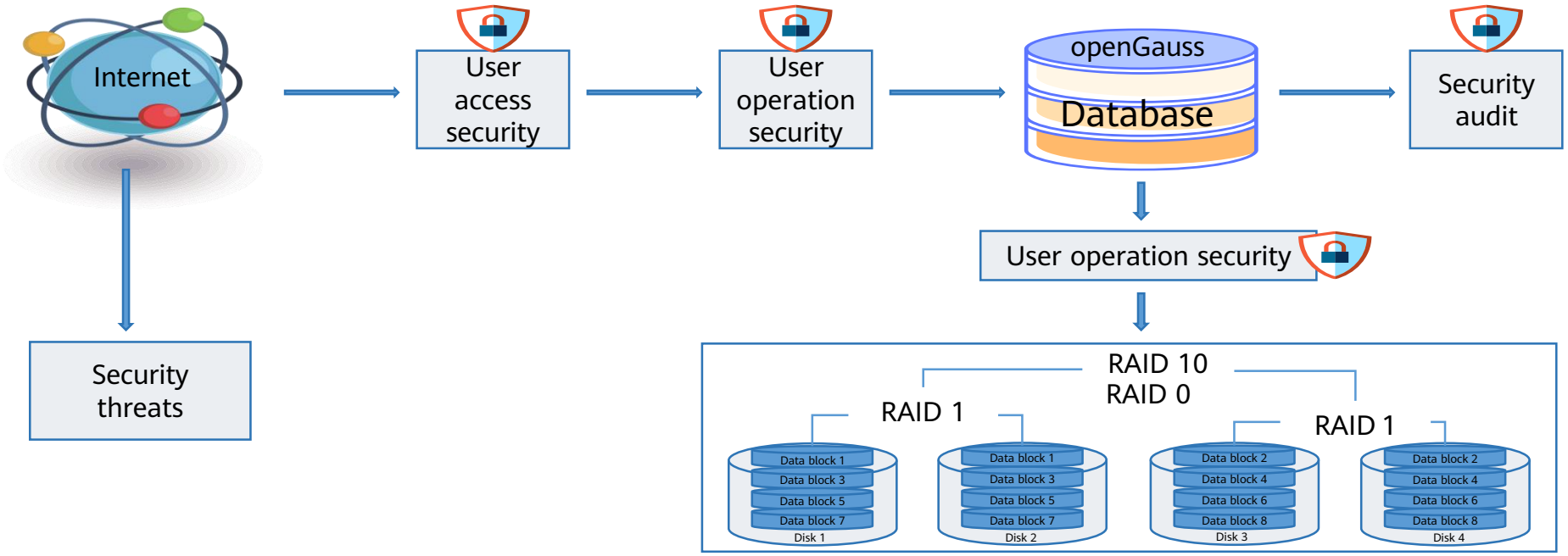
Contents

1. **Client Access Authentication**
2. Users and Roles
3. Database Audit
4. Encrypted Equality Query
5. Ledger Database
6. TDE

HUAWEI

Client Access Authentication

- There are many factors that affect database security, such as user access security, user operation security, data storage security, and security audit. Database user access security (client access authentication) is one of the key security measures to ensure that only authorized clients can access the database system.



Access Control Rules (1)

- Users must be authenticated before they can access a database. During the authentication, the database verifies the username, password, and the IP address of the client host. If SSL encryption is enabled, the database also verifies the SSL certificate.
- Before accessing a database, you need to configure client details and authentication mode in the **pg_hba.conf** configuration file on the server.
  - In the **pg_hba.conf** configuration file, each line specifies an authentication rule. Earlier authentication rules have higher priorities. The format of a rule is as follows:
  - Format: TYPE DATABASE USER ADDRESS METHOD
  - Example: **host postgres omm 192.168.0.1/32 sha256**
  - **TYPE**: database connection type. It can be **local**, **host**, **hostssl**, or **hostnossl**.

HUAWEI

Access Control Rules (2)

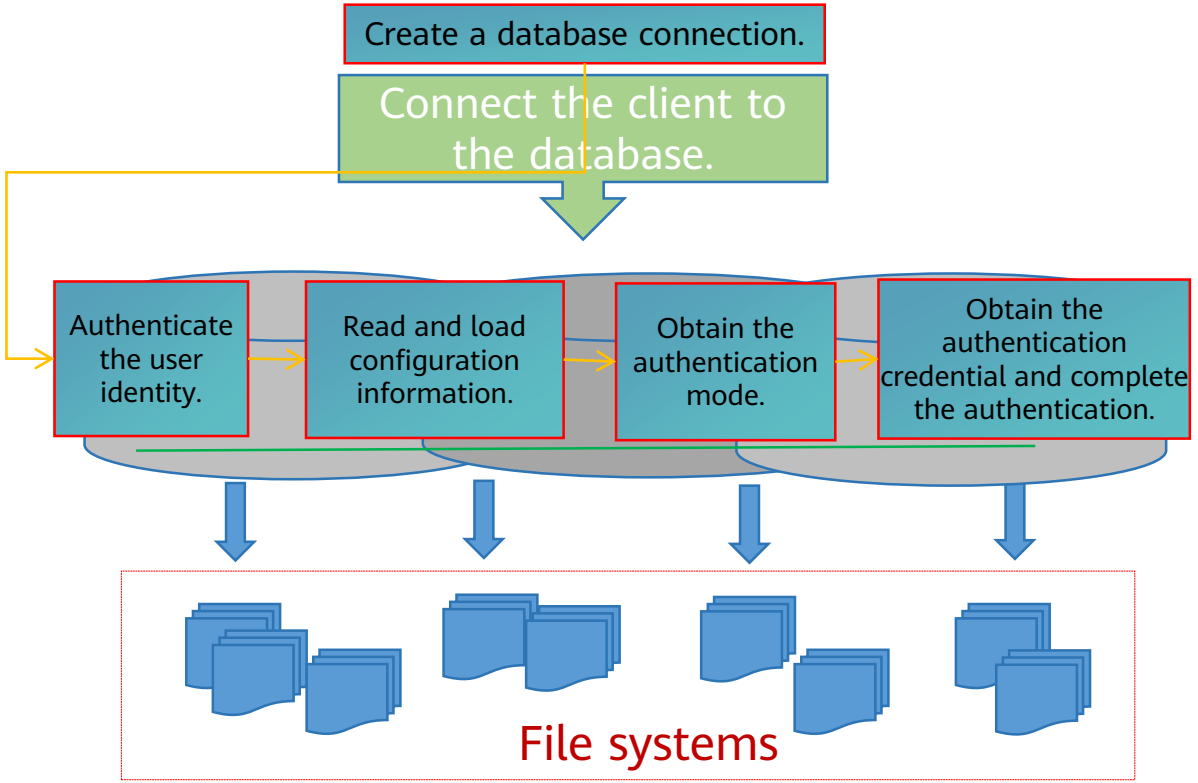
- **DATABASE**: name of the database that matches the record. Multiple databases are separated with commas (,). The value **all** indicates that all databases are matched.
- **USER**: database user that matches the record. Multiple usernames are separated with commas (,). The value **all** indicates that all users are matched.
- **ADDRESS**: IP address that matches the record. Both IPv4 and IPv6 addresses are supported.
- **METHOD**: authentication method used for connection. The value can be **sha256** (recommended), **cert** (certificate authentication), or **reject** (blacklist).

- You can configure authentication rules for **pg_hba.conf** using gs_guc.
  - gs_guc reload -Z coordinator -N all -I all -h "host postgres user 192.168.0.1/32 sha256"

HUAWEI

Client Access Authentication

- openGauss access authentication includes authentication configuration file management, user identification, and password verification. The purpose of authentication is for anti-spoofing. The following figure shows the client access authentication process.



Client Access Authentication (1)

- openGauss access authentication configuration (**pg_hba.conf**)
- In the **pg_hba.conf** configuration file, each line specifies an authentication rule. All whitespace and comment lines started with a number sign (#) will be ignored. Each authentication rule consists of multiple fields separated by spaces, slashes (/), or tab characters. If a field is enclosed with quotation marks ("), it can contain spaces. A rule cannot span multiple lines.

HUAWEI

Client Access Authentication (2)

- The following figure shows the format of the **pg_hba.conf** configuration file.

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD

# "local" is for Unix domain socket connections only
local   all             all                                     trust
host    all     omm     7.213.197.105/32    trust
host    all     omm     7.213.202.227/32    trust
host    all     omm     7.213.208.176/32    trust
# IPv4 local connections:
host    all             all             127.0.0.1/32            trust
host    all     all     7.213.197.105/32    sha256
host    all     all     7.213.202.227/32    sha256
host    all     all     7.213.208.176/32    sha256
host all all 0.0.0.0/0 md5
# IPv6 local connections:
host    all             all             ::1/128                 trust
# Allow replication connections from localhost, by a user with the
# replication privilege.
#local   replication     omm                                     trust
#host    replication     omm         127.0.0.1/32                trust
#host    replication     omm         ::1/128                     trust
```


HUAWEI

Client Access Authentication (3)

- When a client accesses the database, the authentication modes listed in the following table are used.

Authentication Mode	Description
SHA-256	Cryptographic hash function that converts input data into a 256-bit (32-byte) hash value.
MD5	Common hash function that converts input data into a 128-bit (16-byte) hash value.
GSSAPI	Kerberos authentication based on GSSAPI. It is a common security service API used to provide authentication and secure communication services.
Cert	Digital certificate, which is usually generated and verified using an asymmetric encryption algorithm.
Reject	Specifies that the connection from a specific host is rejected unconditionally.
Trust	Set of methods and techniques for calculating or inferring trust, credibility, or credit.

HUAWEI

Client Access Authentication (4)

- openGauss supports the SSL standard protocol (TLS 1.2) and can be used for secure TCP/IP connections. SSL (Secure Sockets Layer) is a robust security protocol that facilitates two-way authentication between the server and client. It achieves this through the use of digital signatures and digital certificates, ensuring secure data transmission.

Classification of SSL Authentication	Description	Client Environment Variable
Two-way authentication	The client verifies the server's certificate and the server verifies the client's certificate. Connection can be set up after the verification is successful.	Set the following environment variables: *PGSSLCERT*, *PGSSLKEY*, *PGSSLROOTCERT*, *PGSSLMODE*
One-way authentication	The client verifies the server certificate, but the server does not verify the client certificate. The server loads the certificate information and sends it to the client. The client verifies the server's certificate according to the root certificate.	Set the following environment variables: *PGSSLROOTCERT* *PGSSLMODE*

Client Access Authentication (5)

- Precautions:
  - You are required to use the SHA-256 authentication method when you remotely access the primary node of the database.
  - For connections between internal servers, the trust authentication mode is used, and IP address whitelist authentication is supported.
  - To use SSL connections, you must apply for a formal server certificate, client certificate, and key from the CA in advance.

HUAWEI

Authentication Mechanism Verification (1)

- Check whether the IP address of a client is valid.
  - The client IP address should be set in the **pg_hba.conf** configuration file on the server and the authentication method is not **reject**.
- Check whether a user being authenticated exists or is locked.
  - If the user does not exist or is automatically or manually locked, the authentication fails.
- Check whether a password is correct.
  - A password authentication method based on RFC5802 should be used. A client encrypts the password using HMAC and sends the ciphertext to the server. The server calculates the ciphertext using HMAC and then compares it with the ciphertext stored in the system catalog. If they are the same, the authentication is successful.

HUAWEI

Authentication Mechanism Verification (2)

- Check the setting of **DATABASE**.
  - Check whether **DATABASE** is specified in the **pg_hba.conf** configuration file. If it is not specified, login authentication fails.
- Check the account validity period
  - **VALID BEGIN** and **VALID UNTIL** in the CREATE ROLE or ALTER ROLE syntax can be used to set the validity period for an account. If the time is not within the validity period, authentication fails. If the parameters are not set, the account will be always valid.
- Check whether the maximum number of connections to a database has been reached.
  - The maximum number of connections allowed for a database can be set by user, database, or CN/DN instance. If the number of connections to a database reaches the maximum allowed, new connections cannot be established.

HUAWEI

User Password Policy (1)

- Accounts can be automatically locked and unlocked.
  - An account will be locked if the number of failed password attempts exceeds the value of **failed_login_attempts**. The account is automatically unlocked after a period of time equal to the preset value of **password_lock_time**. Administrators can also lock or unlock accounts manually.
- The password encryption type can be configured.
  - It is specified by the parameter **password_encryption_type**. By default, SHA-256 is used for password encryption. MD5 encryption is supported, but is not recommended.

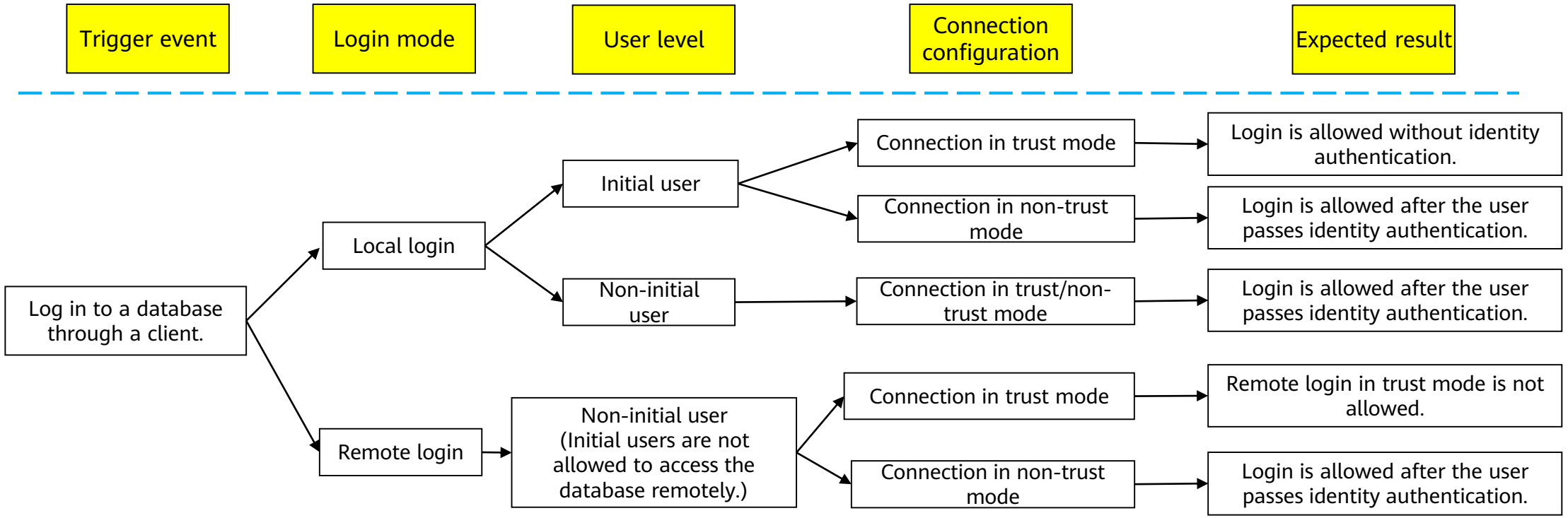
HUAWEI

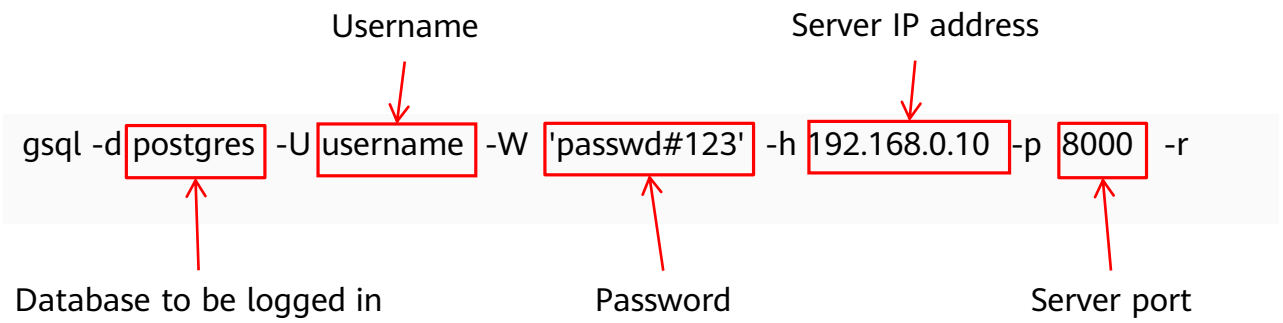
User Password Policy (2)

- Check password complexity.
  - The parameter specifies the password complexity requirements. A password must contain at least eight characters and cannot be the same as the username or the username spelled backwards. A password must contain at least three types of the following characters: uppercase letters, lowercase letters, digits, and special characters.
- Check whether a password is weak.
  - You can run the **CREATE WEAK PASSWORD DICTIONARY** statement to create a weak password.
- Set the number of days before which a password cannot be reused or the number of password changes required before the current password can be reused.
- Set the password validity period.

HUAWEI

Logging in to a Database Through a Client





You can connect to a database using the gsql, JDBC, or ODBC client. For details, see "Application Development Guide" in the user manual.

HUAWEI

Configuring the SSL Connection (1)

- Generate the SSL certificate.
  - An SSL certificate file is generated when the database is installed. You can use the default certificate or replace the SSL certificate as required. For details about how to generate a certificate, see section "Managing SSL Certificates" in *Developer Guide*.
- Configure server parameters.

Parameter	Description	Default Value
ssl	Specifies whether to enable SSL.	**on**, indicating that SSL is enabled
ssl_cert_file	Specifies certificate files of the server, including the public key.	**server.crt**
ssl_key_file	Specifies the private key file for the server, used to decrypt data encrypted using the public key.	**server.key**
ssl_ca_file	Specifies the root certificate of the CA server.	**cacert.pem**

HUAWEI

Configuring the SSL Connection (2)

- Configure environment variables for a client.
  - For the gsql client, you need to configure the *PGSSLMODE* environment variable to specify whether to negotiate the SSL connection with the server. If this parameter is not set, the default value **prefer** is used, indicating that the SSL connection is preferred. If the connection fails, a non-SSL connection is used.
  - For a JDBC client, you need to configure the parameter **sslmode**. You are advised to set this parameter to **prefer** or **require**. **require** indicates that the system only attempts to set up an SSL connection.
  - For details about other clients and parameter options, see *Developer Guide*.

HUAWEI

Contents

1. Client Access Authentication
2. **Users and Roles**
3. Database Audit
4. Encrypted Equality Query
5. Ledger Database
6. TDE

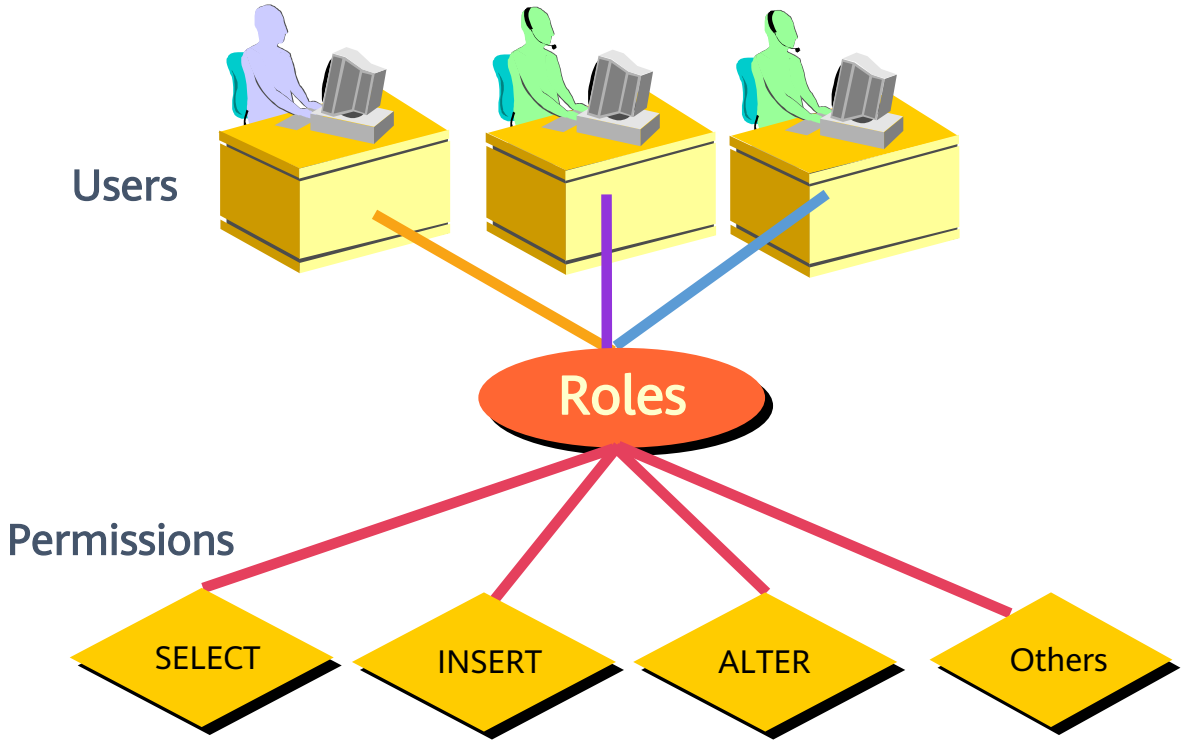
HUAWEI

Users and Roles (1)

- Database user: an entity that is authorized to access and manage the database system, that is, a person who uses and manages database resources. Each database management system has its own user management policy to manage the access permissions, roles, and functions of database users.
- Database role: an important concept used to manage and organize database user permissions. A role allows a database administrator to assign a set of permissions to a role and then assign the role to one or more database users. This simplifies permission management and improves security and maintainability.
- Database permission: capability of controlling users or roles to perform specific operations on database objects (such as tables, views, and stored procedures) in the database management system. Database permissions play an important role for ensuring database security and data protection. Database permissions are classified into system permissions and object permissions.

HUAWEI

Users and Roles (2)



HUAWEI

Role Management (1)

- Create a user or role.

```
CREATE USER/ROLE role_name [ [ WITH ] option [ ... ] ] [ ENCRYPTED | UNENCRYPTED ] {
PASSWORD | IDENTIFIED BY } { 'password' [EXPIRED] | DISABLE };
```

  - Note: **UNENCRYPTED** is prohibited.

- Modify a user or role.

```
ALTER USER/ROLE role_name [ [ WITH ] option [ ... ] ];
```

- Delete a user or role.

```
DROP USER/ROLE [ IF EXISTS ] role_name [, ...];
```

- Grant or revoke a user or role.

```
GRANT role_ADMINname TO role_name [ WITH ADMIN OPTION ];
REVOKE [OPTION FOR ] role_name FROM role_name [ CASCADE | RESTRICT ];
```


HUAWEI

Role Management (2)

- The syntax of the option clause is as follows:

```
{SYSADMIN | NOSYSADMIN}
| {MONADMIN | NOMONADMIN}
| {OPRADMIN | NOOPRADMIN}
| {POLADMIN | NOPOLADMIN}
| {AUDITADMIN | NOAUDITADMIN}
| {CREATEDB | NOCREATEDB}
| {USEFT | NOUSEFT}
| {CREATEROLE | NOCREATEROLE}
| {INHERIT | NOINHERIT}
| {LOGIN | NOLOGIN}
| {REPLICATION | NOREPLICATION}
| {INDEPENDENT | NOINDEPENDENT}
| {VCADMIN | NOVCADMIN}
| {PERSISTENCE | NOPERSISTENCE}
```


HUAWEI

Role Management - Examples (1)

- Create roles **role1** and **role2** and set their passwords to **GaussDB@2012**.

```
postgres=# CREATE ROLE role1 WITH password 'GaussDB@2012';
CREATE ROLE
postgres=# CREATE ROLE role2 WITH password 'GaussDB@2012';
CREATE ROLE
```

- Check the system catalog pg_authid or system view pg_roles to obtain role information.

```
postgres=# SELECT oid,rolname,rolcanlogin,rolpassword FROM pg_roles WHERE rolname='role1'
or rolname='role2';
   oid    | rolname  | rolcanlogin | rolpassword
----------+----------+-------------+-------------------
 16442 |  role2      | f                | ********
 16439 |  role1      | t                | ********
```


HUAWEI

Role Management - Examples (2)

- Set the LOGIN attribute for **role1**.

```
postgres=# ALTER ROLE role1 WITH LOGIN;
ALTER ROLE
```

- Grant **role1** to **role2**. In this case, **role2** belongs to group **role1** and inherits the permissions of **role1**. You can obtain the role membership from the pg_auth_members system catalog.

```
postgres=# GRANT role1 TO role2;
GRANT ROLE
postgres=# SELECT * FROM pg_auth_members;
  roleid  | member | grantor  | admin_option
----------+------------+-----------+------------------
  16439  |  16442    |        10   | f
```

- Delete roles **role1** and **role2**.

```
postgres=# DROP ROLE role1,role2;
DROP ROLE
```


HUAWEI

System Permissions (1)

- System permissions, also known as properties of a user, refer to the permissions of a user to use a database, such as connecting to the database (LOGIN), creating a database (CREATEDB), and creating a role (CREATEROLE).
- System permissions are specified by using CREATE USER/ROLE and ALTER USER/ROLE. System permissions cannot be inherited by a role.
- Create **role1** and grant the permissions for creating users and databases to **role1**.

```
CREATE ROLE role1 WITH CREATEROLE CREATEDB password "openGauss@2023";
```

- Grant the permissions of monitor administrators to **role1** and revoke the permission for creating databases.

```
ALTER ROLE role1 WITH MONADMIN NOCREATEDB;
```

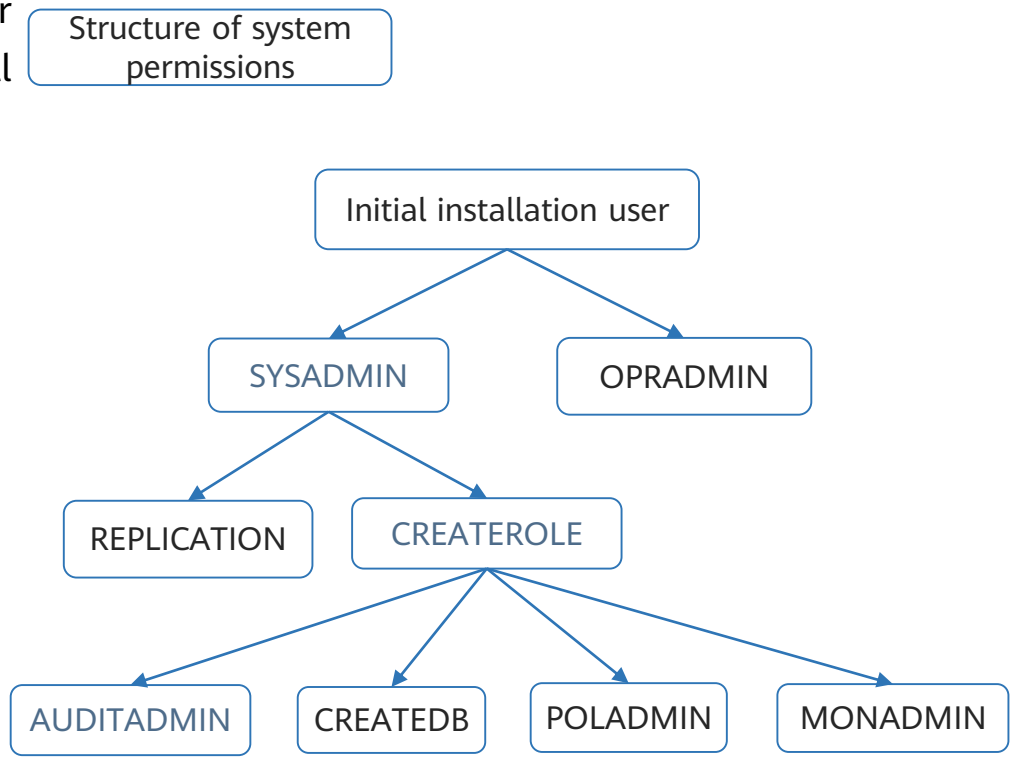
- Check the system catalog pg_authid or system view pg_roles to obtain role information.

```
postgres=# SELECT rolname,rolcreaterole,rolcreatedb,rolmonitoradmin FROM pg_roles
WHERE rolname='role1';
 rolname | rolcreaterole | rolcreatedb | rolmonitoradmin
-------------+------------------+----------------+----------------------
 user1       | t                    | f                   | t
```


HUAWEI

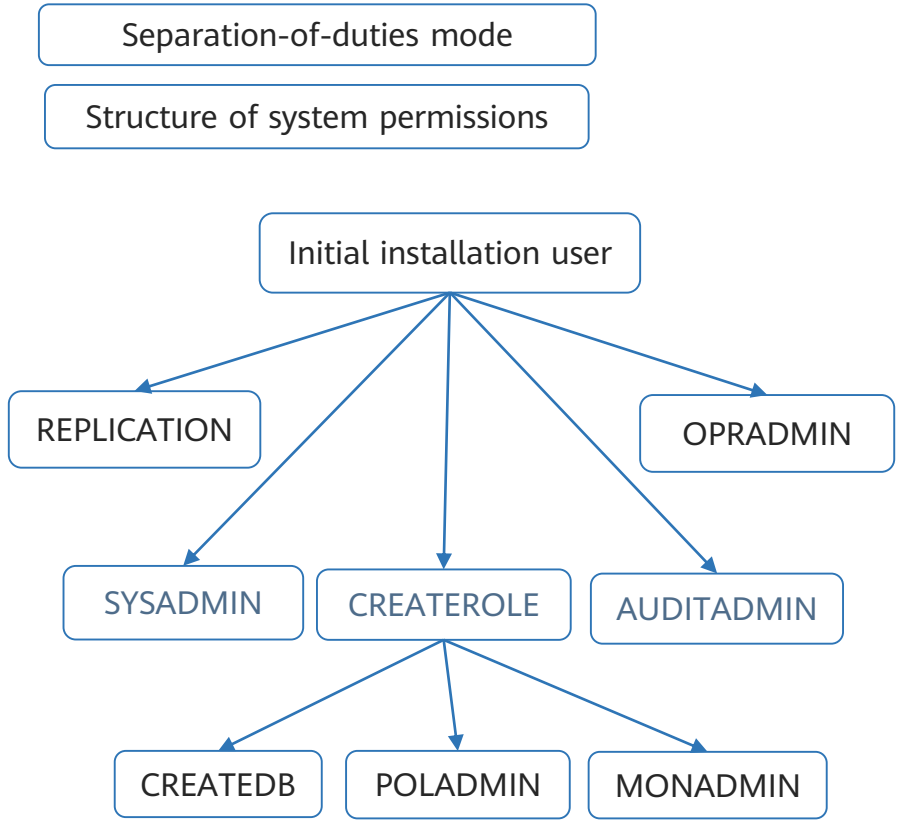
System Permissions (2)

- Initial installation user: an account automatically generated during cluster installation. This account has the full system permissions. It can perform all operations.
  - SYSADMIN: system administrator permissions, which are inferior only to those of the initial installation user. By default, a system administrator has the same permissions as the object owner, excluding the permissions of monitor administrators and O&M administrators.
  - MONADMIN: monitor administrator permissions, including the permissions to access and grant views and functions in the monitor schema DBE_PERF.
  - OPRADMIN: O&M administrator permissions, including the permission to use Roach to perform backup and restoration.
  - CREATEROLE: security administrator permissions, including the permissions to create, modify, and delete users and roles.
  - AUDITADMIN: audit administrator permissions, including the permissions to view and maintain database audit logs.
  - CREATEDB: permission to create databases.
  - POLADMIN: security policy administrator permissions, including the permissions to create resource labels, dynamic data masking policies, and unified audit policies.



System Permissions - Separation of Duties

- Set the GUC parameter **enableSeparationOfDuty** to **on**.
  - SYSADMIN: system administrator permission. A user with this attribute no longer has the permissions to create, modify, or delete users or roles, or view or maintain database audit logs.
  - CREATEROLE: security administrator permissions, including the permissions to create, modify, and delete users and roles.
  - AUDITADMIN: audit administrator permissions, including the permissions to view and maintain database audit logs.
  - A user or role can have only one of the system permissions SYSADMIN, CREATEROLE, and AUDITADMIN.



Users and Roles - Separation of Duties

- In actual service management, you can set separation of duties to prevent system administrators from having excessive centralized permissions, which may cause high risks. Some permissions of a system administrator are transferred to the security administrator and audit administrator, implementing separation of duties among the system administrator, security administrator, and audit administrator.

Role	Permission
Database system administrator	Manages and maintains the database system, including creating, configuring, maintaining, backing up, and restoring the database. Generally, a database system administrator has the highest permission and can access and modify all data and structures of a database.
Security administrator	Manages and maintains database security. Major responsibilities and tasks include access control, identity authentication and authorization, and vulnerability management. This ensures that sensitive information stored in the database system is protected against unauthorized access, data leakage, and other potential security threats.
Audit administrator	Plans, implements, and maintains database audit policies. Major responsibilities and tasks include audit policy formulation, audit setting, audit log management, audit event monitoring, and audit data analysis. This helps detect and cope with potential database security threats in a timely manner.

HUAWEI

Users and Roles (1)

- Run the **GRANT** statement to grant system permissions, object permissions, and roles to a user. Ensure that you have the permissions to be granted. Run the **REVOKE** statement to revoke system permissions, object permissions, and roles that have been granted to a user (or role). Ensure that you have the permissions to grant them.
- **GRANT**: grants permissions to roles and users. You can run the **GRANT** command to grant permissions of a role or user to other roles or users, and grant database object permissions to a role or user.
- **REVOKE**: revokes permissions from roles, database objects, specified columns in tables, specified database, specified schema, or specified tablespace.

HUAWEI

Users and Roles (2)

- Examples of permission granting and revoking:
  - Grant permissions on a database.

```
grant all privileges on database mydb to user3;
```

  - Grant permissions on a tablespace.

```
grant all privileges on tablespace tbs1 to user3;
```

  - Grant permissions on a schema.

```
grant usage on schema sch1 to user3;
```

  - Grant permissions on a function.

```
grant grant execute on function proc_emp(var_empno int, OUT v_name varchar, OUT v_job varchar) to user3;
grant execute on function func_add_sql(int,int) to user3;
```


HUAWEI

Users and Roles (3)

- Grant permissions on a table.

```
grant select (empno,ename,sal),update (comm) on emp TO user3;
grant select,insert,update,delete on all tables in schema sch1 to user3;
grant all privileges on emp to user3 with grant option;
```

- Grant the **jack** role to **user1**. **user1** can grant permissions to other users.

```
grant jack to user1 with admin option;
```

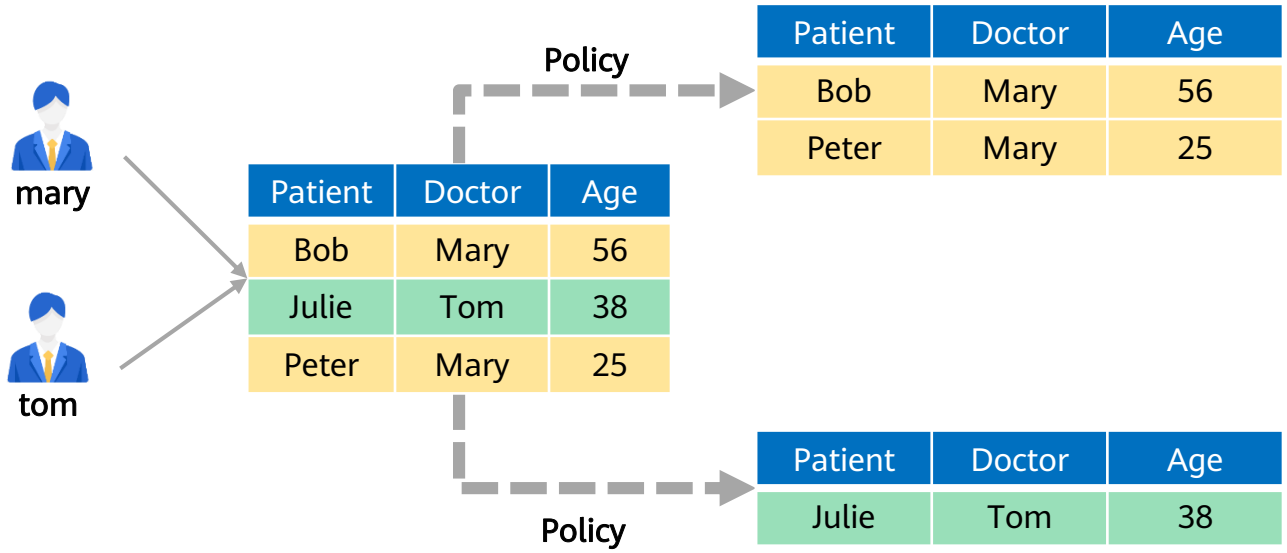
- Revoke permissions.

```
revoke jack from user1;
```


HUAWEI

Row-Level Security

- Row-level security enables database access to be controlled at the row level and allows users to view, update, and delete specific row data.



HUAWEI

Users and Roles (1)

- Database row-level security is a security mechanism used to restrict database users' access to data rows. It allows database administrators or system administrators to define which users can view, modify, or delete specific data rows, enhancing database security and privacy.
- -- Create test data.

```
create user alice password 'gauss@123';
create table all_data(id int, role varchar(100), data varchar(100));
insert into all_data values(1, 'alice', 'alice data');
insert into all_data values(2, 'bob', 'bob data' );
insert into all_data values(3, 'peter', 'peter data');
```


HUAWEI

Users and Roles (2)

- -- Create a row-level security policy.

```
grant select on all_data to alice;
alter table all_data enable row level security;
create row level security policy all_data_rls on all_data using(role = current_user);
```

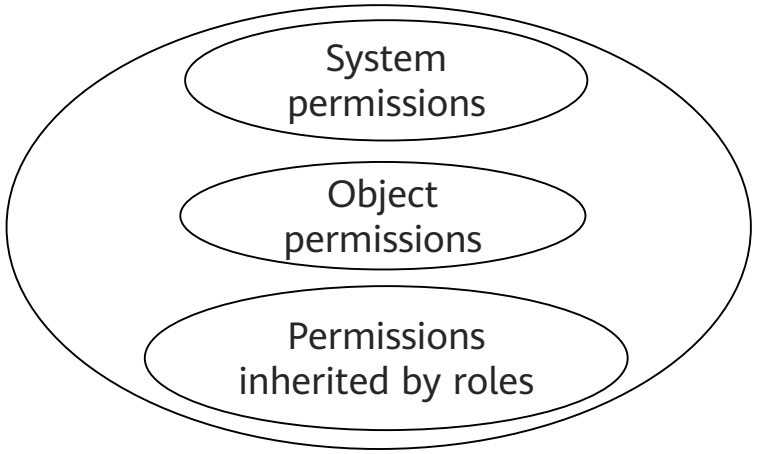
- -- Access the test as user **alice**.

```
\c - alice
select * from all_data;
explain select * from public.all_data;
```


HUAWEI

Permission Management Suggestions

- All permissions of a user or role



- Comply with the principle of least privilege: Assign users only the least permissions required to complete their tasks.
- Revoke permissions from users if the permissions are no longer required.

Contents

1. Client Access Authentication
2. Users and Roles
3. **Database Audit**
4. Encrypted Equality Query
5. Ledger Database
6. TDE

HUAWEI

Database Audit (1)

- openGauss provides the security audit function to monitor and audit the database itself and user behaviors. openGauss audit administrator performs audit-related operations in the audit center.
- Audit principle: The audit affects the database performance. Therefore, the audit scope must be limited to reduce the number of audit events. You need to refer to the following suggestions when formulating audit policies:
  - Specify the audit purpose.
  - Ensure audits as less as possible.
  - Perform audits from large scope to small scope in the following sequence: commands, objects, and users.
  - Protect audit trails.

HUAWEI

Database Audit (2)

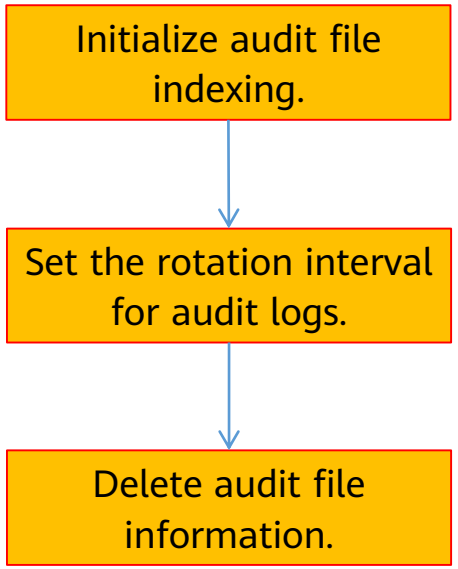
- Audit administrator: Only an audit administrator has the permission to enable and disable the audit function.
- Enabling the audit function: An audit administrator connects to the database server to enable the audit function.
- Disabling the audit function: An audit administrator connects to the database server to disable the audit function.
- Performing audit configurations: Only an audit administrator has the permission to perform audit configurations.
- Clearing audit configurations: openGauss provides the function of clearing outdated audit configurations to delete all audit configuration records from the system catalog.

HUAWEI

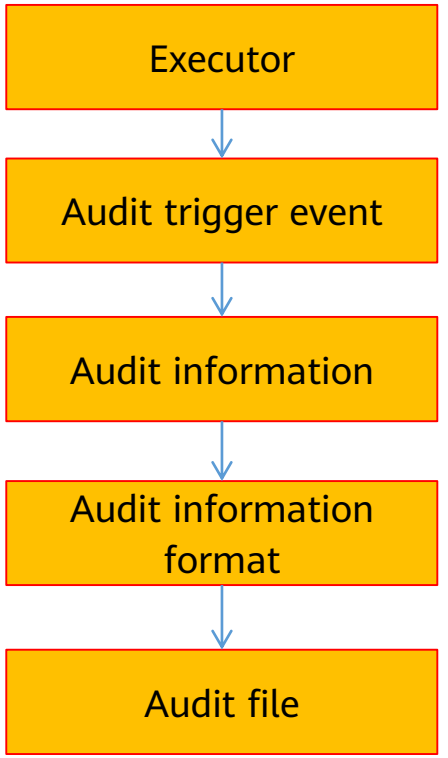
Database Audit - Audit Process

- Audit mechanism
  - Includes managing audit logs, tracing database management and service events, and conducting audits.
- Audit log management
  - Includes creating, rotating, and clearing audit logs.
- Audit log tracing
  - Includes recording events and using an API to query audit information.

Audit log management



Audit logging



Audit Configuration Management - Audit Items (1)

- Set the audit parameter **audit_enabled** to enable or disable system-level audit. **audit_enabled** can be dynamically loaded.
- Each audit configuration item is controlled by an audit parameter. A total of 11 configuration items are provided and can be dynamically loaded.

Configuration Item	Audit Parameter	Description	Default Value
User login and logout audit	audit_login_logout	The default value is **7**, which indicates that auditing user login and logout is enabled. **0** indicates that auditing user login and logout is disabled. Other values are not recommended.	7
Database startup, stop, recovery, and switchover audit	audit_database_process	The default value is **1**, which indicates that auditing database startup, stop, recovery, and switchover is enabled.	1
User locking and unlocking audit	audit_user_locked	The default value is **1**, which indicates that auditing user locking and unlocking is enabled.	1
Unauthorized access audit	audit_user_violation	The default value is **0**, which indicates that auditing unauthorized access is disabled.	0
Permission granting and revoking audit	audit_grant_revoke	The default value is **1**, which indicates that auditing permission granting and revoking is enabled.	1

HUAWEI

Audit Configuration Management - Audit Items (2)

- Modify the parameter value as follows: **gs_guc set -Z coordinator -Z datanode –N all –I all -c "audit_set_parameter = 0"**

Configuration Item	Audit Parameter	Description	Default Value
Audit of CREATE, ALTER, and DROP operations on database objects	audit_system_object	The default value is **12295**, which indicates that only CREATE, ALTER, and DROP operations on databases, schemas, users, data sources, and node groups are audited.	12295
Audit of INSERT, UPDATE, and DELETE operations on a specific table	audit_dml_state	The default value is **0**, which indicates that auditing DML operations (except SELECT) on a specific table is disabled.	0
SELECT audit	audit_dml_state_select	The default value is **0**, which indicates that auditing SELECT operations is disabled.	0
COPY audit	audit_copy_exec	The default value is **1**, which indicates that auditing COPY operations is enabled.	1
Audit of executing stored procedures and user-defined functions	audit_function_exec	The default value is **0**, which indicates that auditing the execution of stored procedures and user-defined functions is disabled.	0
SET audit	audit_set_parameter	The default value is **1**, which indicates that auditing SET operations is enabled.	1

Database Audit - Audit Items

Audit Item	Description
User login or logout	**audit_login_logout**: The default value is **7**, which indicates that auditing user login and logout is enabled. **0** indicates that auditing user login and logout is disabled. Other values are not recommended.
Database startup, stop, recovery, and switchover	**audit_database_process**: The default value is **1**, which indicates that auditing database startup, stop, recovery, and switchover is enabled.
User lock and unlock	**audit_user_locked**: The default value is **1**, indicating that auditing user locking and unlocking is enabled.
Unauthorized user access	**audit_user_violation**: The default value is **0**, which indicates that auditing unauthorized operations is disabled.
Permission granting and revoking	**audit_grant_revoke**: The default value is **1**, which indicates that auditing permission granting and revoking is enabled.

HUAWEI

Database Audit - Examples

- Enable the database audit.

```
gs_guc set -N all -I all -c "audit_enabled=on"
gs_guc reload -N all -I all -c "audit_system_object=12295" -- Audit objects include databases, schemas, users, and data sources.
```

- View audit-related parameters.

```
postgres=# select name,setting,short_desc
from pg_settings
where name like '%audit%';
```

- Query audit information.

```
postgres=# SELECT time,type,result,username,database,object_name,detail_info
FROM pg_query_audit(' '2023-09-18 08:00:00',' '2023-09-18 15:30:00');
```

- Delete audit logs.

```
postgres=# SELECT pg_delete_audit(' '2023-09-18 08:00:00','2023-09-18 15:30:00');
```


HUAWEI

Database Audit - Audit Trail Management

- Audit trails record operations performed on the database based on valid audit configurations. In openGauss, audit trails are recorded in system catalogs. By analyzing audit trails, you can locate the source of suspicious operations in the database.
  - Viewing audit trails: Only audit administrators have the permission to view audit trails. You can directly query the system catalog, which records information such as the executed commands, users who execute the commands, time, and command execution results.
  - Clearing audit trails: After the sources of suspicious operations on the database are identified, the corresponding audit trails do not exist.
- Use audit to investigate the following suspicious operations:
  - Invalid SELECT operations, for example, users who do not have the SELECT permission on the table can perform SELECT on the table.
  - Invalid changes on user passwords.
  - Invalid update of data in the table.
  - Slow database browsing operations and deletion of some table indexes.

HUAWEI

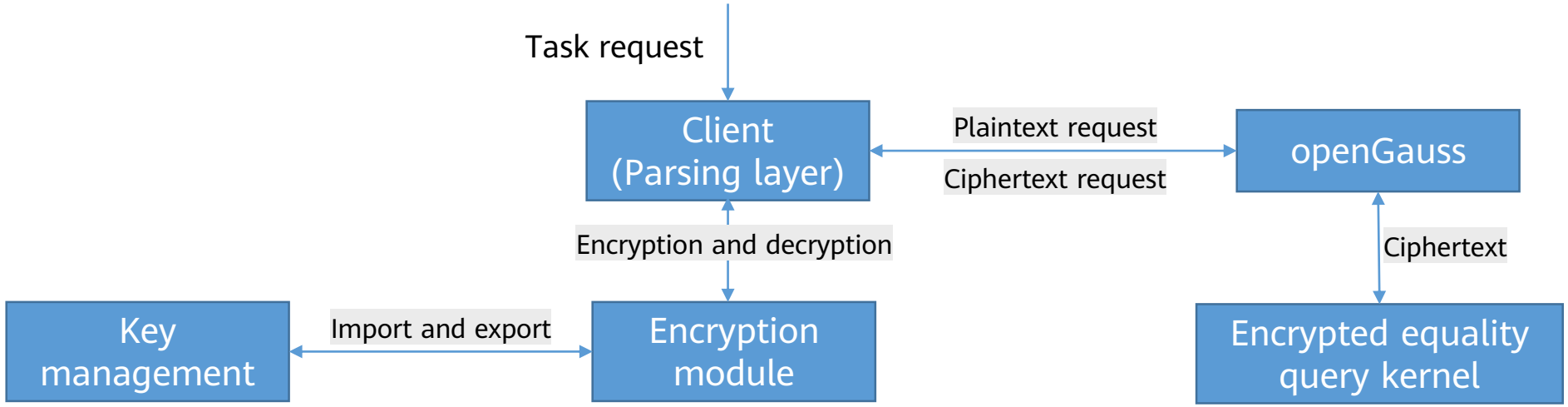
Contents

1. Client Access Authentication
2. Users and Roles
3. Database Audit
4. **Encrypted Equality Query**
5. Ledger Database
6. TDE

HUAWEI

Overall Encrypted Equality Query Solution

- Encrypted equality query belongs to the first phase of the encrypted database solution, but complies with the overall architecture of the encrypted database. The complete form of an encrypted database includes the cryptography solution and the combination of software and hardware.
- Encrypted equality query involves only the software part. Therefore, only the software part of the overall architecture of the encrypted database needs to be integrated. The overall implementation solution is as follows.



Process Analysis (1)

- In the overall process, data is encrypted on the client and sent to the openGauss server in ciphertext. That is, an encryption and decryption module needs to be constructed on the client.
  - The encryption and decryption module depends on the key management module which generates the root key (RK) and client master key (CMK).
  - With the CMK, you can use the SQL syntax to define the column encryption key (CEK). The CMK is encrypted by the RK and stored in the key store file (KSF). The CMK and RK are managed by the keytool in a unified manner.
  - The CEK is encrypted by the CMK and then stored on the server (the symmetric encryption algorithm AES-256 is used) and SM-based algorithm SM2.
- The client uses the symmetric encryption algorithms AES (including AES-128 and AES-256) and SM-based algorithm SM4 to encrypt data based on the generated CEK. The encrypted data is stored on the database server. After the ciphertext calculation, the server returns the ciphertext result set and decrypts the data on the client to obtain the final result.

HUAWEI

Process Analysis (2)

- Users can define encryption attributes for data based on service requirements. Data that does not need to be encrypted is sent to the server in the original plaintext format. After a query task is initiated, the client needs to parse the current query. If the query statement involves encrypted columns, the parameters related to encrypted columns need to be encrypted. (The encryption must be deterministic. Otherwise, the corresponding query cannot be supported.) If no encrypted column is involved in the query statement, the query statement is directly sent to the server, and no additional operation is required.
- On the database server, data in the encrypted columns is always stored in ciphertext, and the entire query is implemented in ciphertext. The first phase of the encrypted equality query solution utilizes deterministic encryption methods, which generates identical ciphertext for identical plaintext data to support equality calculations.
- The encrypted database allows the client to encrypt sensitive data in the client application. During the query, the entire service data flow exists in ciphertext during data processing.

HUAWEI

Feature Description (1)

- From the perspective of users, the encrypted equality query function is divided into three parts, which are implemented by the newly added keytool and the enhanced openGauss gsql client.
- This feature provides the client key management function. Users can use keytool to generate, destroy, and update CMKs, and import and export keys. With the import and export functions of keytool, CMKs can be transmitted between different clients. In addition, the keytool implements key management on a single client. By configuring management files, you can store and update keys.

HUAWEI

Feature Description (2)

- In addition, this feature provides the key creation and encrypted table creation functions. The SQL syntaxes CREATE CLIENT MASTER KEY and CREATE COLUMN ENCRYPTION KEY are added to record and manage CMK and CEK metadata in the database. The CMK and CEK information is recorded in the new system catalog. CREATE TABLE and encryption algorithms are extended to facilitate subsequent data ciphertext storage.
- This feature supports the encrypted equality query function, which is the core of the entire feature. Although users are unaware of the ciphertext query, the query of sensitive data is restricted by the specifications of the current encrypted equality query.

HUAWEI

Restrictions (1)

- Data is encrypted at the column level, and encryption policies cannot be differentiated by row.
- Except the RENAME operation, the ALTER TABLE syntax cannot be used to change encrypted columns (including the conversion between encrypted and unencrypted columns). The ADD and DROP operations can be used to add and delete encrypted columns, respectively.
- The CHECK(COLUMN IS NOT NULL) syntax can be used, but most check syntax cannot be set for encrypted columns.
- When **support_extended_features** is set to **off**, PRIMARY KEY and UNIQUE constraints cannot be used for encrypted columns. When **support_extended_features** is set to **on**, PRIMARY KEY and UNIQUE constraints can be used for deterministically encrypted columns only.
- Different data types cannot be implicitly converted.

HUAWEI

Restrictions (2)

- The collection operation cannot be performed between ciphertexts of different data types.
- No range partition can be created from encrypted columns.
- Only the repeat and empty_blob() functions can be used to operate encrypted columns.
- The current version supports only gsql and JDBC (deployed on a Linux OS) clients. Other clients such as ODBC do not support encrypted equality query.
- When a JDBC client is used, this feature cannot be enabled together with the load balancing or automatic primary node selection feature.
- Data can only be imported to an encrypted table by running the **copy from stdin**, **\copy**, or **insert into values (...)** command on the client.

HUAWEI

Restrictions (3)

- Copying an encrypted table to a file is not supported.
- Except encrypted equality query, other encrypted queries, such as sorting, range query, and fuzzy query are not supported.
- The syntaxes of some encrypted functions and stored procedures are supported.
- Non-encrypted table data cannot be inserted into an encrypted table using the INSERT INTO... SELECT... or MERGE INTO syntax.
- Any change to the CEK information on the server can be detected only after cache update is triggered (for example, the user is changed or the encrypted column fails to be decrypted) and the connection to the server is re-established.
- Encrypted equality query is not supported on columns encrypted using a random encryption algorithm.

HUAWEI

Restrictions (4)

- An error is reported if the two attribute conditions used for comparison in the encrypted equality query use different data encryption keys.
- Encrypted equality query is not supported in time series tables and foreign tables. Ustore is not supported.
- If the database service configuration (such as the pg_settings system catalog, permission, key, and encrypted column) is changed, you need to re-establish a JDBC connection to make the configuration take effect.
- Multiple SQL statements cannot be executed at the same time. This restriction does not apply to the scenario where the INSERT INTO statement is executed in multiple batches.
- The encrypted database does not encrypt empty strings whose length is zero.
- Deterministic encryption is prone to frequency attacks. Therefore, it is not recommended that deterministic encryption be used in scenarios where the plaintext frequency is obviously distributed.

HUAWEI

Using gsql to Operate an Encrypted Database (1)

- Run the following command to enable the encryption function and connect to the encrypted database:

```
gsql -p PORT postgres -r -C
```

- Create a CMK and a CEK.

```
openGauss=# CREATE CLIENT MASTER KEY ImgCMK1 WITH (KEY_STORE = localkms, KEY_PATH =
"key_path_value1", ALGORITHM = RSA_2048);
openGauss=# CREATE CLIENT MASTER KEY ImgCMK WITH (KEY_STORE = localkms, KEY_PATH =
"key_path_value2", ALGORITHM = RSA_2048);
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK1 WITH VALUES (CLIENT_MASTER_KEY = ImgCMK1,
ALGORITHM  = AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK WITH VALUES (CLIENT_MASTER_KEY = ImgCMK,
ALGORITHM  = AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
```


HUAWEI

Using gsql to Operate an Encrypted Database (2)

- Query the system catalog that stores the key information. The result is as follows:

```
openGauss=# SELECT * FROM gs_client_global_keys;
 global_key_name | key_namespace | key_owner | key_acl |        create_date
-----------------+---------------+-----------+---------+---------------------------
 imgcmk1         |          2200 |        10 |         | 2021-04-21 11:04:00.656617
 imgcmk          |          2200 |        10 |         | 2021-04-21 11:04:05.389746
(2 rows)
```


HUAWEI

Using gsql to Operate an Encrypted Database (3)

```
openGauss=# SELECT column_key_name,column_key_distributed_id ,global_key_id,key_owner  FROM gs_column_keys;
 column_key_name | column_key_distributed_id | global_key_id | key_owner
-----------------+---------------------------+---------------+-----------
 imgcek1         |                 760411027 |         16392 |        10
 imgcek          |                3618369306 |         16398 |        10
(2 rows)
```

- Create an encrypted table.

```
openGauss=# CREATE TABLE creditcard_info (id_number    int, name         text encrypted with (column_encryption_key
= ImgCEK, encryption_type = DETERMINISTIC),
credit_card  varchar(19) encrypted with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC));
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id_number' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
```


HUAWEI

Using gsql to Operate an Encrypted Database (4)

- Query the detailed information about a table. If the value of **Modifiers** is **encrypted**, the queried column is encrypted.

```
openGauss=# \d creditcard_info
        Table "public.creditcard_info"
   Column    |       Type        | Modifiers
-------------+-------------------+-----------
 id_number   | integer           |
 name        | text              |  encrypted
 credit_card | character varying |  encrypted
```


HUAWEI

Using gsql to Operate an Encrypted Database (5)

```
openGauss=# INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info VALUES (2, 'joy','6219985678349800033');
INSERT 0 1
openGauss=# select * from creditcard_info where name = 'joe';
 id_number | name |     credit_card
-----------+------+--------------------
         1 | joe  | 6217986500001288393
(1 row)
## Note: The data in the encrypted table is displayed in ciphertext when you use a non-encrypted client to view the data.
openGauss=# select id_number,name from creditcard_info;
 id_number |                                                
```

Using gsql to Operate an Encrypted Database (6)

```
openGauss=# ALTER TABLE creditcard_info ADD COLUMN age int ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY =
ImgCEK, ENCRYPTION_TYPE = DETERMINISTIC);
ALTER TABLE
openGauss=# \d creditcard_info
        Table "public.creditcard_info"
   Column    |       Type        | Modifiers
-------------+-------------------+-----------
 id_number   | integer           |
 name        | text              |  encrypted
 credit_card | character varying |  encrypted
 age         | integer           |  encrypted
openGauss=# ALTER TABLE creditcard_info DROP COLUMN age;
ALTER TABLE
openGauss=# update creditcard_info set credit_card = '800000000111111111' where name = 'joy';
UPDATE 1
openGauss=# select * from creditcard_info  where name = 'joy';
 id_number | name |    credit_card
-----------+------+-------------------
         2 | joy  | 800000000111111111
(1 row)
```


HUAWEI

Contents

1. Client Access Authentication
2. Users and Roles
3. Database Audit
4. Encrypted Equality Query
5. **Ledger Database**
6. TDE

HUAWEI

Ledger Database Definition

- A ledger database records user operations on a specified tamper-proof table and verifies data consistency based on the operation records, ensuring that the user data cannot be maliciously tampered with.
- When a user performs DML operations on a tamper-proof table, the system adds a small amount of additional row-level verification information to the table and records the SQL statements and data changes. The feature provides a verification API for users to check whether the data in the tamper-proof table is consistent with the operation information recorded by the system.
- The ledger database uses the ledger schema to isolate ordinary tables from tamper-proof user tables. If a row-store table created in the ledger schema has the tamper-proof attribute, it is a tamper-proof user table.
  - When data is inserted into a tamper-proof user table, the system automatically generates a small amount of row-level verification information.
  - When a user executes DML operations, the system records user operations in the GS_GLOBAL_CHAIN table and records data changes in the user history table corresponding to the user table. The data in operation records, data change records, and the user history table must be the same.
  - The ledger database provides a high-performance verification API for users to verify data consistency. If the consistency verification fails, the data may be tampered with. In this case, contact the audit administrator to trace the operation history.

HUAWEI

Context

- The ledger database, which integrates a blockchain idea, records a user operation in two types of historical tables: a user history table and a global blockchain table.
  - When a user creates a tamper-proof user table, the system automatically adds a hash column to the table to save the hash digest of each row of data. In blockchain mode, the system creates a user history table to record the change behavior of each data record in the user table.
  - A modification performed by the user on the tamper-proof user table is recorded in the global blockchain table. Because the history table can only be appended and cannot be modified, the records in the history table are concatenated to form the modification history of the tamper-proof user table.

User history table (blockchain.<*schemaname*>_<*tablename*>_hist)

Column	Type	Description
rec_num	bigint	Sequence number of a row-level modification operation in the history table.
hash_ins	hash16	Hash value of the data row inserted by the INSERT or UPDATE operation.
hash_del	hash16	Hash value of the data row deleted by the DELETE or UPDATE operation.
pre_hash	hash32	Summary of the data in the history table of the current user.

Mapping between hash_ins and hash_del

	hash_ins	hash_del
INSERT	(√) Hash value of the inserted row.	Empty.
DELETE	Empty.	(√) Hash value of the deleted row.
UPDATE	(√) Hash value of the newly inserted data.	(√) Hash value of the row before deletion.

Ledger Database Restrictions (1)

- In a tamper-proof schema, row-store tables are tamper-proof, while temporary tables, unlogged tables, column-store tables, time series tables, and Ustore tables are not.
- The structure of tamper-proof user tables cannot be modified. Tamper-proof tables cannot be truncated. A tamper-proof user table cannot be switched to a common schema and a non-tamper-proof table cannot be switched to a tamper-proof schema, either.
- A level-2 partitioned table cannot be specified as a tamper-proof table. If a tamper-proof table is a partitioned table, operations such as EXCHANGE PARTITION, DROP PARTITION, and TRUNCATE PARTITION are not supported.
- Functions and triggers cannot be used to modify data in a tamper-proof user table.
- When a tamper-proof user table is created, no column can be named **hash**.

HUAWEI

Ledger Database Restrictions (2)

- A common user can call the verification API to verify only tables that they have the permission to query.
- Only an audit administrator and the initial user can query the global blockchain table and tables in the BLOCKCHAIN schema. Common users do not have the permission to access and all users do not have the permission to modify the tables.
- According to the naming rules of historical tables, if the name of the schema or table to be created ends or starts with an underscore (_), the name of the corresponding historical table may conflict with that of an existing table. In this case, you need to rename the table.
- Currently, the hash digest of user row-level data in the ledger database is used only to ensure data consistency. It cannot prevent attackers from directly tampering with data files.
- Level-2 partitioned tables are not supported in a tamper-proof schema.
- The index of a table in a tamper-proof schema cannot reference the **hash** column.

HUAWEI

Creating and Operating Tamper-Proof User Tables (1)

- Create tamper-proof schema **ledgernsp**.

```
openGauss=# CREATE SCHEMA ledgernsp WITH BLOCKCHAIN;
```

- Create tamper-proof user table **ledgernsp.usertable**.

```
openGauss=# CREATE TABLE ledgernsp.usertable(id int, name text);
```

- Check the structure of the tamper-proof user table and the corresponding user history table.

```
openGauss=# \d+ ledgernsp.usertable;
                    Table "ledgernsp.usertable"
 Column |  Type   | Modifiers | Storage  | Stats target | Description
--------+---------+-----------+----------+--------------+-------------
 id     | integer |           | plain    |              |
 name   | text    |           | extended |              |
 hash   | hash16  |           | plain    |              |
Has OIDs: no
Options: orientation=row, compression=no
History table name: ledgernsp_usertable_hist
```


HUAWEI

Creating and Operating Tamper-Proof User Tables (2)

```
openGauss=# \d+ blockchain.ledgernsp_usertable_hist;
            Table "blockchain.ledgernsp_usertable_hist"
  Column  |  Type  | Modifiers | Storage | Stats target | Description
----------+--------+-----------+---------+--------------+-------------
 rec_num  | bigint |           | plain   |              |
 hash_ins | hash16 |           | plain   |              |
 hash_del | hash16 |           | plain   |              |
 pre_hash | hash32 |           | plain   |              |
Indexes:
    "gs_hist_16388_index" PRIMARY KEY, btree (rec_num int4_ops) TABLESPACE pg_default
Has OIDs: no
Options: internal_mask=263
```

- Modify the data in the tamper-proof user table. Execute **INSERT**, **UPDATE**, and **DELETE** on the tamper-proof user table.

```
openGauss=# INSERT INTO ledgernsp.usertable VALUES(1, 'alex'), (2, 'bob'), (3, 'peter');
INSERT 0 3
openGauss=# SELECT *, hash FROM ledgernsp.usertable ORDER BY id;
 id | name  |       hash
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob   | 8fcd74a8a6a4b484
  3 | peter | f51b4b1b12d0354b
(3 rows)
```


HUAWEI

Creating and Operating Tamper-Proof User Tables (3)

```
openGauss=# UPDATE ledgernsp.usertable SET name = 'bob2' WHERE id = 2;
UPDATE 1
openGauss=# SELECT *, hash FROM ledgernsp.usertable ORDER BY id;
 id | name  |       hash
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob2  | 437761affbb7c605
  3 | peter | f51b4b1b12d0354b
(3 rows)

openGauss=# DELETE FROM ledgernsp.usertable WHERE id = 3;
DELETE 1
openGauss=# SELECT *, hash FROM ledgernsp.usertable ORDER BY id;
 id | name |       hash
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```


HUAWEI

Viewing the Operation History of a Ledger Database (1)

- Prerequisites
  - You are an audit administrator or a role that has the audit administrator permissions.
  - The database is running properly, and a series of addition, deletion, and modification operations have been performed on the tamper-proof database to ensure that operation records are generated in the ledger for query.
- Procedure
  - View historical operation records in the ledger as a user with the AUDITADMIN attribute.
  - Query the global blockchain table GS_GLOBAL_CHAIN.

HUAWEI

Viewing the Operation History of a Ledger Database (2)

```
openGauss=# SELECT * FROM gs_global_chain;
blocknum |  dbname  | username |          starttime          | relid |  relnsp   |  relname  |     relhash      |
globalhash            |
                txcommand
0 | postgres | omm      | 2021-04-14 07:00:46.32757+08  | 16393 | ledgernsp | usertable | a41714001181a294 |
6b5624e039e8aee36bff3e8295c75b40 | insert into ledge
rnsp.usertable values(1, 'alex'), (2, 'bob'), (3, 'peter');
       1 | postgres | omm      | 2021-04-14 07:01:19.767799+08 | 16393 | ledgernsp | usertable | b3a9ed0755131181 |
328b48c4370faed930937869783c23e0 | update ledgernsp.
usertable set name = 'bob2' where id = 2;
       2 | postgres | omm      | 2021-04-14 07:01:29.896148+08 | 16393 | ledgernsp | usertable | 0ae4b4e4ed2fcab5 |
aa8f0a236357cac4e5bc1648a739f2ef | delete from ledge
rnsp.usertable where id = 3;
```


HUAWEI

Viewing the Operation History of a Ledger Database (3)

- The query result indicates that user **omm** has consecutively executed three DML commands, including **INSERT**, **UPDATE**, and **DELETE**.
- Query the user history table in the BLOCKCHAIN schema. For example, the schema of the user table is **ledgernsp**, the table name is **usertable**, and the name of the corresponding user history table is **blockchain.ledgernsp_usertable_hist**.

```
openGauss=# SELECT * FROM blockchain.ledgernsp_usertable_hist;

rec_num |     hash_ins     |     hash_del     |            pre_hash
---------+------------------+------------------+--------------------------------
       0 | 1f2e543c580cb8c5 |                  | e1b664970d925d09caa295abd38d9b35
       1 | 8fcd74a8a6a4b484 |                  | dad3ed8939a141bf3682043891776b67
       2 | f51b4b1b12d0354b |                  | 53eb887fc7c4302402343c8914e43c69
       3 | 437761affbb7c605 | 8fcd74a8a6a4b484 | c2868c5b49550801d0dbbbaa77a83a10
       4 |                  | f51b4b1b12d0354b | 9c512619f6ffef38c098477933499fe3
```

- The query result shows that user **omm** inserts three rows of data to the **ledgernsp.usertable** table, updates one row of data, deletes one row of data, and leaves two rows of data, and the hash values are **1f2e543c580cb8c5** and **437761affbb7c605**.

Viewing the Operation History of a Ledger Database (4)

- Query user table data and hash verification columns.

```
openGauss=# SELECT *, hash FROM ledgernsp.usertable;
```

- The query result is as follows:

```
 id | name |       hash
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```

- The query result indicates that the remaining two records in the user table are the same as those in the previous step.

Checking Data Consistency of a Ledger Database (1)

- Currently, the ledger database provides two verification APIs: ledger_hist_check(text, text) and ledger_gchain_check(text, text). When a common user calls a verification API, only the tables that the user has the permission to access can be verified.
  - The pg_catalog.ledger_hist_check API is used to verify the tamper-proof user table and user history table. The API call returns **t** if the verification is passed, and **f** if it is not.
  - The pg_catalog.ledger_gchain_check API is used to verify the consistency between the tamper-proof user table, user history table, and global blockchain table. The API call returns **t** if the verification is passed, and **f** if it is not.

HUAWEI

Checking Data Consistency of a Ledger Database (2)

- Check whether the tamper-proof user table **ledgernsp.usertable** is consistent with the corresponding user history table.
  - The query result shows that the results recorded in the tamper-proof user table and user history table are consistent.
- Check whether the records in the tamper-proof table **ledgernsp.usertable** are the same as those in the corresponding user history table and global blockchain table.
  - The query result shows that the records of **ledgernsp.usertable** in the preceding three tables are consistent and no tampering occurs.

```
openGauss=# SELECT
pg_catalog.ledger_hist_check('ledgernsp',
'usertable');
ledger_hist_check
------------------
 t
(1 row)
```

```
openGauss=#  SELECT
pg_catalog.ledger_gchain_check('ledgernsp',
'usertable');
ledger_gchain_check
--------------------
 t
(1 row)
```


HUAWEI

Archiving a Ledger Database (1)

- There are two APIs for verifying the ledger database archive result: ledger_hist_archive and ledger_gchain_archive. Only audit administrators have the permission to call the ledger database APIs.
  - pg_catalog.ledger_hist_archive is used to verify the archive of the user history table. The API call returns **t** if the archive is successful, and **f** if it is not.
  - pg_catalog.ledger_gchain_archive is used to verify the archive of the global blockchain table. The API call returns **t** if the archive is successful, and **f** if it is not.

HUAWEI

Archiving a Ledger Database (2)

- Archive a specified user history table.

```
openGauss=# SELECT pg_catalog.ledger_hist_archive('ledgernsp', 'usertable');
ledger_hist_archive
--------------------
 t
(1 row)
```

- The user history table is archived to one record. The result indicates that the user history table on the current node is exported successfully.

```
openGauss=# SELECT * FROM blockchain.ledgernsp_usertable_hist;
 rec_num |     hash_ins     |     hash_del     |             pre_hash
---------+------------------+------------------+----------------------------------
       3 | e78e75b00d396899 | 8fcd74a8a6a4b484 |
fd61cb772033da297d10c4e658e898d7
(1 row)
```


HUAWEI

Archiving a Ledger Database (3)

- Export the global blockchain table.

```
openGauss=# SELECT pg_catalog.ledger_gchain_archive();
 ledger_gchain_archive
----------------------
 t
(1 row)
```

- The global history table will be archived to *N* (number of user tables) data records by user table.

```
openGauss=# SELECT * FROM gs_global_chain;
 blocknum |  dbname  | username |          starttime          | relid |  relnsp   |  relname  |     relhash      |
globalhash            | txcommand
----------+----------+----------+-----------------------------+-------+-----------+-----------+------------------+--------
-------------------------+-----------
        1 | postgres | libc     | 2021-05-10 19:59:38.619472+08 | 16388 | ledgernsp | usertable |
57c101076694b415 | be82f98ee68b2bc4e375f69209345406 | Archived.
(1 row)
```

- The result indicates that the global blockchain table of the current node is successfully exported.

HUAWEI

Repairing a Ledger Database (1)

- When an exception occurs or a table is damaged, you need to use the ledger_gchain_repair(text, text) or ledger_hist_repair(text, text) API in a ledger database to restore the global blockchain table or user history table. After the restoration, the result of calling the global blockchain table or user history table verification API is **true**.
  - The API for repairing the user history table is pg_catalog.ledger_hist_repair. If the repair is successful, the API call returns the hash increment of the user history table during the repair.
  - The API for archiving the global blockchain table is pg_catalog.ledger_gchain_repair. If the repair is successful, the API call returns the hash increment of the global blockchain table during the repair.

HUAWEI

Repairing a Ledger Database (2)

- Repair a specified user history table. The query result indicates that the user history table on the current node is successfully repaired. The hash increment of the user history table is **84e8bfc3b974e9cf**.

```
openGauss=# SELECT pg_catalog.ledger_hist_repair('ledgernsp', 'usertable');
ledger_hist_repair
-------------------
 84e8bfc3b974e9cf
(1 row)
```

- kchain table is successfully repaired and a piece of repair data is inserted. The hash value is **a41714001181a294**.

```
openGauss=# SELECT pg_catalog.ledger_gchain_repair('ledgernsp', 'usertable');
ledger_gchain_repair
---------------------
 a41714001181a294
(1 row)
```


HUAWEI

Contents

1. Client Access Authentication
2. Users and Roles
3. Database Audit
4. Encrypted Equality Query
5. Ledger Database
6. **TDE**

HUAWEI

Definition and Benefits

- Definition: Transparent data encryption (TDE) is used to encrypt data when the database writes the data to the storage media and automatically decrypts the data when reading the data from the storage media. This prevents attackers from reading data in the data file without database authentication, solving the static data leakage problem. This function is almost transparent to the application layer. You can determine whether to enable the TDE function as required.
- Benefits:
  - To prevent attackers from reading data files without authentication, you can use the TDE function to encrypt data files in the database. This ensures that users can read decrypted data only after starting and connecting to a database properly.
- Dependencies:
  - The key management service is provided by the external KMS. Currently, Huawei Cloud KMS is supported.

HUAWEI

Key Management Mechanism

- Management mechanism:
  - In TDE, data encryption and decryption depend on a secure and reliable key management mechanism. The three-layer key structure is used to implement the key management mechanism, including the RK, CMK, and DEK. CMKs are encrypted and protected by RKs, and DEKs are encrypted and protected by CMKs.
  - DEKs are used to encrypt and decrypt user data. Each table corresponds to a DEK.
  - RKs and CMKs are stored in KMS. You can send a request to the KMS for creating DEKs. After the creation is successful, the key plaintext and ciphertext are returned.
  - The DEK plaintext is cached in a hash table in the memory to reduce the KMS access frequency and improve performance. The key plaintext is stored only in the memory and is not flushed to disks. In addition, the key plaintext that is not frequently used can be automatically deleted. Only the key plaintext used in the last one day is stored.
  - The DEK ciphertext is stored in the database and flushed to the disk for persistence. When encrypting or decrypting user table data, if the corresponding key plaintext does not exist in the memory, send a request to the KMS for decrypting the DEK before using it.

HUAWEI

Table-Level Encryption Solution

- Table-level encryption is supported. When creating a table, you can specify whether to encrypt the table and the encryption algorithm to be used. The encryption algorithm can be AES_128_CTR or SM4_CTR, which cannot be changed once specified. If an encrypted table is created, the database automatically applies for a DEK for the table and saves the encryption algorithm, key ciphertext, and corresponding CMK ID in the **reloptions** column of the pg_class system catalog in keyword=value format.
- You can switch an encrypted table to a non-encrypted table or switch a non-encrypted table to an encrypted table. If the encryption function is not enabled when a table is created, the table cannot be switched to an encrypted table.
- For encrypted tables, DEK rotation is supported. After the key rotation, the data encrypted using the old key is decrypted using the old key, and the newly written data is encrypted using the new key. The encryption algorithm is not changed during key rotation.
- For a row-store table, the minimum data unit for each encryption and decryption is an 8 KB page. Each time the page is encrypted, an information value (IV) is generated through the secure random number API, and the IV, key ciphertext, and CMK ID are stored on the page and written to the storage media. For an encrypted table, the encryption key information needs to be saved on the page. The occupied storage space increases by about 2.5% compared with that when the table is not encrypted.

HUAWEI

Restrictions (1)

- Currently, Huawei Cloud KMS is used for table-level key storage and row-store table encryption. The specifications are as follows:
  - Encryption of a row-store table stored as a heap is supported.
  - Column-store encryption, materialized view encryption, and Ustore-based encryption are not supported.
  - Indexes, sequences, Xlogs, MOTs, and system catalogs cannot be encrypted.
  - You can specify an encryption algorithm when creating a table. Once specified, the encryption algorithm cannot be changed. If **enable_tde** is set to **on** but the encryption algorithm **encrypt_algo** is not specified when a table is created, the AES_128_CTR encryption algorithm is used by default.

HUAWEI

Restrictions (2)

- If the encryption function is not enabled or the encryption algorithm is not specified when a table is created, the table cannot be switched to an encrypted table.
- For a table that has been assigned an encryption key, switching between the encrypted and unencrypted table states does not change the key or encryption algorithm.
- Data key rotation is supported only when the table encryption function is enabled.
- Cross-region primary/standby synchronization of multiple copies in a single cluster is not supported. Cross-region scale-out of a single cluster is not supported. Cross-region backup and restoration, cluster DR, and data migration are not supported.
- In hybrid cloud scenarios, if the Huawei Cloud KMS and management plane functions are used, TDE is supported. For other KMS services, TDE is not supported if no compatible API is available.
- The query performance of encrypted tables is lower than that of non-encrypted tables. If high performance is required, exercise caution when enabling the encryption function.

HUAWEI

Parameters

- **enable_tde**
  - Description: Specifies whether to enable the TDE function. You need to set this parameter to **on** before creating an encrypted table. If this parameter is set to **off**, encrypted tables cannot be created. The created encrypted table is decrypted only when data is read and is not encrypted when the data is written.
- **tde_cmk_id**
  - Description: Specifies the ID of the database instance CMK used by the TDE function, which is generated by KMS. The CMK of the database instance is used to encrypt the DEK. When the DEK needs to be decrypted, a request packet needs to be sent to KMS. The DEK ciphertext and the ID of the corresponding CMK are sent to KMS.

HUAWEI

Quiz

1. (True or False) For encrypted equality query, the data in encrypted columns always exists in ciphertext, and the entire query is also implemented in ciphertext. Non-deterministic encryption is required for the solution in the first phase.

   A. True

   B. False

HUAWEI

Summary

- After learning this chapter, you will be proficient in configuring parameters related to client access authentication, generating and replacing certificates, master concepts related to separation of duties, setting user and role permissions, setting security policies, and performing database audit operations (including viewing, deleting, and backing up audit logs), be proficient in using gsql to perform encrypted equality query, master related theories and concepts, and understand the working principles of ledger databases and TDE.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/en/
- openGauss Help (5.0.0)
  - https://docs-opengauss.osinfra.cn/en/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI